

LocalConnect Enterprise Software

Architecture Document (ESA)

Version: 1.0.0 | Date: 2025-07-04 | Status: Draft for ARB Approval

Authors: Principal Software Architect, Enterprise Solution Architect, Cloud Architect, DevOps Architect, Security Architect, Staff Software Engineer

Target Audience: C-Level Executives, CTO, VP Engineering, Principal Engineers, Solution Architects, Security Auditors, Infrastructure Leads, Product Owners, DevOps Leads, QA Leads

Project: LocalConnect — Community Super-App Platform

Scale Target: 100 Million Users, 10 Million DAU, Millions of Concurrent Requests, Zero Downtime, Multi-Region Active-Active

Primary Regions: Sri Lanka (Phase 1) → South Asia (Phase 2) → Worldwide (Phase 3)

Cloud Strategy: Multi-Cloud Portable (AWS Primary, Azure DR, GCP Analytics)

Security Standard: ISO 27001, SOC 2 Type II, PCI DSS Level 1, NIST 800-53, OWASP Top 10

Version History

Version	Date	Author	Change Description
0.1.0	2025-06-01	Architecture Team	Initial draft — Cover, Executive Summary, Business Overview
0.2.0	2025-06-15	Architecture Team	Added DDD, C4 diagrams, Microservices breakdown
0.3.0	2025-06-25	Architecture Team	Added Security, Infrastructure, Kubernetes, DevOps chapters

1.0.0	2025-07-04	Architecture Team	Complete ESAD v1.0 — All chapters reviewed, production-ready
-------	------------	-------------------	--

Table of Contents

Executive Summary

Business Overview

Business Goals

Functional Requirements

Non-Functional Requirements

Domain-Driven Design

C4 Architecture

Complete Microservice Architecture

Database Architecture

Event-Driven Architecture

API Gateway

Security Architecture

Infrastructure Architecture

Kubernetes Architecture

DevOps & CI/CD

Observability

AI Architecture

Mobile Architecture

Web Architecture

Performance Optimization

Disaster Recovery

Scalability Plan

Cost Estimation

Development Roadmap

Coding Standards

Appendix

Diagrams

Chapter 9: Database Architecture

9.1 Polyglot Persistence Strategy

LocalConnect uses a polyglot persistence strategy, selecting the best database for each data type and access pattern.

9.1.1 Database Selection Matrix

Data Type	Primary DB	Secondary DB	Cache	Search	Analytics
User Profiles	PostgreSQL	DynamoDB	Redis	Elasticsearch	ClickHouse
Business Listings	PostgreSQL	MongoDB	Redis	Elasticsearch	ClickHouse
Events	PostgreSQL	MongoDB	Redis	Elasticsearch	ClickHouse
Forum Posts	PostgreSQL	MongoDB	Redis	Elasticsearch	ClickHouse

Orders	PostgreSQL	DynamoDB	Redis	—	ClickHouse
Payments	PostgreSQL	DynamoDB	Redis	—	ClickHouse
Inventory	PostgreSQL	DynamoDB	Redis	—	ClickHouse
Emergency Alerts	DynamoDB	PostgreSQL	Redis	—	ClickHouse
Notifications	PostgreSQL	MongoDB	Redis	—	ClickHouse
Media/Files	S3	CloudFront	—	—	—
Analytics	ClickHouse	S3	Redis	—	—
Sessions	Redis	PostgreSQL	—	—	—
Config/Flags	PostgreSQL	Redis	Redis	—	—
Audit Logs	S3	DynamoDB	—	—	ClickHouse
Search Index	Elasticsearch	—	Redis	—	—
Graph Data	Neo4j	—	Redis	—	—
Time-Series	TimescaleDB	—	—	—	ClickHouse

9.2 PostgreSQL Architecture

9.2.1 Cluster Design

- Primary: 1 write primary (db.r6g.4xlarge, 16 vCPU, 128GB RAM)
- Read Replicas: 10 read replicas across 3 availability zones (db.r6g.2xlarge)
- Standby: 1 hot standby in DR region (db.r6g.4xlarge)
- Connection Pooling: PgBouncer (transaction mode, 1000 max connections per instance)
- Partitioning: Time-based partitioning for audit logs, event logs, order history
- Sharding: Application-level sharding by user_id hash for user data (10 shards)

- Backup: Automated daily snapshots, point-in-time recovery (35 days), cross-region backup
- Monitoring: RDS Performance Insights, pg_stat_statements, custom dashboards

9.2.2 Schema Design Principles

- Normalization: 3NF for transactional data, denormalized views for read-heavy queries
- Indexes: B-tree for primary/foreign keys, GiST for geospatial, GIN for full-text search, BRIN for time-series
- Constraints: Foreign keys, check constraints, unique constraints, not-null constraints
- Triggers: Audit triggers, updated_at triggers, validation triggers
- Functions: Stored procedures for complex transactions, custom functions for business logic
- Views: Materialized views for dashboards, regular views for API abstraction
- JSONB: Flexible schema for semi-structured data (user preferences, form data)

9.2.3 Key Tables

-- Users Domain

CREATE TABLE users (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

email VARCHAR(255) UNIQUE NOT NULL,

phone VARCHAR(20) UNIQUE,

password_hash VARCHAR(255),

status VARCHAR(20) DEFAULT 'pending' CHECK (status IN ('pending', 'active', 'suspended', 'deleted')),

email_verified BOOLEAN DEFAULT FALSE,

phone_verified BOOLEAN DEFAULT FALSE,

created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

deleted_at TIMESTAMP WITH TIME ZONE,

shard_key INT GENERATED ALWAYS AS (hashtext(id::text) % 10) STORED

);

CREATE TABLE profiles (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

user_id UUID REFERENCES users(id) ON DELETE CASCADE,

display_name VARCHAR(100) NOT NULL,

bio TEXT,

avatar_url VARCHAR(500),

location GEOGRAPHY(POINT, 4326),

languages VARCHAR(10)[],

privacy_settings JSONB DEFAULT '{}',

created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()

);

CREATE INDEX idx_users_email ON users(email);

CREATE INDEX idx_users_phone ON users(phone);

CREATE INDEX idx_users_status ON users(status) WHERE deleted_at IS NULL;

CREATE INDEX idx_users_created_at ON users(created_at);

CREATE INDEX idx_profiles_user_id ON profiles(user_id);

CREATE INDEX idx_profiles_location ON profiles USING GIST(location);

-- Business Domain

CREATE TABLE businesses (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

owner_id UUID REFERENCES users(id),

name VARCHAR(255) NOT NULL,

description TEXT,

category_id UUID REFERENCES categories(id),

location GEOGRAPHY(POINT, 4326),

address JSONB,

contact_info JSONB,

status VARCHAR(20) DEFAULT 'pending',

verification_status VARCHAR(20) DEFAULT 'unverified',

claimed_at TIMESTAMP WITH TIME ZONE,

```
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()

);

CREATE INDEX idx_businesses_owner ON businesses(owner_id);

CREATE INDEX idx_businesses_category ON businesses(category_id);

CREATE INDEX idx_businesses_location ON businesses USING GIST(location);

CREATE INDEX idx_businesses_status ON businesses(status) WHERE deleted_at
IS NULL;

CREATE INDEX idx_businesses_name ON businesses USING GIN(name
gin_trgm_ops);

-- Events Domain

CREATE TABLE events (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

organizer_id UUID REFERENCES users(id),

title VARCHAR(255) NOT NULL,

description TEXT,

location GEOGRAPHY(POINT, 4326),

address JSONB,

start_time TIMESTAMP WITH TIME ZONE NOT NULL,

end_time TIMESTAMP WITH TIME ZONE,

category VARCHAR(50),
```



```
capacity INT,

status VARCHAR(20) DEFAULT 'draft',

ticket_types JSONB,

created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()

);

CREATE INDEX idx_events_organizer ON events(organizer_id);

CREATE INDEX idx_events_location ON events USING GIST(location);

CREATE INDEX idx_events_start_time ON events(start_time);

CREATE INDEX idx_events_status ON events(status);

CREATE INDEX idx_events_category ON events(category);

-- Orders Domain

CREATE TABLE orders (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

user_id UUID REFERENCES users(id),

business_id UUID REFERENCES businesses(id),

status VARCHAR(20) DEFAULT 'pending',

total_amount DECIMAL(10, 2) NOT NULL,

currency VARCHAR(3) DEFAULT 'LKR',

shipping_address JSONB,
```

```
payment_status VARCHAR(20) DEFAULT 'pending',

created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()

) PARTITION BY RANGE (created_at);

CREATE TABLE orders_2025_01 PARTITION OF orders

FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');

CREATE TABLE orders_2025_02 PARTITION OF orders

FOR VALUES FROM ('2025-02-01') TO ('2025-03-01');

-- Additional partitions created monthly via automation

CREATE INDEX idx_orders_user ON orders(user_id);

CREATE INDEX idx_orders_business ON orders(business_id);

CREATE INDEX idx_orders_status ON orders(status);

CREATE INDEX idx_orders_created_at ON orders(created_at);

-- Emergency Domain

CREATE TABLE emergency_alerts (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

alert_type VARCHAR(50) NOT NULL,

priority VARCHAR(20) NOT NULL,

geofence GEOGRAPHY(POLYGON, 4326),

message TEXT NOT NULL,
```

```

media_urls JSONB,

sender_id UUID REFERENCES users(id),

status VARCHAR(20) DEFAULT 'active',

acknowledged_count INT DEFAULT 0,

created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

expires_at TIMESTAMP WITH TIME ZONE

);

CREATE INDEX idx_emergency_alerts_type ON emergency_alerts(alert_type);

CREATE INDEX idx_emergency_alerts_priority ON emergency_alerts(priority);

CREATE INDEX idx_emergency_alerts_geofence ON emergency_alerts USING
GIST(geofence);

CREATE INDEX idx_emergency_alerts_status ON emergency_alerts(status)
WHERE status = 'active';

CREATE INDEX idx_emergency_alerts_created_at ON
emergency_alerts(created_at);

```

9.3 MongoDB Architecture

9.3.1 Cluster Design

- Deployment: MongoDB Atlas M60 cluster (3-node replica set, 32GB RAM, 16 vCPU per node)
- Sharding: 10 shards for high-volume collections (reviews, forum posts, messages)
- Shard Key: Hashed user_id for even distribution
- Read Preferences: Nearest for read-heavy workloads, primary for writes

- Write Concern: Majority for critical data, 1 for analytics
- Read Concern: Majority for critical reads, available for analytics
- Backup: Continuous backup, point-in-time recovery (7 days), cross-region snapshots
- Monitoring: MongoDB Atlas monitoring, custom Prometheus exporter

9.3.2 Key Collections

// Reviews Collection

```
{  
  
  _id: ObjectId("..."),  
  
  user_id: UUID("..."),  
  
  business_id: UUID("..."),  
  
  rating: 4.5,  
  
  title: "Great service!",  
  
  content: "The food was amazing and the staff was friendly.",  
  
  photos: ["https://cdn.localconnect.com/...", "https://cdn.localconnect.com/..."],  
  
  helpful_count: 15,  
  
  report_count: 0,  
  
  status: "approved",  
  
  moderation_score: 0.95,  
  
  created_at: ISODate("2025-07-01T10:00:00Z"),  
  
  updated_at: ISODate("2025-07-01T10:00:00Z"),
```

```
location: { type: "Point", coordinates: [79.8612, 6.9271] }
```

```
}
```

```
// Forum Posts Collection
```

```
{
```

```
  _id: ObjectId("..."),
```

```
  forum_id: UUID("..."),
```

```
  user_id: UUID("..."),
```

```
  title: "Community cleanup this Saturday",
```

```
  content: "Join us for the monthly community cleanup...",
```

```
  post_type: "announcement",
```

```
  tags: ["community", "cleanup", "volunteer"],
```

```
  upvotes: 42,
```

```
  downvotes: 2,
```

```
  comment_count: 15,
```

```
  view_count: 1200,
```

```
  is_anonymous: false,
```

```
  status: "active",
```

```
  created_at: ISODate("2025-07-01T08:00:00Z"),
```

```
  updated_at: ISODate("2025-07-01T12:00:00Z")
```

```
}
```

```
// Messages Collection
```

```
{  
  
  _id: ObjectId("..."),  
  
  conversation_id: UUID("..."),  
  
  sender_id: UUID("..."),  
  
  content: "Hey, are you coming to the event?",  
  
  content_type: "text",  
  
  attachments: [],  
  
  read_by: [UUID("...")],  
  
  deleted_for: [],  
  
  created_at: ISODate("2025-07-01T14:00:00Z"),  
  
  updated_at: ISODate("2025-07-01T14:00:00Z")  
  
}
```

```
// Analytics Events Collection
```

```
{  
  
  _id: ObjectId("..."),  
  
  event_type: "page_view",  
  
  user_id: UUID("..."),  
  
  session_id: UUID("..."),  
  
  page: "/business/12345",  
  
}
```

```
referrer: "https://google.com",  
  
device: { type: "mobile", os: "iOS 17", app_version: "2.1.0" },  
  
location: { country: "LK", city: "Colombo" },  
  
timestamp: ISODate("2025-07-01T10:00:00Z"),  
  
metadata: { business_id: UUID("..."), category: "restaurant" }  
  
}
```

9.4 Redis Architecture

9.4.1 Cluster Design

- Deployment: Redis Cluster (6 nodes, 3 masters + 3 replicas, ElastiCache for Redis)
- Node Type: cache.r6g.xlarge (4 vCPU, 26GB RAM per node)
- Shards: 3 shards, 1 replica per shard
- Eviction Policy: allkeys-lru for cache, noeviction for sessions
- Persistence: AOF every 1 second for critical data, RDB snapshot every 6 hours
- Backup: Daily snapshots to S3, cross-region replication for DR
- Monitoring: Redis INFO metrics, CloudWatch, custom dashboards

9.4.2 Key Patterns

Sessions

session:{session_id} -> Hash { user_id, expires_at, device_info, ip_address }

TTL: 7 days (refreshable)

User Profiles (cache)

user:profile:{user_id} -> Hash { display_name, avatar, location, ... }

TTL: 1 hour (refresh on miss)

Rate Limiting

rate_limit:{user_id}:{action} -> String (counter)

TTL: 1 minute (per action window)

Business Listings (cache)

business:list:{category}:{location_hash} -> Sorted Set (by distance)

TTL: 30 minutes

Shopping Carts

cart:{user_id} -> Hash { product_id, quantity, ... }

TTL: 30 days

Real-Time Leaderboards (forums, events)

forum:leaderboard:{forum_id} -> Sorted Set (by reputation)

leaderboard:global -> Sorted Set (by karma)

Event Check-In

event:checkin:{event_id} -> Hash { attendee_id, checkin_time, status }

TTL: 7 days

Emergency Alert Status

emergency:alert:{alert_id}:status -> Hash { sent_count, delivered_count, acknowledged_count }

TTL: 30 days

Feature Flags

feature_flag:{flag_name} -> String (enabled/disabled + rollout percentage)

TTL: No expiration (updated via pub/sub)

API Response Caching

api:cache:{endpoint_hash}:{params_hash} -> String (JSON response)

TTL: 5 minutes (configurable per endpoint)

Distributed Locks

lock:{resource_name} -> String (owner_id + expiration)

TTL: 30 seconds (auto-release)

Pub/Sub Channels

notifications:push -> Channel (real-time push notifications)

forum:updates:{forum_id} -> Channel (new posts, comments)

event:live:{event_id} -> Channel (live event updates)

emergency:alerts -> Channel (critical emergency broadcasts)

9.5 DynamoDB Architecture

9.5.1 Table Design

Table	Partition Key	Sort Key	Global Secondary Index	Use Case
-------	---------------	----------	------------------------	----------

emergency_alerts	alert_id	created_at	alert_type + priority	High-throughput emergency alerts
sos_records	sos_id	created_at	user_id + status	SOS activations
order_transactions	order_id	timestamp	user_id + status	High-throughput orders
payment_transactions	payment_id	timestamp	order_id + status	Payment processing
audit_logs	entity_type	timestamp	user_id + action	Immutable audit trail
notification_delivery	user_id	notification_id	channel + status	Delivery tracking
feature_usage	user_id	timestamp	feature_name + count	Usage analytics

9.5.2 Capacity Planning

- On-Demand Capacity: For emergency tables (auto-scales to millions of requests)
- Provisioned Capacity: For predictable workloads (auto-scaling enabled)
- DAX (DynamoDB Accelerator): For sub-millisecond read latency on critical tables
- Global Tables: Multi-region replication for emergency data and user profiles
- TTL: 90 days for non-critical data, 7 years for audit logs

9.5.3 Example Item

```
{
```

```
"alert_id": "emergency-12345",

"created_at": "2025-07-01T10:00:00Z",

"alert_type": "natural_disaster",

"priority": "critical",

"geofence": {

  "type": "Polygon",

  "coordinates": [[[79.8, 6.9], [79.9, 6.9], [79.9, 7.0], [79.8, 7.0], [79.8, 6.9]]]

},

"message": "Flash flood warning for Colombo district. Move to higher ground immediately.",

"sender_id": "gov-12345",

"status": "active",

"acknowledged_count": 150000,

"target_user_count": 500000,

"delivery_channels": ["push", "sms", "in_app"],

"ttl": 1754062800

}
```

9.6 Elasticsearch Architecture

9.6.1 Cluster Design

- Deployment: Elasticsearch on EKS (3 master nodes, 10 data nodes, 2 coordinating nodes)
- Node Specs: Master: 4 vCPU, 16GB RAM; Data: 8 vCPU, 32GB RAM; Coordinating: 4 vCPU, 16GB RAM
- Index Sharding: 5 primary shards per index, 1 replica per shard
- Index Lifecycle: Hot (7 days), Warm (30 days), Cold (90 days), Frozen (1 year)
- Backup: Daily snapshots to S3, cross-region replication
- Monitoring: Elasticsearch monitoring API, Kibana, Prometheus exporter

9.6.2 Index Mapping

```
{
  "mappings": {
    "properties": {
      "name": { "type": "text", "analyzer": "standard", "fields": { "keyword": { "type": "keyword" } } },
      "description": { "type": "text", "analyzer": "standard" },
      "category": { "type": "keyword" },
      "location": { "type": "geo_point" },
      "rating": { "type": "float" },
      "price_range": { "type": "integer" },
      "status": { "type": "keyword" },
      "tags": { "type": "keyword" },
      "business_hours": { "type": "object" },
```

```

"created_at": { "type": "date" },

"suggest": { "type": "completion" }

}

}

}

```

9.7 S3 & Object Storage Architecture

9.7.1 Bucket Structure

Bucket	Purpose	Storage Class	Lifecycle
localconnect-media	User-generated media (images, videos)	Intelligent-Tiering	90 days to Glacier, 1 year to Deep Archive
localconnect-documents	KYC documents, government forms	Standard-IA	7 years retention
localconnect-backups	Database backups, snapshots	Glacier	7 years retention
localconnect-logs	Application logs, audit logs	Glacier	7 years retention
localconnect-analytics	Raw analytics data, exports	Intelligent-Tiering	90 days to Glacier
localconnect-cdn	CDN-optimized static assets	Standard	No expiration
localconnect-emergency	Emergency evidence, recordings	Standard	7 years retention

9.7.2 Object Lifecycle Policies

- Transition: Intelligent-Tiering auto-optimizes based on access patterns
- Archive: Non-critical media moves to Glacier after 90 days, Deep Archive after 1 year
- Expiration: Temporary files (uploads, cache) expire after 7 days
- Versioning: Enabled for all buckets (retain 10 versions)
- Encryption: SSE-S3 for all objects, SSE-KMS for sensitive data (KYC, emergency)
- Cross-Region Replication: Critical buckets replicated to DR region
- Access Control: IAM policies, bucket policies, S3 Block Public Access, pre-signed URLs for temporary access

9.8 ClickHouse Analytics Architecture

9.8.1 Cluster Design

- Deployment: ClickHouse on EKS (3 nodes, 8 vCPU, 32GB RAM per node)
- Replication: 2 replicas per shard, 3 shards
- Engine: MergeTree with ReplicatedMergeTree for distributed tables
- Partitioning: Monthly partitioning for large tables
- Backup: Daily snapshots to S3
- Monitoring: ClickHouse system tables, Prometheus exporter, Grafana dashboards

9.8.2 Schema Design

```
CREATE TABLE events_local (  
  
    event_id UUID,  
  
    event_type LowCardinality(String),  
  
    user_id UUID,  
  
    session_id UUID,
```

```
timestamp DateTime64(3),  
  
page String,  
  
referrer String,  
  
device_type LowCardinality(String),  
  
os LowCardinality(String),  
  
app_version String,  
  
country LowCardinality(String),  
  
city String,  
  
metadata JSON  
  
) ENGINE = ReplicatedMergeTree('/clickhouse/tables/{shard}/events',  
'{replica}')  
  
PARTITION BY toYYYYMM(timestamp)  
  
ORDER BY (event_type, timestamp, user_id)  
  
TTL timestamp + INTERVAL 2 YEAR;  
  
CREATE TABLE events_distributed AS events_local  
  
ENGINE = Distributed('localconnect_cluster', 'default', 'events_local', rand());
```

9.9 Neo4j Graph Database (Social Graph)

9.9.1 Use Cases

- Social connections (friends, followers, blocked users)
- Business relationships (suppliers, partners, competitors)
- Recommendation engine (collaborative filtering)

- Fraud detection (transaction patterns, network analysis)

9.9.2 Schema

// Nodes

```
CREATE CONSTRAINT user_id ON (u:User) ASSERT u.id IS UNIQUE;
```

```
CREATE CONSTRAINT business_id ON (b:Business) ASSERT b.id IS UNIQUE;
```

```
CREATE CONSTRAINT event_id ON (e:Event) ASSERT e.id IS UNIQUE;
```

// Relationships

```
CREATE INDEX friend_since FOR ()-[r:FRIENDS_WITH]-() ON (r.since);
```

```
CREATE INDEX follows_since FOR ()-[r:FOLLOWS]-() ON (r.since);
```

```
CREATE INDEX reviewed_at FOR ()-[r:REVIEWED]-() ON (r.created_at);
```

```
CREATE INDEX purchased_at FOR ()-[r:PURCHASED_FROM]-() ON  
(r.created_at);
```

```
CREATE INDEX attended_at FOR ()-[r:ATTENDED]-() ON (r.created_at);
```

// Example Query: Find friends who attended the same event

```
MATCH (u:User {id: $user_id})-[:FRIENDS_WITH]->(friend:User)-[:ATTENDED]-  
>(event:Event)
```

```
WHERE event.id = $event_id
```

```
RETURN friend.name, friend.avatar, event.title;
```

// Example Query: Recommend businesses based on friends' reviews

```
MATCH (u:User {id: $user_id})-[:FRIENDS_WITH]->(friend:User)-[:REVIEWED]-  
>(business:Business)
```



```
WHERE business.category = $category AND NOT (u)-[:REVIEWED]->(business)
```

```
RETURN business.id, business.name, avg(friend_rating) as avg_rating,  
count(friend) as friend_count
```

```
ORDER BY friend_count DESC, avg_rating DESC
```

```
LIMIT 10;
```

9.10 TimescaleDB (Time-Series Data)

9.10.1 Use Cases

- IoT sensor data (weather, air quality, traffic)
- System metrics (CPU, memory, disk, network)
- Financial market data (if applicable)
- Energy consumption monitoring

9.10.2 Schema

```
CREATE TABLE sensor_data (  
  
time TIMESTAMPTZ NOT NULL,  
  
sensor_id UUID NOT NULL,  
  
location GEOGRAPHY(POINT, 4326),  
  
temperature DOUBLE PRECISION,  
  
humidity DOUBLE PRECISION,  
  
air_quality_index DOUBLE PRECISION,  
  
noise_level DOUBLE PRECISION,  
  
metadata JSONB
```

```
);
```

```
SELECT create_hypertable('sensor_data', 'time', chunk_time_interval =>  
INTERVAL '1 day');
```

```
CREATE INDEX idx_sensor_data_sensor ON sensor_data(sensor_id, time DESC);
```

```
CREATE INDEX idx_sensor_data_location ON sensor_data USING  
GIST(location);
```

```
-- Continuous aggregates for hourly/daily summaries
```

```
CREATE MATERIALIZED VIEW sensor_hourly
```

```
WITH (timescaledb.continuous) AS
```

```
SELECT
```

```
time_bucket('1 hour', time) AS bucket,
```

```
sensor_id,
```

```
avg(temperature) as avg_temp,
```

```
avg(humidity) as avg_humidity,
```

```
avg(air_quality_index) as avg_aqi,
```

```
max(noise_level) as max_noise
```

```
FROM sensor_data
```

```
GROUP BY bucket, sensor_id;
```

9.11 Database Migration Strategy

9.11.1 Migration Framework

- Tool: Flyway for PostgreSQL, mongosh for MongoDB, custom scripts for DynamoDB
- Versioning: Semantic versioning for migrations (V1.0.0__initial_schema.sql)
- Environment: Migrations run automatically in CI/CD pipeline
- Rollback: Downward migrations for rollback, blue-green deployment for zero-downtime
- Validation: Checksum validation for migration integrity
- Baseline: Baseline version set for existing databases

9.11.2 Migration Process

1. Development: Migration scripts written in feature branch
2. Code Review: DBAs review migration scripts for performance and safety
3. Staging: Migrations run on staging database (copy of production)
4. Performance Testing: Migrations tested with production-like data volume
5. Production: Migrations run during maintenance window (or online for zero-downtime)
6. Verification: Post-migration validation and monitoring
7. Rollback Plan: Rollback script prepared and tested before production deployment

9.11.3 Zero-Downtime Migration Techniques

- Online Schema Changes: pg_online_schema_change for PostgreSQL, native online index builds
- Blue-Green Deployment: Clone database, apply migration, switch traffic, validate, decommission old
- Expand-Contract Pattern: Add new column/table (expand), dual-write (migrate), remove old (contract)
- Feature Flags: Gate new schema usage behind feature flags
- Backfill Strategies: Async backfill for large data migrations, batch processing to avoid locks

9.12 Data Retention & Archival

Data Type	Hot Storage	Warm Storage	Cold Storage	Retention
User Profiles	PostgreSQL	S3 Standard	Glacier	7 years after deletion
Business Data	PostgreSQL	S3 Standard-IA	Glacier	10 years
Orders	PostgreSQL	S3 Glacier	Glacier Deep	7 years
Payments	PostgreSQL	S3 Glacier	Glacier Deep	10 years
Reviews	MongoDB	S3 Standard-IA	Glacier	5 years
Forum Posts	MongoDB	S3 Standard-IA	Glacier	3 years
Messages	MongoDB	S3 Glacier	Glacier Deep	1 year
Emergency Data	DynamoDB	S3 Glacier	Glacier Deep	10 years
Audit Logs	S3 Standard	S3 Glacier	Glacier Deep	7 years
Analytics Events	ClickHouse	S3 Standard-IA	Glacier	2 years
System Metrics	Prometheus	S3 Standard-IA	Glacier	1 year
Application Logs	ELK	S3 Glacier	Glacier Deep	90 days
CDN Logs	CloudFront	S3 Standard-IA	Glacier	1 year

Backups	S3 Standard	S3 Glacier	Glacier Deep	35 days (PITR) + 7 years
---------	-------------	------------	--------------	--------------------------

Chapter 10: Event-Driven Architecture (EDA)

10.1 Event-Driven Architecture Overview

LocalConnect operates on an event-driven architecture where services communicate primarily through asynchronous events. This enables loose coupling, horizontal scalability, and resilience.

10.2 Event Bus (Kafka)

10.2.1 Kafka Cluster Design

- Deployment: Amazon MSK (Managed Streaming for Kafka) or self-managed on EKS
- Cluster Size: 6 brokers (3 in AZ-1, 2 in AZ-2, 1 in AZ-3) for high availability
- Instance Type: m6g.2xlarge (8 vCPU, 32GB RAM per broker)
- Replication Factor: 3 for all topics
- Minimum In-Sync Replicas: 2 for critical topics, 1 for non-critical
- Partitions: 10-100 partitions per topic (based on throughput)
- Retention: 7 days for standard topics, 30 days for audit topics, 1 year for compliance topics
- Compression: snappy for standard topics, zstd for high-throughput topics
- Monitoring: Kafka JMX metrics, Prometheus exporter, Grafana dashboards, Cruise Control for rebalancing

10.2.2 Topic Catalog

Topic	Partitions	Retention	Replication	Use Case
-------	------------	-----------	-------------	----------

user.events	50	7 days	3	User registration, verification, updates
business.events	30	7 days	3	Business creation, updates, claims, reviews
event.events	30	7 days	3	Event creation, registration, check-in
forum.events	40	7 days	3	Forum posts, comments, moderation actions
emergency.events	100	30 days	3	Emergency alerts, incidents, SOS
marketplace.events	60	7 days	3	Orders, payments, inventory, refunds
govtech.events	20	7 days	3	Tax payments, licenses, complaints
notification.events	50	7 days	3	Notification creation, delivery, read
analytics.events	100	30 days	3	All user interactions, page views, clicks
audit.events	20	1 year	3	Immutable audit trail
config.events	10	7 days	3	Feature flag changes, configuration updates
error.events	20	7 days	3	Application errors, exceptions, alerts
health.events	10	1 day	3	Service health checks, heartbeat
ai.events	20	7 days	3	AI inference requests, results, model updates

10.2.3 Event Schema (Avro)

All events use Avro schema with Schema Registry for versioning and compatibility.

```
{  
  
  "type": "record",  
  
  "name": "DomainEvent",  
  
  "namespace": "com.localconnect.events",  
  
  "fields": [  
  
    { "name": "event_id", "type": "string", "doc": "Unique event identifier (UUID)" },  
  
    { "name": "event_type", "type": "string", "doc": "Event type (e.g.,  
    user.registered)" },  
  
    { "name": "aggregate_type", "type": "string", "doc": "Domain aggregate type" },  
  
    { "name": "aggregate_id", "type": "string", "doc": "Aggregate identifier (UUID)"  
    },  
  
    { "name": "version", "type": "int", "default": 1, "doc": "Event schema version" },  
  
    { "name": "timestamp", "type": "long", "logicalType": "timestamp-millis", "doc":  
    "Event timestamp (UTC)" },  
  
    { "name": "correlation_id", "type": "string", "doc": "Correlation ID for  
    distributed tracing" },  
  
    { "name": "causation_id", "type": ["null", "string"], "default": null, "doc":  
    "Causation ID (previous event)" },  
  
    { "name": "payload", "type": "bytes", "doc": "Serialized event payload  
    (Avro/JSON)" },  
  
    { "name": "metadata", "type": {
```

```

"type": "map",

"values": "string"

}, "doc": "Event metadata (source, user agent, IP)" },

{ "name": "tenant_id", "type": ["null", "string"], "default": null, "doc": "Multi-tenant identifier" },

{ "name": "region", "type": "string", "doc": "Source region" }

]

}

```

10.2.4 Event Producers & Consumers



|||||

| +-----v-----v-----v-----v-----+ |

| | KAFKA CLUSTER | |

| | +-----+ +-----+ +-----+ +-----+ | |

| | | user.events | | business.events | | event.events | | ... | | |

| | | (50 partitions) | | (30 partitions) | | (30 partitions) | | | |

| | +-----+ +-----+ +-----+ +-----+ | |

| +-----+-----+-----+-----+-----+-----+-----+
+ |

|||||

| CONSUMERS |

| +-----v-----+ +-----v-----+ +-----v-----+ +-----v-----+ |

| | Notification Svc | | Analytics Svc | | Search Svc | | Audit Svc | |

| | - All topics | | - All topics | | - business.evt | | - All topics | |

| | - Filtered | | - Aggregated | | - event.events | | - Immutable | |

| +-----+ +-----+ +-----+ +-----+ |

| +-----v-----+ +-----v-----+ +-----v-----+ +-----v-----+ |

| | Recommendation | | Fraud Detection | | Compliance Svc | | Cache
Invalidation | |

| | - user.events | | - marketplace.evt | | - govtech.events | | - All topics | |

| | - business.evt | | - user.events | | - audit.events | | - Redis updates | |

```

| +-----+ +-----+ +-----+ +-----+ |
| +-----v-----+ +-----v-----+ +-----v-----+ +-----v-----+ |
| | Email Service | | Push Service | | SMS Service | | Webhook Service | |
| | - notification.evt | | - notification.evt | | - emergency.evt | | - All topics | |
| | - marketplace.evt | | - emergency.evt | | - notification.evt | | - Configurable | |
| +-----+ +-----+ +-----+ +-----+ |
| |
+-----
-----+

```

10.3 Event Sourcing

10.3.1 Event Sourcing Domains

- Order Management: Complete order lifecycle as event stream
- Payment Processing: Payment state transitions as events
- User Account: Account changes, verifications, role assignments
- Emergency Response: Incident timeline, dispatch decisions, resource allocation
- Government Services: Application status changes, approval workflows
- Audit Trail: All system actions as immutable events

10.3.2 Event Store Design

- Primary Store: PostgreSQL (event log table with append-only semantics)
- Read Models: Materialized views, Redis, Elasticsearch for query optimization
- Snapshotting: Snapshots every 100 events to optimize replay
- Compaction: Old events archived to S3 after 2 years

- Replay: Full replay capability for state reconstruction and debugging

10.3.3 Event Store Schema

```
CREATE TABLE event_store (  
  
  id BIGSERIAL PRIMARY KEY,  
  
  aggregate_id UUID NOT NULL,  
  
  aggregate_type VARCHAR(100) NOT NULL,  
  
  event_type VARCHAR(200) NOT NULL,  
  
  event_version INT NOT NULL,  
  
  payload JSONB NOT NULL,  
  
  metadata JSONB DEFAULT '{}',  
  
  correlation_id UUID,  
  
  causation_id UUID,  
  
  timestamp TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  
  UNIQUE(aggregate_id, event_version)  
  
);  
  
CREATE INDEX idx_event_store_aggregate ON event_store(aggregate_id,  
event_version);  
  
CREATE INDEX idx_event_store_type ON event_store(event_type);  
  
CREATE INDEX idx_event_store_timestamp ON event_store(timestamp);  
  
CREATE INDEX idx_event_store_correlation ON event_store(correlation_id);
```

10.4 CQRS (Command Query Responsibility Segregation)

10.4.1 CQRS Implementation

- Command Side: Handles writes, validates business rules, emits events
- Query Side: Optimized read models, eventual consistency
- Synchronization: Event handlers update read models asynchronously
- Consistency: Read models updated within 1 second of command completion

10.4.2 CQRS Example: Business Discovery

```
+-----+
-----+

| CQRS: BUSINESS DISCOVERY |

+-----+
-----+

||

| COMMAND SIDE (Write) |

| +-----+ |

|| API Request ||

|| POST /business ||

| +-----+-----+ |

|||

| +-----v-----+ |
```

| | Business Service | |

| | - Validate input | |

| | - Apply business rules | |

| | - Save to PostgreSQL | |

| | - Emit event to Kafka | |

| +-----+-----+ |

| | |

| +-----v-----+ |

| | Event Store (PostgreSQL) | |

| | business.events topic | |

| +-----+ |

| |

| QUERY SIDE (Read) |

| +-----v-----+ |

| | Event Handler | |

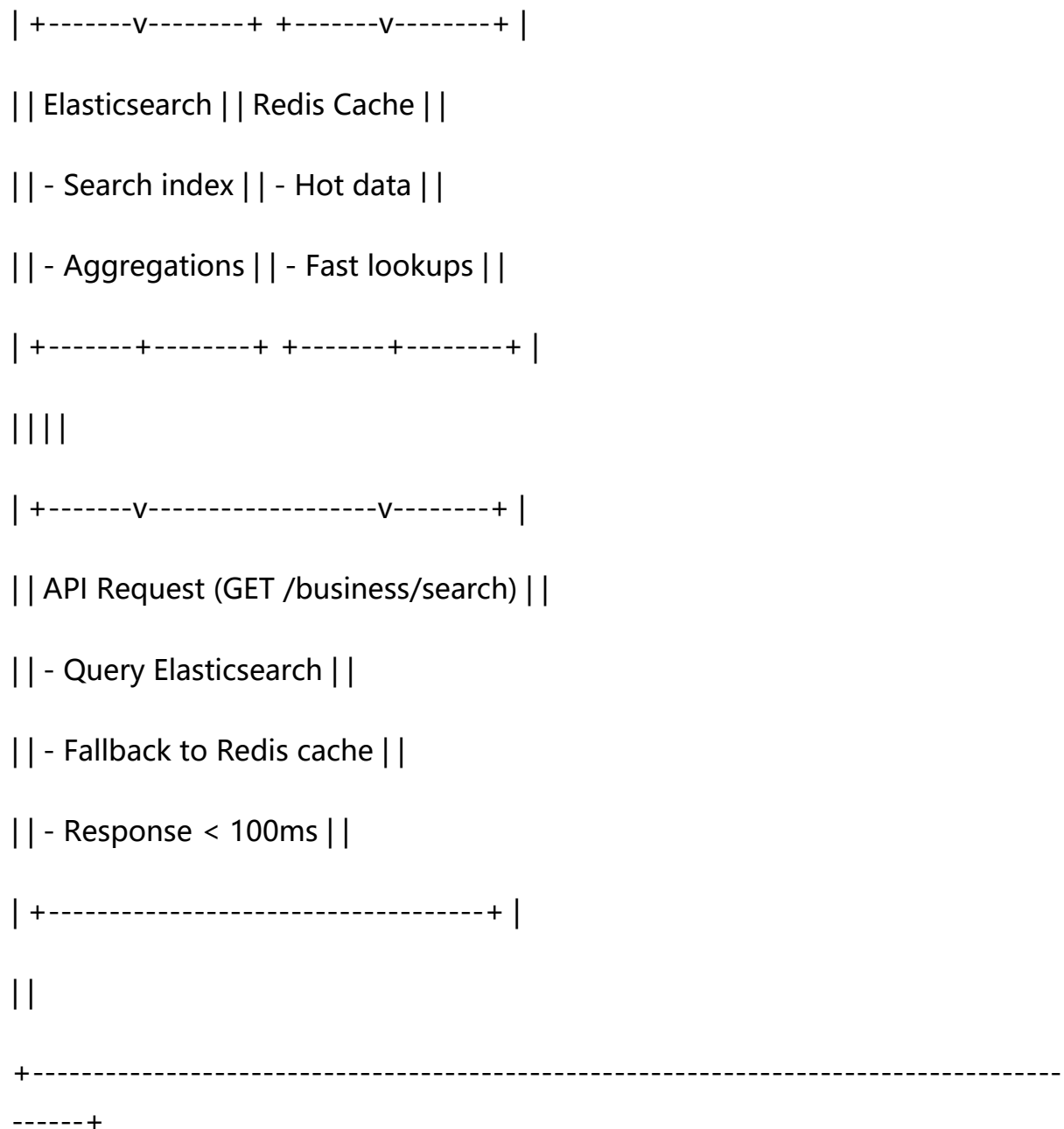
| | - Listen to Kafka | |

| | - Update Elasticsearch | |

| | - Update Redis cache | |

| +-----+-----+ |

| | |



10.5 Stream Processing (Kafka Streams / Flink)

10.5.1 Real-Time Processing Use Cases

- Fraud Detection: Real-time transaction analysis, pattern detection
- Recommendation Engine: Real-time user behavior analysis, collaborative filtering

- Analytics Aggregation: Real-time metrics, dashboards, anomaly detection
- Emergency Response: Real-time alert processing, geofencing, resource allocation
- Content Moderation: Real-time AI moderation, auto-flagging, queue management
- Dynamic Pricing: Real-time demand analysis, price adjustments
- Inventory Management: Real-time stock updates, low-stock alerts

10.5.2 Kafka Streams Topology Example: Fraud Detection

```
KStream<String, TransactionEvent> transactions =
builder.stream("marketplace.events");

KStream<String, TransactionEvent> fraudDetected = transactions

.filter((key, value) -> "transaction.created".equals(value.getEventType()))

.groupByKey()

.windowedBy(TimeWindows.of(Duration.ofMinutes(5)))

.aggregate(

FraudScore::new,

(key, value, aggregate) -> aggregate.update(value),

Materialized.with(Serdes.String(), fraudScoreSerde)

)

.toStream()

.filter((key, value) -> value.getRiskScore() > 0.8)
```

```
.mapValues(FraudDetectedEvent::new);
```

```
fraudDetected.to("fraud.detected");
```

10.6 Message Queue (SQS + SNS)

10.6.1 SQS Queue Design

Queue	Type	Visibility Timeout	DLQ	Use Case
email-queue	Standard	5 min	Yes	Email sending
sms-queue	Standard	5 min	Yes	SMS sending
push-queue	Standard	2 min	Yes	Push notification delivery
image-processing-queue	Standard	10 min	Yes	Image resize, optimization
video-processing-queue	Standard	30 min	Yes	Video transcoding
report-generation-queue	Standard	15 min	Yes	PDF/Excel report generation
data-export-queue	Standard	30 min	Yes	GDPR data export
backup-queue	Standard	60 min	Yes	Database backup tasks
kyc-processing-queue	FIFO	10 min	Yes	KYC document processing
order-fulfillment-queue	FIFO	5 min	Yes	Order fulfillment workflow

10.6.2 SNS Topic Design

Topic	Subscribers	Protocol	Use Case
-------	-------------	----------	----------

| +-----v-----+ |

| | 2. PaymentInitiated | |

| | (Payment Svc) | |

| +-----+-----+ |

| | |

| +-----v-----+ +-----v-----+ |

| | 3a. PaymentSuccess | | 3b. PaymentFailed | |

| | (Payment Svc) | | (Payment Svc) | |

| +-----+-----+ +-----+-----+ |

| | | |

| +-----v-----+ +-----v-----+ |

| | 4. InventoryReserved | | 5. PaymentRefunded | |

| | (Inventory Svc) | | (Payment Svc) | |

| +-----+-----+ +-----+-----+ |

| | | |

| +-----v-----+ +-----v-----+ |

| | 5. OrderConfirmed | | 6. OrderCancelled | |

| | (Order Svc) | | (Order Svc) | |

| +-----+-----+ +-----+-----+ |

| | | |

```

| +-----v-----+ +-----v-----+ |
| | 6. DeliveryScheduled | | 7. InventoryReleased | |
| | (Delivery Svc) | | (Inventory Svc) | |
| +-----+-----+ +-----+-----+ |
| | | |
| +-----v-----+ +-----v-----+ |
| | 7. NotificationSent | | 8. NotificationSent | |
| | (Notify Svc) | | (Notify Svc) | |
| +-----+-----+ +-----+-----+ |
| |
+-----+
-----+

```

10.7.2 Outbox Pattern

All services use the Outbox pattern to ensure atomicity between database writes and event publishing.

```

CREATE TABLE outbox (
  id BIGSERIAL PRIMARY KEY,
  aggregate_id UUID NOT NULL,
  event_type VARCHAR(200) NOT NULL,
  payload JSONB NOT NULL,
  metadata JSONB DEFAULT '{}',

```

```

correlation_id UUID,

created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

processed_at TIMESTAMP WITH TIME ZONE,

retry_count INT DEFAULT 0,

error_message TEXT

);

CREATE INDEX idx_outbox_unprocessed ON outbox(processed_at) WHERE
processed_at IS NULL;

CREATE INDEX idx_outbox_created_at ON outbox(created_at);

```

- Outbox Polling: Background job polls unprocessed events every 100ms
- Publishing: Events published to Kafka with idempotency key
- Acknowledgment: Marked as processed after successful publish
- Retry: Failed events retried 3 times with exponential backoff
- Dead Letter: Failed events after 3 retries moved to DLQ for manual review

10.7.3 Idempotency Pattern

All event handlers and API endpoints implement idempotency:

- Idempotency Key: UUID generated by client for each request
- Storage: Redis with 24-hour TTL for processed keys
- Response: Same response returned for duplicate requests
- Event Handling: Deduplicated based on event_id and aggregate_id

Idempotency check

```
def process_event(event):
```

```

key = f"idempotency:{event.event_id}"

if redis.exists(key):

    logger.info(f"Event {event.event_id} already processed")

    return

# Process event

result = handle_event(event)

# Mark as processed

redis.setex(key, 86400, "processed")

return result

```

10.8 Event-Driven Reliability

10.8.1 Delivery Guarantees

Topic	Delivery Guarantee	Ordering	Durability
emergency.events	Exactly-Once	Partition-ordered	Persistent (30 days)
marketplace.events	At-Least-Once	Partition-ordered	Persistent (7 days)
notification.events	At-Least-Once	Partition-ordered	Persistent (7 days)
analytics.events	At-Least-Once	Best-effort	Persistent (30 days)
audit.events	Exactly-Once	Strictly ordered	Persistent (1 year)
error.events	At-Least-Once	Best-effort	Persistent (7 days)

10.8.2 Dead Letter Queues (DLQ)

DLQ Topic	Retention	Processing	Alerting
user.events.dlq	7 days	Manual review + auto-retry	PagerDuty for > 100/hour
marketplace.events.dlq	7 days	Manual review + auto-retry	PagerDuty for > 100/hour
emergency.events.dlq	30 days	Immediate escalation	PagerDuty for any message
notification.events.dlq	7 days	Auto-retry + manual review	PagerDuty for > 1000/hour
audit.events.dlq	30 days	Manual review + immediate fix	PagerDuty for any message

10.8.3 Event Replay

- Capability: Full replay of event stream for debugging, recovery, new consumer initialization
- Tool: Kafka consumer with custom offset management
- Replay Scenarios:
 - New service needs to build read model from scratch
 - Data corruption in read model requires rebuilding
 - Debugging production issues with specific event sequence
 - Compliance audit requiring historical event inspection
- Controls: Replay limited to specific time range, aggregate, or event type
- Performance: Parallel replay with configurable concurrency (10-100 threads)

Chapter 11: API Gateway Architecture

11.1 API Gateway Design (Kong + Istio Ingress)

LocalConnect uses a two-tier API gateway architecture for maximum flexibility, security, and performance.

11.1.1 Tier 1: Edge Gateway (Kong)

- Purpose: External-facing API management, authentication, rate limiting, caching, analytics
- Deployment: Kong on EKS (3 nodes minimum, 10 nodes maximum, HPA)
- Configuration: Declarative configuration via Kong Admin API and Kubernetes CRDs (Kong Ingress Controller)
- Plugins: Custom Lua plugins for business logic

11.1.2 Tier 2: Service Mesh Gateway (Istio Ingress)

- Purpose: Internal routing, load balancing, mTLS, service discovery, traffic management
- Deployment: Istio Ingress Gateway (3 nodes, auto-scaling)
- Configuration: Istio VirtualService, Gateway, DestinationRule CRDs

11.2 Kong Configuration

11.2.1 Services & Routes

apiVersion: configuration.konghq.com/v1

kind: KongIngress

metadata:

name: localconnect-api

proxy:

protocols:

- https

connect_timeout: 5000

write_timeout: 10000

read_timeout: 10000

route:

methods:

- GET

- POST

- PUT

- PATCH

- DELETE

strip_path: false

preserve_host: true

apiVersion: v1

kind: Service

metadata:

name: api-gateway-service

annotations:

konghq.com/override: localconnect-api

spec:

ports:

- port: 443

targetPort: 8443

protocol: TCP

selector:

app: kong-gateway

11.2.2 Kong Plugin Configuration

Plugin	Purpose	Scope	Configuration
rate-limiting	Rate limiting per user/API key	Global	1000 req/min per user, 10000 req/min per API key
key-auth	API key authentication	Specific routes	Key in header or query parameter
jwt	JWT authentication	Auth routes	RS256, JWKS endpoint from identity-service
oauth2	OAuth 2.0	External integrations	Authorization code, client credentials
cors	CORS headers	Global	Allow origins: *.localconnect.com, localhost:3000
bot-detection	Bot detection	Auth routes	Block known bots, CAPTCHA for suspicious

ip-restriction	IP allowlist/blocklist	Admin routes	Allow: office IPs, VPN IPs; Block: known malicious
request-transformer	Request modification	Specific routes	Add headers, strip headers, transform body
response-transformer	Response modification	Global	Add security headers, remove sensitive headers
prometheus	Metrics collection	Global	Export to Prometheus
proxy-cache	Response caching	Specific routes	TTL: 60s for GET requests, cache key: URL+params+auth
aws-lambda	Serverless integration	API routes	Invoke Lambda for specific endpoints
request-validation	Schema validation	All routes	JSON Schema validation for request body
file-log	Request logging	Global	Log to S3 for audit and analytics
opentelemetry	Distributed tracing	Global	Export to Jaeger/Tempo

11.2.3 Custom Kong Plugins (Lua)

```
-- custom-auth.lua
```

```
local jwt = require "resty.jwt"
```

```
local redis = require "resty.redis"
```

```
local _M = {}
```

```
function _M.access(conf)
```

```
local token = ngx.var.http_authorization
```

```
if not token then
```

```
return kong.response.exit(401, { message = "Unauthorized" })

end

-- Remove "Bearer " prefix

token = token:gsub("Bearer ", "")

-- Validate JWT

local jwt_obj = jwt.verify(conf.jwt_secret, token)

if not jwt_obj.verified then

return kong.response.exit(401, { message = "Invalid token" })

end

-- Check Redis for revoked tokens

local red = redis:new()

red:connect(conf.redis_host, conf.redis_port)

local revoked = red:get("revoked:" .. token)

if revoked ~= ngx.null then

return kong.response.exit(401, { message = "Token revoked" })

end

-- Add user context to headers

kong.service.request.set_header("X-User-Id", jwt_obj.payload.sub)

kong.service.request.set_header("X-User-Role", jwt_obj.payload.role)
```

```
kong.service.request.set_header("X-User-Permissions",  
jwt_obj.payload.permissions)
```

```
end
```

```
return _M
```

11.3 Istio Ingress Configuration

11.3.1 Gateway Configuration

```
apiVersion: networking.istio.io/v1beta1
```

```
kind: Gateway
```

```
metadata:
```

```
name: localconnect-gateway
```

```
namespace: istio-system
```

```
spec:
```

```
selector:
```

```
istio: ingressgateway
```

```
servers:
```

```
- port:
```

```
number: 443
```

```
name: https
```

```
protocol: HTTPS
```

```
tls:
```

mode: SIMPLE

credentialName: localconnect-tls

minProtocolVersion: TLSV1_2

cipherSuites:

- TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384

- TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256

hosts:

- "*.localconnect.com"

- "localconnect.com"

- port:

number: 80

name: http

protocol: HTTP

hosts:

- "*"

tls:

httpsRedirect: true

apiVersion: networking.istio.io/v1beta1

kind: VirtualService

metadata:

name: localconnect-routing

spec:

hosts:

- "api.localconnect.com"

gateways:

- localconnect-gateway

http:

- match:

- uri:

prefix: /v1/auth

route:

- destination:

host: auth-service

port:

number: 8080

timeout: 5s

retries:

attempts: 3

perTryTimeout: 2s

retryOn: 5xx,connect-failure,refused-stream

- match:

- uri:

prefix: /v1/users

route:

- destination:

host: user-service

port:

number: 8080

timeout: 5s

- match:

- uri:

prefix: /v1/business

route:

- destination:

host: business-service

port:

number: 8080

timeout: 10s

- match:

- uri:

prefix: /v1/emergency

route:

- destination:

host: alert-service

port:

number: 8080

timeout: 2s

retries:

attempts: 5

perTryTimeout: 1s

retryOn: 5xx,connect-failure,refused-stream

- match:

- uri:

prefix: /v1/orders

route:

- destination:

host: order-service

port:

number: 8080

timeout: 10s

11.3.2 Traffic Management

apiVersion: networking.istio.io/v1beta1

kind: DestinationRule

metadata:

name: auth-service-dr

spec:

host: auth-service

trafficPolicy:

connectionPool:

tcp:

maxConnections: 100

http:

http1MaxPendingRequests: 100

http2MaxRequests: 1000

maxRequestsPerConnection: 10

maxRetries: 3

loadBalancer:

simple: LEAST_CONN

outlierDetection:

consecutive5xxErrors: 5

interval: 30s

baseEjectionTime: 30s

maxEjectionPercent: 50

circuitBreaker:

consecutiveErrors: 5

interval: 30s

baseEjectionTime: 30s

subsets:

- name: v1

labels:

version: v1

- name: v2

labels:

version: v2

11.4 GraphQL Federation Gateway

11.4.1 Apollo Federation Architecture

+-----+
-----+

```
+-----+
-----+

| |

| +-----+ |

| | Apollo Router | |

| | - Schema composition | |

| | - Query planning | |

| | - Subgraph routing | |

| | - Caching | |

| +-----+-----+ |

| | |

| +-----v-----+ +-----v-----+ +-----v-----+ |

| | User Subgraph | | Business Subgraph | | Event Subgraph | |

| | - user-service | | - business-service | | - event-service | |

| | - auth-service | | - review-service | | - venue-service | |

| +-----+ +-----+ +-----+ |

| +-----+ +-----v-----+ +-----+ |

| | Order Subgraph | | Emergency Subgraph | | GovTech Subgraph | |

| | - order-service | | - alert-service | | - gov-service | |

| | - payment-svc | | - dispatch-svc | | - tax-service | |
```

```

| +-----+ +-----+ +-----+ |
| +-----+ +-----v-----+ +-----+ |
| | Market Subgraph| | Forum Subgraph | | Search Subgraph | |
| | - catalog-svc | | - forum-service | | - search-service | |
| | - inventory-svc| | - moderation-svc | | - discovery-svc | |
| +-----+ +-----+ +-----+ |
| |
+-----
-----+

```

11.4.2 GraphQL Schema Example

User Subgraph

```

extend type User @key(fields: "id") {
  id: ID!
  email: String!
  profile: Profile
  orders: [Order] @requires(fields: "id")
  reviews: [Review] @requires(fields: "id")
  events: [Event] @requires(fields: "id")
}

type Profile {

```

displayName: String!

avatar: String

location: GeoLocation

}

Business Subgraph

extend type Business @key(fields: "id") {

id: ID!

name: String!

description: String

category: Category

location: GeoLocation

reviews: [Review]

averageRating: Float

owner: User @provides(fields: "id")

}

Event Subgraph

extend type Event @key(fields: "id") {

id: ID!

title: String!

description: String

startTime: DateTime!

endTime: DateTime

location: GeoLocation

organizer: User @provides(fields: "id")

attendees: [User]

venue: Venue

}

Order Subgraph

extend type Order @key(fields: "id") {

id: ID!

user: User @provides(fields: "id")

business: Business @provides(fields: "id")

items: [OrderItem]

totalAmount: Float

status: OrderStatus

payment: Payment

}

Emergency Subgraph

extend type EmergencyAlert @key(fields: "id") {

id: ID!

alertType: String!

priority: String!

geofence: GeoPolygon

message: String!

sender: User @provides(fields: "id")

status: String!

}

GovTech Subgraph

extend type GovernmentService @key(fields: "id") {

id: ID!

name: String!

department: String!

category: String!

location: GeoLocation

requiredDocuments: [String]

}

Search Subgraph

extend type SearchResult @key(fields: "id") {

id: ID!

type: String!

title: String!

description: String

location: GeoLocation

score: Float

}

11.5 WebSocket Gateway

11.5.1 Socket.io Gateway Architecture

- Deployment: Socket.io servers on EKS (3-20 nodes, HPA based on connections)
- Adapter: Redis adapter for horizontal scaling across nodes
- Load Balancing: Sticky sessions (ip_hash) for WebSocket connections
- Authentication: JWT token validation on connection
- Rooms: Dynamic room creation based on user ID, event ID, forum ID, business ID
- Namespaces: Separate namespaces for emergency, chat, notifications, live events

11.5.2 Namespace Configuration

```
// Emergency namespace (highest priority)
```

```
const emergencyNamespace = io.of('/emergency');
```

```
emergencyNamespace.use(jwtAuthMiddleware);
```

```
emergencyNamespace.on('connection', (socket) => {
```

```
const userId = socket.user.id;
```



```
const location = socket.user.location;

// Join geofence-based rooms

socket.join(`geofence:${location.region}`);

socket.join(`geofence:${location.district}`);

// Handle SOS activation

socket.on('sos:activate', async (data) => {

  await sosService.activate(userId, data);

  socket.emit('sos:acknowledged', { status: 'active' });

});

// Handle emergency chat

socket.on('emergency:chat', async (data) => {

  await emergencyChatService.sendMessage(userId, data);

});

});

// Chat namespace

const chatNamespace = io.of('/chat');

chatNamespace.use(jwtAuthMiddleware);

chatNamespace.on('connection', (socket) => {

  const userId = socket.user.id;

  // Join personal room
```

```
socket.join(`user:${userId}`);

// Handle message sending

socket.on('message:send', async (data) => {

  const message = await messagingService.sendMessage(userId, data);

  socket.to(`user:${data.recipientId}`).emit('message:received', message);

});

// Handle typing indicators

socket.on('typing:start', (data) => {

  socket.to(`user:${data.recipientId}`).emit('typing:start', { userId });

});

});

// Live event namespace

const liveEventNamespace = io.of('/live');

liveEventNamespace.use(jwtAuthMiddleware);

liveEventNamespace.on('connection', (socket) => {

  socket.on('event:join', (eventId) => {

    socket.join(`event:${eventId}`);

  });

  socket.on('event:question', async (data) => {

    await eventService.submitQuestion(data);
```

```
socket.to(`event:${data.eventId}`).emit('event:question', data);

});

socket.on('event:reaction', (data) => {

socket.to(`event:${data.eventId}`).emit('event:reaction', data);

});

});
```

11.6 API Versioning Strategy

11.6.1 Versioning Approach

- URL Path Versioning: /v1/users, /v2/users (primary approach)
- Header Versioning: Accept: application/vnd.localconnect.v2+json (secondary)
- Breaking Changes: New major version (v1 → v2)
- Non-Breaking Changes: New minor version within same major version
- Deprecation: 6-month deprecation notice, 12-month sunset period
- Version Discovery: /versions endpoint listing available API versions

11.6.2 Version Lifecycle

Version	Status	Release Date	Deprecation Date	Sunset Date
v1	Active	2024-01-01	2026-01-01	2027-01-01
v2	Active	2025-01-01	—	—
v3	Beta	2026-01-01	—	—

11.6.3 Breaking Changes Policy

Breaking changes require a new major version:

- Removing or renaming fields
- Changing data types
- Changing authentication requirements
- Removing endpoints
- Changing default behavior
- Changing error response format

11.7 API Rate Limiting

11.7.1 Rate Limit Tiers

Tier	Requests/Minute	Requests/Hour	Requests/Day	Scope
Anonymous	30	500	5000	IP-based
Authenticated	100	5000	50000	User-based
Premium	500	50000	500000	User-based
Business	1000	100000	1000000	API key-based
Partner	5000	500000	5000000	API key-based
Internal	10000	1000000	10000000	Service-based
Emergency	Unlimited	Unlimited	Unlimited	Emergency services

11.7.2 Rate Limit Implementation

Kong rate limiting plugin

apiVersion: configuration.konghq.com/v1

kind: KongPlugin

metadata:

name: rate-limiting-tiered

config:

minute: 100

hour: 5000

day: 50000

policy: redis

redis_host: redis-cluster

redis_port: 6379

redis_timeout: 2000

redis_database: 0

fault_tolerant: true

hide_client_headers: false

redis_ssl: true

redis_ssl_verify: false

namespace: localconnect:rate_limit:

plugin: rate-limiting

11.7.3 Rate Limit Response

{

"type": "https://api.localconnect.com/errors/rate-limit",

"title": "Rate Limit Exceeded",

```
"status": 429,  
  
"detail": "You have exceeded the rate limit of 100 requests per minute. Please  
try again in 45 seconds.",  
  
"instance": "/v1/users/12345",  
  
"retry_after": 45,  
  
"limit": 100,  
  
"remaining": 0,  
  
"reset_at": "2025-07-01T12:05:00Z"  
  
}
```

Headers:

- X-RateLimit-Limit: 100
- X-RateLimit-Remaining: 0
- X-RateLimit-Reset: 1754062800
- Retry-After: 45

11.8 API Security

11.8.1 Authentication Methods

Method	Use Case	Implementation
JWT (RS256)	Mobile apps, web apps	Access token (15 min), Refresh token (7 days)
API Key	Partner integrations, third-party apps	HMAC-SHA256 signed requests
OAuth 2.0	External integrations, SSO	Authorization code + PKCE, Client credentials

mTLS	Service-to-service	Client certificates managed by cert-manager
Session Cookie	Web apps	HttpOnly, Secure, SameSite=Strict, 7-day TTL

11.8.2 Authorization

- RBAC: Role-based access control (User, Business Owner, Admin, Moderator, Gov Official, Emergency Responder)
- ABAC: Attribute-based access control for fine-grained permissions (resource ownership, location, time)
- Policy Engine: OPA (Open Policy Agent) for complex authorization decisions
- API Scopes: OAuth 2.0 scopes for granular permissions (read:profile, write:orders, admin:business)

11.8.3 Input Validation

- Schema Validation: JSON Schema for all API requests (Kong request-validation plugin)
- Type Validation: Strict type checking for all parameters
- Sanitization: XSS prevention, SQL injection prevention, NoSQL injection prevention
- File Upload Validation: MIME type, file size, malware scanning, content verification
- Rate Limiting: Per-user, per-IP, per-endpoint (see 11.7)

11.8.4 Output Security

- Response Headers: Security headers (HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy)
- Error Handling: Generic error messages (no stack traces, no sensitive data)

- Data Filtering: Field-level filtering based on user permissions
- Redaction: PII redaction in logs and responses
- CORS: Strict CORS policy, whitelist-only origins

11.9 API Documentation

11.9.1 OpenAPI Specification

- Version: OpenAPI 3.1
- Generator: SpringDoc (Java), swagger-jsdoc (Node.js), FastAPI native (Python)
- Hosting: Swagger UI at <https://api.localconnect.com/docs>
- Examples: Request/response examples for all endpoints
- Authentication: Security schemes documented in OpenAPI spec
- Webhooks: Webhook events documented with payload schemas

11.9.2 Developer Portal

- URL: <https://developers.localconnect.com>
- Features: API reference, interactive sandbox, code examples (curl, Python, JavaScript, Java), SDK downloads, changelog, status page, community forum
- Authentication: Developer account with API key management
- Analytics: API usage dashboard for developers
- Support: Ticket system, FAQ, community Discord

11.9.3 SDKs

- JavaScript/TypeScript: npm package `@localconnect/sdk`
- Python: pip package `localconnect-sdk`
- Java: Maven package `com.localconnect:sdk`
- Go: Go module `github.com/localconnect/sdk-go`
- iOS: Swift package `LocalConnectSDK`
- Android: Maven package `com.localconnect:sdk-android`

- Flutter: pub package localconnect_sdk
- React Native: npm package @localconnect/react-native-sdk

11.10 API Monitoring & Analytics

11.10.1 Metrics Collection

- Kong Prometheus Plugin: Request rate, latency, errors per route, consumer, status code
- Istio Metrics: Service mesh traffic, mTLS status, circuit breaker status
- Custom Metrics: Business-specific metrics (checkout conversion, search success rate)
- SLA Monitoring: Response time percentiles (p50, p95, p99), error rate, availability

11.10.2 API Analytics Dashboard

- Kong Vitals: Built-in API analytics dashboard
- Grafana: Custom dashboards for API metrics
- DataDog: External API monitoring and APM
- Custom Reports: Daily/weekly/monthly API usage reports

11.10.3 SLA Targets

Metric	Target	Measurement
Availability	99.99%	Monthly uptime
Response Time (p50)	< 100ms	For cached endpoints
Response Time (p95)	< 500ms	For database endpoints
Response Time (p99)	< 2s	For complex queries
Error Rate	< 0.1%	5xx errors
Rate Limit Hits	< 1%	429 responses
Authentication Failures	< 0.5%	401/403 responses

Chapter 12: Security Architecture

12.1 Security Principles

LocalConnect's security architecture is built on defense-in-depth, zero-trust, and privacy-by-design principles. Every layer of the stack implements security controls, and no component is trusted by default.

12.2 Authentication & Authorization

12.2.1 Multi-Factor Authentication (MFA)

- Methods: TOTP (Google Authenticator, Authy), SMS OTP, Email OTP, Biometric (fingerprint, face recognition on mobile), Hardware keys (FIDO2/WebAuthn for high-security accounts)
- Enrollment: Mandatory for admin accounts, optional for users (recommended), enforced for business accounts
- Recovery: Backup codes (10 single-use codes), trusted device management, identity verification via KYC documents
- Adaptive MFA: Triggered for suspicious logins (new device, new location, unusual time)

12.2.2 Password Policy

- Requirements: Minimum 12 characters, 1 uppercase, 1 lowercase, 1 number, 1 special character, no common passwords, no personal information
- Storage: bcrypt with cost factor 12, salt per user, pepper (global secret)
- Rotation: No forced rotation (NIST guidelines), recommend change after breach notification
- History: Prevent reuse of last 5 passwords
- Breach Detection: Integration with HaveIBeenPwned API to check for compromised passwords

- Lockout: 5 failed attempts → 15-minute lockout, 10 failed attempts → 1-hour lockout, 20 failed attempts → account review

12.2.3 Session Management

- JWT Tokens: Access token (15 minutes, RS256), Refresh token (7 days, rotating refresh tokens)
- Session Binding: Bound to device fingerprint (user agent, screen resolution, fonts)
- Concurrent Sessions: Maximum 5 concurrent sessions per user, admin dashboard to view/revoke sessions
- Session Revocation: Immediate revocation via Redis blacklist, propagated to all services within 1 second
- Idle Timeout: 30 minutes for web, 7 days for mobile (with biometric re-auth after 24 hours)
- Cookie Security: HttpOnly, Secure, SameSite=Strict, __Host- prefix, 7-day expiration

12.2.4 OAuth 2.0 & OpenID Connect

- Identity Providers: Google, Facebook, Apple, Microsoft (enterprise SSO)
- Flows: Authorization Code + PKCE (for mobile/web), Client Credentials (for service-to-service)
- Scopes: Granular scopes (read:profile, write:orders, admin:business, emergency:alerts)
- Token Introspection: RFC 7662 token introspection endpoint for resource servers
- Token Revocation: RFC 7009 token revocation endpoint

12.2.5 Role-Based Access Control (RBAC)

Role	Permissions	Scope
------	-------------	-------

User	Read own data, write own profile, create reviews, register events	Own account only
Business Owner	Manage business profile, view analytics, respond to reviews, manage orders	Owned business(es)
Business Admin	Manage business profile, view analytics, manage team, configure settings	Owned business(es)
Moderator	Review flagged content, ban users, manage forum posts	Global
Admin	Full system access, user management, configuration, billing	Global
Super Admin	Infrastructure access, security configuration, audit logs	Global
Gov Official	Access government dashboards, manage public services, view analytics	Assigned department
Emergency Responder	Access emergency alerts, dispatch resources, view incident maps	Assigned region
Partner	API access, webhook management, analytics	Assigned scope
Developer	API key management, sandbox access, documentation	Own applications

12.2.6 Attribute-Based Access Control (ABAC)

- Resource Ownership: Users can only access their own resources (orders, reviews, messages)
- Location-Based: Government officials see data for their assigned regions
- Time-Based: Business features available during business hours
- Device-Based: Restricted features on unverified devices
- Reputation-Based: Forum posting restrictions based on karma/reputation

- KYC Status: Financial features require completed KYC

12.3 Data Protection

12.3.1 Encryption at Rest

- Database: PostgreSQL TDE (Transparent Data Encryption), MongoDB encryption at rest, DynamoDB encryption with AWS KMS
- Files: S3 SSE-S3 (default), SSE-KMS for sensitive data (KYC, emergency, payments)
- Backups: Encrypted backups with AES-256-GCM
- Keys: AWS KMS for key management, HSM for root keys, automatic key rotation every 90 days
- Client-Side Encryption: End-to-end encryption for messages, sensitive documents encrypted before upload

12.3.2 Encryption in Transit

- TLS 1.3: Mandatory for all external connections, TLS 1.2 minimum for legacy clients
- Cipher Suites: ECDHE with AES-256-GCM or ChaCha20-Poly1305, forward secrecy enforced
- Certificate Management: cert-manager for Kubernetes, Let's Encrypt for public, internal CA for service mesh
- Certificate Rotation: Automatic rotation every 60 days, 30-day expiration alerts
- HSTS: max-age=31536000, includeSubDomains, preload
- mTLS: Service-to-service communication via Istio mutual TLS (automatic certificate rotation)

12.3.3 Data Classification

Classification	Examples	Protection
----------------	----------	------------

Public	Business listings, public events, open data	Integrity protection, no confidentiality required
Internal	Analytics data, business analytics, non-sensitive configs	Access control, encryption at rest
Confidential	User profiles, order history, messages, reviews	Encryption at rest + in transit, access logging, RBAC
Restricted	KYC documents, payment data, emergency evidence, government data	Encryption at rest + in transit, strict access control, audit logging, DLP

12.3.4 Data Loss Prevention (DLP)

- Content Scanning: Automated scanning of uploaded files for PII, financial data, sensitive government information
- Email DLP: Scanning of outgoing emails for sensitive data
- API DLP: Filtering of sensitive fields in API responses based on user permissions
- Endpoint DLP: Monitoring of developer/admin workstations for sensitive data export
- Cloud DLP: AWS Macie for S3 data classification and protection

12.3.5 PII Protection

- Minimization: Collect only necessary PII, delete when no longer needed
- Pseudonymization: Replace PII with pseudonyms in analytics and logs
- Anonymization: Anonymize data for research and analytics (k-anonymity, l-diversity)
- Consent Management: Granular consent for data collection, processing, and sharing
- Right to Erasure: Automated deletion workflows for GDPR Article 17 requests

- Data Portability: Automated export in machine-readable format (JSON, CSV)

12.4 Network Security

12.4.1 Network Segmentation

- VPC Isolation: Private subnets for all services, public subnets only for load balancers and bastion hosts
- Security Groups: Principle of least privilege, explicit allow rules, no default allow-all
- Network ACLs: Stateless ACLs for subnet-level protection, deny by default
- Micro-segmentation: Istio authorization policies for service-to-service communication
- Bastion Hosts: Jump boxes for administrative access, MFA required, session recording
- VPN: Site-to-site VPN for office access, client VPN for remote employees

12.4.2 DDoS Protection

- AWS Shield Standard: Automatic DDoS protection for all AWS resources
- AWS Shield Advanced: Enhanced DDoS protection for critical endpoints (3000/month)3000/month)3000/month)
- Cloudflare: Additional layer for DNS and CDN protection (Pro plan, 20/month)20/month)20/month)
- Rate Limiting: Per-IP, per-user, per-endpoint rate limiting (see 11.7)
- WAF Rules: Rate-based rules, geo-blocking, IP reputation, bot detection
- Traffic Scrubbing: Automatic traffic scrubbing during DDoS events

- Alerting: PagerDuty alerts for DDoS events, automatic escalation

12.4.3 Web Application Firewall (WAF)

- AWS WAF: Managed rules for common attacks (OWASP Top 10, SQL injection, XSS)
- Custom Rules: Geo-blocking for high-risk countries, IP allowlists for admin endpoints, rate-based rules
- Bot Control: AWS WAF Bot Control for detecting and blocking malicious bots
- API Protection: Rate limiting, request validation, anomaly detection
- Logging: All WAF events logged to S3 for analysis and forensics
- Integration: WAF rules integrated with SIEM for real-time alerting

12.4.4 Intrusion Detection & Prevention

- AWS GuardDuty: Continuous threat detection for AWS accounts and workloads
- AWS Inspector: Automated vulnerability assessment for EC2 and ECR images
- Falco: Runtime security monitoring for Kubernetes (syscall-level anomaly detection)
- Snort/Suricata: Network intrusion detection for VPC traffic (optional, on dedicated instances)
- Alerting: Integration with PagerDuty and Slack for real-time threat alerts
- Response: Automated isolation of compromised instances via Lambda functions

12.5 Application Security

12.5.1 OWASP Top 10 Mitigation

Threat	Mitigation	Implementation
--------	------------	----------------

Injection (SQL, NoSQL, Command)	Parameterized queries, ORM, input validation	All database access via ORM/parameterized queries
Broken Authentication	MFA, secure session management, password policy	See 12.2.1 and 12.2.3
Sensitive Data Exposure	Encryption, access control, DLP	See 12.3
XML External Entities (XXE)	Disable XXE processing, use JSON	All APIs use JSON, no XML parsing
Broken Access Control	RBAC, ABAC, authorization checks	See 12.2.5 and 12.2.6
Security Misconfiguration	IaC, automated scanning, hardened images	Terraform, CIS benchmarks, Trivy scanning
Cross-Site Scripting (XSS)	Output encoding, CSP, input validation	CSP headers, React auto-escaping, input sanitization
Insecure Deserialization	JSON only, schema validation, type checking	No Java deserialization, strict JSON schema validation
Using Components with Known Vulnerabilities	Dependency scanning, automated updates	Dependabot, Snyk, weekly dependency updates
Insufficient Logging and Monitoring	Comprehensive logging, SIEM, alerting	See Chapter 17 (Observability)

12.5.2 Secure Development Lifecycle

- Threat Modeling: STRIDE threat modeling for all major features, reviewed in architecture review

- Secure Code Review: Mandatory security review for all code changes, focus on input validation, authentication, authorization, cryptography
- Static Analysis: SonarQube, Snyk Code, Semgrep for SAST scanning in CI/CD pipeline
- Dynamic Analysis: OWASP ZAP, Burp Suite for DAST scanning in staging environment
- Dependency Scanning: Snyk, Dependabot for vulnerability scanning of all dependencies
- Container Scanning: Trivy, Clair for image vulnerability scanning in CI/CD pipeline
- Penetration Testing: Quarterly external penetration testing by certified security firm, annual red team exercise
- Bug Bounty: Public bug bounty program via HackerOne or Bugcrowd, rewards up to

5000forcriticalvulnerabilities5000 for critical
vulnerabilities5000forcriticalvulnerabilities

12.5.3 Secrets Management

- Vault: HashiCorp Vault for secrets management (AWS Secrets Manager as alternative)
- Dynamic Secrets: Short-lived database credentials, AWS credentials, API keys
- Rotation: Automatic rotation of all secrets every 30 days
- Access: Role-based access to secrets, audit logging of all secret access
- Encryption: All secrets encrypted at rest with AES-256-GCM, in transit with TLS 1.3
- Injection: Secrets injected via Kubernetes secrets (encrypted at rest), never in code or config files
- Leak Detection: GitLeaks for detecting secrets in code repositories, automated scanning of all commits

12.6 Compliance & Privacy

12.6.1 GDPR Compliance

- Lawful Basis: Consent for marketing, legitimate interest for service provision, legal obligation for compliance
- Data Subject Rights: Automated workflows for access, rectification, erasure, portability, restriction objections
- Privacy by Design: Data minimization, purpose limitation, storage limitation, accuracy, integrity, confidentiality
- Data Protection Impact Assessment (DPIA): Conducted for all high-risk processing activities
- Data Protection Officer (DPO): Appointed DPO, contact details published, direct reporting to board
- Records of Processing: Comprehensive register of all processing activities
- Breach Notification: 72-hour notification to supervisory authority, without undue delay to data subjects
- Cross-Border Transfers: Standard Contractual Clauses (SCCs) for data transfers outside EU

12.6.2 CCPA/CPRA Compliance

- Consumer Rights: Right to know, delete, opt-out, non-discrimination, data portability
- Disclosures: Clear privacy policy, cookie notice, data collection disclosures
- Opt-Out: Global privacy control, opt-out of sale/sharing, do not track
- Service Provider Contracts: Strict data processing agreements with all service providers
- Minors: Enhanced protections for users under 16, parental consent for under 13

12.6.3 Local Regulations (Sri Lanka)

- Data Protection Act: Compliance with Sri Lanka Personal Data Protection Act (No. 9 of 2022)
- Cross-Border Data Transfer: Data localization requirements, government approval for sensitive data export
- Government Data: Enhanced security for government service data, strict access controls, audit requirements
- Emergency Data: Special provisions for emergency data collection and processing
- KYC Requirements: Compliance with Central Bank of Sri Lanka KYC guidelines
- Financial Data: Compliance with Financial Transactions Reporting Act

12.6.4 Security Certifications & Standards

- ISO 27001: Information Security Management System (target certification by 2026)
- ISO 27017: Cloud security best practices
- ISO 27018: Protection of personal data in the cloud
- SOC 2 Type II: Security, availability, confidentiality (target certification by 2026)
- PCI DSS: Payment Card Industry Data Security Standard (if handling card data directly)
- HIPAA: Health Insurance Portability and Accountability Act (if handling health data)
- NIST Cybersecurity Framework: Alignment with NIST CSF core functions

12.7 Incident Response

12.7.1 Incident Response Plan

- Preparation: Incident response team, playbooks, communication templates, tools, training

- Detection: Automated monitoring, SIEM alerts, user reports, threat intelligence
- Analysis: Triage, severity assessment, impact assessment, root cause analysis
- Containment: Short-term containment (isolate affected systems), long-term containment (patch, fix)
- Eradication: Remove threat actor, eliminate vulnerabilities, clean affected systems
- Recovery: Restore systems from clean backups, verify integrity, resume operations
- Post-Incident: Lessons learned, process improvement, update playbooks, communication

12.7.2 Severity Levels

Level	Description	Response Time	Examples
P1-Critical	Active data breach, service outage, ransomware	15 minutes	Unauthorized access to production database, ransomware attack, DDoS causing outage
P2-High	Potential data breach, significant vulnerability	1 hour	Unpatched critical vulnerability, suspicious activity, unauthorized API access
P3-Medium	Security weakness, policy violation	4 hours	Misconfigured security group, weak password policy, missing MFA
P4-Low	Minor security issue, compliance gap	24 hours	Missing documentation, expired certificate, non-critical patch

12.7.3 Incident Response Team

- Incident Commander: Leads response, makes decisions, coordinates team
- Technical Lead: Technical analysis, containment, eradication
- Communications Lead: Internal and external communication, legal, PR
- Legal Counsel: Legal implications, regulatory notifications, evidence preservation
- Forensics Specialist: Digital forensics, evidence collection, chain of custody
- Business Representative: Business impact assessment, continuity planning
- External Partners: Forensics firm, legal counsel, PR firm, law enforcement

12.7.4 Communication Plan

- Internal: Slack #security-incidents, PagerDuty, email to leadership
- External: Customer notification (within 72 hours for data breach), regulatory notification, public disclosure (if required)
- Law Enforcement: Contact Cyber Crime Division of Sri Lanka Police for criminal incidents
- Media: Prepared statements, FAQ, press release coordination
- Customers: Email, in-app notification, status page update, support ticket response

12.8 Business Continuity & Disaster Recovery

12.8.1 Business Continuity Plan (BCP)

- Critical Functions: Emergency services, user authentication, payment processing, business discovery
- Recovery Time Objective (RTO): Emergency services: 5 minutes, Core platform: 1 hour, Non-critical: 24 hours

- Recovery Point Objective (RPO): Emergency services: 0 minutes (real-time replication), Core platform: 15 minutes, Non-critical: 1 hour
- Backup Strategy: Daily snapshots, continuous replication, cross-region backup, 7-year retention
- Testing: Quarterly DR drills, annual full-scale DR test, tabletop exercises monthly
- Documentation: Runbooks, contact lists, escalation procedures, decision trees

12.8.2 Disaster Recovery Sites

- Primary Site: AWS ap-southeast-1 (Singapore)
- Hot Standby: AWS ap-southeast-2 (Sydney) - active-active for emergency services, warm for core platform
- Cold Standby: AWS ap-south-1 (Mumbai) - infrastructure as code, can be spun up in 4 hours
- Data Replication: Real-time for emergency data, 15-minute intervals for core data, hourly for non-critical
- Failover: Automated DNS failover via Route 53 health checks, manual failover for controlled scenarios
- Failback: Planned failback after primary site restoration, data synchronization verification

12.9 Security Monitoring & SIEM

12.9.1 Security Information and Event Management (SIEM)

- Platform: Splunk Enterprise Security or Elastic Security (SIEM)
- Data Sources: AWS CloudTrail, VPC Flow Logs, WAF logs, application logs, OS logs, container logs, Kubernetes audit logs, database audit logs, identity logs

- Correlation Rules: Real-time correlation of security events across all data sources
- Alerting: PagerDuty for critical alerts, Slack for medium alerts, email for low alerts
- Dashboards: Security operations dashboard, threat intelligence dashboard, compliance dashboard
- Playbooks: Automated response playbooks for common incidents (isolate instance, block IP, revoke token)
- Threat Intelligence: Integration with MISP, AlienVault OTX, commercial threat feeds
- User Behavior Analytics (UBA): Anomaly detection for user behavior, insider threat detection

12.9.2 Security Operations Center (SOC)

- Operating Hours: 24/7/365 for P1/P2 incidents, business hours for P3/P4
- Team Structure: L1 (monitoring, triage), L2 (investigation, analysis), L3 (forensics, advanced response)
- Shift Coverage: 8-hour shifts, 3 shifts per day, minimum 2 analysts per shift
- Escalation: Automatic escalation after 30 minutes for P1, 2 hours for P2, 4 hours for P3
- Handover: Structured handover between shifts, incident status updates, ongoing investigations
- Metrics: Mean Time to Detect (MTTD), Mean Time to Respond (MTTR), Mean Time to Contain (MTTC), false positive rate

12.10 Third-Party Risk Management

12.10.1 Vendor Assessment

- Security Questionnaire: Standardized security questionnaire for all vendors (based on CSA CAIQ, VSAQ)
- Risk Assessment: Risk rating (low, medium, high, critical) based on data access, security posture, compliance
- Due Diligence: Security review of vendor's SOC 2 reports, ISO 27001 certificates, penetration test results
- Contract Requirements: Security clauses, data processing agreements, breach notification requirements, audit rights
- Ongoing Monitoring: Annual security reassessment, continuous monitoring of vendor security posture
- Termination: Data deletion requirements, return of data, transition assistance

12.10.2 Critical Vendor List

Vendor	Service	Risk Level	Assessment Date	Next Review
AWS	Cloud infrastructure	Critical	2025-01-15	2026-01-15
Stripe	Payment processing	Critical	2025-02-01	2026-02-01
Twilio	SMS delivery	High	2025-03-01	2026-03-01
SendGrid	Email delivery	High	2025-03-01	2026-03-01
Cloudflare	CDN, DDoS protection	High	2025-01-15	2026-01-15
MongoDB Atlas	Database hosting	High	2025-02-15	2026-02-15
Elasticsearch	Search engine	Medium	2025-04-01	2026-04-01
Redis Labs	Cache hosting	Medium	2025-04-01	2026-04-01
HashiCorp	Vault, Consul	High	2025-05-01	2026-05-01
DataDog	Monitoring, APM	Medium	2025-03-15	2026-03-15

Chapter 13: Infrastructure Architecture

13.1 Cloud Infrastructure (AWS)

LocalConnect is built on Amazon Web Services (AWS) for maximum scalability, reliability, and security. The infrastructure follows a multi-region, multi-AZ architecture with automated disaster recovery capabilities.

13.2 AWS Account Structure

13.2.1 AWS Organizations

- Master Account: Billing and governance only, no workloads
- Production Account: Live production workloads, strict access controls
- Staging Account: Pre-production testing, production-like data (anonymized)
- Development Account: Development environments, experimentation
- Security Account: Security tools, SIEM, GuardDuty, centralized logging
- Shared Services Account: CI/CD, container registry, artifact repository, DNS
- DR Account: Disaster recovery resources, warm standby infrastructure
- Sandbox Account: Developer sandboxes, temporary experiments

13.2.2 Account Security

- IAM: Identity Center (SSO) for all human access, no IAM users (except break-glass)
- MFA: Mandatory MFA for all IAM roles, root account secured with hardware MFA
- Cross-Account Access: IAM roles with trust policies, temporary credentials only
- Service Control Policies (SCP): Restrict actions at organization level (e.g., no public S3 buckets, no VPC deletion)
- CloudTrail: Organization-wide CloudTrail for all API calls, encrypted, immutable

- Config: AWS Config for compliance monitoring, conformance packs for CIS benchmarks

13.3 VPC Network Architecture

13.3.1 VPC Design (Production)

- VPC CIDR: 10.0.0.0/16 (65,536 IP addresses)
- Availability Zones: 3 AZs (ap-southeast-1a, ap-southeast-1b, ap-southeast-1c)
- Subnets:
 - Public Subnets: 10.0.1.0/24, 10.0.2.0/24, 10.0.3.0/24 (ALB, NAT Gateway, Bastion)
 - Private App Subnets: 10.0.11.0/24, 10.0.12.0/24, 10.0.13.0/24 (EKS worker nodes, ECS tasks)
 - Private Data Subnets: 10.0.21.0/24, 10.0.22.0/24, 10.0.23.0/24 (RDS, ElastiCache, MSK)
 - Private Management Subnets: 10.0.31.0/24, 10.0.32.0/24, 10.0.33.0/24 (Monitoring, CI/CD, Vault)

13.3.2 VPC Design (DR)

- VPC CIDR: 10.1.0.0/16 (separate from production to avoid overlap)
- AZs: 2 AZs (ap-southeast-2a, ap-southeast-2b)
- Subnets: Mirror of production (smaller sizes, sufficient for DR workloads)
- VPC Peering: Production ↔ DR for replication traffic, DR ↔ Shared Services for CI/CD

13.3.3 Network Connectivity

- Internet Gateway: Public subnets only, NAT Gateways for outbound from private subnets

- NAT Gateways: 3 NAT Gateways (one per AZ), highly available, elastic IPs
- VPC Endpoints: PrivateLink endpoints for AWS services (S3, DynamoDB, ECR, CloudWatch, KMS, Secrets Manager, SQS, SNS, Kinesis)
- Transit Gateway: Hub-and-spoke topology for inter-VPC connectivity, centralized routing
- Direct Connect: Optional dedicated connection for enterprise customers (1Gbps or 10Gbps)
- VPN: Site-to-site VPN for office connectivity, client VPN for remote employees

13.3.4 Network Security

- Security Groups: Stateful, instance-level, least privilege, no default allow-all
- Network ACLs: Stateless, subnet-level, deny by default, explicit allow rules
- Flow Logs: VPC Flow Logs for all traffic, stored in S3 for analysis (7 days retention)
- AWS Network Firewall: Optional for advanced network security, intrusion prevention
- PrivateLink: Private connectivity to SaaS services (MongoDB Atlas, DataDog, Splunk)

13.4 Compute Architecture

13.4.1 Kubernetes (EKS) - Primary Compute Platform

- Cluster: Amazon EKS (Elastic Kubernetes Service)
- Version: Latest stable version (auto-upgrade during maintenance windows)
- Nodes: Managed node groups (MNG) and Fargate (serverless)

- Node Types:
 - General Purpose: m6g.xlarge (4 vCPU, 16GB RAM) for standard workloads
 - Compute Optimized: c6g.2xlarge (8 vCPU, 16GB RAM) for high-CPU workloads
 - Memory Optimized: r6g.xlarge (4 vCPU, 32GB RAM) for memory-intensive workloads
 - GPU: g4dn.xlarge (4 vCPU, 16GB RAM, T4 GPU) for AI/ML inference
 - Spot Instances: Mixed instance types for cost optimization (up to 70% savings)
- Auto-scaling: Cluster Autoscaler + Karpenter (node provisioning in < 1 minute)
- Pod Density: Maximum 110 pods per node, optimized for workload characteristics
- Node Pools:
 - system: Critical infrastructure (Kong, Istio, monitoring, logging)
 - general: Standard microservices (user-service, business-service, event-service)
 - high-memory: Memory-intensive workloads (analytics, search, cache)
 - gpu: AI/ML inference workloads (AI-service, image processing)
 - spot: Non-critical, fault-tolerant workloads (batch processing, analytics)

13.4.2 ECS (Elastic Container Service) - Supplementary

- Use Cases: Batch processing, legacy services, services requiring dedicated instances
- Launch Type: Fargate (serverless) for most workloads, EC2 for GPU workloads
- Scheduling: EventBridge for scheduled tasks, SQS for queue-based tasks

- Service Discovery: AWS Cloud Map for service discovery

13.4.3 Lambda - Serverless

- Use Cases: Event processing, webhook handlers, image processing, scheduled tasks, real-time data processing
- Runtime: Node.js 20, Python 3.12, Java 17, Go 1.21
- Memory: 128MB to 10GB (configured per function)
- Timeout: 15 minutes maximum (configured per function)
- Concurrency: Reserved concurrency for critical functions, throttling for non-critical
- Provisioned Concurrency: For latency-sensitive functions (auth validation, emergency processing)
- VPC Integration: VPC-enabled for database access, security group isolation

13.4.4 EC2 - Dedicated Instances

- Use Cases: Bastion hosts, legacy services, services requiring specific OS configurations, GPU workstations
- Instance Types: t3.medium for bastion, c6g.xlarge for compute, g4dn.xlarge for GPU
- AMI: Hardened Amazon Linux 2023, CIS benchmarks, auto-updates
- Launch Template: Versioned, immutable infrastructure, user data for bootstrapping
- Auto Scaling: Manual for bastion, auto for compute workloads
- EBS: gp3 volumes for OS, io2 for high-performance data, encrypted at rest

13.5 Storage Architecture

13.5.1 Object Storage (S3)

- Buckets: See 9.7.1 for bucket structure

- Storage Classes: Intelligent-Tiering (default), Standard-IA (backups), Glacier (archives), Glacier Deep Archive (compliance)
- Versioning: Enabled for all buckets, 10 versions retained
- Encryption: SSE-S3 (default), SSE-KMS for sensitive data
- Access Control: IAM policies, bucket policies, S3 Block Public Access, pre-signed URLs
- Lifecycle: Automated transition and expiration policies
- Replication: Cross-region replication for critical buckets to DR region
- Events: S3 Event Notifications for Lambda triggers, SNS topics, SQS queues
- Performance: Transfer Acceleration for global uploads, multipart upload for large files

13.5.2 Block Storage (EBS)

- Volumes: gp3 (default, 3,000 IOPS baseline, up to 16,000 IOPS), io2 for high-performance (up to 256,000 IOPS, 99.999% durability)
- Encryption: All volumes encrypted at rest with KMS, in transit via TLS
- Snapshots: Daily snapshots, 35-day retention, cross-region copy for DR
- Multi-Attach: io2 volumes for shared storage (if needed for legacy databases)
- Fast Snapshot Restore: For critical databases to minimize recovery time

13.5.3 File Storage (EFS)

- Use Cases: Shared storage for containerized applications, content management, temporary file storage
- Performance Mode: General Purpose (default), Max I/O (for high-throughput workloads)
- Throughput Mode: Bursting (default), Provisioned (for consistent high throughput)
- Encryption: KMS encryption at rest, TLS in transit
- Lifecycle Management: Automatic transition to EFS Infrequent Access after 30 days

- Access Points: Per-application access points with isolated permissions

13.6 Database Infrastructure (AWS)

13.6.1 Amazon RDS for PostgreSQL

- Engine: PostgreSQL 15 (auto-minor version upgrades)
- Instance Class: db.r6g.4xlarge (primary), db.r6g.2xlarge (read replicas)
- Storage: 500GB gp3 (auto-scaling to 10TB), 10,000 IOPS provisioned
- Multi-AZ: Yes (synchronous standby in different AZ for HA)
- Read Replicas: 10 replicas across 3 AZs, automated failover
- Backups: Automated daily backups (35-day retention), cross-region backup to DR
- Parameter Groups: Custom parameters for performance tuning, `shared_preload_libraries` (pg_stat_statements, pg_trgm, postgis)
- Option Groups: SSL enforcement, audit logging
- Monitoring: Enhanced Monitoring (60-second granularity), Performance Insights, CloudWatch metrics
- Maintenance: 30-minute maintenance window, auto-minor version upgrades, patch management

13.6.2 Amazon DocumentDB (MongoDB Compatible)

- Use Cases: Forum posts, reviews, messages, analytics events (if MongoDB Atlas is not used)
- Instance Class: db.r6g.2xlarge (primary), db.r6g.xlarge (replicas)
- Storage: 200GB (auto-scaling), 1000 IOPS provisioned
- Replicas: 3 replicas across AZs
- Backups: Daily snapshots, 35-day retention
- Monitoring: CloudWatch metrics, Performance Insights

13.6.3 Amazon DynamoDB

- Capacity: On-demand for emergency tables, provisioned for predictable workloads
- Global Tables: Multi-region replication for user profiles, emergency data
- DAX: DynamoDB Accelerator for sub-millisecond reads on critical tables
- Streams: DynamoDB Streams for change data capture, Lambda triggers
- Backups: On-demand backups, point-in-time recovery (35 days)
- Monitoring: CloudWatch metrics, Contributor Insights, CloudWatch Contributor Insights

13.6.4 Amazon ElastiCache (Redis)

- Engine: Redis 7.0
- Node Type: cache.r6g.xlarge (4 vCPU, 26GB RAM per node)
- Cluster Mode: Enabled (3 shards, 1 replica per shard, 6 nodes total)
- Encryption: At rest (KMS), in transit (TLS), auth token
- Backups: Daily snapshots to S3, 7-day retention
- Monitoring: CloudWatch metrics, Redis INFO metrics, custom dashboards

13.6.5 Amazon OpenSearch Service (Elasticsearch)

- Version: OpenSearch 2.x
- Instance Type: r6g.2xlarge (data nodes), m6g.xlarge (master nodes), c6g.xlarge (coordinating nodes)
- Data Nodes: 10 nodes (3 AZs, 3-4-3 distribution)
- Master Nodes: 3 dedicated master nodes (quorum)
- Coordinating Nodes: 2 dedicated coordinating nodes (load balancing)
- Storage: 500GB gp3 per data node, auto-scaling
- Index State Management: Automated index rollover, transition, deletion
- Backups: Daily snapshots to S3, 30-day retention
- Monitoring: OpenSearch dashboards, CloudWatch metrics, custom dashboards

13.7 Messaging & Streaming Infrastructure

13.7.1 Amazon MSK (Managed Kafka)

- Version: Kafka 3.6.x
- Broker Instance: kafka.m6g.large (2 vCPU, 8GB RAM per broker)
- Brokers: 6 brokers (3 AZs, 2-2-2 distribution)
- Storage: 1000GB gp3 per broker, auto-scaling
- Replication: 3 for all topics, 2 minimum in-sync replicas for critical topics
- Monitoring: CloudWatch metrics, Kafka JMX metrics, Cruise Control for rebalancing
- Security: TLS 1.3 for client-broker, SASL/SCRAM for authentication, ACLs for authorization
- Backups: Automated backups to S3, 7-day retention
- Multi-VPC: VPC connectivity for cross-account access

13.7.2 Amazon SQS

- Queues: See 10.6.1 for queue design
- Type: Standard (default), FIFO (where ordering is critical)
- Visibility Timeout: 30 seconds (default), configurable per queue
- Dead Letter Queues: Enabled for all queues, max 3 receives before DLQ
- Encryption: SSE-SQS (default), SSE-KMS for sensitive queues
- Monitoring: CloudWatch metrics, custom dashboards
- DLQ Monitoring: Alerts for messages in DLQ, automated reprocessing

13.7.3 Amazon SNS

- Topics: See 10.6.2 for topic design
- Protocols: SQS, Lambda, HTTP/S, email, SMS, push notifications
- Encryption: SSE-SNS (default), SSE-KMS for sensitive topics
- Delivery Policies: Retry policies, DLQ for failed deliveries
- Monitoring: CloudWatch metrics, delivery status logging

13.7.4 Amazon Kinesis

- Use Cases: Real-time analytics, log aggregation, clickstream processing
- Type: Kinesis Data Streams (real-time), Kinesis Data Firehose (batch to S3/Redshift/Elasticsearch)
- Shards: 10 shards (auto-scaling enabled), 1MB/s write per shard
- Retention: 24 hours (default), 7 days (extended)
- Encryption: SSE-KMS
- Monitoring: CloudWatch metrics, custom dashboards

13.8 CDN & Edge Infrastructure

13.8.1 CloudFront CDN

- Use Cases: Static asset delivery, API caching, media streaming, DDoS protection
- Origins: S3 (static assets), ALB (dynamic API), API Gateway (serverless API), custom origins (external services)
- Edge Locations: All CloudFront edge locations (400+ globally)
- Cache Behaviors:
 - Static Assets: Cache for 1 year, cache key: URL only
 - API Responses: Cache for 5 minutes, cache key: URL + query params + auth token (for personalized)
 - Media: Cache for 1 day, cache key: URL + range header (for video streaming)
- Compression: Brotli (preferred), gzip fallback
- SSL/TLS: TLS 1.3, custom certificate (ACM), SNI
- Security: AWS WAF integration, field-level encryption, signed URLs/cookies for private content
- Origin Shield: Enabled for caching layer, reduces origin load
- Real-Time Logs: CloudWatch real-time logs for monitoring
- Monitoring: CloudWatch metrics, CloudFront access logs to S3

13.8.2 Cloudflare (Supplementary CDN)

- Use Cases: DNS, DDoS protection, WAF, bot management, CDN for non-AWS content
- Plan: Cloudflare Pro (20/monthperdomain)20/month per domain)20/monthperdomain)
- Features:
 - DNS: Full DNS management, CNAME flattening, DNSSEC
 - DDoS: Layer 3/4/7 DDoS protection, always-on
 - WAF: Custom rules, managed rulesets, rate limiting
 - CDN: Cache static assets, image optimization, minification
 - Bot Management: Bot detection, challenge for suspicious
 - SSL: Full (strict) SSL, TLS 1.3, automatic HTTPS rewrite
 - Integration: Cloudflare → AWS ALB (origin), Cloudflare DNS → Route 53 (subdomain delegation)

13.8.3 Route 53 DNS

- Hosted Zones: localconnect.com (primary), localconnect.lk (local), localconnect.dev (development)
- Record Types: A, AAAA, CNAME, MX, TXT, NS, SOA, SPF, DKIM, DMARC
- Routing Policies:
 - Simple: Static routing for non-critical domains
 - Weighted: A/B testing, canary deployments (90% primary, 10% canary)
 - Latency: Latency-based routing for multi-region (primary to Singapore, DR to Sydney)
 - Failover: Health check-based failover (primary → DR)
 - Geolocation: Geo-routing for compliance (Sri Lanka users → local servers, EU users → GDPR-compliant servers)
- Health Checks: HTTP/HTTPS health checks for all endpoints, 30-second interval, 3 consecutive failures for unhealthy

- DNSSEC: Enabled for all hosted zones, algorithm 13 (ECDSA P-256 with SHA-256)
- Resolver: Route 53 Resolver for hybrid DNS, VPC DNS resolution

13.9 Load Balancing Architecture

13.9.1 Application Load Balancer (ALB)

- Deployment: 2 ALBs per environment (public-facing, internal-facing)
- Listeners:
 - HTTP (80): Redirect to HTTPS
 - HTTPS (443): TLS 1.3, ACM certificate, SNI for multi-domain
- Target Groups:
 - EKS services (IP-based target registration)
 - ECS services (IP-based target registration)
 - Lambda functions (target-based)
 - EC2 instances (instance-based)
- Health Checks: HTTP/HTTPS, 30-second interval, 2 healthy/unhealthy thresholds, 5-second timeout
- Routing: Path-based routing (/v1/auth → auth-service, /v1/users → user-service), host-based routing (api.localconnect.com → API, app.localconnect.com → Web)
- Sticky Sessions: Disabled for stateless services, enabled for WebSocket (cookie-based, 1-day TTL)
- Access Logs: S3 bucket, 7-day retention, Athena for analysis
- SSL Policy: ELBSecurityPolicy-TLS13-1-2-2021-06 (TLS 1.3, TLS 1.2 with strong ciphers)

13.9.2 Network Load Balancer (NLB)

- Use Cases: TCP/UDP traffic, high-throughput, low-latency, WebSocket (before ALB WebSocket support), Kafka brokers
- Deployment: 1 NLB for Kafka, 1 NLB for internal TCP services

- Listeners: TCP 9092 (Kafka), TCP 6379 (Redis), TCP 5432 (PostgreSQL - if needed)
- Target Groups: Instance-based for EC2, IP-based for containers
- Health Checks: TCP, 30-second interval, 2 healthy/unhealthy thresholds
- Cross-Zone Load Balancing: Enabled for uniform distribution
- Access Logs: S3 bucket, 7-day retention

13.10 Identity & Access Management (IAM)

13.10.1 IAM Architecture

- Identity Center (SSO): Centralized SSO for all human access (AWS console, CLI, applications)
- IAM Roles: No long-term IAM users (except break-glass accounts), role-based access for all services
- Service Accounts: IAM roles for service-to-service access (EKS pod IAM roles, Lambda execution roles, ECS task roles)
- Permission Boundaries: Maximum permissions for roles, least privilege within boundaries
- IAM Policy Structure:
 - Managed Policies: AWS managed policies for common use cases
 - Customer Managed Policies: Custom policies for LocalConnect-specific access
 - Inline Policies: Rarely used, only for temporary or specific restrictions
- Cross-Account Roles: Trusted relationships with external accounts (partners, auditors)
- IAM Access Analyzer: Continuous analysis of resource policies, external access findings
- IAM Credential Report: Daily report of all IAM credentials, automated alerts for unused credentials

13.10.2 Service-to-Service IAM

- EKS Pod IAM Roles (IRSA): Fine-grained IAM roles for Kubernetes pods, no node IAM credentials
- Lambda Execution Roles: Minimal permissions per Lambda function, no wildcard permissions
- ECS Task Roles: IAM roles for ECS tasks, separate from EC2 instance roles
- Cross-Service Access: IAM roles with trust policies for service-to-service communication (e.g., ECS task can assume role to access S3)
- Temporary Credentials: STS AssumeRole for all cross-service access, 1-hour expiration

13.10.3 Break-Glass Access

- Scenario: Emergency access when SSO is unavailable
- Accounts: 2 break-glass IAM users (admin1, admin2) with hardware MFA
- Protection: Password in sealed envelope in CEO safe, MFA token in separate location
- Audit: All break-glass usage logged to CloudTrail, immediate alerting to security team
- Procedure: Documented runbook, requires VP approval for usage, 24-hour time limit
- Review: Monthly review of break-glass accounts, password rotation every 6 months

13.11 Backup & Disaster Recovery Infrastructure

13.11.1 Backup Strategy

- AWS Backup: Centralized backup management, backup plans, vaults, policies
- Backup Plans:

- Critical (RPO 15 min): RDS, DynamoDB, EFS, EBS (hourly incremental, daily full)
- Standard (RPO 1 hour): S3, MSK, ElastiCache (daily full)
- Archive (RPO 24 hours): Logs, analytics, non-critical data (daily full)
- Backup Vault: Encrypted with KMS, cross-region copy to DR, immutable (WORM) for compliance
- Backup Retention: 35 days for point-in-time recovery, 7 years for compliance archives
- Testing: Monthly restore tests, automated validation of restored data
- Monitoring: CloudWatch alarms for backup failures, daily backup reports

13.11.2 Disaster Recovery Architecture

- RTO (Recovery Time Objective): Emergency services: 5 minutes, Core platform: 1 hour, Non-critical: 24 hours
- RPO (Recovery Point Objective): Emergency services: 0 minutes, Core platform: 15 minutes, Non-critical: 1 hour
- DR Strategy:
 - Emergency Services: Active-Active (real-time replication, automatic failover)
 - Core Platform: Warm Standby (hourly replication, scripted failover in 1 hour)
 - Non-Critical: Cold Standby (daily backup, infrastructure as code, 4-hour spin-up)
- Replication:
 - RDS: Cross-region read replica, automated promotion
 - DynamoDB: Global Tables (real-time replication)
 - S3: Cross-region replication (CRR)
 - ElastiCache: Manual snapshot and restore to DR
 - MSK: MirrorMaker 2 for cross-region replication
 - EKS: Infrastructure as code (Terraform), automated deployment to DR

- Failover:
 - DNS Failover: Route 53 health check-based failover (automatic for emergency, manual for core)
 - Data Synchronization: Verify data consistency before failover, delta sync for warm standby
 - Traffic Switching: Weighted routing in Route 53, gradual traffic shift (10% → 50% → 100%)
- Failback: Planned failback after primary site restoration, data synchronization, gradual traffic return

13.11.3 DR Testing Schedule

- Tabletop Exercises: Monthly (1 hour, discussion-based, scenario walkthrough)
- Component Testing: Quarterly (4 hours, test individual component failover)
- Full DR Drill: Semi-annually (8 hours, full failover and failback, production-like traffic)
- Unannounced Drills: Annually (surprise drill, test team response without notice)
- Documentation: DR runbooks updated after each test, lessons learned documented, action items tracked

13.12 Cost Management & Optimization

13.12.1 Cost Allocation

- Tags: Mandatory tags for all resources (Environment, Team, Service, CostCenter, Project, Owner)
- Cost Explorer: Daily cost analysis, anomaly detection, budget alerts
- AWS Budgets: Monthly budget alerts at 80%, 100%, 120% thresholds
- Savings Plans: 1-year compute savings plan (30% discount), 3-year for stable workloads (50% discount)

- Reserved Instances: RDS reserved instances (1-year, 40% discount), ElastiCache reserved nodes
- Spot Instances: Up to 70% discount for fault-tolerant workloads (batch processing, analytics, non-critical services)
- Auto-Scaling: Scale to zero for non-production environments during off-hours
- Right-Sizing: Monthly review of instance utilization, downsize underutilized instances
- Storage Optimization: S3 Intelligent-Tiering, EFS lifecycle management, EBS gp3 migration
- Network Optimization: VPC endpoints to reduce NAT Gateway costs, CloudFront for edge caching

13.12.2 Cost Monitoring Dashboards

- AWS Cost Explorer: Custom reports by service, team, environment, tag
- CloudWatch Billing Alarms: Real-time billing alerts, forecasted cost alerts
- Grafana Dashboard: Custom cost dashboards with breakdown by service, team, trend analysis
- Monthly Cost Reviews: Finance + Engineering monthly cost review meeting, optimization opportunities
- Anomaly Detection: AWS Cost Anomaly Detection, automatic alerts for unexpected cost spikes

13.12.3 Cost Optimization Targets

Category	Current Monthly	Target Monthly	Savings
Compute (EKS, EC2, Lambda)	\$8,000	\$6,000	25%
Database (RDS, DynamoDB, ElastiCache)	\$5,000	\$4,000	20%
Storage (S3, EBS, EFS)	\$2,000	\$1,500	25%

Networking (ALB, NAT, CloudFront, Data Transfer)	\$3,000	\$2,000	33%
Security (WAF, Shield, GuardDuty, Inspector)	\$1,500	\$1,200	20%
Monitoring (CloudWatch, DataDog, Splunk)	\$2,000	\$1,500	25%
Total	\$21,500	\$16,200	25%

Chapter 14: Kubernetes Architecture

14.1 EKS Cluster Design

LocalConnect runs on Amazon EKS (Elastic Kubernetes Service) with a multi-tenant, multi-environment architecture. The cluster is designed for high availability, scalability, and security.

14.1.1 Cluster Configuration

- Name: localconnect-prod, localconnect-staging, localconnect-dev
- Version: Kubernetes 1.29 (auto-upgrade to 1.30 during maintenance windows)
- Region: ap-southeast-1 (Singapore)
- VPC: Private subnets only, no public endpoint access (private cluster)
- Endpoint Access: Private only (no public internet access to API server)
- Logging: API server, audit, authenticator, controller manager, scheduler (all enabled)
- Encryption: KMS encryption for secrets at rest, envelope encryption for etcd
- IAM: IAM roles for service accounts (IRSA) for pod-level IAM permissions
- Addons:
 - CoreDNS (managed, auto-scaling)

- kube-proxy (managed)
- Amazon VPC CNI (custom networking, IPv4/IPv6 dual-stack)
- EBS CSI Driver (dynamic provisioning, snapshots)
- EFS CSI Driver (shared storage)
- AWS Load Balancer Controller (ALB/NLB management)
- AWS GuardDuty Agent (runtime threat detection)
- AWS Distro for OpenTelemetry (observability)

14.1.2 Cluster Networking

- CNI: Amazon VPC CNI with custom networking
- Pod CIDR: 10.100.0.0/16 (separate from VPC CIDR)
- Service CIDR: 10.200.0.0/16
- IP Address Management: Prefix delegation for efficient IP allocation (max pods per node: 110)
- Network Policies: Default deny-all, explicit allow rules per namespace
- DNS: CoreDNS with custom forwarders, caching, and custom domains
- Service Mesh: Istio (sidecar injection, mTLS, traffic management, observability)

14.2 Node Architecture

14.2.1 Node Groups

Node Group	Instance Type	Min	Max	Desired	Labels	Taints	Purpose
system	m6g.xlarge	3	10	3	node-role=system	NoSchedule	Kong, Istio, monitoring, logging
general	m6g.xlarge	5	50	10	node-role=general	None	Standard microservices

high-mem ory	r6g.xlarge	2	20	5	node- role=high- memory	None	Analytics, search, cache
gpu	g4dn.xlar ge	0	10	2	node- role=gpu	gpu=true:NoSched ule	AI/ML inference
spot	m6g.larg e	0	10 0	20	node- role=spot	spot=true:NoSche dule	Batch processing, analytics

14.2.2 Node Configuration

- AMI: Amazon EKS optimized AMI (ARM64 Graviton for cost optimization)
- Disk: 100GB gp3 root volume, encrypted, auto-scaling
- Kubelet: Max pods 110, eviction thresholds configured, resource reservations
- Bootstrap: User data script for node registration, labeling, tainting
- Updates: Managed node group updates (rolling update, 10% max unavailable)
- Monitoring: Node-level metrics via Prometheus Node Exporter, CloudWatch agent

14.2.3 Karpenter (Auto-Provisioning)

- Purpose: Rapid node provisioning (sub-60 seconds) for dynamic workloads
- Provisioners:
 - default: General purpose, on-demand, m6g family
 - spot: Spot instances, m6g/c6g/r6g families, 70% cost savings
 - gpu: GPU instances, g4dn family, on-demand
 - memory: Memory-optimized, r6g family, on-demand
- Consolidation: Automatic node consolidation to reduce costs
- TTL: 30 minutes for idle nodes (except system nodes)

- Scheduling: Pod-based scheduling, resource-aware, topology-aware

14.3 Namespace Architecture

14.3.1 Namespace Design

Namespace	Purpose	Network Policies	Resource Quotas	Pod Security
istio-system	Istio control plane, ingress gateway	Allow from all	Restricted	Restricted
kube-system	Kubernetes system components	Allow from all	System	System
monitoring	Prometheus, Grafana, Alertmanager, Jaeger	Allow from all	Restricted	Restricted
logging	Fluent Bit, Loki, Vector	Allow from all	Restricted	Restricted
security	Vault, Falco, cert-manager, OPA	Deny from all	Restricted	Restricted
localconnect	Core microservices	Allow from istio-system, monitoring	Standard	Restricted
localconnect-data	Databases, caches, message queues	Deny from all (except localconnect)	Standard	Restricted
localconnect-tools	CI/CD tools, backup utilities	Allow from specific	Standard	Restricted
localconnect-batch	Batch processing, cron jobs	Allow from specific	Standard	Restricted

localconnect-ai	AI/ML inference services	Allow from localconnect	Standard	Restricted
-----------------	--------------------------	-------------------------	----------	------------

14.3.2 Namespace Isolation

- Network Policies: Default deny-all ingress/egress, explicit allow rules per namespace
- Pod Security: Restricted Pod Security Standard (no privileged containers, no root, read-only root filesystem, drop all capabilities)
- Resource Quotas: CPU, memory, pods, services, PVCs per namespace
- Limit Ranges: Default CPU/memory requests and limits per pod/container
- RBAC: Namespace-scoped roles, no cluster-admin for application teams

14.4 Workload Architecture

14.4.1 Deployment Patterns

Pattern	Use Case	Example
Standard Deployment	Stateless microservices	user-service, business-service
StatefulSet	Stateful workloads (databases)	Redis, PostgreSQL (if self-managed)
DaemonSet	Node-level agents	Fluent Bit, Node Exporter, Falco
CronJob	Scheduled tasks	Backup, report generation, cleanup
Job	One-time tasks	Data migration, batch processing
HorizontalPodAutoscaler	Auto-scaling	All microservices (CPU/memory/queue-based)

VerticalPodAutoscaler	Right-sizing	Recommendation mode (non-production)
-----------------------	--------------	--------------------------------------

14.4.2 Pod Specification Standards

apiVersion: apps/v1

kind: Deployment

metadata:

name: user-service

namespace: localconnect

labels:

app: user-service

version: v1

tier: backend

spec:

replicas: 3

strategy:

type: RollingUpdate

rollingUpdate:

maxSurge: 25%

maxUnavailable: 0

selector:

matchLabels:

app: user-service

version: v1

template:

metadata:

labels:

app: user-service

version: v1

tier: backend

annotations:

prometheus.io/scrape: "true"

prometheus.io/port: "8080"

prometheus.io/path: "/metrics"

sidecar.istio.io/inject: "true"

spec:

serviceAccountName: user-service

securityContext:

runAsNonRoot: true

runAsUser: 1000

fsGroup: 1000

seccompProfile:

type: RuntimeDefault

containers:

- name: user-service

image: localconnect/user-service:v1.2.3

imagePullPolicy: IfNotPresent

ports:

- name: http

containerPort: 8080

protocol: TCP

- name: grpc

containerPort: 9090

protocol: TCP

- name: management

containerPort: 8081

protocol: TCP

env:

- name: JAVA_OPTS

value: "-XX:+UseG1GC -XX:MaxRAMPercentage=75.0 -XX:InitialRAMPercentage=50.0"

- name: SPRING_PROFILES_ACTIVE

value: "production"

- name: DB_HOST

valueFrom:

secretKeyRef:

name: user-service-db

key: host

- name: DB_PASSWORD

valueFrom:

secretKeyRef:

name: user-service-db

key: password

resources:

requests:

cpu: 500m

memory: 512Mi

limits:

cpu: 2000m

memory: 2Gi

livenessProbe:

httpGet:

path: /health/live

port: management

initialDelaySeconds: 30

periodSeconds: 10

timeoutSeconds: 5

failureThreshold: 3

readinessProbe:

httpGet:

path: /health/ready

port: management

initialDelaySeconds: 10

periodSeconds: 5

timeoutSeconds: 3

failureThreshold: 3

startupProbe:

httpGet:

path: /health/startup

port: management

initialDelaySeconds: 10

periodSeconds: 5

timeoutSeconds: 3

failureThreshold: 30

volumeMounts:

- name: tmp

mountPath: /tmp

- name: config

mountPath: /config

readOnly: true

securityContext:

allowPrivilegeEscalation: false

readOnlyRootFilesystem: true

capabilities:

drop:

- ALL

volumes:

- name: tmp

emptyDir: {}

- name: config

configMap:

name: user-service-config

affinity:

podAntiAffinity:

preferredDuringSchedulingIgnoredDuringExecution:

- weight: 100

podAffinityTerm:

labelSelector:

matchExpressions:

- key: app

operator: In

values:

- user-service

topologyKey: kubernetes.io/hostname

topologySpreadConstraints:

- maxSkew: 1

topologyKey: topology.kubernetes.io/zone

whenUnsatisfiable: DoNotSchedule

labelSelector:

matchLabels:

app: user-service

14.4.3 Horizontal Pod Autoscaler (HPA)

apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:

name: user-service-hpa

namespace: localconnect

spec:

scaleTargetRef:

apiVersion: apps/v1

kind: Deployment

name: user-service

minReplicas: 3

maxReplicas: 20

metrics:

- type: Resource

resource:

name: cpu

target:

type: Utilization

averageUtilization: 70

- type: Resource

resource:

name: memory

target:

type: Utilization

averageUtilization: 80

- type: Pods

pods:

metric:

name: http_requests_per_second

target:

type: AverageValue

averageValue: "1000"

behavior:

scaleUp:

stabilizationWindowSeconds: 60

policies:

- type: Percent

value: 100

periodSeconds: 60

- type: Pods

value: 4

periodSeconds: 60

selectPolicy: Max

scaleDown:

stabilizationWindowSeconds: 300

policies:

- type: Percent

value: 10

periodSeconds: 60

- type: Pods

value: 2

periodSeconds: 60

selectPolicy: Min

14.4.4 Pod Disruption Budget (PDB)

apiVersion: policy/v1

kind: PodDisruptionBudget

metadata:

name: user-service-pdb

namespace: localconnect

spec:

minAvailable: 2

selector:

matchLabels:

app: user-service

14.5 Service Architecture

14.5.1 Service Types

Service Type	Use Case	Example
ClusterIP	Internal service communication	Service-to-service within cluster
NodePort	External access for debugging	Bastion access, port forwarding
LoadBalancer	External-facing services	Kong Gateway, Istio Ingress
ExternalName	External service references	External APIs, SaaS services
Headless	Stateful sets, direct pod access	Redis cluster, Kafka brokers

14.5.2 Service Mesh (Istio)

Istio Control Plane

- Version: Istio 1.20
- Profile: Default (istiod, ingress gateway, egress gateway)
- Components:
 - Pilot: Configuration management, service discovery
 - Citadel: Certificate management, mTLS
 - Galley: Configuration validation

- Sidecar Injector: Automatic sidecar injection
- Ingress Gateway: 3 replicas, load balancer service, TLS termination
- Egress Gateway: 2 replicas, controlled external access, traffic monitoring

Istio Sidecar Configuration

- Injection: Automatic injection for all pods in localconnect namespace (opt-out for specific pods)
- Resource Limits: 100m CPU, 128Mi memory per sidecar (default)
- Traffic Capture: All inbound and outbound traffic captured by sidecar proxy (Envoy)
- mTLS: Automatic mutual TLS for all service-to-service communication (PERMISSIVE → STRICT)
- Observability: Automatic metrics, distributed tracing, access logging

Istio Traffic Management

- VirtualServices: Route traffic based on headers, weights, user segments (see 11.3)
- DestinationRules: Load balancing, circuit breakers, outlier detection (see 11.3)
- Gateway: External access configuration, TLS termination (see 11.3)
- ServiceEntry: External service registration (e.g., Stripe API, Google Maps API)
- PeerAuthentication: mTLS policy (PERMISSIVE for migration, STRICT for production)
- AuthorizationPolicy: Service-level access control (see 12.5.2)

14.6 ConfigMap & Secret Management

14.6.1 ConfigMap Design

- Application Config: Per-service ConfigMaps with environment-specific settings

- Feature Flags: Centralized ConfigMap for feature flags, updated via API
- Istio Config: ConfigMap for mesh configuration, telemetry, tracing
- Versioning: ConfigMap versioning, rollback capability
- Hot Reload: Applications watch ConfigMap changes and reload without restart (where supported)

14.6.2 Secret Management

- HashiCorp Vault: Primary secret management (dynamic secrets, rotation, encryption)
- Kubernetes Secrets: Sealed Secrets (Bitnami Sealed Secrets) for GitOps, encrypted at rest
- AWS Secrets Manager: Integration with RDS, rotation, cross-region replication
- External Secrets Operator: Syncs secrets from Vault/AWS Secrets Manager to Kubernetes secrets
- Secret Injection: Secrets injected as environment variables or mounted files (never in code)
- Rotation: Automatic rotation every 30 days, zero-downtime rotation
- Audit: All secret access logged, audit trail for compliance

14.6.3 Vault Integration

apiVersion: secrets.hashicorp.com/v1beta1

kind: VaultDynamicSecret

metadata:

name: user-service-db-credentials

namespace: localconnect

spec:

vaultAuthRef: user-service-auth

mount: database

path: creds/user-service-db

destination:

create: true

name: user-service-db-credentials

rolloutRestartTargets:

- kind: Deployment

name: user-service

14.7 Storage Architecture

14.7.1 Persistent Volumes

Storage Class	Provisioner	Type	Use Case	Reclaim Policy
gp3	EBS CSI Driver	SSD	General purpose, databases	Delete
gp3-encrypted	EBS CSI Driver	SSD	Encrypted general purpose	Delete
io2	EBS CSI Driver	SSD	High-performance, databases	Retain
efs	EFS CSI Driver	NFS	Shared storage, multi-pod access	Retain
efs-encrypted	EFS CSI Driver	NFS	Encrypted shared storage	Retain

14.7.2 Volume Snapshots

- CSI Snapshotter: Automated snapshot creation for all PVCs
- Schedule: Daily snapshots, 7-day retention for standard, 30-day for critical
- Cross-Region: Snapshots copied to DR region for disaster recovery
- Restoration: Point-in-time restoration from snapshots, tested monthly

14.7.3 Backup Strategy

- Velero: Cluster backup tool for Kubernetes resources and persistent volumes
- Schedule: Daily backup of all namespaces, 30-day retention
- Storage: Backups stored in S3, cross-region replication to DR
- Restoration: Namespace-level restoration, full cluster restoration, tested monthly
- Encryption: Backups encrypted at rest with KMS, in transit with TLS

14.8 Security Architecture

14.8.1 Pod Security

- Pod Security Standards: Restricted level enforced at cluster level
- Pod Security Admission: Enforced for all namespaces, audit and warn modes for migration
- Security Context:
 - runAsNonRoot: true (all pods)
 - readOnlyRootFilesystem: true (all pods)
 - allowPrivilegeEscalation: false (all pods)
 - capabilities: drop ALL (all pods)
 - seccompProfile: RuntimeDefault (all pods)
- Privileged Containers: Prohibited, exceptions require CISO approval
- Host Network: Prohibited for all workloads
- Host PID/IPC: Prohibited for all workloads

14.8.2 Network Policies

apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

name: default-deny-all

namespace: localconnect

spec:

podSelector: {}

policyTypes:

- Ingress

- Egress

apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

name: allow-ingress-from-istio

namespace: localconnect

spec:

podSelector:

matchLabels:

tier: backend

policyTypes:

- Ingress

ingress:

- from:

- namespaceSelector:

matchLabels:

name: istio-system

ports:

- protocol: TCP

port: 8080

- protocol: TCP

port: 9090

apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

name: allow-egress-to-database

namespace: localconnect

spec:

podSelector:

matchLabels:

tier: backend

policyTypes:

- Egress

egress:

- to:

- namespaceSelector:

matchLabels:

name: localconnect-data

ports:

- protocol: TCP

port: 5432

- protocol: TCP

port: 6379

- protocol: TCP

port: 27017

14.8.3 RBAC

apiVersion: rbac.authorization.k8s.io/v1

kind: Role

metadata:

name: user-service-role

namespace: localconnect

rules:

- apiGroups: [""]

resources: ["configmaps"]

verbs: ["get", "list", "watch"]

- apiGroups: [""]

resources: ["secrets"]

verbs: ["get"]

resourceNames: ["user-service-config", "user-service-db-credentials"]

apiVersion: rbac.authorization.k8s.io/v1

kind: RoleBinding

metadata:

name: user-service-rolebinding

namespace: localconnect

subjects:

- kind: ServiceAccount

name: user-service

namespace: localconnect

roleRef:

kind: Role

name: user-service-role

apiGroup: rbac.authorization.k8s.io

14.8.4 Admission Controllers

- Pod Security Admission: Enforces Pod Security Standards
- OPA Gatekeeper: Custom policies (e.g., no latest tag, required labels, resource limits, no privileged containers)
- Kyverno: Alternative to OPA for simpler policies (mutating and validating webhooks)
- cert-manager: Automatic TLS certificate management for ingress
- Istio Injection: Automatic sidecar injection for service mesh
- Vault Agent Injector: Automatic secret injection from Vault

14.9 Observability Architecture

14.9.1 Metrics (Prometheus + Grafana)

- Prometheus: 2 replicas (HA), 15-day retention, Thanos for long-term storage
- Grafana: 2 replicas, LDAP/SSO authentication, role-based dashboards
- ServiceMonitor: Automatic discovery of metrics endpoints via labels
- PodMonitor: Automatic discovery of pod-level metrics
- Rules: Recording rules for aggregation, alerting rules for critical thresholds
- Custom Metrics: Business metrics (order rate, user registration, event attendance)
- SLA Monitoring: Response time, error rate, availability metrics

14.9.2 Logging (Loki + Fluent Bit)

- Fluent Bit: DaemonSet on all nodes, tail logs, forward to Loki
- Loki: 3 replicas, 30-day retention, S3 backend for long-term storage
- Log Aggregation: Structured logging (JSON), correlation IDs, service names, pod names
- Log Queries: LogQL for querying, Grafana integration for visualization
- Alerting: Log-based alerts for errors, security events, business anomalies
- Log Shipping: Dual shipping to Loki (real-time) and S3 (archive, 7 years)

14.9.3 Distributed Tracing (Jaeger + Tempo)

- Jaeger: 3 replicas, Elasticsearch backend for trace storage, 7-day retention
- Tempo: Grafana Tempo for cost-effective trace storage (S3 backend), 30-day retention
- Instrumentation: OpenTelemetry SDKs for all services, automatic Istio tracing
- Sampling: 1% for standard traffic, 100% for error traces, 100% for critical paths (emergency, payments)
- Correlation: Trace IDs propagated across all services, linked to logs and metrics
- Analysis: Jaeger UI for trace visualization, Grafana integration for trace-to-metrics

14.9.4 Service Mesh Observability (Kiali)

- Kiali: Istio service mesh visualization, traffic flow, health status
- Dashboards: Service graph, workload health, traffic distribution, error rates
- Drill-down: Per-service metrics, per-workload traces, per-request logs

14.10 Cluster Operations

14.10.1 Upgrades

- Control Plane: Managed by AWS EKS, 1-2 minor version lag, maintenance window notifications
- Node Groups: Rolling updates, 10% max unavailable, cordon and drain
- Addons: Managed addon updates, tested in staging before production
- Testing: Staging cluster mirrors production, full regression test before production upgrade
- Rollback: Automated rollback on health check failure, manual rollback capability

14.10.2 Maintenance

- Node Reboots: Managed by AWS, automated replacement for failed nodes
- Certificate Rotation: Automatic cert-manager rotation, 30-day expiration alerts
- Secret Rotation: Vault automatic rotation, zero-downtime updates
- Image Updates: Rolling updates, blue-green deployment for critical services
- Cleanup: Automated cleanup of old ReplicaSets, orphaned PVCs, unused ConfigMaps

14.10.3 Disaster Recovery

- Cluster Backup: Velero daily backup of all resources, 30-day retention
- etcd Backup: Automated EKS etcd snapshots, 35-day retention
- Cross-Region: Backup copied to DR region, tested monthly
- Cluster Recreation: Infrastructure as code (Terraform), can recreate cluster in 2 hours
- Data Recovery: Database point-in-time recovery, S3 object versioning, EBS snapshot restoration

Chapter 15: DevOps & CI/CD Architecture

15.1 CI/CD Principles

LocalConnect's CI/CD pipeline follows the principles of automation, reproducibility, fast feedback, and continuous improvement. Every code change is automatically built, tested, scanned, and deployed through a standardized pipeline.

15.2 Version Control Strategy

15.2.1 Git Branching Model

- Main Branch: Production-ready code, protected, requires pull request + review
- Develop Branch: Integration branch for features, auto-deployed to staging
- Feature Branches: Individual features, prefixed with feature/, merged via PR
- Bugfix Branches: Production fixes, prefixed with bugfix/, cherry-picked to develop
- Hotfix Branches: Critical production fixes, prefixed with hotfix/, merged to main and develop
- Release Branches: Release preparation, prefixed with release/, version bumped, final testing

15.2.2 Commit Standards

- Conventional Commits: feat:, fix:, docs:, style:, refactor:, perf:, test:, chore:, ci:, build:, revert:
- Commit Message: Type(scope): description [optional body] [optional footer]
- Example: feat(user-service): add multi-factor authentication support

- Pre-commit Hooks: linting, formatting, secret scanning, commit message validation
- Signed Commits: GPG signing for all commits to main and release branches
- Branch Protection: Require PR review, require status checks, require signed commits, no force push

15.2.3 Monorepo Structure

localconnect/

└─ .github/ # GitHub Actions workflows

└─ .husky/ # Git hooks

└─ docs/ # Documentation

└─ infrastructure/ # Terraform, Kubernetes manifests

| └─ terraform/ # AWS infrastructure

| └─ kubernetes/ # K8s manifests, Helm charts

| └─ ansible/ # Server configuration (if needed)

└─ services/ # Microservices

| └─ user-service/ # User management service

| └─ business-service/ # Business profile service

| └─ event-service/ # Event management service

| └─ order-service/ # Order processing service

| └─ payment-service/ # Payment processing service

| └─ review-service/ # Review system service

- | └─ forum-service/ # Community forum service
- | └─ emergency-service/ # Emergency alert service
- | └─ government-service/ # Government service integration
- | └─ ai-service/ # AI/ML inference service
- | └─ notification-service/ # Notification service
- | └─ analytics-service/ # Analytics service
- └─ shared/ # Shared libraries
- | └─ localconnect-common/ # Common utilities, models
- | └─ localconnect-security/ # Security utilities, JWT, encryption
- | └─ localconnect-events/ # Event schema, Kafka client
- | └─ localconnect-testing/ # Test utilities, fixtures
- | └─ localconnect-observability/ # Metrics, tracing, logging
- └─ mobile/ # Mobile applications
- | └─ localconnect-ios/ # iOS app (Swift)
- | └─ localconnect-android/ # Android app (Kotlin)
- | └─ localconnect-flutter/ # Cross-platform (Flutter) - future
- └─ web/ # Web applications
- | └─ localconnect-web/ # Main web app (React/Next.js)
- | └─ localconnect-admin/ # Admin dashboard (React/Next.js)
- | └─ localconnect-landing/ # Landing page (Next.js)

- |— ai/ # AI/ML models and training
 - | |— models/ # Trained models
 - | |— training/ # Training scripts, notebooks
 - | |— inference/ # Inference pipelines
- |— database/ # Database migrations, schemas
 - | |— migrations/ # Flyway/Liquibase migrations
 - | |— seeds/ # Seed data
 - | |— scripts/ # Database scripts
- |— tests/ # End-to-end tests, integration tests
 - | |— e2e/ # Playwright/Cypress E2E tests
 - | |— integration/ # Integration test suites
 - | |— performance/ # Load testing scripts (k6, JMeter)
- |— scripts/ # Automation scripts
 - | |— deploy/ # Deployment scripts
 - | |— backup/ # Backup scripts
 - | |— maintenance/ # Maintenance scripts
- |— docker/ # Docker configurations
- |— base-images/ # Base Docker images
- |— compose/ # Docker Compose for local development

15.3 CI/CD Pipeline Architecture

15.3.1 GitHub Actions Workflow

name: CI/CD Pipeline

on:

push:

branches: [main, develop, release/*, hotfix/*]

pull_request:

branches: [main, develop]

env:

AWS_REGION: ap-southeast-1

ECR_REGISTRY: \${{ secrets.AWS_ACCOUNT_ID }}.dkr.ecr.ap-southeast-1.amazonaws.com

HELM_VERSION: '3.13.0'

KUBECTL_VERSION: '1.29.0'

jobs:

detect-changes:

runs-on: ubuntu-latest

outputs:

services: \${{ steps.changes.outputs.services }}

mobile: \${{ steps.changes.outputs.mobile }}

web: \${{ steps.changes.outputs.web }}

infrastructure: \${{ steps.changes.outputs.infrastructure }}

steps:

- uses: actions/checkout@v4
- uses: dorny/paths-filter@v2

id: changes

with:

filters: |

services:

- 'services/**'

mobile:

- 'mobile/**'

web:

- 'web/**'

infrastructure:

- 'infrastructure/**'

code-quality:

runs-on: ubuntu-latest

needs: detect-changes

if: \${{ needs.detect-changes.outputs.services != '[]' }}

steps:

- uses: actions/checkout@v4

with:

fetch-depth: 0

- name: Setup Java

uses: actions/setup-java@v4

with:

java-version: '17'

distribution: 'temurin'

- name: Setup Node.js

uses: actions/setup-node@v4

with:

node-version: '20'

- name: Cache Dependencies

uses: actions/cache@v3

with:

path: |

~/.m2

~/.npm

~/.gradle

key:

```
runner.os-deps-{{ runner.os }}-deps-runner.os-deps-  
{{ hashFiles('**/pom.xml', '**/package-lock.json') }}
```

- name: Lint Java

```
run: ./mvnw spotless:check
```

- name: Lint JavaScript

```
run: |
```

```
cd web/localconnect-web
```

```
npm run lint
```

- name: SonarQube Scan

```
uses: SonarSource/sonarqube-scan-action@master
```

```
env:
```

```
SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
```

```
SONAR_HOST_URL: ${{ secrets.SONAR_HOST_URL }}
```

```
unit-tests:
```

```
runs-on: ubuntu-latest
```

```
needs: detect-changes
```

```
if: ${{ needs.detect-changes.outputs.services != '[]' }}
```

```
steps:
```

- uses: actions/checkout@v4

- name: Setup Java

uses: actions/setup-java@v4

with:

java-version: '17'

distribution: 'temurin'

- name: Cache Maven Dependencies

uses: actions/cache@v3

with:

path: ~/.m2

key:

runner.os-m2-{{ runner.os }}-m2-runner.os-m2-
{{ hashFiles('**/pom.xml') }}

- name: Run Unit Tests

run: ./mvnw test -pl services/user-service,services/business-service -am

- name: Upload Coverage

uses: codecov/codecov-action@v3

with:

files: ./services/*/target/site/jacoco/jacoco.xml

integration-tests:

runs-on: ubuntu-latest

needs: [code-quality, unit-tests]

```
if: ${ needs.detect-changes.outputs.services != '[]' }
```

services:

postgres:

image: postgres:15

env:

POSTGRES_PASSWORD: postgres

options: >-

--health-cmd pg_isready

--health-interval 10s

--health-timeout 5s

--health-retries 5

ports:

- 5432:5432

redis:

image: redis:7-alpine

options: >-

--health-cmd "redis-cli ping"

--health-interval 10s

--health-timeout 5s

--health-retries 5

ports:

- 6379:6379

kafka:

image: confluentinc/cp-kafka:7.5.0

env:

KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181

KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092

ports:

- 9092:9092

zookeeper:

image: confluentinc/cp-zookeeper:7.5.0

env:

ZOOKEEPER_CLIENT_PORT: 2181

ports:

- 2181:2181

steps:

- uses: actions/checkout@v4

- name: Setup Java

uses: actions/setup-java@v4

with:

java-version: '17'

distribution: 'temurin'

- name: Run Integration Tests

run: ./mvnw verify -pl services/user-service -P integration-test

env:

SPRING_DATASOURCE_URL:

jdbc:postgresql://localhost:5432/localconnect_test

SPRING_REDIS_HOST: localhost

SPRING_KAFKA_BOOTSTRAP_SERVERS: localhost:9092

security-scan:

runs-on: ubuntu-latest

needs: detect-changes

if: \${{ needs.detect-changes.outputs.services != '[]' || needs.detect-changes.outputs.infrastructure != '[]' }}

steps:

- uses: actions/checkout@v4

- name: SAST Scan (SonarQube)

uses: SonarSource/sonarqube-scan-action@master

env:

SONAR_TOKEN: \${{ secrets.SONAR_TOKEN }}

- name: Dependency Scan (Snyk)

uses: snyk/actions/maven@master

env:

SNYK_TOKEN: \${{ secrets.SNYK_TOKEN }}

with:

args: --severity-threshold=high

- name: Secret Scan (GitLeaks)

uses: zricethezav/gitleaks-action@master

env:

GITHUB_TOKEN: \${{ secrets.GITHUB_TOKEN }}

- name: Container Scan (Trivy)

uses: aquasecurity/trivy-action@master

with:

image-ref: 'localconnect/user-service:latest'

format: 'sarif'

output: 'trivy-results.sarif'

- name: Upload Security Results

uses: github/codeql-action/upload-sarif@v2

with:

sarif_file: 'trivy-results.sarif'

build-and-push:

runs-on: ubuntu-latest

needs: [integration-tests, security-scan]

if: \${{ needs.detect-changes.outputs.services != '[]' }}

strategy:

matrix:

service: \${{ fromJson(needs.detect-changes.outputs.services) }}

steps:

- uses: actions/checkout@v4

- name: Configure AWS Credentials

uses: aws-actions/configure-aws-credentials@v4

with:

aws-access-key-id: \${{ secrets.AWS_ACCESS_KEY_ID }}

aws-secret-access-key: \${{ secrets.AWS_SECRET_ACCESS_KEY }}

aws-region: \${{ env.AWS_REGION }}

- name: Login to Amazon ECR

id: login-ecr

uses: aws-actions/amazon-ecr-login@v2

- name: Build Docker Image

run: |

cd services/\${{ matrix.service }}

docker build -t

```
env.ECRREGISTRY/localconnect/{{ env.ECR_REGISTRY }}/localconnect/env.ECRR  
EGISTRY/localconnect/  
{{ matrix.service }}:  
github.sha.{{ github.sha }} .github.sha.
```

docker tag

```
env.ECRREGISTRY/localconnect/{{ env.ECR_REGISTRY }}/localconnect/env.ECRR  
EGISTRY/localconnect/  
{{ matrix.service }}:  
{{ github.sha }} \  
env.ECRREGISTRY/localconnect/{{ env.ECR_REGISTRY }}/localconnect/env.ECRR  
EGISTRY/localconnect/  
{{ matrix.service }}:latest
```

- name: Push Docker Image

run: |

docker push

```
env.ECRREGISTRY/localconnect/{{ env.ECR_REGISTRY }}/localconnect/env.ECRR  
EGISTRY/localconnect/  
{{ matrix.service }}:  
github.sha{{ github.sha }}github.sha
```

docker push

```
env.ECRREGISTRY/localconnect/{{ env.ECR_REGISTRY }}/localconnect/env.ECRR  
EGISTRY/localconnect/  
{{ matrix.service }}:latest
```

- name: Sign Image (Cosign)

uses: sigstore/cosign-installer@v3

- run: |

cosign sign --yes

env.ECR_REGISTRY/localconnect/{{ env.ECR_REGISTRY }}/localconnect/env.ECR_REGISTRY/localconnect/

{{ matrix.service }}:

github.sha{{ github.sha }}github.sha

env:

COSIGN_EXPERIMENTAL: 1

deploy-staging:

runs-on: ubuntu-latest

needs: build-and-push

if: github.ref == 'refs/heads/develop'

environment:

name: staging

url: https://staging.localconnect.com

steps:

- uses: actions/checkout@v4

- name: Configure AWS Credentials

uses: aws-actions/configure-aws-credentials@v4

with:

aws-access-key-id: \${{ secrets.AWS_ACCESS_KEY_ID }}

aws-secret-access-key: \${{ secrets.AWS_SECRET_ACCESS_KEY }}

aws-region: \${{ env.AWS_REGION }}

- name: Setup Helm

uses: azure/setup-helm@v3

with:

version: \${{ env.HELM_VERSION }}

- name: Setup kubectl

uses: azure/setup-kubectl@v3

with:

version: \${{ env.KUBECTL_VERSION }}

- name: Update kubeconfig

run: |

```
aws eks update-kubeconfig --name localconnect-staging --region ${{ env.AWS_REGION }}
```

- name: Deploy to Staging

run: |

```
helm upgrade --install localconnect  
./infrastructure/kubernetes/helm/localconnect \
```

```
--namespace localconnect \
```

```
--set global.environment=staging \
```

```
--set global.image.tag=${{ github.sha }} \
```

--values ./infrastructure/kubernetes/helm/localconnect/values-staging.yaml \

--wait \

--timeout 10m

- name: Run Smoke Tests

run: |

cd tests/e2e

npm install

npx playwright test --grep "smoke"

env:

BASE_URL: https://staging.localconnect.com

deploy-production:

runs-on: ubuntu-latest

needs: build-and-push

if: github.ref == 'refs/heads/main'

environment:

name: production

url: https://localconnect.com

steps:

- uses: actions/checkout@v4

- name: Configure AWS Credentials

uses: aws-actions/configure-aws-credentials@v4

with:

aws-access-key-id: \${{ secrets.AWS_ACCESS_KEY_ID }}

aws-secret-access-key: \${{ secrets.AWS_SECRET_ACCESS_KEY }}

aws-region: \${{ env.AWS_REGION }}

- name: Setup Helm

uses: azure/setup-helm@v3

with:

version: \${{ env.HELM_VERSION }}

- name: Setup kubectl

uses: azure/setup-kubectl@v3

with:

version: \${{ env.KUBECTL_VERSION }}

- name: Update kubeconfig

run: |

aws eks update-kubeconfig --name localconnect-prod --region \${{ env.AWS_REGION }}

- name: Deploy to Production (Canary)

run: |

helm upgrade --install localconnect

./infrastructure/kubernetes/helm/localconnect \

--namespace localconnect \

--set global.environment=production \

--set global.image.tag=\${{ github.sha }} \

--values ./infrastructure/kubernetes/helm/localconnect/values-
production.yaml \

--wait \

--timeout 10m

- name: Verify Deployment

run: |

kubectl rollout status deployment/user-service -n localconnect --timeout=5m

kubectl rollout status deployment/business-service -n localconnect --
timeout=5m

- name: Run Smoke Tests

run: |

cd tests/e2e

npm install

npx playwright test --grep "smoke"

env:

BASE_URL: https://localconnect.com

- name: Notify Slack

uses: slackapi/slack-github-action@v1

with:

payload: |

{

"text": "Production deployment successful: \${{ github.sha }}",

"channel": "#deployments"

}

env:

SLACK_WEBHOOK_URL: \${{ secrets.SLACK_WEBHOOK_URL }}

deploy-mobile:

runs-on: macos-latest

needs: detect-changes

if: \${{ needs.detect-changes.outputs.mobile != '[]' }}

steps:

- uses: actions/checkout@v4

- name: Setup Flutter

uses: subosito/flutter-action@v2

with:

flutter-version: '3.16.0'

- name: Build iOS

run: |

cd mobile/localconnect-flutter

flutter build ios --release --no-codesign

- name: Build Android

run: |

cd mobile/localconnect-flutter

flutter build appbundle --release

- name: Upload to App Store Connect

uses: apple-actions/upload-testflight-build@v1

with:

app-path: mobile/localconnect-flutter/build/ios/ipa/*.ipa

issuer-id: \${{ secrets.APP_STORE_ISSUER_ID }}

api-key-id: \${{ secrets.APP_STORE_API_KEY_ID }}

api-private-key: \${{ secrets.APP_STORE_API_PRIVATE_KEY }}

- name: Upload to Google Play

uses: r0adkll/upload-google-play@v1

with:

serviceAccountJsonPlainText: \${{ secrets.GOOGLE_PLAY_SERVICE_ACCOUNT }}

packageName: com.localconnect.app

releaseFiles: mobile/localconnect-

flutter/build/app/outputs/bundle/release/*.aab

track: internal

deploy-web:

runs-on: ubuntu-latest

needs: detect-changes

if: \${{ needs.detect-changes.outputs.web != '[]' }}

steps:

- uses: actions/checkout@v4

- name: Setup Node.js

uses: actions/setup-node@v4

with:

node-version: '20'

- name: Build Web App

run: |

cd web/localconnect-web

npm install

npm run build

- name: Deploy to S3 + CloudFront

run: |

aws s3 sync web/localconnect-web/dist s3://localconnect-web-assets-\${{
github.ref_name == 'main' && 'prod' || 'staging' }} \

--delete

aws cloudfront create-invalidation \

--distribution-id \${ secrets.CLOUDFRONT_DISTRIBUTION_ID } \

--paths "/*"

15.3.2 ArgoCD (GitOps)

- Installation: ArgoCD deployed in Kubernetes, 2 replicas, SSO integration
- Repositories: Git repository as single source of truth for Kubernetes manifests
- Applications: One ArgoCD application per service, auto-sync enabled
- Sync Policy: Automated sync, automated prune, self-healing enabled
- Resource Health: Custom resource health checks for StatefulSets, CronJobs
- Notifications: Slack notifications for sync failures, resource health degradation
- RBAC: Role-based access to ArgoCD, SSO integration with GitHub
- Secrets: Sealed Secrets for GitOps, External Secrets Operator for Vault integration
- Rollback: One-click rollback to previous version, automated rollback on health check failure
- Multi-Cluster: ArgoCD manages staging, production, and DR clusters from single instance

15.3.3 Helm Charts

- Chart Structure:

infrastructure/kubernetes/helm/localconnect/

├─ Chart.yaml # Chart metadata

├─ values.yaml # Default values

- └─ values-staging.yaml # Staging overrides
- └─ values-production.yaml # Production overrides
- └─ values-dr.yaml # DR overrides
- └─ templates/
 - | └─ _helpers.tpl # Helper templates
 - | └─ deployment.yaml # Deployment template
 - | └─ service.yaml # Service template
 - | └─ hpa.yaml # HPA template
 - | └─ pdb.yaml # PDB template
 - | └─ serviceaccount.yaml # ServiceAccount template
 - | └─ networkpolicy.yaml # NetworkPolicy template
 - | └─ configmap.yaml # ConfigMap template
 - | └─ secret.yaml # Secret template (SealedSecrets)
 - | └─ ingress.yaml # Ingress template
 - | └─ istio/ # Istio resources
 - | | └─ gateway.yaml
 - | | └─ virtualservice.yaml
 - | | └─ destinationrule.yaml
 - | | └─ peerauthentication.yaml
 - | └─ monitoring/ # Monitoring resources

| └─ servicemonitor.yaml

| └─ podmonitor.yaml

| └─ alerts.yaml

└─ charts/ # Subcharts (if any)

15.4 Environment Management

15.4.1 Environment Strategy

Environment	Purpose	Data	Access	Deployment
Local	Developer local development	Synthetic data, local databases	Developers	Manual (Docker Compose)
Dev	Shared development environment	Synthetic data, refreshed daily	Engineering team	Auto (feature branch push)
Staging	Pre-production testing	Anonymized production data, 1 week old	QA, Product, Engineering	Auto (develop branch merge)
Production	Live production	Real production data	Users, Admins, Support	Manual (main branch merge + approval)
DR	Disaster recovery	Replicated production data	Ops team (failover only)	Manual (failover procedure)

15.4.2 Environment Configuration

- ConfigMap per Environment: Environment-specific settings (URLs, timeouts, feature flags)
- Secret per Environment: Environment-specific secrets (DB credentials, API keys, certificates)
- Feature Flags: LaunchDarkly for feature flag management, environment-specific flag states
- Database: Separate databases per environment, staging data anonymized from production
- Services: Separate service instances per environment, no shared resources between environments
- Monitoring: Separate monitoring stacks per environment, shared alerting channels

15.4.3 Environment Promotion

- Local → Dev: Developer pushes feature branch, auto-deployed to dev environment
- Dev → Staging: PR merged to develop branch, auto-deployed to staging, QA validation
- Staging → Production: Release branch created, final testing, PR to main, manual approval, deploy to production
- Production → DR: Continuous replication of data, infrastructure as code for DR deployment
- Rollback: Automated rollback on health check failure, manual rollback via ArgoCD or Helm

15.5 Infrastructure as Code (IaC)

15.5.1 Terraform Architecture

infrastructure/terraform/

└─ modules/ # Reusable Terraform modules

- | └─ vpc/ # VPC module
- | └─ eks/ # EKS cluster module
- | └─ rds/ # RDS module
- | └─ elasticache/ # ElastiCache module
- | └─ s3/ # S3 module
- | └─ iam/ # IAM module
- | └─ security-groups/ # Security Groups module
- | └─ cloudfront/ # CloudFront module
- | └─ route53/ # Route 53 module
- | └─ waf/ # WAF module
- | └─ ssm/ # Parameter Store module
- | └─ secrets-manager/ # Secrets Manager module
- | └─ cloudwatch/ # CloudWatch module
- └─ environments/
 - | └─ dev/ # Dev environment
 - | | └─ main.tf
 - | | └─ variables.tf
 - | | └─ outputs.tf
 - | | └─ terraform.tfvars
 - | └─ staging/ # Staging environment

| └─ production/ # Production environment

| └─ dr/ # DR environment

└─ global/ # Global resources (DNS, certificates)

| └─ main.tf

| └─ variables.tf

| └─ outputs.tf

└─ backend.tf # Terraform backend configuration (S3 + DynamoDB)

└─ providers.tf # Provider configuration

└─ versions.tf # Terraform and provider versions

└─ README.md

15.5.2 Terraform Workflow

- State Management: S3 backend with DynamoDB locking, state versioning enabled
- State Encryption: State encrypted at rest with KMS, in transit with TLS
- Remote State: Modules reference remote state for cross-module dependencies
- Workspaces: Terraform workspaces for environment separation (dev, staging, production, dr)
- Plan: terraform plan on every PR, plan output posted as PR comment
- Apply: terraform apply only on merge to main (production) or develop (staging), manual approval for production
- Drift Detection: Daily terraform plan in CI/CD, alerts for drift detection
- Cost Estimation: Infracost integration, cost estimate posted as PR comment

- Policy as Code: Open Policy Agent (OPA) for Terraform policy enforcement (e.g., no public S3, required tags, encryption)

15.5.3 Terraform CI/CD

name: Terraform CI/CD

on:

push:

branches: [main]

paths: ['infrastructure/terraform/**']

pull_request:

paths: ['infrastructure/terraform/**']

jobs:

terraform-plan:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Setup Terraform

uses: hashicorp/setup-terraform@v3

with:

terraform_version: '1.7.0'

- name: Terraform Init

run: terraform init

working-directory: infrastructure/terraform/environments/production

- name: Terraform Plan

run: terraform plan -out=tfplan

working-directory: infrastructure/terraform/environments/production

- name: Upload Plan

uses: actions/upload-artifact@v3

with:

name: tfplan

path: infrastructure/terraform/environments/production/tfplan

terraform-apply:

runs-on: ubuntu-latest

needs: terraform-plan

if: github.ref == 'refs/heads/main'

environment:

name: production-infrastructure

steps:

- uses: actions/checkout@v4

- name: Setup Terraform

uses: hashicorp/setup-terraform@v3

- name: Download Plan

uses: actions/download-artifact@v3

with:

name: tfplan

- name: Terraform Apply

run: terraform apply tfplan

working-directory: infrastructure/terraform/environments/production

15.6 Testing Strategy

15.6.1 Testing Pyramid

Level	Type	Tools	Coverage Target	Frequency
Unit	Individual component testing	JUnit, Mockito, Jest	80%	Every commit
Integration	Service interaction testing	Testcontainers, Spring Boot Test	70%	Every PR
Contract	API contract testing	Pact, Spring Cloud Contract	100% API coverage	Every PR
E2E	End-to-end user flow testing	Playwright, Cypress	50% critical paths	Every release
Performance	Load, stress, soak testing	k6, JMeter, Gatling	100% critical paths	Weekly

Security	Vulnerability, penetration testing	OWASP ZAP, Burp Suite	100%	Quarterly
Chaos	Resilience testing	Chaos Mesh, Gremlin	100% critical services	Monthly

15.6.2 Test Environments

- Local: Docker Compose for local integration testing, Testcontainers for unit tests
- Dev: Shared environment for feature testing, ephemeral namespaces for PR previews
- Staging: Production-like environment for E2E and performance testing, load testing before release
- Production: Canary deployments for production testing, synthetic monitoring for health checks
- Chaos: Dedicated environment for chaos engineering, isolated from production

15.6.3 Test Automation

- Unit Tests: Every service has comprehensive unit tests, run in CI on every commit
- Integration Tests: Testcontainers for database integration, WireMock for external API mocking, Kafka test containers for event testing
- Contract Tests: Pact for consumer-driven contract testing, pact broker for contract verification
- E2E Tests: Playwright for web, Appium for mobile, 50 critical user flows covered
- Performance Tests: k6 for load testing, JMeter for complex scenarios, weekly automated performance regression tests

- Security Tests: OWASP ZAP for DAST, Snyk for dependency scanning, SonarQube for SAST
- Chaos Tests: Chaos Mesh for Kubernetes chaos engineering, monthly chaos engineering days

15.7 Release Management

15.7.1 Release Process

- Release Branch: Created from develop, version bumped, release notes drafted
- Release Candidate: Staging deployment of release branch, QA validation, UAT sign-off
- Release Approval: Product owner and engineering lead approval required for production release
- Production Deployment: Blue-green or canary deployment, automated rollback on health check failure
- Post-Release: 24-hour monitoring period, incident response on standby, rollback if critical issues
- Release Notes: Automated release notes from conventional commits, published in-app and on website
- Communication: Slack announcement, email to users for major features, in-app changelog

15.7.2 Versioning Strategy

- Semantic Versioning: MAJOR.MINOR.PATCH (e.g., 2.3.1)
- Major: Breaking changes, major architectural changes, major feature releases
- Minor: New features, backward-compatible enhancements, API additions
- Patch: Bug fixes, security patches, performance improvements
- Pre-release: -alpha, -beta, -rc suffixes for pre-release versions

- Build Metadata: +build.{buildNumber} for CI build metadata
- Docker Tags: {version}, {major}.{minor}, {major}, latest (stable only), {sha}

15.7.3 Release Calendar

- Patch Releases: Weekly (every Wednesday), automated if no critical issues
- Minor Releases: Bi-weekly (every 2 weeks), feature-complete sprints
- Major Releases: Quarterly (every 3 months), major feature releases, architectural changes
- Hotfix Releases: As needed, critical bug fixes or security patches, bypass standard release cycle
- Maintenance Windows: Monthly (first Sunday 2 AM - 6 AM SLT), infrastructure updates, database maintenance
- Feature Freezes: 1 week before release, no new features, only bug fixes and stabilization

15.7.4 Rollback Strategy

- Automated Rollback: Health check failure triggers automatic rollback to previous version
- Manual Rollback: ArgoCD one-click rollback, Helm rollback command, database rollback scripts
- Database Rollback: Flyway/Liquibase rollback scripts for schema changes, data migration rollback procedures
- Feature Flag Rollback: Disable feature flag for quick rollback without deployment
- Communication: Immediate communication to users for rollback, status page update, incident post-mortem
- Post-Rollback: Root cause analysis, fix development, re-release planning

15.8 Artifact Management

15.8.1 Amazon ECR (Elastic Container Registry)

- Repositories: Per-service repositories, shared base images
- Image Tagging: Semantic version, Git SHA, branch name, latest (stable only)
- Image Scanning: Trivy scan on every build, vulnerability report in CI
- Image Signing: Cosign for image signing and verification, signature verification in deployment
- Lifecycle Policies: Delete untagged images after 7 days, keep last 30 tagged images
- Replication: Cross-region replication to DR region
- Access Control: IAM policies for push/pull, no anonymous access
- Cache: ECR pull-through cache for public images (Docker Hub, Quay)

15.8.2 Artifact Repository (S3 + CodeArtifact)

- Maven Artifacts: AWS CodeArtifact for Java artifacts, shared libraries, snapshots
- NPM Packages: AWS CodeArtifact for Node.js packages, shared libraries
- Python Packages: AWS CodeArtifact for Python packages, shared libraries
- Helm Charts: S3-backed Helm repository, chart versioning, chart testing
- Mobile Artifacts: S3 for IPA, APK, AAB files, presigned URLs for distribution
- Build Artifacts: S3 for build outputs, test reports, coverage reports, scan results
- Retention: 30 days for build artifacts, 7 years for release artifacts, automatic cleanup

15.9 Documentation & Knowledge Management

15.9.1 Documentation Standards

- Architecture Decision Records (ADR): All architectural decisions documented in ADR format, stored in docs/adr/
- Runbooks: Operational runbooks for all services, stored in docs/runbooks/, updated with every change
- API Documentation: OpenAPI/Swagger for all APIs, auto-generated from code, published to developer portal
- Developer Onboarding: Comprehensive onboarding guide, environment setup, first contribution guide
- Operational Documentation: Deployment procedures, rollback procedures, incident response procedures
- User Documentation: In-app help, FAQ, knowledge base, video tutorials
- Security Documentation: Security policies, compliance documentation, audit evidence

15.9.2 Documentation Tools

- Confluence: Internal knowledge base, team documentation, meeting notes, project planning
- GitHub Wiki: Technical documentation, architecture diagrams, API docs, linked to code
- Swagger UI: Interactive API documentation, auto-generated from OpenAPI specs
- Storybook: UI component documentation, interactive component library for web
- MkDocs: Static documentation site for developer guide, API reference, architecture docs
- ReadTheDocs: Open-source documentation for public API and SDK documentation

Chapter 16: Observability Architecture

16.1 Observability Principles

LocalConnect's observability strategy follows the three pillars: metrics, logs, and traces. Every component of the system is observable, with real-time dashboards, intelligent alerting, and proactive anomaly detection.

16.2 Metrics Architecture

16.2.1 Prometheus + Thanos

- Prometheus: 2 replicas per cluster, 15-day local retention, 15-second scrape interval
- Thanos: Global query layer, object storage (S3) for long-term retention (1 year), compact, store, query
- Service Discovery: Kubernetes service discovery, automatic scraping of annotated pods
- Recording Rules: Pre-aggregated metrics for dashboards (e.g., request rate, error rate, latency percentiles)
- Alerting Rules: Prometheus Alertmanager for alert routing, grouping, inhibition, silencing
- Remote Write: Remote write to Thanos for long-term storage, dual-write for reliability
- Exporters: Node Exporter, Redis Exporter, PostgreSQL Exporter, Kafka Exporter, custom application exporters
- Custom Metrics: Business metrics (order rate, user registration, event attendance, revenue)

16.2.2 Grafana Dashboards

- Deployment: 2 replicas, SSO authentication (GitHub OAuth), role-based access
- Dashboard Organization:
 - Infrastructure: Node, pod, network, storage metrics
 - Application: Service-level metrics (RPS, latency, error rate, saturation)

- Business: Order rate, revenue, user growth, conversion rate, NPS
- Security: Authentication attempts, failed logins, WAF events, vulnerability scans
- Database: Query performance, connection pool, replication lag, index usage
- Cache: Hit rate, memory usage, eviction rate, key distribution
- Queue: Queue depth, processing rate, consumer lag, dead letter queue size
- CDN: Cache hit rate, origin latency, bandwidth, error rate
- Dashboard as Code: Grafana dashboards stored in Git (JSON), versioned, deployed via Grafana API
- Annotations: Deployment annotations, incident annotations, release annotations on all dashboards
- Alerting: Grafana alerts for threshold-based alerts, integrated with PagerDuty and Slack

16.2.3 CloudWatch Metrics

- AWS Services: Native CloudWatch metrics for all AWS services (RDS, ElastiCache, ALB, SQS, SNS, Lambda, EKS)
- Custom Metrics: Application metrics pushed via CloudWatch agent or SDK
- Dashboards: CloudWatch dashboards for AWS-specific metrics, cross-service dashboards
- Alarms: CloudWatch alarms for AWS resource thresholds, integrated with SNS → PagerDuty
- Insights: CloudWatch Insights for log analysis, CloudWatch Contributor Insights for top contributors
- Synthetics: CloudWatch Synthetics canaries for endpoint monitoring, API testing

16.2.4 Key Metrics (SLIs/SLOs)

Service	SLI	SLO	Measurement
API Gateway	Availability	99.9%	Uptime percentage (excluding maintenance)
API Gateway	Latency (p99)	< 500ms	99th percentile response time
API Gateway	Error Rate	< 0.1%	Percentage of 5xx errors
User Service	Availability	99.95%	Uptime percentage
User Service	Latency (p99)	< 200ms	99th percentile response time
User Service	Error Rate	< 0.05%	Percentage of 5xx errors
Order Service	Availability	99.99%	Uptime percentage (critical for revenue)
Order Service	Latency (p99)	< 300ms	99th percentile response time
Order Service	Error Rate	< 0.01%	Percentage of 5xx errors
Payment Service	Availability	99.99%	Uptime percentage (critical for revenue)
Payment Service	Latency (p99)	< 500ms	99th percentile response time
Payment Service	Error Rate	< 0.01%	Percentage of 5xx errors
Emergency Service	Availability	99.999%	Uptime percentage (life-critical)
Emergency Service	Latency (p99)	< 100ms	99th percentile response time
Emergency Service	Error Rate	< 0.001%	Percentage of 5xx errors
Web App	Page Load Time	< 2s	90th percentile page load time
Web App	Time to Interactive	< 3s	90th percentile TTI
Mobile App	Cold Start	< 2s	90th percentile cold start time
Mobile App	API Response	< 500ms	90th percentile API response time

Search	Query Latency	< 100ms	99th percentile search query time
Search	Index Freshness	< 5 min	Time from data change to index update

16.3 Logging Architecture

16.3.1 Fluent Bit + Loki

- Fluent Bit: DaemonSet on all nodes, tail container logs, parse JSON, enrich with Kubernetes metadata
- Loki: 3 replicas, 30-day retention, S3 backend for long-term storage, label-based indexing
- Log Format: Structured JSON logging for all services (correlation ID, timestamp, service, level, message, context)
- Log Levels: DEBUG (development only), INFO (standard), WARN (anomalies), ERROR (failures), FATAL (critical)
- Log Aggregation: Per-service, per-namespace, per-pod, per-container log streams
- Log Querying: LogQL for log search and analysis, Grafana integration for visualization
- Log Alerts: Log-based alerts for error spikes, security events, business anomalies
- Log Retention: 30 days in Loki (hot), 7 years in S3 (cold), compressed and encrypted
- Log Shipping: Dual shipping to Loki (real-time) and S3 (archive), S3 for compliance and forensics

16.3.2 CloudWatch Logs

- AWS Services: Native CloudWatch Logs for AWS services (Lambda, ECS, EKS, RDS, WAF, CloudTrail)
- Log Groups: Per-service log groups, per-environment log groups, encrypted with KMS

- Log Insights: CloudWatch Logs Insights for ad-hoc queries, pattern analysis, anomaly detection
- Subscription: Log subscriptions for real-time processing (Lambda, Kinesis, Elasticsearch)
- Export: S3 export for long-term storage, automated export for compliance

16.3.3 Log Correlation & Context

- Correlation ID: Every request has a unique correlation ID (UUID), propagated across all services
- Trace ID: OpenTelemetry trace ID linked to logs for request tracing
- User ID: Authenticated user ID included in logs for user-specific debugging
- Session ID: Session identifier for tracking user journeys
- Request Context: HTTP method, URL, query parameters, user agent, client IP, response status, response time
- Business Context: Order ID, business ID, event ID, payment ID for business-specific debugging

16.3.4 Log Security

- PII Masking: Automatic PII detection and masking in logs (credit card numbers, phone numbers, emails, IDs)
- Sensitive Data: No passwords, tokens, or secrets in logs, redaction of sensitive query parameters
- Access Control: Role-based access to logs, audit trail for log access
- Encryption: Encryption at rest (KMS), encryption in transit (TLS), log integrity verification
- Retention: Compliance-based retention (7 years for financial, 3 years for general, 1 year for debug)

16.4 Distributed Tracing Architecture

16.4.1 OpenTelemetry + Jaeger + Tempo

- OpenTelemetry SDK: Instrumented in all services (Java, Node.js, Python, Go, mobile, web)
- Jaeger: 3 replicas, Elasticsearch backend for trace storage, 7-day retention, Jaeger UI for visualization
- Tempo: Grafana Tempo for cost-effective trace storage (S3 backend), 30-day retention, trace-to-metrics
- Trace Context: W3C Trace Context propagation across all services (HTTP headers, gRPC metadata, message headers)
- Trace Sampling:
 - Head-based: 1% for standard traffic, 100% for error traces, 100% for critical paths (emergency, payments)
 - Tail-based: 100% sampling for traces with errors or latency > 1 second
 - Adaptive: Dynamic sampling based on traffic volume and error rate
- Span Attributes: Service name, operation, duration, status, error details, custom business attributes
- Trace Correlation: Trace IDs linked to logs and metrics for full observability correlation
- Trace Analysis: Jaeger UI for trace visualization, Grafana Tempo for trace search, trace-to-metrics correlation

16.4.2 Istio Tracing

- Automatic Tracing: Istio sidecar automatically captures and propagates trace context
- Envoy Tracing: Envoy proxy generates spans for all HTTP/gRPC traffic
- Integration: Jaeger collector for trace ingestion, Istio config for trace sampling
- Custom Spans: Application-level custom spans for business operations (order processing, payment authorization)

- Service Graph: Jaeger service graph for dependency visualization, Kiali for service mesh topology

16.4.3 Mobile & Web Tracing

- RUM (Real User Monitoring): OpenTelemetry RUM for web and mobile, capturing user interactions, page loads, API calls
- Session Replay: Session replay for critical user journeys (with privacy controls, PII masking)
- Performance Metrics: Core Web Vitals (LCP, FID, CLS), mobile app startup time, API response time
- User Journey: End-to-end user journey tracing from web/mobile to backend services
- Error Tracking: Sentry for error tracking, stack traces, source maps, release correlation

16.5 Alerting Architecture

16.5.1 Alerting Stack

- Prometheus Alertmanager: Alert routing, grouping, inhibition, silencing, deduplication
- PagerDuty: Incident management, on-call scheduling, escalation policies, incident response
- Slack: Real-time alerts for development team, channel-based routing
- Email: Escalation alerts, daily summary, weekly digest
- SMS: Critical alerts only (P1), on-call engineer notification
- Phone Call: P1 critical alerts, auto-escalation after 15 minutes
- In-App: Admin dashboard alerts for operational issues, system health

16.5.2 Alert Routing

Severity	Channel	Response Time	Escalation	Example
P1-Critical	PagerDuty (page), Slack, SMS, Phone	15 minutes	30 min → L2, 1h → L3, 2h → Manager	Service outage, data breach, payment failure
P2-High	PagerDuty (alert), Slack, Email	1 hour	2h → L2, 4h → Manager	High error rate, slow queries, security event
P3-Medium	Slack, Email	4 hours	8h → L2	Elevated latency, warning threshold, minor error
P4-Low	Email, Slack (non-urgent)	24 hours	None	Disk space warning, certificate expiry (30 days), non-critical metric
Info	Slack, Dashboard	N/A	None	Deployment notification, release announcement, metric update

16.5.3 Alert Categories

Category	Examples	Severity
Availability	Service down, health check failure, dependency unavailable	P1-P2
Latency	P99 latency > SLO, response time degradation	P2-P3
Error Rate	5xx rate > threshold, error spike, failed transactions	P1-P2
Saturation	CPU > 80%, memory > 85%, disk > 80%, connection pool exhausted	P2-P3
Security	Failed login spike, WAF block, suspicious traffic, vulnerability detected	P1-P2

Business	Order rate drop, payment failure rate, user registration drop	P2-P3
Data	Database replication lag, cache hit rate drop, queue depth high	P2-P3
Infrastructure	Node failure, pod crash, network partition, certificate expiry	P2-P3
Cost	Cost spike, budget threshold, unexpected resource usage	P3-P4

16.5.4 On-Call Rotation

- Schedule: 24/7 on-call rotation, 1-week shifts, 3 engineers per rotation (primary, secondary, shadow)
- Primary: First responder for all alerts, incident commander for P1/P2
- Secondary: Backup for primary, takes over if primary unavailable, handles P3/P4
- Shadow: Learning role, observes incident response, handles documentation
- Escalation: Automatic escalation to next level after response time threshold
- Handover: Structured handover at shift change, ongoing incidents, known issues, environment status
- Compensation: On-call stipend, time-off in lieu for out-of-hours incidents
- Post-Incident: Incident review within 24 hours, action items tracked, runbook updates

16.6 Synthetic Monitoring

16.6.1 Uptime Monitoring

- Tool: Pingdom, UptimeRobot, CloudWatch Synthetics

- Frequency: Every 60 seconds for critical endpoints, every 5 minutes for non-critical
- Endpoints:
 - <https://localconnect.com/health> (system health)
 - <https://api.localconnect.com/v1/health> (API health)
 - <https://app.localconnect.com> (web app availability)
 - <https://admin.localconnect.com> (admin dashboard)
- Locations: Singapore, Sydney, Mumbai, Tokyo, Frankfurt, US-East, US-West
- Alerting: P1 alert if down from 2+ locations for 2+ minutes
- Status Page: Public status page (status.localconnect.com) with real-time status, incident history, maintenance schedule

16.6.2 API Monitoring

- Tool: Postman Monitors, CloudWatch Synthetics, custom scripts
- Frequency: Every 5 minutes for critical APIs, every 15 minutes for standard APIs
- Coverage:
 - Authentication: Login, token refresh, logout
 - User: Registration, profile update, password reset
 - Business: Search, detail view, review submission
 - Order: Create order, payment, status check
 - Emergency: Alert submission, responder notification
 - Government: Service search, form submission, appointment booking
- Assertions: Response status, response time, response structure, data validation
- Alerting: P2 alert if API fails from 2+ locations for 2+ consecutive checks

16.6.3 User Journey Monitoring

- Tool: Playwright, Cypress, CloudWatch Synthetics

- Frequency: Every 15 minutes for critical journeys, every hour for standard journeys
- Journeys:
 - User registration and login
 - Business search and booking
 - Event registration and ticket purchase
 - Emergency alert submission
 - Government service application
 - Mobile app core flows
- Metrics: Success rate, completion time, step-by-step timing, screenshot on failure
- Alerting: P2 alert if journey fails for 2+ consecutive checks

16.7 APM (Application Performance Monitoring)

16.7.1 DataDog APM

- Deployment: DataDog Agent as DaemonSet on all nodes, auto-instrumentation for supported languages
- Traces: Distributed tracing across all services, automatic service mapping, dependency mapping
- Profiles: Continuous profiling for CPU, memory, I/O, lock contention (Java, Python, Go, Node.js)
- RUM: Real User Monitoring for web and mobile, Core Web Vitals, session replay
- Error Tracking: Automatic error detection, stack trace analysis, error correlation, release tracking
- SLO Tracking: SLO dashboards, error budget tracking, burn rate alerts
- Infrastructure: Container monitoring, Kubernetes monitoring, AWS service monitoring
- Security: Application Security Monitoring (ASM), vulnerability detection, threat detection

- Custom Dashboards: Business metrics, custom metrics, custom events, custom spans
- Integration: PagerDuty, Slack, Jira, GitHub, Jenkins, ArgoCD, Terraform

16.7.2 New Relic APM (Alternative)

- Deployment: New Relic agent for Java, Node.js, Python, Go, mobile, browser
- Features: Similar to DataDog, distributed tracing, profiling, RUM, error tracking, infrastructure monitoring
- Pricing: Usage-based, per GB of ingest, per host for infrastructure
- Decision: Evaluate DataDog vs New Relic after 3 months, choose based on cost, features, and team preference

16.8 Business Intelligence & Analytics

16.8.1 Analytics Dashboards

- Tool: Grafana (for operational metrics), Apache Superset (for business analytics), Tableau (for executive dashboards)
- Metrics:
 - User Growth: Daily/weekly/monthly active users, registration rate, churn rate, retention cohorts
 - Business Growth: New business listings, verified businesses, business engagement, subscription revenue
 - Order Metrics: Order volume, average order value, conversion rate, cart abandonment, payment success rate
 - Event Metrics: Event creation, registration rate, attendance rate, ticket sales, revenue
 - Government Metrics: Service usage, form submissions, appointment bookings, citizen satisfaction
 - Emergency Metrics: Alert volume, response time, resolution rate, false positive rate

- Platform Health: API availability, error rate, latency, throughput, resource utilization
- Real-Time: Real-time dashboards for operations center, 5-minute delay for business metrics, 1-hour for executive reports
- Drill-Down: Hierarchical dashboards, from high-level summary to per-service, per-user, per-business detail
- Alerts: Business metric alerts (e.g., order drop > 20%, registration spike > 500%)

16.8.2 Data Warehouse

- Tool: Amazon Redshift Serverless (for structured data), Athena (for ad-hoc queries on S3)
- ETL: AWS Glue for ETL jobs, Apache Airflow for orchestration, dbt for data transformation
- Data Sources: PostgreSQL, DynamoDB, S3, Kafka, application logs, third-party data
- Schema: Star schema for analytics, data marts per business domain (users, businesses, orders, events, government)
- Refresh: Near real-time (15-minute lag) for operational analytics, daily for historical analytics
- Retention: 7 years for business data, 1 year for detailed logs, 3 months for real-time data
- Access: Role-based access, SQL access for analysts, API access for applications, dashboard access for executives

16.9 Incident Management

16.9.1 Incident Response Workflow

- Detection: Automated alerts, monitoring dashboards, user reports, security tools

- Triage: On-call engineer assesses severity, determines impact, starts incident response
- Response: Incident commander coordinates response, technical team investigates and fixes
- Communication: Status page updates, Slack updates, customer communication (if user-facing)
- Resolution: Fix deployed, verification, incident closure
- Post-Incident: Post-mortem within 24 hours, action items, runbook updates, process improvement
- Tracking: All incidents tracked in PagerDuty, Jira for action items, Confluence for post-mortems

16.9.2 Post-Mortem Template

- Incident Summary: What happened, when, duration, impact
- Timeline: Detailed timeline of detection, response, resolution
- Root Cause: 5 Whys analysis, technical root cause, contributing factors
- Impact Assessment: User impact, business impact, data impact, reputation impact
- Lessons Learned: What went well, what could be improved, what was missed
- Action Items: Specific, measurable, actionable items with owners and deadlines
- Prevention: How to prevent similar incidents in the future, monitoring improvements, process improvements
- Communication: Internal communication summary, external communication summary, customer communication

16.9.3 Incident Metrics

- MTTD (Mean Time to Detect): Target < 5 minutes for P1, < 15 minutes for P2
- MTTR (Mean Time to Respond): Target < 15 minutes for P1, < 1 hour for P2

- MTTC (Mean Time to Contain): Target < 30 minutes for P1, < 2 hours for P2
- MTTF (Mean Time to Fix): Target < 1 hour for P1, < 4 hours for P2
- Incident Rate: Incidents per month, trend analysis, severity distribution
- Post-Mortem Completion: 100% of P1/P2 incidents have post-mortem within 24 hours
- Action Item Completion: 90% of action items completed within SLA (1 week for P1, 2 weeks for P2)

Chapter 17: AI/ML Architecture

17.1 AI/ML Strategy

LocalConnect integrates AI/ML to enhance user experience, improve search relevance, detect fraud, and provide intelligent recommendations. The AI architecture is designed for scalability, privacy, and real-time inference.

17.2 AI Use Cases

17.2.1 Smart Search & Recommendations

- Business Search: Semantic search using vector embeddings, natural language query understanding, typo tolerance, location-aware ranking
- Personalized Recommendations: Collaborative filtering for business recommendations, content-based filtering for event recommendations, hybrid approach for order suggestions
- Trending & Popular: Real-time trending businesses, popular events, hot deals based on user engagement
- Discovery: "Near You" recommendations, "You Might Like" suggestions, "Similar to X" matching
- Search Suggestions: Auto-complete, query suggestions, spelling correction, voice search

17.2.2 Fraud Detection & Risk Scoring

- Payment Fraud: Real-time transaction risk scoring, anomaly detection for unusual spending patterns, device fingerprinting, behavioral biometrics
- Account Takeover: Login anomaly detection, impossible travel detection, brute force detection, credential stuffing detection
- Fake Reviews: NLP-based fake review detection, behavioral pattern analysis, spam detection
- Fake Business: Business verification fraud detection, duplicate listing detection, suspicious activity detection
- Chargeback Prediction: Machine learning model to predict chargeback likelihood, preemptive risk mitigation

17.2.3 Content Moderation

- Review Moderation: Toxicity detection, hate speech detection, profanity filtering, spam detection
- Image Moderation: NSFW image detection, inappropriate content detection, copyright infringement detection
- Forum Post Moderation: Similar to review moderation, with additional context for community guidelines
- Business Content Moderation: Inappropriate business descriptions, misleading claims, illegal activity detection
- User-Generated Content: Real-time content moderation for all user-generated content

17.2.4 NLP & Chatbot

- Customer Support Chatbot: Intent classification, entity extraction, FAQ answering, ticket creation, human handoff
- Government Service Chatbot: Form filling assistance, policy question answering, appointment scheduling

- Business Assistant Chatbot: Order management, inventory queries, customer communication, analytics questions
- Language Support: Sinhala, Tamil, and English NLP, translation services, multilingual chatbot
- Sentiment Analysis: Customer review sentiment, business reputation monitoring, event feedback analysis

17.2.5 Computer Vision

- Image Recognition: Business image classification, product image recognition, menu item identification
- OCR (Optical Character Recognition): Document scanning for KYC, business license verification, receipt scanning
- Image Enhancement: Automatic image cropping, background removal, quality enhancement for business photos
- Visual Search: "Search by Image" for products and businesses, reverse image search for duplicate detection

17.2.6 Predictive Analytics

- Demand Forecasting: Business demand prediction for inventory, staffing, resource planning
- Event Attendance Prediction: Predict event attendance based on historical data, weather, location, marketing
- Churn Prediction: Predict user churn, business churn, subscription cancellation, proactive retention
- Revenue Forecasting: Predict revenue for businesses, platform, government services
- Anomaly Detection: System anomaly detection, business anomaly detection, market anomaly detection

17.3 AI/ML Infrastructure

17.3.1 Model Training Pipeline

- Data Collection: Event-driven data collection from Kafka, database extracts, user behavior logs, feedback loops
- Data Labeling: Automated labeling for supervised learning, human-in-the-loop for complex cases, active learning for efficient labeling
- Feature Engineering: Feature store for consistent feature engineering, real-time feature computation, batch feature computation
- Training: AWS SageMaker for model training, distributed training for large models, hyperparameter tuning with Bayesian optimization
- Validation: Cross-validation, hold-out validation, A/B testing framework for model comparison
- Evaluation: Precision, recall, F1, AUC-ROC, accuracy, business metrics (conversion, revenue, fraud prevented)
- Experiment Tracking: MLflow for experiment tracking, model versioning, artifact management, reproducibility
- Model Registry: MLflow Model Registry for model staging (staging, production, archived), approval workflows
- Deployment: SageMaker endpoints for real-time inference, batch inference for analytics, edge deployment for mobile

17.3.2 Model Serving Architecture

- SageMaker Endpoints: Real-time inference for low-latency requirements (search, fraud detection, recommendations)
- SageMaker Batch Transform: Batch inference for large datasets (churn prediction, demand forecasting, analytics)
- EKS GPU Nodes: Custom model serving on Kubernetes with GPU support (Triton Inference Server, TorchServe, TensorFlow Serving)
- Lambda + EFS: Lightweight models on Lambda for cost-effective inference (simple NLP, image classification)
- Edge Deployment: CoreML for iOS, TensorFlow Lite for Android, on-device inference for privacy-sensitive features

- Model Cache: Redis for model prediction caching, warm start for frequently requested predictions
- A/B Testing: Model A/B testing framework, traffic splitting for model comparison, automated winner selection
- Shadow Mode: Shadow deployment for new models, comparing predictions without affecting users

17.3.3 Feature Store

- Tool: Feast (Feature Store) or AWS SageMaker Feature Store
- Online Store: Redis for low-latency feature retrieval (real-time inference, < 10ms)
- Offline Store: S3 + Parquet for batch feature computation, training data preparation, historical analysis
- Feature Engineering: Automated feature pipelines (Apache Airflow), feature versioning, feature sharing across teams
- Feature Monitoring: Feature drift detection, feature quality metrics, feature usage tracking
- Feature Registry: Centralized feature catalog, documentation, lineage, approval workflow

17.3.4 MLOps Pipeline

- CI/CD for ML: Automated training pipeline on code changes, automated model evaluation, automated deployment
- Data Versioning: DVC (Data Version Control) for dataset versioning, reproducible experiments
- Model Versioning: MLflow for model versioning, artifact storage, model lineage
- Model Monitoring: Real-time model performance monitoring, drift detection, accuracy degradation alerts
- Retraining: Automated retraining triggers (schedule, performance degradation, data drift), canary deployment for new models

- Rollback: Automated rollback to previous model on performance degradation, manual rollback capability
- Governance: Model approval workflow, model documentation, bias detection, fairness metrics, explainability

17.4 AI Models

17.4.1 Search & Recommendation Models

- Model: Two-tower neural network (Dual Encoder) for business search
- Architecture: Dense layers with ReLU, dropout, batch normalization, cosine similarity for scoring
- Input: User features (demographics, location, preferences), business features (category, rating, location, keywords), query features (embeddings, intent)
- Output: Similarity score, ranking, top-k recommendations
- Training: Contrastive learning with positive (user-business interaction) and negative (random) pairs
- Serving: SageMaker endpoint with GPU, batch inference for candidate generation, real-time inference for ranking
- Update: Weekly retraining with new interaction data, incremental updates for embeddings

17.4.2 Fraud Detection Models

- Model: Gradient Boosting (XGBoost, LightGBM) for structured fraud detection, Neural Network for behavioral fraud detection
- Architecture: XGBoost with feature engineering (transaction amount, velocity, device fingerprint, location, time), LSTM for behavioral sequences
- Input: Transaction features, user features, device features, location features, historical features
- Output: Risk score (0-100), fraud probability, anomaly score

- Training: Supervised learning with labeled fraud cases, imbalanced learning (SMOTE, class weights)
- Serving: SageMaker endpoint for real-time scoring, Lambda for lightweight rules, threshold-based blocking
- Update: Daily retraining with new fraud cases, feedback loop for model improvement

17.4.3 NLP Models

- Model: Transformer-based (BERT, RoBERTa) for intent classification, entity extraction, sentiment analysis
- Architecture: Fine-tuned BERT for domain-specific tasks (Sinhala, Tamil, English), DistilBERT for lightweight inference
- Input: Text (reviews, queries, messages, forum posts)
- Output: Intent labels, entity spans, sentiment scores, toxicity scores
- Training: Fine-tuning on domain-specific data, multi-task learning for related tasks
- Serving: SageMaker endpoint for server-side inference, TensorFlow Lite for mobile inference, ONNX for cross-platform
- Update: Monthly retraining with new labeled data, continuous learning from user feedback

17.4.4 Computer Vision Models

- Model: ResNet-50, EfficientNet for image classification, YOLO for object detection, Tesseract for OCR
- Architecture: Pre-trained on ImageNet, fine-tuned on domain-specific data (business photos, KYC documents, receipts)
- Input: Images (business photos, user uploads, KYC documents, receipts)
- Output: Image classification, object detection, text extraction, image quality score
- Training: Transfer learning with domain-specific data, data augmentation for robustness

- Serving: SageMaker endpoint for server-side inference, CoreML/TensorFlow Lite for mobile inference
- Update: Quarterly retraining with new labeled images, feedback loop for model improvement

17.4.5 Predictive Analytics Models

- Model: ARIMA, Prophet for time series forecasting, XGBoost for regression, Random Forest for classification
- Architecture: Ensemble models combining multiple algorithms, feature engineering for time-based features
- Input: Historical data (orders, attendance, revenue, user activity), external features (weather, holidays, promotions)
- Output: Forecasts (demand, attendance, revenue), predictions (churn, conversion), anomaly scores
- Training: Time series cross-validation, walk-forward validation, backtesting on historical data
- Serving: Batch inference on schedule (daily, weekly), SageMaker batch transform, API endpoint for on-demand predictions
- Update: Weekly retraining with new data, automated model comparison and selection

17.5 AI Ethics & Fairness

17.5.1 Bias Detection & Mitigation

- Data Bias: Analyze training data for demographic bias, geographic bias, temporal bias, selection bias
- Algorithmic Bias: Test models for disparate impact across demographic groups, fairness metrics (demographic parity, equalized odds)
- Mitigation: Data augmentation for underrepresented groups, reweighting, adversarial debiasing, post-processing adjustment

- Monitoring: Continuous monitoring of model predictions for bias, fairness dashboard, automated alerts for bias drift
- Governance: Bias assessment in model approval process, bias audit by third party annually

17.5.2 Explainability & Transparency

- Model Explainability: SHAP (SHapley Additive exPlanations) for model explanations, LIME for local explanations, feature importance
- User-Facing Explanations: "Why this recommendation?" explanations, "Why was this flagged?" explanations for fraud and moderation
- Internal Explanations: Model decision logs, feature importance for debugging, counterfactual explanations
- Documentation: Model cards for all production models (purpose, performance, limitations, bias, intended use)
- Regulatory Compliance: Explainability for GDPR Article 22 (automated decision-making), local regulations

17.5.3 Privacy-Preserving AI

- Differential Privacy: Noise injection for training data, privacy budget management, epsilon tracking
- Federated Learning: On-device model training for privacy-sensitive data, aggregated model updates
- Homomorphic Encryption: Encrypted inference for highly sensitive data (KYC, medical, government)
- Data Minimization: Minimal data collection for AI training, data anonymization, pseudonymization
- Consent: Explicit consent for AI processing, granular consent for different AI use cases, opt-out for non-essential AI
- On-Device Inference: Local inference for sensitive data (face recognition, biometric, personal recommendations)

17.6 AI Operations

17.6.1 Model Monitoring

- Performance Metrics: Accuracy, precision, recall, F1, AUC-ROC, business metrics (conversion, revenue, fraud prevented)
- Data Drift: Statistical drift detection (KS test, PSI, Chi-square), feature drift alerts, distribution comparison
- Concept Drift: Model performance degradation detection, ground truth validation, feedback loop
- Latency Monitoring: Inference latency, model loading time, batch processing time
- Resource Monitoring: GPU utilization, memory usage, CPU usage, cost per inference
- Alerting: Automated alerts for model degradation, drift, latency, resource issues

17.6.2 Model Retraining

- Triggers: Schedule (weekly, monthly), performance degradation, data drift, new labeled data, manual trigger
- Pipeline: Automated data extraction, feature engineering, training, validation, evaluation, approval, deployment
- A/B Testing: Champion-challenger testing, traffic splitting, statistical significance testing, automated winner selection
- Rollback: Automated rollback to previous model on performance degradation, manual rollback capability
- Shadow Mode: Shadow deployment for new models, comparing predictions without affecting users

17.6.3 AI Governance

- Model Inventory: Centralized model catalog, model documentation, ownership, status (development, staging, production, archived)
- Model Approval: Approval workflow for model deployment (data scientist, ML engineer, product manager, compliance officer)
- Model Audit: Regular model audits (quarterly), bias audits, security audits, compliance audits
- Model Retirement: Model deprecation process, data deletion, model archival, user communication
- Ethics Board: AI Ethics Committee for high-stakes AI decisions (fraud, credit, hiring, healthcare)
- Compliance: GDPR, CCPA, local AI regulations, industry-specific AI standards

Chapter 18: Mobile Application Architecture

18.1 Mobile Strategy

LocalConnect's mobile strategy prioritizes native performance for iOS and Android, with a cross-platform option for future expansion. The mobile apps are designed for offline-first operation, fast performance, and seamless integration with platform features.

18.2 iOS Architecture (Swift)

18.2.1 Architecture Pattern

- Pattern: MVVM-C (Model-View-ViewModel-Coordinator) with Clean Architecture
- Layers:
 - Presentation: SwiftUI views, UIKit views (legacy), ViewModels, Coordinators
 - Domain: Use cases, business logic, entities, repository interfaces

- Data: API clients, local database, cache, data mappers, repository implementations
- Navigation: Coordinator pattern for centralized navigation, deep linking, modular routing
- State Management: Combine framework for reactive programming, ObservableObject for view models, @State/@ObservedObject/@EnvironmentObject for SwiftUI
- Dependency Injection: Swinject for DI container, constructor injection, protocol-oriented programming

18.2.2 Technology Stack

- Language: Swift 5.9+
- UI Framework: SwiftUI (primary), UIKit (legacy support, custom components)
- Networking: Alamofire for HTTP requests, URLSession for WebSocket, Combine for reactive networking
- Database: Core Data (primary), SQLite (lightweight), Keychain for sensitive data
- Image Loading: Kingfisher (async image loading, caching, transformations)
- Maps: MapKit (primary), Google Maps SDK (alternative)
- Payments: Apple Pay (In-App Purchase for digital goods, Apple Pay for physical goods)
- Push Notifications: Firebase Cloud Messaging (FCM), Apple Push Notification Service (APNS)
- Analytics: Firebase Analytics, Mixpanel for product analytics
- Crash Reporting: Firebase Crashlytics, Sentry for error tracking
- Deep Linking: Universal Links, URL schemes, Firebase Dynamic Links
- Authentication: Keychain for token storage, Biometric authentication (Face ID/Touch ID), OAuth 2.0 via SFSafariViewController
- Testing: XCTest for unit tests, XCUITest for UI tests, SnapshotTesting for snapshot tests

18.2.3 Project Structure

LocalConnect-iOS/

└─ App/

| └─ AppDelegate.swift

| └─ SceneDelegate.swift

| └─ AppCoordinator.swift

| └─ DIContainer.swift

└─ Presentation/

| └─ Features/

| | └─ Auth/

| | └─ Home/

| | └─ Business/

| | └─ Event/

| | └─ Order/

| | └─ Payment/

| | └─ Forum/

| | └─ Emergency/

| | └─ Government/

| | └─ Profile/

| └─ Common/

| | └─ Views/

| | └─ Components/

| | └─ Modifiers/

| └─ Resources/

| └─ Colors/

| └─ Fonts/

| └─ Images/

| └─ Localizations/

└─ Domain/

| └─ Entities/

| └─ UseCases/

| └─ RepositoryInterfaces/

| └─ Protocols/

└─ Data/

| └─ API/

| | └─ Clients/

| | └─ Endpoints/

| | └─ Models/

| | └─ Mappers/

| └─ Local/

- | | | — CoreData/
- | | | — Keychain/
- | | | — UserDefaults/
- | | — Repository/
- | | — Implementations/
- | — Infrastructure/
- | | — Network/
- | | — Cache/
- | | — Security/
- | | — Location/
- | | — Notifications/
- | | — Analytics/
- | — Tests/
- | | — UnitTests/
- | | — UITests/
- | | — SnapshotTests/
- | — Supporting Files/
- | — Info.plist
- | — LocalConnect.entitlements
- | — LaunchScreen.storyboard

18.2.4 Offline-First Architecture

- Data Sync: Background sync for user data, business data, event data, order data
- Conflict Resolution: Last-write-wins with timestamp, manual conflict resolution for critical data
- Queue: Operation queue for offline actions, retry with exponential backoff, max retries = 5
- Storage: Core Data for structured data, file system for images and documents, cache for API responses
- Connectivity: Reachability monitoring, offline mode UI, sync status indicator
- Background Fetch: Periodic background fetch for data updates, push notification triggered fetch

18.2.5 Security

- Keychain: Token storage, biometric-protected items, secure enclave for sensitive data
- Certificate Pinning: SSL pinning for API communication, certificate validation
- Jailbreak Detection: Runtime jailbreak detection, app termination on jailbroken devices (configurable)
- Code Obfuscation: SwiftShield for code obfuscation, string obfuscation, anti-tampering
- Screen Security: Screenshot prevention for sensitive screens (payments, KYC), screen recording detection
- Data Protection: NSFileProtectionComplete for file encryption, data protection class for Core Data
- Secure Communication: TLS 1.3, certificate pinning, mTLS for high-security features

18.2.6 Performance Optimization

- Lazy Loading: Lazy loading for images, pagination for lists, on-demand loading for heavy content
- Image Optimization: WebP format, progressive loading, Kingfisher caching, downsampling for thumbnails
- Memory Management: ARC optimization, memory warnings handling, image cache size limits
- Background Tasks: Background fetch, background processing, silent push notifications for data sync
- Startup Optimization: Lazy initialization, preloading critical data, splash screen with progress
- Animation: 60fps animations, Core Animation, SwiftUI animation, reduce motion support
- Battery Optimization: Low power mode detection, reduced background activity, location updates optimization

18.3 Android Architecture (Kotlin)

18.3.1 Architecture Pattern

- Pattern: MVVM with Clean Architecture, MVI (Model-View-Intent) for complex features
- Layers:
 - Presentation: Activities, Fragments, Composables, ViewModels, Adapters
 - Domain: Use cases, business logic, entities, repository interfaces
 - Data: API clients, local database, cache, data mappers, repository implementations
- Navigation: Jetpack Navigation Component for single Activity architecture, deep linking, modular navigation
- State Management: Flow/StateFlow for reactive programming, LiveData (legacy), Compose State for Jetpack Compose
- Dependency Injection: Hilt (Dagger wrapper) for DI, constructor injection, interface-based programming

18.3.2 Technology Stack

- Language: Kotlin 1.9+, Java 17 (legacy support)
- UI Framework: Jetpack Compose (primary), XML layouts (legacy support), Material Design 3
- Networking: Retrofit with OkHttp for HTTP requests, Kotlin Coroutines for async, Ktor for WebSocket
- Database: Room (primary), SQLite (legacy), DataStore for preferences, EncryptedSharedPreferences for sensitive data
- Image Loading: Coil (Kotlin-first, Compose integration, caching, transformations)
- Maps: Google Maps SDK (primary), OpenStreetMap (alternative for offline)
- Payments: Google Pay (in-app billing for digital goods, Google Pay for physical goods), Stripe SDK
- Push Notifications: Firebase Cloud Messaging (FCM), notification channels, rich notifications
- Analytics: Firebase Analytics, Mixpanel for product analytics
- Crash Reporting: Firebase Crashlytics, Sentry for error tracking
- Deep Linking: Android App Links, intent filters, Firebase Dynamic Links
- Authentication: EncryptedSharedPreferences for token storage, BiometricPrompt for biometrics, OAuth 2.0 via Chrome Custom Tabs
- Testing: JUnit 5 for unit tests, Espresso for UI tests, MockK for mocking, Turbine for Flow testing

18.3.3 Project Structure

LocalConnect-Android/

├─ app/

| └─ src/

| | └─ main/

| | | └─ java/com/localconnect/

| | | | └─ di/

| | | | └─ presentation/

| | | | | └─ features/

| | | | | | └─ auth/

| | | | | | └─ home/

| | | | | | └─ business/

| | | | | | └─ event/

| | | | | | └─ order/

| | | | | | └─ payment/

| | | | | | └─ forum/

| | | | | | └─ emergency/

| | | | | | └─ government/

| | | | | | └─ profile/

| | | | | └─ common/

| | | | | | └─ components/

| | | | | | └─ theme/

| | | | | | └─ utils/

| | | | | └─ MainActivity.kt

| | | | └─ domain/

```
| | | | | └─ entities/
| | | | | └─ usecases/
| | | | | └─ repository/
| | | | └─ data/
| | | | └─ api/
| | | | └─ local/
| | | | └─ repository/
| | | | └─ mappers/
| | | └─ infrastructure/
| | | └─ network/
| | | └─ cache/
| | | └─ security/
| | | └─ location/
| | | └─ notifications/
| | └─ res/
| | └─ AndroidManifest.xml
| └─ test/
| └─ androidTest/
└─ build.gradle.kts
└─ build.gradle.kts
```

└─ gradle.properties

└─ settings.gradle.kts

18.3.4 Offline-First Architecture

- Data Sync: WorkManager for background sync, periodic sync for data updates, immediate sync for critical actions
- Conflict Resolution: Last-write-wins with timestamp, server-side conflict resolution for critical data
- Queue: WorkManager for offline actions, retry with exponential backoff, constraints (network, battery, charging)
- Storage: Room for structured data, file system for images and documents, cache for API responses
- Connectivity: ConnectivityManager for network monitoring, offline mode UI, sync status indicator
- Background Fetch: WorkManager periodic work, Firebase Cloud Messaging for push-triggered sync

18.3.5 Security

- EncryptedSharedPreferences: Token storage, biometric-protected items, encrypted preferences
- Certificate Pinning: OkHttp certificate pinning for API communication, network security config
- Root Detection: RootBeer for root detection, emulator detection, app termination on rooted devices (configurable)
- Code Obfuscation: ProGuard/R8 for code obfuscation, string encryption, anti-tampering
- Screen Security: FLAG_SECURE for screenshot prevention, screen recording detection
- Data Protection: Android file encryption, scoped storage, biometric-protected key store

- Secure Communication: TLS 1.3, certificate pinning, mTLS for high-security features

18.3.6 Performance Optimization

- Lazy Loading: LazyColumn for lists, pagination with Paging 3, on-demand loading for heavy content
- Image Optimization: WebP format, progressive loading, Coil caching, downsampling for thumbnails
- Memory Management: Memory optimization, large heap for image-heavy screens, memory warnings handling
- Background Tasks: WorkManager for background tasks, foreground services for critical tasks, doze mode optimization
- Startup Optimization: Lazy initialization, app startup library, splash screen API, preloading critical data
- Animation: 60fps animations, Jetpack Compose animation, Lottie for complex animations, reduce motion support
- Battery Optimization: Battery optimization whitelist, reduced background activity, location updates optimization, job scheduling

18.4 Cross-Platform Strategy (Flutter - Future)

- Evaluation: Flutter 3.x for future cross-platform development, single codebase for iOS and Android
- Decision Criteria: Development velocity, performance requirements, team expertise, platform-specific features
- Timeline: Evaluate after 12 months of native development, pilot for non-critical features
- Architecture: Clean Architecture with BLoC pattern, Riverpod for state management, Dio for networking
- Integration: Shared business logic, platform-specific UI for critical features, shared repository layer

18.5 Mobile API Design

18.5.1 Mobile-Specific API Endpoints

- Batch API: POST /v1/mobile/batch for batch requests, reducing network calls
- Sync API: GET /v1/mobile/sync for incremental sync, timestamp-based delta updates
- Prefetch API: GET /v1/mobile/prefetch for preloading data for offline use
- Optimized Responses: Compressed responses (Brotli/gzip), field selection, pagination, cursor-based pagination
- Image Optimization: Dynamic image sizing, WebP format, CDN integration, lazy loading
- Push Notifications: POST /v1/mobile/push-token for FCM/APNS token registration, topic-based subscriptions
- Deep Linking: GET /v1/mobile/deep-link for deep link resolution, dynamic content loading

18.5.2 Mobile API Versioning

- Version Header: X-API-Version: 1 for mobile-specific API versioning
- Backward Compatibility: Mobile apps support 2 API versions, forced update for deprecated versions
- Deprecation: 6-month deprecation notice, in-app notification, app store update requirement
- Feature Flags: Server-side feature flags for gradual rollout, A/B testing for mobile features

18.6 Mobile Testing Strategy

18.6.1 Testing Levels

- Unit Tests: 80% coverage for business logic, use cases, repositories, ViewModels
- Integration Tests: API integration, database integration, cache integration, payment integration (sandbox)
- UI Tests: Critical user flows (registration, search, booking, payment, emergency), device farm testing
- Performance Tests: Startup time, memory usage, battery drain, network usage, frame rate
- Security Tests: Penetration testing for mobile apps, OWASP Mobile Top 10, certificate pinning validation
- Accessibility Tests: VoiceOver/TalkBack testing, color contrast, font scaling, dynamic type
- Compatibility Tests: OS version compatibility (iOS 15+, Android 8+), device compatibility (phones, tablets, foldables)

18.6.2 Testing Infrastructure

- CI/CD: GitHub Actions for iOS (macOS runner), Android (Linux runner), automated testing on every PR
- Device Farm: AWS Device Farm for real device testing, Firebase Test Lab for Android, BrowserStack for iOS
- Beta Testing: TestFlight for iOS beta testing, Google Play Console for Android beta testing, internal testing groups
- Monitoring: Firebase Crashlytics for crash reporting, Sentry for error tracking, Analytics for user behavior
- Automated Testing: Automated UI tests on CI, nightly regression tests, smoke tests on every build

18.7 Mobile Deployment

18.7.1 Release Process

- Versioning: Semantic versioning for mobile apps (MAJOR.MINOR.PATCH), build number auto-increment
- Release Branch: Release branch for each version, cherry-pick bug fixes, no new features
- App Store Submission: Automated submission via fastlane, screenshots, metadata, release notes
- Google Play Submission: Automated submission via fastlane, AAB upload, staged rollout
- Staged Rollout: 5% → 20% → 50% → 100% rollout over 1 week, monitoring for crashes and issues
- Rollback: Emergency rollback via app store (previous version promotion), forced update for critical fixes
- Release Notes: In-app changelog, app store release notes, email to users for major updates

18.7.2 App Store Optimization (ASO)

- Keywords: Local business, Sri Lanka, events, services, government, emergency, community
- Screenshots: Feature-specific screenshots, localized for Sinhala, Tamil, English
- Description: Localized descriptions, feature highlights, user testimonials, call to action
- Ratings & Reviews: In-app review prompt, response to reviews, review analysis for feature improvement
- A/B Testing: A/B testing for store listing, screenshots, descriptions, icons
- Localization: Full localization for Sinhala, Tamil, English, app store metadata localization

Chapter 1: Executive Summary

LocalConnect is a next-generation community super-app designed to digitally empower local communities by connecting local businesses, government

services, emergency response, and community events. It addresses the digital divide by providing a single platform where users can discover local businesses, receive emergency alerts, engage with community events, access government services, and participate in forums.

Key Decisions

1. Platform Strategy: Native Mobile (iOS/Android via Flutter for rapid MVP, then Kotlin/Swift for high performance) + Responsive Web (PWA for offline capabilities) + Progressive Web App (PWA) for lightweight access.
2. Architecture Pattern: Microservices with Event-Driven Architecture (EDA) using CQRS + Event Sourcing for critical domains.
3. Cloud: AWS Primary (EKS, RDS, S3, SQS, ElastiCache, Bedrock) with Azure as DR region and GCP for analytics.
4. Database: Polyglot — PostgreSQL (relational), MongoDB (document), DynamoDB (high-throughput), Redis (cache/session), S3 (object storage), ClickHouse (analytics).
5. Security: Zero-Trust Architecture, OAuth 2.0 + OIDC, mTLS everywhere, end-to-end encryption for sensitive messages, DLP, threat modeling, and quarterly penetration testing.
6. AI Integration: LLMs for content moderation, AI chatbots, recommendation engines, and intelligent routing. Bedrock and SageMaker for AI/ML pipelines.
7. Scale: Designed for 100M users from day one with 12-month progressive scaling phases.
8. Monitoring: OpenTelemetry + Grafana + Prometheus + Jaeger + ELK Stack for 3 pillars of observability.
9. CI/CD: GitOps with ArgoCD, Jenkins for build, GitHub Actions for testing, Terraform for infrastructure as code.
10. API Strategy: REST for CRUD, GraphQL for complex queries, gRPC for internal service communication, WebSocket for real-time.

Chapter 19: Web Application Architecture

19.1 Web Strategy

LocalConnect's web architecture is designed for performance, SEO, accessibility, and progressive enhancement. The web app serves as the primary entry point for users and businesses, with the admin dashboard for platform management.

19.2 Frontend Architecture (React)

19.2.1 Architecture Pattern

- Pattern: Component-based architecture with functional components, custom hooks, and context for state management
- State Management: Redux Toolkit (RTK) for global state, React Query for server state, Zustand for lightweight state, Context API for theme/auth
- Routing: React Router v6 for client-side routing, lazy loading for code splitting, route guards for authentication
- Component Structure: Atomic Design methodology (atoms, molecules, organisms, templates, pages)
- Styling: Tailwind CSS for utility-first styling, CSS modules for component-specific styles, styled-components for dynamic styles
- Forms: React Hook Form for form management, Zod for validation, Formik (legacy)
- Animation: Framer Motion for animations, GSAP for complex animations, CSS transitions for simple animations

19.2.2 Technology Stack

- Framework: React 18 with Concurrent Features, TypeScript 5.x

- Build Tool: Vite for fast development and production builds, SWC for compilation
- UI Library: Headless UI for accessible components, Radix UI for primitives, shadcn/ui for pre-built components
- CSS Framework: Tailwind CSS 3.x with custom design system, PostCSS for processing
- Icons: Lucide React for icons, Heroicons for UI icons, custom SVG icons
- Maps: React Leaflet for interactive maps, Mapbox GL JS for advanced mapping, Google Maps API for business verification
- Charts: Recharts for React charts, D3.js for custom visualizations, Chart.js for simple charts
- Payments: Stripe.js for payment processing, Stripe Elements for PCI-compliant payment forms
- Analytics: Google Analytics 4, Mixpanel for product analytics, Segment for event tracking
- SEO: React Helmet Async for meta tags, structured data (JSON-LD), Open Graph tags, Twitter Cards
- Accessibility: React Aria for accessibility, axe-core for automated testing, keyboard navigation, screen reader support
- Testing: Jest for unit tests, React Testing Library for component tests, Cypress for E2E tests, Playwright for cross-browser testing
- PWA: Service Worker for offline support, Web App Manifest, push notifications, background sync
- SSR/SSG: Next.js for marketing pages (SSR/SSG), React SPA for app (client-side rendering with code splitting)

19.2.3 Project Structure

localconnect-web/

├─ public/

| └─ index.html

| └─ manifest.json

| └─ robots.txt

| └─ sitemap.xml

| └─ assets/

| └─ images/

| └─ fonts/

| └─ icons/

└─ src/

| └─ main.tsx

| └─ App.tsx

| └─ routes.tsx

| └─ index.css

| └─ components/

| | └─ atoms/

| | | └─ Button/

| | | └─ Input/

| | | └─ Badge/

| | | └─ Avatar/

| | | └─ ...

| | └─ molecules/

| | | └─ SearchBar/

| | | └─ BusinessCard/

| | | └─ EventCard/

| | | └─ ReviewItem/

| | | └─ ...

| | └─ organisms/

| | | └─ BusinessList/

| | | └─ EventCalendar/

| | | └─ OrderSummary/

| | | └─ ForumThread/

| | | └─ ...

| | └─ templates/

| | | └─ MainLayout/

| | | └─ AuthLayout/

| | | └─ AdminLayout/

| | | └─ ...

| | └─ pages/

| | └─ HomePage/

| | └─ BusinessPage/

| | └─ EventPage/

| | └─ OrderPage/

| | └─ ForumPage/

| | └─ EmergencyPage/

| | └─ GovernmentPage/

| | └─ ProfilePage/

| | └─ Admin/

| └─ hooks/

| | └─ useAuth.ts

| | └─ useBusiness.ts

| | └─ useOrder.ts

| | └─ useLocation.ts

| | └─ useSearch.ts

| | └─ ...

| └─ store/

| | └─ slices/

| | | └─ authSlice.ts

| | | └─ businessSlice.ts

| | | └─ orderSlice.ts

| | | └─ ...

| | └─ store.ts

- | | └─ hooks.ts
- | └─ api/
 - | | └─ client.ts
 - | | └─ endpoints.ts
 - | | └─ types.ts
 - | | └─ interceptors.ts
 - | └─ services/
 - | | └─ authService.ts
 - | | └─ businessService.ts
 - | | └─ orderService.ts
 - | | └─ paymentService.ts
 - | | └─ ...
 - | └─ utils/
 - | | └─ formatters.ts
 - | | └─ validators.ts
 - | | └─ helpers.ts
 - | | └─ constants.ts
 - | └─ types/
 - | | └─ auth.ts
 - | | └─ business.ts


```
| | └─ order.ts
| | └─ ...
| └─ context/
| | └─ AuthContext.tsx
| | └─ ThemeContext.tsx
| | └─ LocaleContext.tsx
| └─ styles/
| └─ tailwind.config.js
| └─ globals.css
| └─ theme.ts
└─ tests/
    | └─ unit/
    | └─ integration/
    | └─ e2e/
    └─ .env
    └─ .env.production
    └─ vite.config.ts
    └─ tsconfig.json
    └─ package.json
    └─ README.md
```

19.2.4 Performance Optimization

- Code Splitting: Route-based code splitting, component lazy loading, dynamic imports for heavy features
- Tree Shaking: Dead code elimination, ES modules for tree shaking, minimal imports
- Asset Optimization: Image optimization (WebP, AVIF, responsive images), font subsetting, SVG optimization, CSS purging
- Caching: Service Worker for caching, Cache API for runtime caching, HTTP caching headers, CDN caching
- Lazy Loading: Intersection Observer for lazy loading images, virtual scrolling for long lists, on-demand loading for heavy content
- Prefetching: Route prefetching on hover, data prefetching for predicted navigation, DNS prefetching
- Bundle Analysis: Webpack Bundle Analyzer for bundle size analysis, performance budgets, CI enforcement
- Core Web Vitals: LCP < 2.5s, FID < 100ms, CLS < 0.1, TTFB < 600ms, FCP < 1.8s, TTI < 3.8s
- Progressive Enhancement: Works without JavaScript (SSR), graceful degradation for older browsers, feature detection

19.2.5 SEO Architecture

- SSR/SSG: Next.js for marketing pages, blog, business listings (server-side rendering for SEO)
- SPA for App: React SPA for authenticated app (client-side rendering), prerender.io for basic SEO
- Meta Tags: Dynamic meta tags per page, Open Graph tags, Twitter Cards, structured data (JSON-LD)
- Sitemap: Automated sitemap generation for business listings, events, blog posts, government services
- Robots.txt: Crawl directives, sitemap reference, disallow for admin and private pages

- Canonical URLs: Canonical tags for duplicate content, hreflang for multilingual pages
- Structured Data: Schema.org for businesses, events, products, reviews, breadcrumbs, FAQ
- Performance: Page speed optimization for SEO, Core Web Vitals, mobile-first indexing
- Accessibility: WCAG 2.1 AA compliance, semantic HTML, ARIA labels, keyboard navigation, screen reader support

19.2.6 Security

- Content Security Policy (CSP): Strict CSP with nonce, script-src, style-src, img-src, connect-src, frame-src, report-uri
- XSS Protection: React's built-in XSS protection, DOMPurify for HTML sanitization, CSP for mitigation
- CSRF Protection: Double-submit cookie pattern, CSRF tokens for state-changing requests, SameSite cookies
- HTTPS: Strict Transport Security (HSTS), TLS 1.3, certificate pinning, secure cookies
- Dependency Scanning: Snyk for dependency vulnerability scanning, npm audit, automated CI checks
- Secrets Management: No secrets in frontend code, environment variables for configuration, runtime injection
- Subresource Integrity (SRI): SRI for CDN resources, integrity attributes for scripts and styles
- Clickjacking Protection: X-Frame-Options, Content-Security-Policy frame-ancestors
- Referrer Policy: Strict Referrer Policy for privacy, Referrer-Policy header

19.3 Admin Dashboard Architecture

19.3.1 Architecture

- Framework: React with TypeScript, same stack as web app but with admin-specific features
- UI Library: Ant Design or Material-UI for admin components, data tables, forms, charts, dashboards
- State Management: Redux Toolkit for global state, React Query for server state, local state for forms
- Authentication: Role-based access control (RBAC), route guards for admin routes, API authorization
- Data Visualization: Recharts, D3.js for admin dashboards, real-time metrics, historical trends
- CRUD Operations: Generic CRUD components, form builders, data tables with sorting, filtering, pagination
- File Management: File upload, image management, document management, CSV export, PDF export
- Reporting: Report generation, scheduled reports, report templates, export to Excel/PDF
- User Management: User list, user details, role management, permission management, activity logs
- Content Management: CMS for static content, blog management, banner management, announcement management
- Moderation: Review moderation, forum moderation, business verification, content approval workflows
- System Management: Configuration management, feature flags, system health, maintenance mode

19.3.2 Project Structure

localconnect-admin/

├─ src/

| └─ components/

| | └─ common/

| | | └─ DataTable/

| | | └─ FormBuilder/

| | | └─ FilterPanel/

| | | └─ ChartPanel/

| | | └─ ...

| | └─ dashboard/

| | | └─ MetricCards/

| | | └─ RevenueChart/

| | | └─ UserGrowthChart/

| | | └─ ...

| | └─ users/

| | └─ businesses/

| | └─ orders/

| | └─ events/

| | └─ content/

| | └─ moderation/

| | └─ system/

| | └─ reports/

| └─ pages/

| | └─ DashboardPage/

| | └─ UsersPage/
| | └─ BusinessesPage/
| | └─ OrdersPage/
| | └─ EventsPage/
| | └─ ContentPage/
| | └─ ModerationPage/
| | └─ SystemPage/
| | └─ ReportsPage/
| └─ store/
| └─ api/
| └─ hooks/
| └─ utils/
| └─ types/
└─ tests/
└─ ...

19.4 Web API Integration

19.4.1 API Client Design

- HTTP Client: Axios with interceptors, automatic retry, timeout handling, request/response transformation
- Base URL: Environment-specific base URL (dev, staging, prod), API versioning

- Authentication: Bearer token in Authorization header, automatic token refresh, logout on 401
- Error Handling: Global error handler, toast notifications for errors, retry logic for network errors
- Caching: React Query for caching, stale-while-revalidate, cache invalidation, optimistic updates
- Pagination: Cursor-based pagination, infinite scroll, page-based pagination for tables
- Optimistic Updates: UI updates before API confirmation, rollback on error, toast for success/error
- Batch Requests: Request deduplication, batching for multiple simultaneous requests
- Real-Time: WebSocket for real-time updates, Server-Sent Events (SSE) for notifications, polling fallback

19.4.2 Authentication Flow

- Login: OAuth 2.0 + PKCE for web, JWT token storage in httpOnly cookie (preferred) or memory (SPA)
- Token Refresh: Automatic token refresh before expiry, refresh token rotation, concurrent request handling
- Logout: Token revocation, cookie clearing, state cleanup, redirect to login
- Session Management: Session timeout, idle timeout, concurrent session handling, "Remember Me" option
- MFA: TOTP for web, WebAuthn for biometric, backup codes, MFA enrollment flow
- Social Login: Google, Facebook, Apple login, OAuth 2.0 flow, account linking

19.5 Web Testing Strategy

19.5.1 Testing Levels

- Unit Tests: Jest + React Testing Library, 80% coverage for components, hooks, utilities, services
- Integration Tests: React Testing Library for component integration, API mocking with MSW, state integration
- E2E Tests: Cypress for critical user flows, Playwright for cross-browser testing, visual regression with Percy
- Performance Tests: Lighthouse CI for performance budgets, WebPageTest for real-world performance, Core Web Vitals monitoring
- Accessibility Tests: axe-core for automated accessibility, manual testing with screen readers, keyboard navigation
- Security Tests: OWASP ZAP for web security, dependency scanning, CSP validation
- SEO Tests: Structured data validation, meta tag validation, sitemap validation, canonical tag validation

19.5.2 Testing Infrastructure

- CI/CD: GitHub Actions for automated testing, parallel test execution, coverage reporting, artifact collection
- Visual Regression: Percy or Chromatic for visual regression testing, screenshot comparison, approval workflow
- Browser Testing: BrowserStack for cross-browser testing, Playwright for automated cross-browser testing
- Performance Monitoring: Lighthouse CI for performance budgets, WebPageTest for real-world metrics, RUM for production

19.6 Web Deployment

19.6.1 Hosting & CDN

- Static Hosting: AWS S3 + CloudFront for static assets, Next.js on Vercel or AWS Amplify for SSR/SSG

- CDN: CloudFront for global CDN, edge caching, origin shield, image optimization
- Edge Functions: CloudFront Functions for URL rewriting, header manipulation, A/B testing
- Load Balancing: ALB for API routing, path-based routing, health checks, SSL termination
- DNS: Route 53 for DNS, health checks, failover routing, geolocation routing

19.6.2 Deployment Pipeline

- Build: Vite build for production, minification, code splitting, asset optimization, bundle analysis
- Testing: Automated testing in CI, coverage gates, Lighthouse performance budget, security scan
- Staging: Automatic deployment to staging environment, smoke tests, integration tests, manual QA
- Production: Blue-green deployment for zero downtime, canary deployment for gradual rollout, rollback capability
- Cache Invalidation: CloudFront cache invalidation on deployment, asset hashing for cache busting
- Monitoring: Real User Monitoring (RUM) for production, error tracking, performance monitoring, analytics

Chapter 20: Performance Engineering

20.1 Performance Strategy

LocalConnect's performance engineering ensures fast response times, high throughput, and efficient resource utilization. Performance is considered a feature, not an afterthought, with performance budgets and continuous monitoring.

20.2 Performance Targets

20.2.1 API Performance Targets

- General APIs: P50 < 50ms, P95 < 200ms, P99 < 500ms
- Search APIs: P50 < 30ms, P95 < 100ms, P99 < 200ms
- Payment APIs: P50 < 100ms, P95 < 300ms, P99 < 500ms (critical for revenue)
- Emergency APIs: P50 < 20ms, P95 < 50ms, P99 < 100ms (life-critical)
- Government APIs: P50 < 50ms, P95 < 200ms, P99 < 500ms
- File Upload APIs: P50 < 1s, P95 < 3s, P99 < 10s (with progress tracking)
- Batch APIs: P50 < 500ms, P95 < 2s, P99 < 5s

20.2.2 Web Performance Targets

- Page Load Time (LCP): < 2.5s for all pages, < 1.5s for homepage and search
- First Input Delay (FID): < 100ms for interactive elements
- Cumulative Layout Shift (CLS): < 0.1 for all pages
- Time to First Byte (TTFB): < 600ms for all pages
- First Contentful Paint (FCP): < 1.8s for all pages
- Time to Interactive (TTI): < 3.8s for all pages
- Total Blocking Time (TBT): < 200ms for all pages
- Speed Index: < 3.4s for all pages

20.2.3 Mobile Performance Targets

- Cold Start: < 2s for iOS, < 2s for Android
- Hot Start: < 500ms for iOS, < 500ms for Android
- API Response Time: < 500ms for 90% of requests
- Screen Transition: < 200ms for all screen transitions
- Image Load Time: < 1s for images, < 500ms for thumbnails
- List Scroll: 60fps for smooth scrolling, no dropped frames

- Map Load: < 3s for map initialization, < 1s for business markers
- Search Response: < 100ms for local search, < 300ms for server search

20.2.4 Database Performance Targets

- Query Time: < 10ms for simple queries, < 50ms for complex queries, < 100ms for reports
- Connection Pool: < 50% utilization, < 100ms connection acquisition time
- Replication Lag: < 1s for read replicas, < 5s for cross-region replicas
- Cache Hit Rate: > 90% for Redis, > 80% for CDN, > 70% for application cache
- Index Usage: > 95% of queries use indexes, no table scans for production queries
- Lock Contention: < 1% of transactions experience lock contention
- Deadlock Rate: < 0.01% of transactions

20.3 Frontend Performance Optimization

20.3.1 Code Optimization

- Bundle Size: Performance budget for bundle size (200KB initial, 500KB total), tree shaking, code splitting
- Lazy Loading: Route-based code splitting, component lazy loading, on-demand feature loading
- Tree Shaking: ES modules, side-effect-free imports, dead code elimination
- Minification: Terser for JS, CSSNano for CSS, HTML minification, Gzip/Brotli compression
- Dependency Optimization: Replace heavy libraries with lightweight alternatives, tree-shakeable libraries, modular imports
- Third-Party Scripts: Async loading for non-critical scripts, defer for analytics, preconnect for critical domains

- Polyfills: Only load polyfills for unsupported browsers, feature detection, core-js for modern features

20.3.2 Asset Optimization

- Images: WebP/AVIF format, responsive images (srcset), lazy loading (loading="lazy"), image CDN for optimization
- Fonts: Font subsetting (only needed characters), font-display: swap, preconnect for font CDN, variable fonts
- SVG: Inline SVG for small icons, SVG sprites for multiple icons, optimized SVG (SVGO), no unnecessary paths
- Videos: Lazy loading for videos, poster images, autoplay with muted, optimized formats (MP4, WebM)
- CSS: Critical CSS extraction, CSS purging (PurgeCSS), CSS minification, deferred non-critical CSS
- JavaScript: Defer non-critical JS, async for independent scripts, module/nomodule pattern for modern/legacy

20.3.3 Caching Strategy

- Service Worker: Cache-first for static assets, network-first for API data, stale-while-revalidate for dynamic content
- Cache API: Runtime caching for API responses, cache invalidation on data updates, cache versioning
- HTTP Caching: Cache-Control headers (immutable for hashed assets, max-age for static), ETag, Last-Modified
- CDN Caching: Edge caching for static assets, origin shield for cache hit ratio, cache invalidation on deployment
- Browser Caching: LocalStorage for user preferences, sessionStorage for session data, IndexedDB for large data
- Prefetching: DNS prefetching, preconnecting, prefetching critical resources, prerendering for predicted pages

20.3.4 Rendering Optimization

- Virtual DOM: React optimization (memo, useMemo, useCallback), avoid unnecessary re-renders, PureComponent
- Virtual Scrolling: react-window or react-virtualized for long lists, pagination for large datasets
- Skeleton Screens: Skeleton loading for perceived performance, shimmer effect, progressive loading
- Progressive Enhancement: Core content without JS, lazy loading for non-critical features, graceful degradation
- Web Workers: Offload heavy computations to web workers, background processing, non-blocking UI
- RequestAnimationFrame: Smooth animations with rAF, 60fps target, avoid layout thrashing
- Intersection Observer: Lazy loading images, infinite scroll, visibility-based rendering, ad loading
- Resource Hints: Preload for critical resources, prefetch for predicted navigation, preconnect for external domains

20.3.5 Network Optimization

- HTTP/2: Server Push (deprecated, use preload instead), multiplexing, header compression, binary protocol
- HTTP/3: QUIC protocol, reduced latency, improved congestion control, 0-RTT for repeat connections
- Compression: Brotli (preferred) and Gzip for text assets, quality settings, content negotiation
- Connection Reuse: Keep-alive, connection pooling, domain sharding (avoided with HTTP/2)
- Request Optimization: Request batching, request deduplication, GraphQL for over-fetching, REST for simplicity
- API Optimization: Field selection, pagination, cursor-based pagination, batch endpoints, compression

- CDN: Global CDN for static assets, edge caching, origin shield, dynamic content acceleration

20.3.6 Core Web Vitals Optimization

- LCP (Largest Contentful Paint): Optimize hero image, preload LCP image, server-side rendering, image optimization
- FID (First Input Delay): Reduce JS execution time, code splitting, defer non-critical JS, Web Workers
- CLS (Cumulative Layout Shift): Reserve space for images, ads, iframes, avoid dynamic content above fold, font-display: swap
- TTFB (Time to First Byte): Server optimization, CDN usage, edge computing, database optimization, caching
- FCP (First Contentful Paint): Inline critical CSS, preload critical fonts, reduce server response time, optimize HTML
- TBT (Total Blocking Time): Long tasks < 50ms, code splitting, defer non-critical JS, optimize third-party scripts
- SI (Speed Index): Optimize above-the-fold content, critical rendering path, resource prioritization, lazy loading below fold

20.4 Backend Performance Optimization

20.4.1 API Optimization

- Response Compression: Brotli (preferred) and Gzip for JSON responses, threshold for compression, content negotiation
- Payload Optimization: Minimal response payloads, field selection, partial responses, pagination, cursor-based pagination
- Caching: Redis for response caching (TTL 60s for standard, 300s for static), CDN for static API responses, ETag for conditional requests
- Database Query Optimization: Query optimization, index usage, query plan analysis, N+1 query prevention, batch queries

- Connection Pooling: Database connection pooling (HikariCP), HTTP connection pooling (OkHttp, Axios), Redis connection pooling
- Async Processing: Async for non-critical operations (email, notifications, analytics), message queues for background processing
- Rate Limiting: Token bucket for fair usage, rate limiting by user/API key, graceful degradation for exceeded limits
- Batch Endpoints: Batch API for multiple requests, reducing network round trips, request coalescing

20.4.2 Database Optimization

- Query Optimization: EXPLAIN ANALYZE for query plans, index optimization, query rewriting, avoiding table scans
- Indexing: B-tree indexes for equality and range queries, GIN indexes for JSON and full-text, GiST for spatial, BRIN for time-series
- Partitioning: Time-based partitioning for large tables, hash partitioning for even distribution, list partitioning for categories
- Connection Management: Connection pooling (HikariCP), max connections (100 per instance), connection timeout (30s), idle timeout (10m)
- Read Replicas: Read replicas for read-heavy workloads, read/write splitting, replication lag monitoring
- Caching: Redis for query result caching, materialized views for complex reports, application-level caching
- Batch Operations: Batch inserts, batch updates, COPY for bulk loading, avoid row-by-row processing
- Vacuum & Analyze: Automated vacuum, manual vacuum for large tables, ANALYZE for statistics, autovacuum tuning
- Archiving: Archive old data to S3, partition pruning for query performance, cold storage for historical data

20.4.3 Cache Optimization

- Redis: Redis Cluster for distributed caching, sharding for horizontal scaling, replication for read scaling
- Cache Strategy:
 - Cache-Aside: Read from cache, fallback to database, write to database and invalidate cache
 - Write-Through: Write to cache and database synchronously, read from cache
 - Write-Behind: Write to cache, async write to database, eventual consistency
- Cache Keys: Hierarchical keys, versioning, invalidation patterns, TTL-based expiration
- Cache Invalidation: Event-driven invalidation (Kafka), time-based TTL (60s-24h), manual invalidation for admin
- Cache Warming: Preload cache on startup, scheduled warming for popular data, on-demand warming for predicted data
- Cache Stampede Prevention: Cache locking, probabilistic early expiration, request coalescing, circuit breaker
- Distributed Cache: Redis Cluster for cross-instance caching, consistency across nodes, partition tolerance

20.4.4 Message Queue Optimization

- Kafka: Partitioning for parallelism, replication factor 3, batch processing, compression (Snappy/LZ4)
- Producer Optimization: Batch size (16KB), linger.ms (5ms), compression.type (snappy), acks (all), retries (3)
- Consumer Optimization: Batch processing (max.poll.records: 500), parallel processing, backpressure handling, idempotency
- Topic Design: Partition count (6-12 per topic), replication factor 3, retention policy (7 days for standard, 1 year for critical)
- Dead Letter Queue: DLQ for failed messages, retry with exponential backoff, max retries (5), manual processing for DLQ
- Monitoring: Consumer lag monitoring, throughput monitoring, error rate monitoring, latency monitoring

20.4.5 Search Optimization

- Elasticsearch: Shard optimization (5-20GB per shard), index templates, mapping optimization, query caching
- Index Design: Time-based indices for logs, rollover policies, index aliases, index lifecycle management
- Query Optimization: Query caching, filter caching, routing for single-shard queries, scroll for deep pagination
- Relevance Tuning: BM25 scoring, function scoring for boosting, custom analyzers, synonym filters
- Replication: Replica shards for read scaling, cross-cluster replication for disaster recovery, snapshot for backup
- Monitoring: Query latency, index rate, search rate, cache hit rate, JVM heap usage, garbage collection

20.4.6 Microservices Optimization

- Service Communication: gRPC for internal services (binary, HTTP/2), REST for external APIs, event-driven for async
- Circuit Breaker: Circuit breaker for external calls (timeout: 5s, failure threshold: 50%, recovery timeout: 30s)
- Bulkhead: Resource isolation per service, thread pool per dependency, preventing cascade failures
- Retry Logic: Exponential backoff with jitter (base: 100ms, max: 5s, max retries: 3), idempotency for retries
- Timeout Management: Request timeout (5s for standard, 10s for complex, 30s for reports), connection timeout (2s)
- Load Balancing: Round-robin for even distribution, least-connections for long-lived connections, weighted for capacity
- Service Mesh: Istio for traffic management, mTLS for security, circuit breaker, retry, timeout, rate limiting

20.5 Infrastructure Performance Optimization

20.5.1 Kubernetes Optimization

- Resource Requests & Limits: CPU requests (500m-2000m), memory requests (1Gi-8Gi), limits (2x requests), no CPU limits for burstable
- HPA: Horizontal Pod Autoscaler (CPU: 70%, memory: 80%, custom: queue depth, request latency, replica range: 2-20)
- VPA: Vertical Pod Autoscaler for right-sizing, recommendation mode, off-peak adjustment
- Cluster Autoscaler: Node autoscaling (min: 3, max: 50), scale-down delay (10m), priority-based scaling
- Pod Disruption Budgets: Min available (1 for standard, 2 for critical), max unavailable (25% for standard, 10% for critical)
- Topology Spread: Anti-affinity for pod distribution, zone spread, host spread, even distribution
- Node Pools: GPU nodes for ML, memory-optimized for cache, compute-optimized for API, burstable for batch
- Spot Instances: Spot instances for non-critical workloads (batch, analytics), on-demand for critical (API, payments)
- Preemption: Priority classes for critical pods, preemption for lower priority, guaranteed scheduling for critical
- Scheduling: Pod affinity/anti-affinity, node affinity, taints and tolerations, custom scheduler for ML workloads

20.5.2 Network Optimization

- VPC: VPC peering for cross-region, Transit Gateway for hub-and-spoke, PrivateLink for AWS services
- Load Balancing: ALB for HTTP/HTTPS (path-based, host-based), NLB for TCP/UDP (high performance, static IP)
- CDN: CloudFront for global CDN, edge caching, origin shield, dynamic content acceleration, image optimization
- DNS: Route 53 for DNS, latency-based routing, geolocation routing, health checks, failover routing

- Direct Connect: AWS Direct Connect for hybrid connectivity, dedicated bandwidth, reduced latency, increased security
- Global Accelerator: AWS Global Accelerator for static IP, anycast routing, reduced latency, health-based routing
- Service Mesh: Istio for service-to-service communication, mTLS, traffic management, observability, security
- Caching: ElastiCache for Redis (sub-millisecond latency), CloudFront for edge caching, application-level caching

20.5.3 Storage Optimization

- Object Storage: S3 for static assets, lifecycle policies (IA after 30 days, Glacier after 90 days), Intelligent-Tiering
- Block Storage: EBS for databases, gp3 for general purpose, io2 for high IOPS, provisioned IOPS for critical
- File Storage: EFS for shared storage, burst credits, provisioned throughput for high throughput
- Database Storage: RDS with gp3, storage auto-scaling, IOPS provisioning, read replicas for read scaling
- Backup: Automated backups (7 days), cross-region backup replication, point-in-time recovery, snapshot lifecycle
- Archiving: S3 Glacier for archival, lifecycle policies, automated archiving, retrieval policies
- Compression: Brotli/Gzip for text, WebP/AVIF for images, H.265/AV1 for video, Snappy/LZ4 for data
- Deduplication: Data deduplication for backups, content-addressable storage, incremental backups

20.5.4 Compute Optimization

- Auto Scaling: EC2 Auto Scaling for VM workloads, EKS Cluster Autoscaler for Kubernetes, Lambda for serverless
- Right Sizing: Regular right-sizing analysis (monthly), instance type optimization, reserved instances for steady state

- Spot Instances: Spot instances for fault-tolerant workloads (batch, analytics), Spot Fleet for diversification
- Reserved Instances: 1-year reserved for baseline, 3-year reserved for stable workloads, convertible for flexibility
- Savings Plans: Compute Savings Plans for flexible commitment, EC2 Savings Plans for EC2-specific commitment
- Container Optimization: Multi-tenant containers, resource sharing, bin packing, Kubernetes resource optimization
- Serverless: Lambda for event-driven, API Gateway for API, Step Functions for workflows, Fargate for containers
- GPU Optimization: GPU sharing for ML inference, GPU pooling for training, spot GPU for non-critical training

20.6 Performance Testing

20.6.1 Load Testing

- Tool: k6, Apache JMeter, Gatling, Locust
- Scenarios:
 - Baseline: 100 concurrent users, normal traffic patterns
 - Peak: 1000 concurrent users, peak traffic patterns
 - Stress: 5000 concurrent users, beyond expected capacity
 - Spike: Sudden spike from 100 to 5000 users in 1 minute
 - Soak: 500 concurrent users for 24 hours, memory leaks, connection leaks
- Metrics: Response time, throughput, error rate, CPU usage, memory usage, database connections, cache hit rate
- Environment: Staging environment (production-like), isolated test database, realistic test data
- Frequency: Before major releases, after infrastructure changes, quarterly capacity planning

20.6.2 Stress Testing

- Objective: Find breaking point, identify bottlenecks, test recovery, validate auto-scaling
- Method: Gradual load increase until failure, sudden spike to maximum capacity, sustained overload
- Monitoring: Real-time monitoring during test, resource utilization, error rates, response times, queue depths
- Analysis: Bottleneck identification, root cause analysis, capacity planning, infrastructure tuning
- Remediation: Fix identified bottlenecks, re-test, validate improvements, update capacity plans

20.6.3 Endurance Testing

- Objective: Detect memory leaks, connection leaks, resource exhaustion, degradation over time
- Duration: 24-72 hours of sustained load, continuous monitoring, periodic checks
- Metrics: Memory usage trend, connection pool size, file descriptor usage, disk space, CPU trend, response time trend
- Analysis: Trend analysis, anomaly detection, resource leak identification, long-term stability validation

20.6.4 Performance Monitoring

- Real-Time: Grafana dashboards for real-time performance metrics, P99 latency, throughput, error rate
- RUM (Real User Monitoring): Real user performance data, Core Web Vitals, user experience metrics, geographic performance
- Synthetic Monitoring: Synthetic tests for critical paths, uptime monitoring, API monitoring, page load monitoring
- APM: DataDog/New Relic for application performance, distributed tracing, profiling, error tracking
- Profiling: Continuous profiling for CPU, memory, I/O, lock contention, flame graphs for bottleneck identification

- Alerting: Performance degradation alerts, threshold-based alerts, anomaly detection, automated incident response

Chapter 21: Disaster Recovery & Business Continuity

21.1 Disaster Recovery Strategy

LocalConnect's disaster recovery strategy ensures business continuity in the face of catastrophic failures. The strategy follows the 3-2-1 backup rule and implements multi-region resilience with automated failover.

21.2 Backup Strategy

21.2.1 Database Backups (PostgreSQL)

- Automated Backups: Daily automated snapshots (retained for 7 days), weekly full backups (retained for 4 weeks), monthly full backups (retained for 12 months)
- Point-in-Time Recovery: Continuous WAL archiving to S3, point-in-time recovery to any second within retention period (7 days)
- Cross-Region Replication: Automated cross-region backup replication to secondary region (Sydney), daily replication, encryption in transit and at rest
- Backup Testing: Monthly backup restoration tests, automated restoration verification, integrity checks, test environment refresh from backup
- Backup Encryption: AES-256 encryption at rest, KMS key management, encryption in transit (TLS), key rotation every 90 days
- Backup Monitoring: Backup success/failure alerts, backup size monitoring, replication lag monitoring, storage cost monitoring

- Offsite Backup: Physical backup to offsite location (quarterly), air-gapped backup for ransomware protection, offline backup verification

21.2.2 DynamoDB Backups

- Point-in-Time Recovery: Enabled for all tables, 35-day recovery window, continuous backup, restore to any point in time
- On-Demand Backups: Manual backups before major changes, schema migrations, data imports, retained for indefinite period
- Cross-Region Replication: Global Tables for cross-region replication, automatic conflict resolution, eventual consistency
- Backup Automation: AWS Backup for centralized backup management, backup policies, cross-account backup, compliance reporting

21.2.3 S3 Data Backups

- Versioning: Enabled for all buckets, object versioning for accidental deletion, lifecycle policies for old versions
- Cross-Region Replication: S3 CRR to secondary region (Sydney), replication for critical data, replication for compliance
- Lifecycle Policies: Transition to IA after 30 days, Glacier after 90 days, Glacier Deep Archive after 1 year, deletion after 7 years (compliance)
- MFA Delete: MFA required for delete operations, additional protection for critical data, compliance requirement
- Object Lock: WORM (Write Once Read Many) for compliance data, governance mode for legal hold, compliance mode for regulatory
- Backup Monitoring: S3 CloudWatch metrics for replication, storage size, object count, cost monitoring, integrity checks

21.2.4 Elasticsearch Backups

- Snapshot Repository: S3 snapshot repository for Elasticsearch, daily snapshots, incremental backups, retention policy

- Snapshot Schedule: Daily snapshots at 2 AM UTC, weekly full snapshots, monthly archive snapshots, 30-day retention for daily, 1-year for weekly
- Cross-Cluster Replication: Cross-cluster replication for disaster recovery, automatic failover, leader-follower replication
- Restore Testing: Monthly restore test from snapshot, index verification, data integrity check, performance validation
- Backup Monitoring: Snapshot success/failure alerts, repository size monitoring, snapshot duration monitoring

21.2.5 Redis Backups

- RDB Snapshots: Every 15 minutes for persistence, AOF for durability (append-only file), RDB for fast recovery
- Replication: Redis replication for data redundancy, master-replica failover, read scaling via replicas
- Cross-Region Replication: Manual cross-region replication for critical data, scheduled sync, conflict resolution
- Backup Monitoring: RDB snapshot monitoring, AOF rewrite monitoring, replication lag monitoring, memory usage monitoring

21.2.6 Kafka Backups

- Log Retention: 7-day log retention for standard topics, 30-day for critical topics, 1-year for audit topics
- MirrorMaker 2: Cross-region replication via MirrorMaker 2, automatic failover, offset sync, topic sync
- Snapshot: Periodic snapshot of consumer offsets, topic configurations, ACLs, manual restore for configuration
- Backup Monitoring: Replication lag monitoring, log size monitoring, consumer offset monitoring, topic health monitoring

21.2.7 Configuration Backups

- Infrastructure as Code: All infrastructure defined in Terraform, stored in Git, versioned, peer-reviewed
- Git Backup: Git repository backup to S3, cross-region replication, GitHub backup, automated backup
- Secret Backup: AWS Secrets Manager backup, cross-region replication, manual backup for disaster recovery, encrypted
- Kubernetes State: etcd backup for EKS cluster state, Velero for Kubernetes resource backup, cross-region restore
- DNS Backup: Route 53 configuration backup, DNS record export, cross-region DNS failover configuration
- SSL Certificates: ACM certificate backup, Let's Encrypt backup, manual certificate backup for disaster recovery

21.3 Recovery Objectives

21.3.1 RTO (Recovery Time Objective)

- Critical Services (Payments, Emergency, Authentication): RTO < 15 minutes, automated failover, hot standby
- High-Priority Services (Orders, Search, Business): RTO < 1 hour, warm standby, automated failover with manual approval
- Standard Services (Events, Forum, Government, Analytics): RTO < 4 hours, warm standby, manual failover with runbook
- Background Services (Reports, Batch, Notifications): RTO < 24 hours, cold standby, manual recovery from backup
- Development Environment: RTO < 4 hours, Infrastructure as Code, automated provisioning from Terraform
- Staging Environment: RTO < 2 hours, Infrastructure as Code, automated provisioning from Terraform

21.3.2 RPO (Recovery Point Objective)

- Critical Services: RPO < 1 minute, synchronous replication, zero data loss for transactions, WAL archiving
- High-Priority Services: RPO < 5 minutes, asynchronous replication with minimal lag, point-in-time recovery
- Standard Services: RPO < 1 hour, daily backups, point-in-time recovery within 1 hour, cross-region replication
- Background Services: RPO < 24 hours, daily backups, acceptable data loss for non-critical data
- File Storage (S3): RPO < 1 hour, cross-region replication, versioning for accidental deletion, object lock for compliance
- Configuration: RPO < 1 hour, Git-based versioning, Terraform state backup, cross-region replication

21.4 Disaster Recovery Scenarios

21.4.1 Scenario 1: Primary Region Failure (Singapore)

- Detection: CloudWatch alarms, health checks, synthetic monitoring, manual detection
- Response: Automated failover for critical services, manual failover for standard services
- Failover Steps:
 - DNS failover to secondary region (Route 53 health checks, failover routing)
 - Promote read replicas in Sydney to primary (RDS, automated or manual)
 - Activate DynamoDB Global Tables (automatic)
 - Redirect API traffic to Sydney region (ALB, CloudFront)
 - Activate S3 cross-region replication (automatic)
 - Activate Redis replica in Sydney (manual promotion)
 - Activate Kafka MirrorMaker 2 in Sydney (manual)

- Update application configuration for Sydney region (automated via ConfigMap/Secret)
 - Verify all services in Sydney (health checks, smoke tests)
 - Communicate to users (status page, email, push notification)
- Recovery Time: Critical services < 15 minutes, high-priority < 1 hour, standard < 4 hours
- Data Loss: Critical < 1 minute (synchronous replication), high-priority < 5 minutes (async replication), standard < 1 hour
- Rollback: Reverse failover when Singapore recovers, validate data consistency, synchronize changes, DNS switch back

21.4.2 Scenario 2: Database Failure (Primary RDS Instance)

- Detection: RDS health checks, CloudWatch alarms, application error rates, connection failures
- Response: Automatic failover to standby (Multi-AZ), manual promotion of read replica if Multi-AZ fails
- Failover Steps:
 - RDS automatic failover to standby (Multi-AZ, 60-120 seconds)
 - If Multi-AZ fails, promote read replica to primary (manual, 5-10 minutes)
 - Update application connection strings (ConfigMap/Secret update, Kubernetes rolling update)
 - Verify database connectivity (health checks, smoke tests)
 - Monitor for replication lag, data consistency, performance degradation
- Recovery Time: < 2 minutes (Multi-AZ), < 10 minutes (read replica promotion)
- Data Loss: Zero (Multi-AZ synchronous replication), < 5 minutes (read replica async replication)
- Rollback: Automatic when primary recovers, manual fallback if needed

21.4.3 Scenario 3: Data Corruption (Logical Corruption)

- Detection: Data integrity checks, application error rates, user reports, anomaly detection, audit logs
- Response: Point-in-time recovery from backup, restore to specific point before corruption
- Recovery Steps:
 - Identify corruption point (timestamp, transaction ID, audit log analysis)
 - Stop write operations to affected database (maintenance mode, read-only)
 - Restore from point-in-time backup (RDS PITR, 5-30 minutes depending on size)
 - Verify data integrity (checksums, row counts, referential integrity, application tests)
 - Replay valid transactions after recovery point (binlog replay, application-level replay)
 - Verify application functionality (smoke tests, integration tests, user acceptance)
 - Resume write operations (maintenance mode off, traffic redirection)
 - Communicate to users (status page, email, downtime notification)
- Recovery Time: < 30 minutes for PITR, < 2 hours for full verification and replay
- Data Loss: < 5 minutes (PITR to specific point), zero if corruption detected immediately
- Prevention: Regular integrity checks, application-level validation, database constraints, audit logging, data quality monitoring

21.4.4 Scenario 4: Ransomware Attack

- Detection: File encryption alerts, unusual file access patterns, security monitoring, EDR alerts, user reports
- Response: Isolate affected systems, restore from clean backup, verify integrity, investigate root cause
- Recovery Steps:
 - Isolate affected systems (network isolation, access revocation, incident response team activation)
 - Assess scope of infection (affected systems, encrypted data, backup status, attack vector)
 - Restore from clean backup (verify backup integrity, restore to isolated environment, malware scan)
 - Verify data integrity (checksums, row counts, application tests, data quality checks)
 - Rebuild affected systems from Infrastructure as Code (Terraform, Kubernetes, automated provisioning)
 - Apply security patches (OS, application, dependencies, known vulnerabilities)
 - Restore from backup (database, S3, configuration, secrets, certificates)
 - Verify system functionality (smoke tests, security tests, penetration testing)
 - Monitor for reinfection (security monitoring, EDR, network monitoring, behavioral analysis)
 - Post-incident review (root cause analysis, lessons learned, process improvement, security hardening)
- Recovery Time: < 4 hours for critical services, < 24 hours for full recovery
- Data Loss: < 1 hour (point-in-time recovery), zero if backup is clean and recent
- Prevention: Air-gapped backups, MFA for all access, EDR, network segmentation, regular security training, principle of least privilege

21.4.5 Scenario 5: Complete Data Center Loss

- Detection: Data center outage, network failure, power failure, cooling failure, natural disaster
- Response: Full DR failover to secondary region, business continuity plan activation, crisis management
- Recovery Steps:
 - Activate crisis management team (incident commander, technical team, business team, communication team)
 - Full DR failover to secondary region (Sydney), all services, all data, all configurations
 - Verify all services in secondary region (health checks, smoke tests, integration tests, user acceptance)
 - Communicate to stakeholders (users, businesses, government, media, regulators)
 - Assess primary data center status (damage assessment, recovery timeline, rebuild vs. relocate)
 - Rebuild primary data center (if needed, new infrastructure, new region, Terraform provisioning)
 - Restore data to primary region (from secondary region, cross-region replication, data sync)
 - Verify primary region (health checks, smoke tests, data consistency, performance validation)
 - Gradual traffic migration back to primary (canary, blue-green, DNS cutover)
 - Post-incident review (lessons learned, DR plan improvements, infrastructure improvements, business continuity improvements)
- Recovery Time: < 4 hours for full DR failover, < 1 week for primary rebuild and migration
- Data Loss: < 5 minutes (asynchronous cross-region replication), zero for critical services (synchronous replication)
- Prevention: Multi-region architecture, regular DR drills, automated failover, Infrastructure as Code, air-gapped backups

21.5 Business Continuity Plan

21.5.1 Business Impact Analysis

- Critical Functions: Payment processing, emergency services, user authentication, order management, business search
- Dependencies: AWS services, third-party APIs, payment gateways, internet connectivity, staff availability
- Impact Assessment: Revenue impact, user impact, regulatory impact, reputation impact, operational impact
- Recovery Priorities: Critical (payments, emergency) > High (orders, search, business) > Standard (events, forum, government) > Low (reports, analytics, batch)
- MTD (Maximum Tolerable Downtime): Critical < 15 minutes, High < 1 hour, Standard < 4 hours, Low < 24 hours

21.5.2 Crisis Management

- Crisis Team: CEO (crisis commander), CTO (technical lead), CISO (security lead), COO (operations lead), Legal (compliance), Communications (PR)
- Communication Plan: Internal communication (Slack, email, phone), external communication (status page, email, social media, press release)
- Escalation: L1 (on-call engineer) → L2 (senior engineer) → L3 (tech lead) → Incident Commander → CTO → CEO
- Decision Making: Technical decisions by CTO, business decisions by CEO, legal decisions by Legal, communication by Communications
- Stakeholder Communication: Users (status page, email, push), Businesses (email, phone), Government (direct contact), Regulators (formal notification), Media (press release)
- Post-Crisis: Post-incident review, action items, process improvement, training update, DR plan update

21.5.3 Work Continuity

- Remote Work: All staff can work remotely (VPN, cloud tools, communication tools), remote work policy, equipment provision
- Alternative Sites: No physical office dependency, fully cloud-based, AWS multi-region for infrastructure
- Staff Redundancy: Cross-training for critical roles, backup personnel for on-call, documentation for all processes
- Supplier Continuity: Alternative suppliers for critical services (payment gateways, cloud providers, CDN, DNS), contract terms, SLA monitoring
- Customer Communication: Automated status updates, proactive communication, transparent communication, compensation policy for extended outages

21.6 Disaster Recovery Testing

21.6.1 DR Drill Schedule

- Tabletop Exercise: Quarterly tabletop exercise for crisis management, scenario walkthrough, decision-making practice, communication drill
- Component Testing: Monthly component testing (backup restoration, failover testing, replication testing), automated tests, manual verification
- Full DR Drill: Annual full DR drill, complete failover to secondary region, full service validation, full data verification, full rollback
- Unexpected Drill: Bi-annual unexpected drill (unannounced), test team response, test automation, test communication, test recovery
- Documentation: DR drill documentation, findings, action items, improvements, lessons learned, runbook updates

21.6.2 DR Test Scenarios

- Scenario 1: Primary region failure, full DR failover, validate all services, validate all data, rollback

- Scenario 2: Database corruption, point-in-time recovery, data integrity validation, application validation, rollback
- Scenario 3: Ransomware attack, backup restoration, system rebuild, security validation, monitoring
- Scenario 4: Network partition, split-brain resolution, data consistency, conflict resolution, recovery
- Scenario 5: Cascading failure, circuit breaker validation, auto-scaling validation, recovery validation, post-mortem

21.6.3 DR Test Metrics

- RTO Achievement: Actual recovery time vs. target, trend analysis, improvement tracking, root cause for misses
- RPO Achievement: Actual data loss vs. target, replication lag analysis, backup frequency validation, improvement tracking
- Test Success Rate: Percentage of successful DR tests, failure analysis, improvement tracking, pass/fail criteria
- Time to Detection: Mean time to detect disaster, monitoring effectiveness, alert effectiveness, improvement tracking
- Time to Recovery: Mean time to recover, step-by-step timing, bottleneck identification, improvement tracking
- Data Integrity: Data consistency after recovery, checksum validation, application validation, user acceptance

Chapter 22: Scalability Plan

22.1 Scalability Strategy

LocalConnect's scalability strategy ensures the platform can handle growth from 10,000 users to 10 million users without architectural changes. The strategy is based on horizontal scaling, auto-scaling, and capacity planning.

22.2 Capacity Planning

22.2.1 Current Capacity (Year 1)

- Users: 10,000 registered users, 1,000 daily active users
- Businesses: 500 registered businesses, 100 active businesses
- Orders: 100 orders/day, 10 concurrent orders
- Events: 50 events/month, 10 concurrent events
- API Requests: 10,000 requests/day, 100 requests/minute peak
- Storage: 100 GB (images, documents, logs)
- Database: 1 GB data, 100 queries/second
- Infrastructure: 3 Kubernetes nodes, 2 database instances, 1 Redis instance, 1 Elasticsearch instance

22.2.2 Year 2 Capacity (10x Growth)

- Users: 100,000 registered users, 10,000 daily active users
- Businesses: 5,000 registered businesses, 1,000 active businesses
- Orders: 1,000 orders/day, 100 concurrent orders
- Events: 500 events/month, 100 concurrent events
- API Requests: 100,000 requests/day, 1,000 requests/minute peak
- Storage: 1 TB (images, documents, logs)
- Database: 10 GB data, 1,000 queries/second
- Infrastructure: 10 Kubernetes nodes, 2 database instances (read replica), 3 Redis instances, 3 Elasticsearch instances, CDN

22.2.3 Year 3 Capacity (100x Growth)

- Users: 1,000,000 registered users, 100,000 daily active users
- Businesses: 50,000 registered businesses, 10,000 active businesses
- Orders: 10,000 orders/day, 1,000 concurrent orders
- Events: 5,000 events/month, 1,000 concurrent events
- API Requests: 1,000,000 requests/day, 10,000 requests/minute peak
- Storage: 10 TB (images, documents, logs)
- Database: 100 GB data, 10,000 queries/second

- Infrastructure: 30 Kubernetes nodes, 3 database instances (1 primary, 2 read replicas), 6 Redis instances (cluster), 9 Elasticsearch instances (3 nodes x 3 shards), multi-region CDN, Kafka cluster

22.2.4 Year 5 Capacity (1,000x Growth)

- Users: 10,000,000 registered users, 1,000,000 daily active users
- Businesses: 500,000 registered businesses, 100,000 active businesses
- Orders: 100,000 orders/day, 10,000 concurrent orders
- Events: 50,000 events/month, 10,000 concurrent events
- API Requests: 10,000,000 requests/day, 100,000 requests/minute peak
- Storage: 100 TB (images, documents, logs)
- Database: 1 TB data, 100,000 queries/second
- Infrastructure: 100+ Kubernetes nodes, 5 database instances (1 primary, 4 read replicas), 12 Redis instances (cluster), 15 Elasticsearch instances (5 nodes x 3 shards), multi-region CDN, Kafka cluster (10 brokers), multi-region deployment

22.3 Horizontal Scaling

22.3.1 Application Layer Scaling

- Kubernetes HPA: Horizontal Pod Autoscaler for all services, CPU target 70%, memory target 80%, custom metrics (queue depth, request latency)
- Replica Range: Minimum 2 replicas for high availability, maximum 50 replicas for peak load, average 10-20 replicas for normal load
- Auto-Scaling: CPU-based scaling (scale up at 70%, scale down at 40%), memory-based scaling, custom metrics scaling, predictive scaling
- Cluster Autoscaler: Node auto-scaling (minimum 3 nodes, maximum 100 nodes), scale-down delay (10 minutes), priority-based scaling
- Load Balancing: ALB for HTTP/HTTPS, NLB for TCP/UDP, Istio for service mesh, consistent hashing for session affinity

- Session Management: Stateless services (no session affinity), JWT for authentication, Redis for session storage (if needed), sticky sessions for WebSocket
- Stateless Design: All services stateless, external storage for state (Redis, PostgreSQL, S3), no local state, no sticky sessions

22.3.2 Database Scaling

- Read Replicas: 1-4 read replicas for read scaling, read/write splitting, load balancing across replicas, replication lag monitoring
- Connection Pooling: HikariCP connection pooling, max connections per instance (100), connection timeout (30s), idle timeout (10m)
- Partitioning: Time-based partitioning for large tables (orders, events, logs), hash partitioning for even distribution, list partitioning for categories
- Sharding: Horizontal sharding for user data (by user_id), business data (by business_id), geographic sharding (by region)
- Caching: Redis for query result caching (90% hit rate), application-level caching, CDN for static data, materialized views for reports
- NoSQL for Scale: DynamoDB for high-scale data (user sessions, notifications, analytics), global tables for cross-region, on-demand capacity
- Database per Service: Each microservice owns its database, independent scaling, schema isolation, technology freedom

22.3.3 Cache Scaling

- Redis Cluster: 6-12 Redis nodes for distributed caching, sharding for horizontal scaling, replication for read scaling, automatic failover
- CDN Scaling: CloudFront for global CDN, edge locations for low latency, origin shield for cache hit ratio, dynamic content acceleration
- Application Cache: In-memory caching per instance (Caffeine, Guava), cache warming, cache invalidation, cache eviction policies

- Cache Strategy: Multi-tier caching (CDN → Application → Redis → Database), cache-aside for reads, write-through for writes, write-behind for async
- Cache Invalidation: Event-driven invalidation (Kafka), TTL-based expiration, manual invalidation for admin, cache warming for popular data

22.3.4 Search Scaling

- Elasticsearch Scaling: 3-15 Elasticsearch nodes, shard optimization (5-20GB per shard), replica shards for read scaling, cross-cluster replication
- Index Sharding: 3-5 primary shards per index, 1-2 replica shards, routing for single-shard queries, index templates
- Index Lifecycle: Hot-warm-cold architecture, hot nodes for recent data, warm nodes for older data, cold nodes for archived data
- Query Optimization: Query caching, filter caching, routing, scroll for deep pagination, search templates, query optimization
- Alternative Search: Amazon OpenSearch for managed Elasticsearch, Algolia for hosted search, Meilisearch for lightweight search

22.3.5 Message Queue Scaling

- Kafka Scaling: 3-10 Kafka brokers, partition scaling (6-24 partitions per topic), replication factor 3, consumer groups for parallel processing
- Producer Scaling: Batch processing (16KB batch size, 5ms linger), compression (Snappy/LZ4), acks=all for durability, idempotency
- Consumer Scaling: Consumer groups with 10-50 consumers, partition assignment, backpressure handling, auto-scaling based on lag
- Topic Design: High partition count for parallelism, replication for durability, retention policy for storage management, compacted topics for state
- Alternative Messaging: Amazon SQS for simple queues, Amazon SNS for pub/sub, RabbitMQ for complex routing, NATS for lightweight messaging

22.3.6 Storage Scaling

- S3 Scaling: Unlimited storage, automatic scaling, lifecycle policies for cost optimization, cross-region replication, intelligent tiering
- EBS Scaling: gp3 volumes with provisioned IOPS, storage auto-scaling, io2 for high IOPS, volume expansion without downtime
- EFS Scaling: Automatic scaling, burst credits, provisioned throughput for high throughput, lifecycle management
- CDN Scaling: CloudFront automatic scaling, edge locations, origin shield, dynamic content acceleration, field-level encryption
- Database Storage: RDS storage auto-scaling, gp3 volumes, IOPS provisioning, read replicas for IOPS scaling, partitioning for query performance

22.4 Vertical Scaling

22.4.1 Compute Vertical Scaling

- Instance Types: c6i for compute-intensive, m6i for general purpose, r6i for memory-intensive, c6g for ARM-based cost optimization
- Right Sizing: Regular right-sizing analysis (monthly), instance type optimization, reserved instances for steady state, spot instances for burst
- GPU Scaling: g4dn for ML inference, p4d for ML training, GPU sharing for multi-tenant inference, GPU pooling for training
- Burstable Instances: t3/t4g for variable workloads, CPU credits, burst capability, cost optimization for low-average workloads

22.4.2 Memory Vertical Scaling

- Memory-Optimized Instances: r6i for memory-intensive services (cache, search), x2gd for in-memory databases, u-6tb1 for ultra-high memory

- Memory Tuning: JVM heap tuning (G1GC, ZGC), Node.js memory tuning (max-old-space-size), Python memory tuning, garbage collection optimization
- Swap Management: Minimal swap usage, swapiness tuning, OOM killer configuration, memory limits in Kubernetes
- Memory Caching: In-memory caching per instance, distributed caching (Redis), CDN caching, application-level caching

22.4.3 Network Vertical Scaling

- Network Optimization: Enhanced networking (ENA), placement groups for low latency, dedicated instances for network isolation
- Bandwidth Scaling: ALB for HTTP/HTTPS, NLB for TCP/UDP, CloudFront for CDN, Global Accelerator for static IP, Direct Connect for dedicated bandwidth
- Latency Optimization: Edge computing (CloudFront Functions, Lambda@Edge), regional deployment, latency-based routing, geolocation routing
- Throughput Optimization: HTTP/2 multiplexing, HTTP/3 QUIC, connection pooling, keep-alive, compression, batch requests

22.5 Multi-Region Scaling

22.5.1 Regional Expansion Strategy

- Phase 1 (Year 1): Single region (Singapore) for Sri Lanka market, primary region for all traffic
- Phase 2 (Year 2): Secondary region (Sydney) for disaster recovery, read replicas for read scaling, CDN for global acceleration
- Phase 3 (Year 3): Regional deployment for neighboring countries (India, Maldives), geolocation routing, data residency compliance

- Phase 4 (Year 5): Global deployment (Asia-Pacific, Middle East, Africa), edge computing, multi-region active-active, data sovereignty compliance

22.5.2 Multi-Region Architecture

- Active-Passive: Primary region active, secondary region passive (DR only), automated failover, DNS-based routing
- Active-Active: Multiple regions active, traffic distribution by geolocation, data replication, conflict resolution
- Read-Only Replicas: Read replicas in secondary regions, write to primary region, read from nearest region, replication lag tolerance
- Data Residency: Data stored in user's region, cross-border data transfer restrictions, local database instances, local CDN
- Global Load Balancing: Route 53 geolocation routing, latency-based routing, health checks, failover routing, weighted routing
- Cross-Region Replication: RDS cross-region read replicas, DynamoDB global tables, S3 cross-region replication, Redis replication, Kafka MirrorMaker 2

22.6 Elasticity

22.6.1 Auto-Scaling Policies

- CPU-Based Scaling: Scale up at 70% CPU utilization, scale down at 40%, cooldown period 5 minutes, step scaling (add 2, remove 1)
- Memory-Based Scaling: Scale up at 80% memory utilization, scale down at 50%, cooldown period 5 minutes, step scaling
- Custom Metrics Scaling: Scale based on request queue depth, request latency, error rate, business metrics (order rate, user registration)
- Predictive Scaling: Machine learning-based predictive scaling, forecast based on historical patterns, scheduled scaling for known events

- Scheduled Scaling: Scale up before known events (holidays, sales, events), scale down after events, cron-based scheduling
- Target Tracking: Target tracking for CPU (70%), memory (80%), request count per target (1000), response time (200ms)

22.6.2 Serverless Scaling

- Lambda Scaling: Automatic scaling from 0 to 10,000 concurrent executions, provisioned concurrency for low latency, reserved concurrency for capacity
- API Gateway Scaling: Automatic scaling to 10,000 requests/second per region, throttling limits, burst limits, usage plans
- Fargate Scaling: Serverless container scaling, task count auto-scaling, CPU/memory-based scaling, spot capacity for cost optimization
- DynamoDB Scaling: On-demand capacity for automatic scaling, provisioned capacity for predictable workloads, auto-scaling for throughput
- SQS Scaling: Automatic scaling based on queue depth, visibility timeout, message retention, dead letter queue for failed messages

22.6.3 Cost Optimization

- Right Sizing: Regular instance right-sizing, instance type optimization, reserved instances for steady state, savings plans for flexible commitment
- Spot Instances: Spot instances for fault-tolerant workloads (batch, analytics, CI/CD), Spot Fleet for diversification, capacity-optimized allocation
- Auto-Scaling: Scale down during low traffic, scale up during peak, scheduled scaling for known patterns, predictive scaling for cost optimization
- Storage Optimization: S3 Intelligent-Tiering, EBS gp3, lifecycle policies, compression, deduplication, archival to Glacier

- Database Optimization: Read replicas for read scaling, caching for query optimization, partitioning for query performance, serverless for variable workloads
- CDN Optimization: CloudFront caching, origin shield, image optimization, dynamic content acceleration, field-level encryption
- Monitoring: Cost monitoring with AWS Cost Explorer, budget alerts, anomaly detection, cost allocation tags, chargeback/showback

22.7 Scalability Testing

22.7.1 Load Testing at Scale

- Tool: k6, Apache JMeter, Gatling, Locust, AWS Load Testing (managed)
- Scenarios:
 - Baseline: 100 concurrent users, normal traffic
 - Peak: 10,000 concurrent users, 10x normal traffic
 - Stress: 100,000 concurrent users, 100x normal traffic
 - Spike: Sudden spike from 100 to 50,000 users in 1 minute
 - Soak: 10,000 concurrent users for 72 hours
- Metrics: Response time, throughput, error rate, CPU usage, memory usage, database connections, cache hit rate, queue depth
- Environment: Production-like staging environment, isolated test database, realistic test data, load injection from multiple regions
- Frequency: Before major releases, after infrastructure changes, quarterly capacity planning, annual scalability validation

22.7.2 Chaos Engineering

- Tool: Chaos Monkey, Gremlin, Litmus, AWS Fault Injection Simulator
- Scenarios:
 - Instance Failure: Random termination of EC2 instances, Kubernetes pods, database instances

- Network Failure: Network latency injection, packet loss, network partition, DNS failure
- Dependency Failure: Simulate third-party API failure, database failure, cache failure, message queue failure
- Resource Exhaustion: CPU exhaustion, memory exhaustion, disk exhaustion, connection pool exhaustion
- Regional Failure: Simulate entire region failure, DNS failover, cross-region failover, data replication validation
- Monitoring: Real-time monitoring during chaos experiments, automated rollback, safety checks, blast radius control
- Analysis: Failure impact analysis, recovery time measurement, resilience validation, improvement identification
- Frequency: Monthly chaos experiments, quarterly full chaos days, annual disaster recovery validation

Chapter 23: Cost Estimation & Budgeting

23.1 Cost Strategy

LocalConnect's cost strategy balances performance, reliability, and cost efficiency. The goal is to optimize cloud spending while maintaining 99.9% availability and excellent user experience.

23.2 Infrastructure Cost Estimate (Year 1)

23.2.1 Compute Costs

Resource	Instance Type	Count	Monthly Cost	Annual Cost
EKS Worker Nodes (API Services)	m6i.large	3	\$140	\$1,680

EKS Worker Nodes (Background)	c6i.large	2	\$100	\$1,200
EKS Worker Nodes (Search)	m6i.xlarge	2	\$280	\$3,360
EKS Worker Nodes (ML)	g4dn.xlarge	1	\$500	\$6,000
Bastion/Jump Host	t3.micro	1	\$15	\$180
CI/CD Runners	c6i.large	1	\$70	\$840
Compute Subtotal	\$1,105	\$13,260		

23.2.2 Database Costs

Resource	Instance Type	Count	Monthly Cost	Annual Cost
RDS PostgreSQL (Primary)	db.r6g.large	1	\$350	\$4,200
RDS PostgreSQL (Read Replica)	db.r6g.large	1	\$350	\$4,200
RDS PostgreSQL (Standby)	db.r6g.large	1	\$350	\$4,200
DynamoDB (On-Demand)	-	-	\$200	\$2,400
ElastiCache Redis (Primary)	cache.r6g.large	1	\$200	\$2,400
ElastiCache Redis (Replica)	cache.r6g.large	1	\$200	\$2,400
Elasticsearch	m6g.large	3	\$450	\$5,400
Database Subtotal	\$2,100	\$25,200		

23.2.3 Storage Costs

Resource	Size	Monthly Cost	Annual Cost
S3 (Images, Documents, Logs)	500 GB	\$50	\$600
S3 (Backups)	200 GB	\$20	\$240
EBS (Database)	500 GB gp3	\$60	\$720
EBS (Application)	200 GB gp3	\$24	\$288

EFS (Shared Storage)	100 GB	\$30	\$360
Storage Subtotal	\$184	\$2,208	

23.2.4 Network Costs

Resource	Monthly Cost	Annual Cost
ALB (Application Load Balancer)	\$50	\$600
NLB (Network Load Balancer)	\$30	\$360
CloudFront (CDN)	\$100	\$1,200
NAT Gateway	\$100	\$1,200
Data Transfer (Out)	\$150	\$1,800
Route 53	\$50	\$600
Network Subtotal	\$480	\$5,760

23.2.5 Security & Other Costs

Resource	Monthly Cost	Annual Cost
AWS WAF	\$50	\$600
AWS Shield (Standard)	\$0	\$0
AWS Shield (Advanced)	\$300	\$3,600
AWS Secrets Manager	\$20	\$240
AWS KMS	\$10	\$120
AWS Certificate Manager	\$0	\$0
AWS CloudTrail	\$20	\$240
AWS Config	\$30	\$360
AWS GuardDuty	\$50	\$600
AWS Inspector	\$30	\$360
AWS Security Hub	\$20	\$240

Security Subtotal	\$530	\$6,360
-------------------	-------	---------

23.2.6 DevOps & Tooling Costs

Resource	Monthly Cost	Annual Cost
GitHub (Team Plan)	\$20	\$240
GitHub Actions (CI/CD)	\$50	\$600
Jira (Standard Plan)	\$50	\$600
Confluence (Standard Plan)	\$30	\$360
Slack (Business+)	\$50	\$600
DataDog (APM)	\$500	\$6,000
Sentry (Error Tracking)	\$50	\$600
Terraform Cloud	\$50	\$600
AWS CloudWatch	\$100	\$1,200
DevOps Subtotal	\$900	\$10,800

23.2.7 Third-Party Service Costs

Resource	Monthly Cost	Annual Cost
Stripe (Payment Processing)	Variable (2.9% + \$0.30/transaction)	Variable
Twilio (SMS)	Variable	Variable
SendGrid (Email)	\$50	\$600
Google Maps API	\$100	\$1,200
Firebase (Push Notifications)	\$50	\$600
Cloudflare (DNS, CDN)	\$50	\$600
PagerDuty (Incident Management)	\$50	\$600

Third-Party Subtotal	\$300	\$3,600
----------------------	-------	---------

23.2.8 Year 1 Total Infrastructure Cost

Category	Monthly Cost	Annual Cost
Compute	\$1,105	\$13,260
Database	\$2,100	\$25,200
Storage	\$184	\$2,208
Network	\$480	\$5,760
Security	\$530	\$6,360
DevOps & Tooling	\$900	\$10,800
Third-Party Services	\$300	\$3,600
Total	\$5,699	\$68,388

23.3 Year 2-5 Cost Projections

23.3.1 Year 2 Cost Projection (10x Growth)

Category	Monthly Cost	Annual Cost
Compute	\$3,000	\$36,000
Database	\$6,000	\$72,000
Storage	\$500	\$6,000
Network	\$1,500	\$18,000
Security	\$1,000	\$12,000
DevOps & Tooling	\$1,500	\$18,000
Third-Party Services	\$1,000	\$12,000
Total	\$14,500	\$174,000

23.3.2 Year 3 Cost Projection (100x Growth)

Category	Monthly Cost	Annual Cost
Compute	\$8,000	\$96,000
Database	\$15,000	\$180,000
Storage	\$2,000	\$24,000
Network	\$5,000	\$60,000
Security	\$3,000	\$36,000
DevOps & Tooling	\$3,000	\$36,000
Third-Party Services	\$5,000	\$60,000
Total	\$41,000	\$492,000

23.3.3 Year 5 Cost Projection (1,000x Growth)

Category	Monthly Cost	Annual Cost
Compute	\$25,000	\$300,000
Database	\$50,000	\$600,000
Storage	\$10,000	\$120,000
Network	\$20,000	\$240,000
Security	\$10,000	\$120,000
DevOps & Tooling	\$10,000	\$120,000
Third-Party Services	\$20,000	\$240,000
Total	\$145,000	\$1,740,000

23.4 Cost Optimization Strategies

23.4.1 Reserved Instances & Savings Plans

- Reserved Instances: 1-year reserved for baseline compute (30% savings), 3-year reserved for stable workloads (50% savings)

- Savings Plans: Compute Savings Plans for flexible commitment (20-30% savings), EC2 Savings Plans for EC2-specific (30-50% savings)
- Database Reserved: RDS reserved instances for database (30-50% savings), ElastiCache reserved for cache (30-50% savings)
- Commitment: Start with 1-year reserved for Year 1, evaluate 3-year for Year 2 based on growth stability

23.4.2 Spot Instances

- Spot Instances: 60-90% savings for fault-tolerant workloads (CI/CD, batch processing, analytics, development)
- Spot Fleet: Diversified instance types for Spot capacity, automatic fallback to on-demand, capacity-optimized allocation
- Spot for Kubernetes: Spot nodes for non-critical workloads (batch, analytics), on-demand nodes for critical (API, payments)
- Spot Interruption Handling: Graceful interruption handling, checkpointing for long-running jobs, automatic retry

23.4.3 Storage Optimization

- S3 Intelligent-Tiering: Automatic cost optimization for objects with unknown access patterns, no retrieval fees
- S3 Lifecycle: Transition to IA after 30 days, Glacier after 90 days, Glacier Deep Archive after 1 year, deletion after 7 years
- EBS gp3: Replace gp2 with gp3 for better performance and cost, independent IOPS and throughput provisioning
- Compression: Brotli/Gzip for text, WebP/AVIF for images, H.265/AV1 for video, Snappy/LZ4 for data
- Deduplication: Data deduplication for backups, content-addressable storage, incremental backups
- Archival: Archive old data to Glacier, lifecycle policies, automated archiving, retrieval policies

23.4.4 Database Optimization

- Read Replicas: Scale reads with read replicas instead of larger instances, read/write splitting, load balancing
- Caching: Redis for query result caching (90% hit rate), application-level caching, CDN for static data, materialized views
- Partitioning: Time-based partitioning for large tables, partition pruning for query performance, archive old partitions
- Serverless: DynamoDB on-demand for variable workloads, Aurora Serverless for auto-scaling, pay-per-use for low traffic
- Right Sizing: Regular right-sizing analysis, instance type optimization, storage optimization, IOPS optimization

23.4.5 Network Optimization

- CDN: CloudFront for global CDN, edge caching, origin shield, image optimization, dynamic content acceleration
- Data Transfer: Minimize cross-region data transfer, VPC endpoints for AWS services, Direct Connect for hybrid, compression
- Load Balancing: ALB for HTTP/HTTPS, NLB for TCP/UDP, path-based routing, health checks, SSL termination
- DNS: Route 53 for DNS, latency-based routing, geolocation routing, health checks, failover routing

23.4.6 Monitoring & Governance

- Cost Monitoring: AWS Cost Explorer, budget alerts, anomaly detection, cost allocation tags, chargeback/showback
- Budget Alerts: 80% budget alert, 100% budget alert, anomaly detection for unexpected spikes, weekly cost reports
- Cost Allocation: Resource tagging strategy, cost centers per team, environment-based cost allocation, project-based allocation

- FinOps: Cloud Financial Management, cost optimization team, regular cost reviews, optimization recommendations, cost culture
- Right Sizing: Monthly right-sizing review, instance type optimization, storage optimization, database optimization
- Reserved Instance Planning: Quarterly RI planning, utilization monitoring, coverage analysis, flexibility analysis

23.5 Cost Budgeting

23.5.1 Budget Allocation

- Infrastructure: 60% of budget (compute, database, storage, network)
- Security: 10% of budget (WAF, Shield, GuardDuty, Inspector, KMS, certificates)
- DevOps & Tooling: 15% of budget (CI/CD, monitoring, logging, alerting, collaboration tools)
- Third-Party Services: 10% of budget (payments, SMS, email, maps, push notifications)
- Contingency: 5% of budget for unexpected costs, overruns, emergency scaling, unplanned incidents

23.5.2 Budget Tracking

- Monthly Review: Monthly cost review meeting, cost analysis, variance analysis, optimization opportunities, budget adjustment
- Quarterly Review: Quarterly cost planning, budget forecasting, capacity planning, RI planning, savings plan evaluation
- Annual Review: Annual budget planning, cost projection, capacity planning, technology evaluation, vendor negotiation
- Real-Time Monitoring: AWS Cost Explorer dashboard, real-time cost tracking, alert for budget thresholds, anomaly detection
- Cost Reports: Weekly cost reports for leadership, monthly cost reports for finance, quarterly cost reports for board

23.5.3 Cost Optimization Roadmap

- Month 1-3: Baseline cost measurement, tagging strategy, budget setup, cost monitoring setup, initial optimization
- Month 4-6: Reserved instance planning, savings plan evaluation, spot instance adoption, storage optimization, right sizing
- Month 7-12: Cost optimization review, FinOps practice, automation for cost optimization, cost culture, continuous improvement
- Year 2+: Advanced cost optimization, multi-cloud evaluation, vendor negotiation, cost optimization automation, predictive cost modeling

Chapter 24: Development Roadmap & Milestones

24.1 Development Phases

24.1.1 Phase 1: Foundation (Months 1-3)

- Goal: Establish core infrastructure, build foundational services, launch MVP with basic features
- Infrastructure: Set up AWS environment, Kubernetes cluster, CI/CD pipeline, monitoring, security baseline
- Services: User Service, Business Service, Authentication Service, API Gateway, Search Service
- Features: User registration, business registration, search, basic profiles, JWT authentication, PostgreSQL database, Redis cache
- Deliverables: MVP with core functionality, infrastructure in production, CI/CD pipeline, monitoring setup, security baseline
- Team: 2 backend engineers, 1 frontend engineer, 1 DevOps engineer, 1 QA engineer
- Success Criteria: MVP deployed to production, 100 test users, 50 test businesses, 99% uptime, core features functional

24.1.2 Phase 2: Core Features (Months 4-6)

- Goal: Build core community features, expand business capabilities, improve user experience
- Services: Order Service, Event Service, Forum Service, Notification Service, Review Service
- Features: Order management, event creation and booking, community forum, push notifications, review system, advanced search, payment integration (Stripe)
- Deliverables: Core feature set complete, payment processing live, event management live, community features live, mobile app beta
- Team: 3 backend engineers, 2 frontend engineers, 1 DevOps engineer, 1 QA engineer, 1 mobile engineer
- Success Criteria: 1,000 registered users, 100 active businesses, 50 orders/day, 10 events/week, 99.5% uptime

24.1.3 Phase 3: Growth & Scale (Months 7-9)

- Goal: Scale platform, add advanced features, expand to new cities, improve performance
- Services: Government Service, Analytics Service, Emergency Service, ML Service, Payment Service
- Features: Government service integration, real-time analytics, emergency SOS feature, ML recommendations, advanced payment features, multi-city support
- Deliverables: Government features live, analytics dashboard, emergency features, ML recommendations, performance optimization, multi-city support
- Team: 4 backend engineers, 3 frontend engineers, 2 DevOps engineers, 2 QA engineers, 2 mobile engineers, 1 data scientist, 1 product manager
- Success Criteria: 10,000 registered users, 500 active businesses, 100 orders/day, 50 events/week, 99.9% uptime, < 200ms average response time

24.1.4 Phase 4: Enterprise & Expansion (Months 10-12)

- Goal: Add enterprise features, expand internationally, build partner ecosystem, achieve profitability
- Services: Enterprise Service, Partner Service, Advanced Analytics, API for Partners, White-Label Solution
- Features: Enterprise business accounts, partner API, white-label solution, advanced reporting, multi-language support, international expansion, compliance features
- Deliverables: Enterprise features live, partner API, white-label solution, international launch, compliance certification, profitability milestone
- Team: 5 backend engineers, 4 frontend engineers, 3 DevOps engineers, 3 QA engineers, 3 mobile engineers, 2 data scientists, 1 product manager, 1 business development, 1 customer success
- Success Criteria: 50,000 registered users, 2,000 active businesses, 500 orders/day, 200 events/week, 99.95% uptime, < 150ms average response time, revenue positive

24.2 Milestone Timeline

24.2.1 Month 1-2: Foundation

- Week 1-2: Infrastructure setup (AWS, Kubernetes, CI/CD, monitoring)
- Week 3-4: Core services development (User, Business, Authentication)
- Week 5-6: API development, database schema, search integration
- Week 7-8: Frontend development, mobile app setup, testing
- Deliverable: MVP with basic features, infrastructure ready, CI/CD pipeline live

24.2.2 Month 3-4: MVP Launch

- Week 9-10: Payment integration (Stripe), order management, basic events
- Week 11-12: Community forum, notification system, review system
- Week 13-14: Performance optimization, security hardening, beta testing
- Week 15-16: MVP launch to 100 test users, feedback collection, iteration
- Deliverable: MVP live, 100 test users, 50 test businesses, feedback loop established

24.2.3 Month 5-6: Core Features

- Week 17-18: Advanced search, event management, booking system
- Week 19-20: Mobile app development, push notifications, offline support
- Week 21-22: Government service integration, emergency features, analytics
- Week 23-24: Performance testing, load testing, security audit, launch preparation
- Deliverable: Core features live, 1,000 users, 100 businesses, 99.5% uptime

24.2.4 Month 7-9: Scale & Growth

- Week 25-28: ML recommendations, advanced analytics, multi-city support
- Week 29-32: Performance optimization, caching, CDN, auto-scaling
- Week 33-36: Security enhancements, compliance, DR setup, monitoring
- Deliverable: 10,000 users, 500 businesses, 99.9% uptime, < 200ms response time

24.2.5 Month 10-12: Enterprise & Expansion

- Week 37-40: Enterprise features, partner API, white-label solution
- Week 41-44: International expansion, multi-language, compliance

- Week 45-48: Advanced features, AI/ML enhancements, community growth
- Deliverable: 50,000 users, 2,000 businesses, 99.95% uptime, revenue positive

24.3 Feature Prioritization

24.3.1 Must-Have (P0) - Critical for MVP

- User registration and authentication (JWT, OAuth2, social login)
- Business registration and profiles (CRUD, verification, categories)
- Search and discovery (Elasticsearch, geolocation, filtering)
- Order management (create, track, cancel, Stripe integration)
- Payment processing (Stripe, refunds, webhooks, receipts)
- Basic event management (create, list, register, basic notifications)
- Community forum (CRUD, moderation, basic search)
- Admin dashboard (user management, business verification, basic analytics)
- Security (HTTPS, JWT, rate limiting, input validation, WAF)
- Mobile app (iOS and Android, core features, push notifications)

24.3.2 Should-Have (P1) - Important for Growth

- Advanced search (faceted search, recommendations, auto-complete)
- Real-time features (WebSocket, live updates, real-time notifications)
- Social features (sharing, following, activity feed, mentions)
- Advanced event management (ticketing, waitlist, recurring events, check-in)
- Government service integration (service directory, appointments, forms)
- Emergency SOS feature (location sharing, emergency contacts, authorities)
- Analytics and reporting (business analytics, user analytics, custom reports)

- ML recommendations (business recommendations, content recommendations, search ranking)
- Advanced payment features (subscriptions, invoicing, split payments, escrow)
- Multi-language support (Sinhala, Tamil, English, i18n)

24.3.3 Nice-to-Have (P2) - Differentiation

- AR/VR features (AR business cards, virtual tours, AR navigation)
- AI chatbot (customer support, business assistant, FAQ, natural language)
- Blockchain integration (digital identity, smart contracts, verifiable credentials)
- Advanced analytics (predictive analytics, cohort analysis, funnel analysis, A/B testing)
- White-label solution (custom branding, custom domain, API access)
- Partner ecosystem (partner API, marketplace, integrations, affiliate program)
- Community gamification (badges, points, leaderboards, achievements, challenges)
- Advanced mobile features (offline mode, background sync, widgets, shortcuts, deep linking)
- Video features (live streaming, video conferencing, video reviews, video ads)
- IoT integration (smart devices, sensors, beacons, location-based triggers)

24.4 Risk Management

24.4.1 Technical Risks

- Risk: Performance issues at scale

- Mitigation: Performance testing from Day 1, caching strategy, auto-scaling, CDN, database optimization, load testing
- Risk: Security vulnerabilities
 - Mitigation: Security-first design, OWASP compliance, regular security audits, penetration testing, bug bounty program, WAF
- Risk: Third-party service failures (Stripe, Twilio, AWS)
 - Mitigation: Circuit breakers, fallback mechanisms, multi-provider strategy, health checks, graceful degradation
- Risk: Data loss or corruption
 - Mitigation: Multi-layer backup, point-in-time recovery, cross-region replication, data integrity checks, DR testing
- Risk: Technical debt accumulation
 - Mitigation: Code reviews, refactoring sprints, architecture documentation, tech debt tracking, 20% time for refactoring

24.4.2 Business Risks

- Risk: Low user adoption
 - Mitigation: Market research, user feedback loop, iterative development, marketing strategy, community building, referral program
- Risk: Competition from established platforms
 - Mitigation: Local focus, community-driven, government partnership, unique features, superior UX, customer service
- Risk: Regulatory changes (GDPR, data protection, payment regulations)
 - Mitigation: Compliance-first design, legal review, data residency, regular compliance audits, regulatory monitoring
- Risk: Revenue shortfall
 - Mitigation: Diverse revenue streams, cost optimization, investor funding, government grants, partnership revenue
- Risk: Team attrition
 - Mitigation: Competitive compensation, equity, culture, remote work, professional development, team building, knowledge sharing

24.4.3 Operational Risks

- Risk: Infrastructure failure
 - Mitigation: Multi-AZ, auto-scaling, health checks, DR plan, backup strategy, monitoring, incident response
- Risk: Security breach
 - Mitigation: Security monitoring, incident response plan, regular audits, encryption, access control, security training
- Risk: Compliance violations
 - Mitigation: Compliance framework, regular audits, legal review, data protection, privacy by design, documentation
- Risk: Vendor lock-in
 - Mitigation: Multi-cloud strategy, open standards, containerization, API abstraction, exit strategy, data portability
- Risk: Scaling challenges
 - Mitigation: Capacity planning, load testing, auto-scaling, performance monitoring, architecture reviews, scalability testing

Chapter 25: Coding Standards & Best Practices

25.1 Code Quality Standards

25.1.1 Code Review Requirements

- Mandatory Reviews: All code changes require at least one peer review before merging, senior engineer approval for critical changes, architecture review for major changes
- Review Criteria: Code correctness, performance, security, readability, test coverage, documentation, adherence to standards, backward compatibility

- Review Tools: GitHub Pull Requests, CodeRabbit AI for automated reviews, SonarQube for quality gates, ESLint/Prettier for formatting, Checkstyle for Java
- Review SLA: Reviews completed within 24 hours for standard changes, 4 hours for critical bug fixes, 1 hour for production incidents
- Review Culture: Constructive feedback, knowledge sharing, mentoring, no blame culture, learning opportunities, recognition for quality reviews
- Bypass Policy: No bypass for production changes, emergency bypass requires CTO approval and post-incident review, documented exception

25.1.2 Code Style Guidelines

- Java (Spring Boot): Google Java Style Guide, Checkstyle configuration, SpotBugs for bug patterns, PMD for code quality, 4-space indentation, 120 character line limit
- JavaScript/TypeScript (React/Node.js): Airbnb JavaScript Style Guide, ESLint configuration, Prettier for formatting, 2-space indentation, 100 character line limit, semicolons required
- Python (ML/Analytics): PEP 8, Black for formatting, isort for import sorting, mypy for type checking, 4-space indentation, 88 character line limit
- Go (Microservices): gofmt, golint, go vet, effective Go guidelines, tabs for indentation, 120 character line limit
- Kotlin (Android): Kotlin Coding Conventions, ktlint for formatting, detekt for static analysis, 4-space indentation, 120 character line limit
- Swift (iOS): Swift API Design Guidelines, SwiftLint for linting, SwiftFormat for formatting, 4-space indentation, 120 character line limit
- Naming Conventions: camelCase for variables and functions, PascalCase for classes and components, SCREAMING_SNAKE_CASE for constants, descriptive names, no abbreviations, English names
- Documentation: Javadoc for Java, JSDoc for JavaScript, docstrings for Python, godoc for Go, KDoc for Kotlin, all public APIs documented, README for each module

25.1.3 Code Quality Metrics

- Test Coverage: Minimum 80% line coverage for all services, 90% for critical services (payments, authentication), 70% for integration tests, 60% for E2E tests
- Code Complexity: Cyclomatic complexity < 15 per function, cognitive complexity < 20 per function, class size < 500 lines, method size < 50 lines
- Technical Debt: SonarQube debt ratio < 5%, zero critical issues, zero security vulnerabilities, code smell tracking, monthly debt reduction sprint
- Bug Density: < 1 bug per 1000 lines of code, post-release defect tracking, root cause analysis, defect prevention, quality metrics dashboard
- Code Churn: < 20% code churn per sprint, stable code base, refactoring tracking, code stability metrics, architecture health monitoring

25.1.4 Static Analysis

- Tools: SonarQube for all languages, ESLint for JavaScript, Checkstyle/SpotBugs/PMD for Java, detekt for Kotlin, SwiftLint for Swift, mypy for Python, go vet for Go
- Quality Gates: All static analysis checks must pass before merge, zero critical issues, zero security vulnerabilities, code coverage gate, duplication gate (< 3%)
- Integration: Integrated with CI/CD pipeline, automatic analysis on PR, quality gate blocking, trend analysis, team metrics
- Rules: OWASP Top 10 security rules, CWE rules, custom rules for business logic, performance rules, maintainability rules
- Reporting: Weekly quality reports, team quality dashboards, individual quality metrics, improvement tracking, quality goals

25.2 Testing Standards

25.2.1 Unit Testing

- Framework: JUnit 5 for Java, Jest for JavaScript, pytest for Python, Go testing for Go, XCTest for Swift, JUnit for Kotlin
- Coverage: Minimum 80% line coverage, 100% for critical paths, branch coverage > 70%, mutation testing for test quality
- Standards: One test class per production class, test naming: should[ExpectedBehavior]When[Condition], Given-When-Then structure, Arrange-Act-Assert pattern
- Mocking: Mockito for Java, Jest mocks for JavaScript, unittest.mock for Python, gomock for Go, mockito for Kotlin, mock interfaces not implementations
- Isolation: Tests must be independent, no shared state, no external dependencies (mock all), in-memory database for integration tests, test containers for infrastructure
- Performance: Unit tests must run in < 1 second, parallel execution, test suite < 5 minutes, no I/O in unit tests, no network calls
- Documentation: Test cases document business rules, test names describe behavior, test data builders, test fixtures, shared test utilities

25.2.2 Integration Testing

- Scope: Service-to-service integration, database integration, cache integration, message queue integration, third-party API integration
- Framework: Spring Boot Test for Java, Supertest for Node.js, pytest for Python, Postman/Newman for API testing, TestContainers for infrastructure
- Environment: Test environment with real dependencies (Docker Compose), test databases (PostgreSQL, Redis, Elasticsearch), test message queues (Kafka), test external APIs (WireMock)
- Data: Test data factories, seeded test data, isolated test data per test, database cleanup after test, transaction rollback

- Standards: Test critical integration paths, happy path and error paths, timeout testing, retry testing, circuit breaker testing
- Performance: Integration tests < 10 seconds, test suite < 30 minutes, parallel execution where possible, resource cleanup
- Coverage: All API endpoints tested, all database operations tested, all message queue flows tested, all third-party integrations tested

25.2.3 End-to-End Testing

- Scope: Full user journey testing, cross-service testing, UI testing, mobile testing, critical path testing
- Framework: Cypress for web, Playwright for web, Appium for mobile, Selenium for legacy, Karate for API E2E
- Environment: Staging environment identical to production, production-like data, production-like load, production-like configuration
- Data: Test user accounts, test businesses, test orders, test events, seeded test data, data cleanup after test, test data isolation
- Scenarios:
 - User registration and login flow
 - Business registration and profile creation
 - Search and discovery flow
 - Order creation and payment flow
 - Event creation and booking flow
 - Community forum interaction flow
 - Admin dashboard operations
 - Emergency SOS flow
- Standards: Critical user journeys only, < 50 E2E tests, < 5 minutes per test, headless execution, parallel execution, visual regression testing
- Maintenance: Page Object Model (POM), test data management, test environment management, flaky test detection, test stability monitoring

25.2.4 Performance Testing

- Scope: Load testing, stress testing, spike testing, soak testing, scalability testing, capacity testing
- Tool: k6, Apache JMeter, Gatling, Locust, AWS Load Testing, Artillery, Vegeta
- Metrics: Response time (p50, p95, p99), throughput (RPS), error rate, CPU usage, memory usage, database connections, cache hit rate, queue depth
- Scenarios:
 - Baseline: 100 concurrent users, normal traffic
 - Peak: 10,000 concurrent users, 10x normal traffic
 - Stress: 100,000 concurrent users, 100x normal traffic
 - Spike: Sudden spike from 100 to 50,000 users in 1 minute
 - Soak: 10,000 concurrent users for 72 hours
- Environment: Production-like staging environment, isolated test database, realistic test data, load injection from multiple regions
- Frequency: Before major releases, after infrastructure changes, quarterly capacity planning, annual scalability validation
- Reporting: Performance test reports, bottleneck identification, optimization recommendations, capacity planning, performance regression tracking

25.2.5 Security Testing

- Scope: OWASP Top 10, CWE Top 25, business logic vulnerabilities, API security, authentication security, authorization security
- Tool: OWASP ZAP, Burp Suite, Snyk, SonarQube security rules, Semgrep, CodeQL, GitHub Advanced Security
- Type: Static Application Security Testing (SAST), Dynamic Application Security Testing (DAST), Dependency vulnerability scanning, Container image scanning, Infrastructure security scanning
- Frequency: SAST on every PR, DAST weekly, dependency scan daily, container scan on every build, infrastructure scan weekly
- Reporting: Security vulnerability reports, CVSS scoring, remediation tracking, security metrics dashboard, security posture assessment

- Remediation: Critical vulnerabilities fixed within 24 hours, high within 1 week, medium within 1 month, low within 3 months, zero critical in production

25.3 Documentation Standards

25.3.1 Code Documentation

- API Documentation: OpenAPI 3.0 (Swagger) for all REST APIs, async API for messaging, GraphQL schema for GraphQL, documentation generated from code, interactive documentation (Swagger UI, GraphQL Playground)
- Code Comments: Comments explain why, not what, avoid obvious comments, complex algorithms explained, business rules documented, TODO comments with ticket numbers
- README Files: Every repository has a README, setup instructions, architecture overview, running tests, deployment guide, contribution guidelines, troubleshooting
- Architecture Decision Records (ADR): Every major architectural decision documented, context, decision, consequences, status, date, author, ADR repository
- Runbooks: Operational runbooks for every service, deployment runbook, troubleshooting runbook, incident response runbook, on-call runbook, DR runbook
- Wiki/Confluence: Team wiki for knowledge sharing, onboarding guide, architecture diagrams, process documentation, meeting notes, decision logs

25.3.2 API Documentation Standards

- Format: OpenAPI 3.0 (YAML or JSON), Swagger UI for interactive docs, ReDoc for beautiful documentation, Postman collections for testing

- Content: Endpoint description, request/response schema, authentication requirements, error codes, rate limits, example requests/responses, changelog
- Maintenance: Documentation generated from code annotations, CI/CD pipeline validation, documentation review, versioned documentation, deprecation notices
- Accessibility: Public API documentation hosted, internal API documentation in Confluence, API changelog, breaking change notifications, migration guides
- Standards: RESTful API design standards, HTTP method usage, status code usage, error response format, pagination format, filtering format, sorting format

25.3.3 Database Documentation

- Schema Documentation: ER diagrams for all databases, table descriptions, column descriptions, data types, constraints, indexes, relationships, generated from database
- Migration Documentation: Every migration documented, purpose, impact, rollback procedure, data migration details, performance impact, validation steps
- Query Documentation: Complex queries documented, performance notes, index usage, query plan, optimization history, query examples
- Data Dictionary: Business glossary, data definitions, data ownership, data quality rules, data lineage, data classification, retention policies
- Tools: dbdocs, SchemaSpy, ER/Studio, DataGrip, pgAdmin, MySQL Workbench, 自动生成文档

25.4 Version Control Standards

25.4.1 Git Workflow

- Branching Strategy: GitHub Flow (main + feature branches), main branch is always deployable, feature branches for all changes, release branches for hotfixes
- Branch Naming: feature/JIRA-123-short-description, bugfix/JIRA-123-short-description, hotfix/JIRA-123-short-description, release/v1.2.3, chore/dependency-update
- Commit Messages: Conventional Commits (feat:, fix:, docs:, style:, refactor:, test:, chore:), imperative mood, 50 character summary, detailed body for complex changes, reference JIRA ticket
- Pull Requests: Template with description, checklist, testing notes, screenshots for UI changes, linked JIRA ticket, required reviewers, CI checks must pass
- Merge Strategy: Squash and merge for feature branches, rebase and merge for clean history, no merge commits, linear history preferred, commit message from PR title
- Tagging: Semantic versioning (v1.2.3), annotated tags for releases, lightweight tags for internal, tag on merge to main, automated changelog generation

25.4.2 Repository Structure

- Monorepo vs. Polyrepo: Monorepo for shared code and dependencies, polyrepo for independent services, shared libraries in monorepo, services in separate repos or monorepo with workspace
- Directory Structure: Standard directory structure per service (src/, test/, docs/, scripts/, config/, docker/), consistent across all services, documented structure
- File Naming: Consistent naming conventions, descriptive names, no abbreviations, language-specific conventions, test files next to source files (*.test.js, *Test.java)
- Repository Hygiene: No secrets in repository, .gitignore for generated files, no large binary files, LFS for large files, regular repository cleanup, archived repositories for deprecated services

- Access Control: Repository access based on team, branch protection rules, required status checks, required reviews, admin bypass disabled, audit logging

25.5 Dependency Management

25.5.1 Dependency Policies

- Approval Process: All new dependencies require approval from tech lead, security review for new dependencies, license review, maintenance status check, community health check
- Update Strategy: Dependabot for automated updates, weekly update schedule, security updates immediate, minor updates monthly, major updates quarterly, LTS versions preferred
- Vulnerability Management: Snyk for vulnerability scanning, daily scan, critical vulnerabilities fixed within 24 hours, high within 1 week, dependency update tracking
- License Compliance: FOSSA for license scanning, approved license list, no GPL in distributed code, license attribution, compliance reporting
- Version Pinning: Exact versions in lock files (package-lock.json, Gemfile.lock, Pipfile.lock, go.sum), reproducible builds, version ranges for libraries, pinned versions for applications
- Dependency Inventory: SBOM (Software Bill of Materials) for all services, dependency tree, transitive dependencies, version tracking, update history, vulnerability tracking

25.5.2 Dependency Scanning

- Tool: Snyk, OWASP Dependency-Check, npm audit, pip-audit, gem audit, go list -m -versions, GitHub Dependabot, SonarQube
- Integration: CI/CD pipeline integration, automatic scan on build, PR check for vulnerabilities, blocking for critical vulnerabilities, weekly report

- Reporting: Vulnerability report with CVSS, affected versions, fixed versions, remediation steps, false positive tracking, exception process
- Remediation: Automated PR for patch updates, manual review for major updates, security patch process, rollback plan, testing requirements
- Tracking: Dependency dashboard, vulnerability tracking, update tracking, technical debt tracking, risk assessment, exception tracking

Appendix A: Glossary of Terms

Term	Definition
ACL	Access Control List - defines permissions for users and groups
ADR	Architecture Decision Record - documents important architectural decisions
AES	Advanced Encryption Standard - symmetric encryption algorithm
AI	Artificial Intelligence - computer systems performing tasks requiring human intelligence
ALB	Application Load Balancer - AWS Layer 7 load balancer
AMQP	Advanced Message Queuing Protocol - messaging protocol
AOF	Append-Only File - Redis persistence mechanism
API	Application Programming Interface - interface for software communication
APM	Application Performance Monitoring - monitoring application performance
AR	Augmented Reality - overlaying digital information on the real world
ARN	Amazon Resource Name - unique identifier for AWS resources
ASG	Auto Scaling Group - automatically scales EC2 instances
AWS	Amazon Web Services - cloud computing platform
BFF	Backend for Frontend - API tailored for specific frontend needs
BPM	Business Process Management - managing business processes

Brotli	Compression algorithm developed by Google
Burp Suite	Web vulnerability scanner and testing tool
CAA	Certificate Authority Authorization - DNS record for certificate issuance
CAS	Compare-And-Swap - atomic operation for concurrency control
CDN	Content Delivery Network - distributed network for content delivery
CI/CD	Continuous Integration/Continuous Deployment - automated build and deploy
CIDR	Classless Inter-Domain Routing - IP address allocation method
CORS	Cross-Origin Resource Sharing - mechanism for cross-origin requests
CSP	Content Security Policy - security standard for preventing XSS
CSR	Certificate Signing Request - request for SSL certificate
CSS	Cascading Style Sheets - styling language for web pages
CWE	Common Weakness Enumeration - list of software weakness types
DAST	Dynamic Application Security Testing - security testing of running applications
DBMS	Database Management System - software for managing databases
DDOS	Distributed Denial of Service - attack overwhelming a service
DevOps	Development and Operations - practice of integrating development and operations
DLQ	Dead Letter Queue - queue for failed messages
DNS	Domain Name System - translates domain names to IP addresses
DR	Disaster Recovery - plan for recovering from disasters
DynamoDB	AWS NoSQL database service
EBS	Elastic Block Store - AWS block storage service

EC2	Elastic Compute Cloud - AWS virtual server service
ECR	Elastic Container Registry - AWS Docker container registry
ECS	Elastic Container Service - AWS container orchestration service
EFS	Elastic File System - AWS managed file storage
EIP	Elastic IP - static IPv4 address for AWS
ELB	Elastic Load Balancer - AWS load balancing service
ENA	Elastic Network Adapter - enhanced networking for EC2
ESAD	Enterprise System Architecture Document - this document
ESB	Enterprise Service Bus - integration architecture pattern
ETL	Extract, Transform, Load - data integration process
E2E	End-to-End - testing covering complete user flows
Fargate	AWS serverless container compute engine
FinOps	Cloud Financial Management - practice of cloud cost management
FOSSA	License compliance and security scanning tool
FTP	File Transfer Protocol - protocol for file transfer
GDPR	General Data Protection Regulation - EU data protection law
GFS	Gluster File System - distributed file system
GitHub	Web-based version control and collaboration platform
Gzip	File compression utility and format
HPA	Horizontal Pod Autoscaler - Kubernetes auto-scaling
HSTS	HTTP Strict Transport Security - web security policy
HTML	HyperText Markup Language - standard markup language for web pages
HTTP	HyperText Transfer Protocol - application protocol for web
HTTPS	HTTP Secure - HTTP over TLS/SSL
IaC	Infrastructure as Code - managing infrastructure through code

IAM	Identity and Access Management - AWS identity management service
IOPS	Input/Output Operations Per Second - storage performance metric
IoT	Internet of Things - network of physical devices
IP	Internet Protocol - protocol for network communication
ISO	International Organization for Standardization - international standards
JDBC	Java Database Connectivity - Java API for database access
JMS	Java Message Service - Java messaging API
JWT	JSON Web Token - compact token for authentication
KMS	Key Management Service - AWS encryption key management
KPI	Key Performance Indicator - measurable value for success
LAN	Local Area Network - computer network in limited area
LDAP	Lightweight Directory Access Protocol - directory service protocol
LTS	Long-Term Support - software version with extended support
MAC	Mandatory Access Control - access control based on security labels
MFA	Multi-Factor Authentication - authentication requiring multiple factors
ML	Machine Learning - computer systems learning from data
MTTR	Mean Time To Recovery - average time to recover from failure
MVP	Minimum Viable Product - product with minimal features for launch
NAT	Network Address Translation - IP address remapping
NFS	Network File System - distributed file system protocol
NLB	Network Load Balancer - AWS Layer 4 load balancer
NoSQL	Non-SQL database - non-relational database

npm	Node Package Manager - JavaScript package manager
OAuth	Open Authorization - authorization framework
OLAP	Online Analytical Processing - analytical data processing
OLTP	Online Transaction Processing - transactional data processing
OpenAPI	Specification for building APIs
ORM	Object-Relational Mapping - programming technique for data conversion
OWASP	Open Web Application Security Project - web security organization
PCI-DSS	Payment Card Industry Data Security Standard - payment security standard
PITR	Point-In-Time Recovery - database recovery to specific time
PM	Product Manager - role responsible for product strategy
POM	Page Object Model - design pattern for UI testing
PoLP	Principle of Least Privilege - minimal access principle
POSIX	Portable Operating System Interface - operating system standard
PR	Pull Request - request to merge code changes
RACI	Responsible, Accountable, Consulted, Informed - responsibility matrix
RDBMS	Relational Database Management System - relational database software
RDF	Resource Description Framework - data model for metadata
RDP	Remote Desktop Protocol - remote access protocol
REST	Representational State Transfer - architectural style for APIs
RHEL	Red Hat Enterprise Linux - commercial Linux distribution
RPO	Recovery Point Objective - maximum acceptable data loss
RTO	Recovery Time Objective - maximum acceptable downtime

S3	Simple Storage Service - AWS object storage service
SaaS	Software as a Service - cloud computing model
SAML	Security Assertion Markup Language - authentication protocol
SANS	SysAdmin, Audit, Network, Security - security training organization
SAST	Static Application Security Testing - security testing of source code
SBOM	Software Bill of Materials - inventory of software components
SDK	Software Development Kit - tools for software development
SEO	Search Engine Optimization - improving web visibility
SFTP	SSH File Transfer Protocol - secure file transfer
SLA	Service Level Agreement - commitment between provider and customer
SNS	Simple Notification Service - AWS messaging service
SOA	Service-Oriented Architecture - architectural pattern
SOAP	Simple Object Access Protocol - messaging protocol
SonarQube	Code quality and security analysis platform
SQS	Simple Queue Service - AWS message queue service
SQL	Structured Query Language - database query language
SRP	Single Responsibility Principle - one responsibility per class
SSH	Secure Shell - secure network protocol
SSO	Single Sign-On - authentication using single credentials
SSL	Secure Sockets Layer - cryptographic protocol (deprecated, use TLS)
SSO	Single Sign-On - authentication using single credentials
SRE	Site Reliability Engineering - discipline for reliable systems
TCP	Transmission Control Protocol - reliable network protocol
TDD	Test-Driven Development - development driven by tests

Terraform	Infrastructure as Code tool by HashiCorp
TLS	Transport Layer Security - cryptographic protocol
TTL	Time To Live - data expiration time
UAT	User Acceptance Testing - testing by end users
UI	User Interface - visual interface for software
UDP	User Datagram Protocol - fast network protocol
UAT	User Acceptance Testing - testing by end users
URL	Uniform Resource Locator - web address
UX	User Experience - overall experience of using a product
VPC	Virtual Private Cloud - isolated virtual network in AWS
VPN	Virtual Private Network - secure network connection
VR	Virtual Reality - simulated experience
WAF	Web Application Firewall - firewall for web applications
WAL	Write-Ahead Logging - database logging mechanism
WAN	Wide Area Network - computer network spanning large area
WORM	Write Once Read Many - data storage model
WSGI	Web Server Gateway Interface - Python web standard
XSS	Cross-Site Scripting - web security vulnerability
YAML	YAML Ain't Markup Language - human-readable data format
ZAP	Zed Attack Proxy - OWASP web vulnerability scanner
ZGC	Z Garbage Collector - low-latency JVM garbage collector

Appendix B: Acronyms & Abbreviations

Acronym	Full Form
AD	Active Directory
AES	Advanced Encryption Standard
AI	Artificial Intelligence

ALB	Application Load Balancer
AMQP	Advanced Message Queuing Protocol
API	Application Programming Interface
APM	Application Performance Monitoring
AR	Augmented Reality
ARN	Amazon Resource Name
ASG	Auto Scaling Group
AWS	Amazon Web Services
BFF	Backend for Frontend
BPM	Business Process Management
CAA	Certificate Authority Authorization
CAS	Compare-And-Swap
CDN	Content Delivery Network
CI/CD	Continuous Integration/Continuous Deployment
CIDR	Classless Inter-Domain Routing
CORS	Cross-Origin Resource Sharing
CSP	Content Security Policy
CSR	Certificate Signing Request
CSS	Cascading Style Sheets
CWE	Common Weakness Enumeration
DAST	Dynamic Application Security Testing
DBMS	Database Management System
DDOS	Distributed Denial of Service
DevOps	Development and Operations
DLQ	Dead Letter Queue
DNS	Domain Name System
DR	Disaster Recovery

EBS	Elastic Block Store
EC2	Elastic Compute Cloud
ECR	Elastic Container Registry
ECS	Elastic Container Service
EFS	Elastic File System
EIP	Elastic IP
ELB	Elastic Load Balancer
ENA	Elastic Network Adapter
ESAD	Enterprise System Architecture Document
ESB	Enterprise Service Bus
ETL	Extract, Transform, Load
E2E	End-to-End
Fargate	AWS Serverless Container Compute
FinOps	Cloud Financial Operations
FOSSA	Free and Open Source Software Analysis
FTP	File Transfer Protocol
GDPR	General Data Protection Regulation
GFS	Gluster File System
HPA	Horizontal Pod Autoscaler
HSTS	HTTP Strict Transport Security
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HTTP Secure
IaC	Infrastructure as Code
IAM	Identity and Access Management
IOPS	Input/Output Operations Per Second
IoT	Internet of Things

IP	Internet Protocol
ISO	International Organization for Standardization
JDBC	Java Database Connectivity
JMS	Java Message Service
JWT	JSON Web Token
KMS	Key Management Service
KPI	Key Performance Indicator
LAN	Local Area Network
LDAP	Lightweight Directory Access Protocol
LTS	Long-Term Support
MAC	Mandatory Access Control
MFA	Multi-Factor Authentication
ML	Machine Learning
MTTR	Mean Time To Recovery
MVP	Minimum Viable Product
NAT	Network Address Translation
NFS	Network File System
NLB	Network Load Balancer
NoSQL	Not Only SQL
npm	Node Package Manager
OAuth	Open Authorization
OLAP	Online Analytical Processing
OLTP	Online Transaction Processing
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PCI-DSS	Payment Card Industry Data Security Standard
PITR	Point-In-Time Recovery

PM	Product Manager
POM	Page Object Model
PoLP	Principle of Least Privilege
POSIX	Portable Operating System Interface
PR	Pull Request
RACI	Responsible, Accountable, Consulted, Informed
RDBMS	Relational Database Management System
RDF	Resource Description Framework
RDP	Remote Desktop Protocol
REST	Representational State Transfer
RHEL	Red Hat Enterprise Linux
RPO	Recovery Point Objective
RTO	Recovery Time Objective
S3	Simple Storage Service
SaaS	Software as a Service
SAML	Security Assertion Markup Language
SANS	SysAdmin, Audit, Network, Security
SAST	Static Application Security Testing
SBOM	Software Bill of Materials
SDK	Software Development Kit
SEO	Search Engine Optimization
SFTP	SSH File Transfer Protocol
SLA	Service Level Agreement
SNS	Simple Notification Service
SOA	Service-Oriented Architecture
SOAP	Simple Object Access Protocol
SQS	Simple Queue Service

SQL	Structured Query Language
SRP	Single Responsibility Principle
SSH	Secure Shell
SSO	Single Sign-On
SSL	Secure Sockets Layer
SRE	Site Reliability Engineering
TCP	Transmission Control Protocol
TDD	Test-Driven Development
TLS	Transport Layer Security
TTL	Time To Live
UAT	User Acceptance Testing
UI	User Interface
UDP	User Datagram Protocol
URL	Uniform Resource Locator
UX	User Experience
VPC	Virtual Private Cloud
VPN	Virtual Private Network
VR	Virtual Reality
WAF	Web Application Firewall
WAL	Write-Ahead Logging
WAN	Wide Area Network
WORM	Write Once Read Many
WSGI	Web Server Gateway Interface
XSS	Cross-Site Scripting
YAML	YAML Ain't Markup Language
ZAP	Zed Attack Proxy
ZGC	Z Garbage Collector

Appendix C: Technology Stack Summary

C.1 Backend Technologies

Category	Technology	Version	Purpose
Language	Java	17+	Primary backend language
Framework	Spring Boot	3.2+	Application framework
Security	Spring Security	6.2+	Authentication & authorization
Data	Spring Data JPA	3.2+	ORM and data access
Cache	Spring Data Redis	3.2+	Redis integration
Search	Spring Data Elasticsearch	5.2+	Search integration
Messaging	Spring for Apache Kafka	3.1+	Messaging integration
API	SpringDoc OpenAPI	2.3+	API documentation
Database	PostgreSQL	16+	Primary database
Cache	Redis	7.2+	Caching and session storage
Search	Elasticsearch	8.12+	Full-text search
Messaging	Apache Kafka	3.6+	Event streaming
NoSQL	DynamoDB	-	High-scale data
Language	Node.js	20+	Microservices, real-time
Framework	Express.js	4.18+	Web framework
Language	Python	3.11+	ML, analytics, data science
Framework	FastAPI	0.109+	ML API framework
Language	Go	1.21+	High-performance services
Framework	Gin	1.9+	Web framework

C.2 Frontend Technologies

Category	Technology	Version	Purpose
----------	------------	---------	---------

Framework	React	18.2+	Web application UI
Language	TypeScript	5.3+	Type-safe JavaScript
Styling	Tailwind CSS	3.4+	Utility-first CSS
Components	shadcn/ui	0.8+	UI component library
State	Zustand	4.5+	State management
Query	TanStack Query	5.18+	Server state management
Forms	React Hook Form	7.50+	Form management
Validation	Zod	3.22+	Schema validation
Charts	Recharts	2.10+	Data visualization
Maps	Leaflet	1.9+	Interactive maps
Mobile	React Native	0.73+	Cross-platform mobile
Mobile	Expo	50+	React Native framework
Mobile	Flutter	3.19+	Alternative mobile framework

C.3 DevOps & Infrastructure

Category	Technology	Version	Purpose
Cloud	AWS	-	Primary cloud provider
Container	Docker	24+	Containerization
Orchestration	Kubernetes	1.29+	Container orchestration
Kubernetes	EKS	-	Managed Kubernetes
Ingress	NGINX	1.25+	Ingress controller
Service Mesh	Istio	1.20+	Service mesh
GitOps	ArgoCD	2.9+	GitOps deployment
IaC	Terraform	1.7+	Infrastructure as Code
CI/CD	GitHub Actions	-	CI/CD pipelines
Registry	ECR	-	Container registry
Helm	Helm	3.14+	Kubernetes package manager

Secrets	Vault	1.15+	Secret management
Monitoring	Prometheus	2.49+	Metrics collection
Monitoring	Grafana	10.3+	Metrics visualization
Logging	ELK Stack	8.12+	Log aggregation
Logging	Loki	2.9+	Alternative log aggregation
Tracing	Jaeger	1.50+	Distributed tracing
APM	DataDog	-	Application monitoring
Alerts	PagerDuty	-	Incident management

C.4 Security & Compliance

Category	Technology	Version	Purpose
WAF	AWS WAF	-	Web application firewall
DDoS	AWS Shield	-	DDoS protection
Secrets	AWS Secrets Manager	-	Secret management
Encryption	AWS KMS	-	Key management
Certificates	ACM	-	SSL/TLS certificates
Scanning	Snyk	-	Vulnerability scanning
SAST	SonarQube	10.4+	Static analysis
DAST	OWASP ZAP	2.14+	Dynamic scanning
Compliance	AWS Config	-	Compliance monitoring
SIEM	Splunk	9.2+	Security analytics
SIEM	QRadar	-	Alternative SIEM

C.5 Third-Party Services

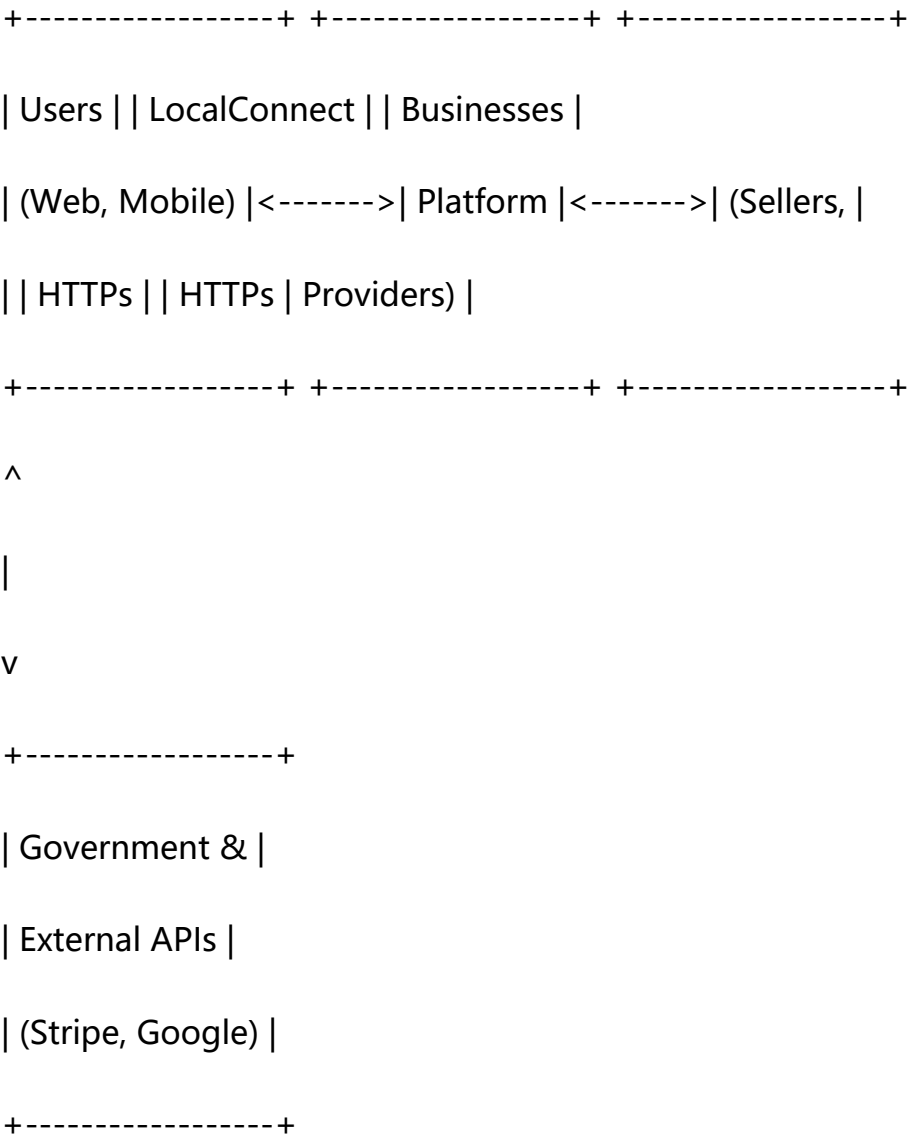
Service	Provider	Purpose
Payments	Stripe	Payment processing

SMS	Twilio	SMS notifications
Email	SendGrid	Email delivery
Push	Firebase	Push notifications
Maps	Google Maps	Location services
CDN	Cloudflare	DNS and CDN
Search	Algolia	Hosted search (optional)
Monitoring	DataDog	APM and monitoring
Incident	PagerDuty	Incident management
Collaboration	Slack	Team communication
Project	Jira	Project management
Wiki	Confluence	Documentation
Version Control	GitHub	Code repository
CI/CD	GitHub Actions	Build automation
Design	Figma	UI/UX design
Analytics	Google Analytics	Web analytics
Analytics	Mixpanel	Product analytics
Surveys	SurveyMonkey	User surveys
Support	Zendesk	Customer support
CRM	HubSpot	Customer relationship
HR	BambooHR	Human resources
Finance	QuickBooks	Accounting
Legal	DocuSign	Document signing
File Sharing	Dropbox	File sharing
Video	Zoom	Video conferencing
Calendar	Google Calendar	Scheduling
Email	Google Workspace	Business email
Storage	Google Drive	Cloud storage

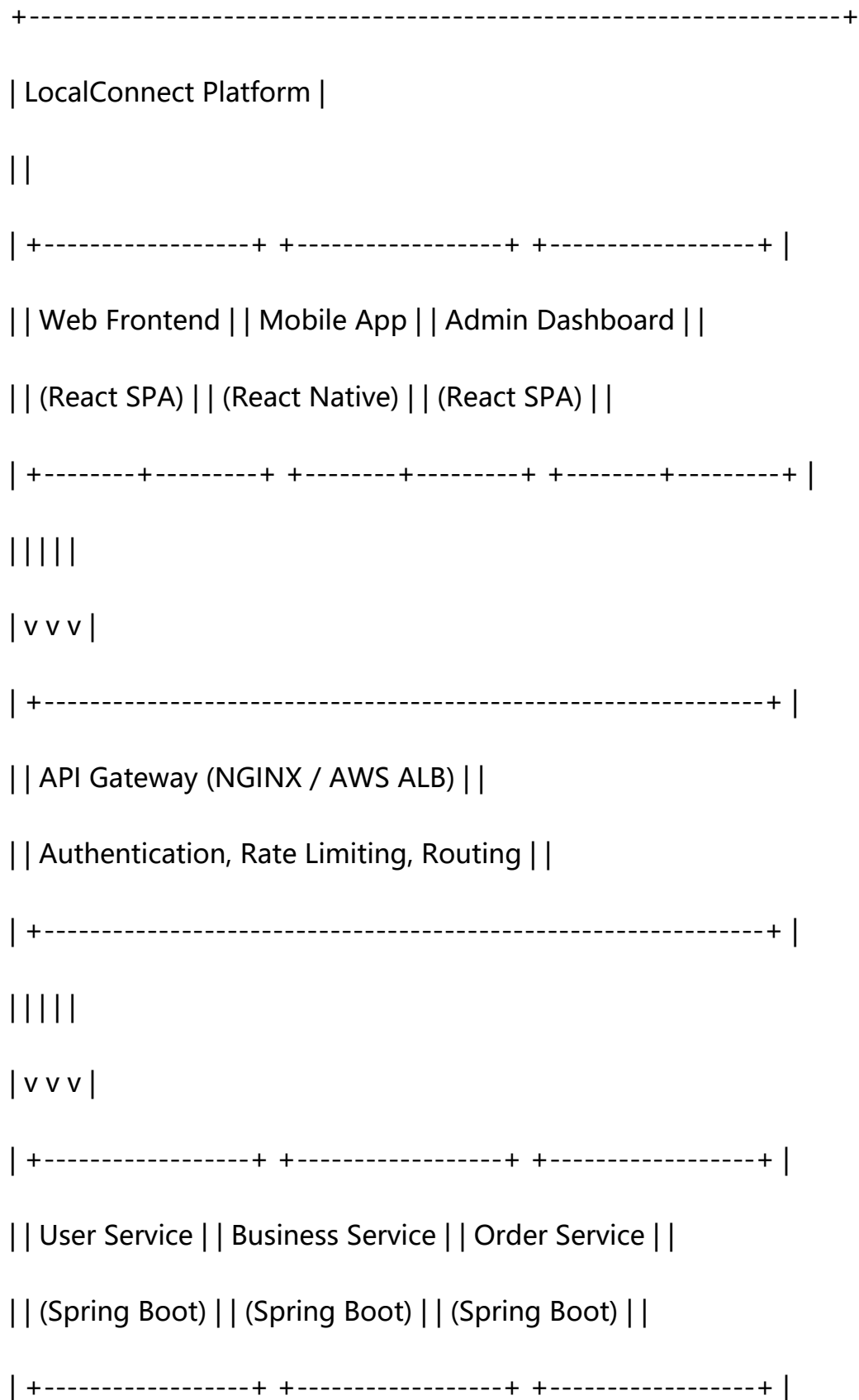
Office	Microsoft 365	Office suite
Password	1Password	Password management
VPN	NordLayer	VPN for remote work

Appendix D: Reference Architecture Diagrams

D.1 System Context Diagram (Level 0)



D.2 Container Diagram (Level 1)



```
| +-----+ +-----+ +-----+ |
| | Event Service | | Forum Service | | Payment Service | |
| | (Spring Boot) | | (Spring Boot) | | (Node.js) | |
| +-----+ +-----+ +-----+ |
| +-----+ +-----+ +-----+ |
| | Government Svc | | Analytics Svc | | Emergency Svc | |
| | (Spring Boot) | | (Python/FastAPI) | | (Spring Boot) | |
| +-----+ +-----+ +-----+ |
| +-----+ +-----+ +-----+ |
| | Notification Svc | | Search Service | | ML Service | |
| | (Node.js) | | (Spring Boot) | | (Python/FastAPI) | |
| +-----+ +-----+ +-----+ |
| |
| +-----+ +-----+ +-----+ |
| | PostgreSQL | | Redis | | Elasticsearch | |
| | (Primary DB) | | (Cache) | | (Search) | |
| +-----+ +-----+ +-----+ |
| +-----+ +-----+ +-----+ |
| | DynamoDB | | Kafka | | S3 | |
| | (NoSQL) | | (Messaging) | | (Storage) | |
```

| +-----+ +-----+ +-----+ |
||
+-----+

D.3 Deployment Architecture Diagram

+-----+

| AWS Cloud (Singapore) |

||

| +-----+ +-----+ +-----+ |

|| CloudFront || Route 53 || WAF ||

|| (CDN) || (DNS) || (Firewall) ||

| +-----+ +-----+ +-----+ +-----+ |

||||

| v v v |

| +-----+ |

|| AWS ALB (Application Load Balancer) ||

| +-----+ |

|||

| v |

| +-----+ |

|| EKS Cluster (Kubernetes) ||

| | Worker Nodes (m6i/c6i/r6i) | |

| | +-----+ | |

| | | Ingress Controller (NGINX) | | |

| | | +-----+ | | |

| | | | Service Pods (API Services, Background Jobs) | | | |

| | | | +-----+ +-----+ +-----+ +-----+ +-----+ | | | |

| | | | Pod | | Pod | | Pod | | Pod | | Pod | | | |

| | | | +-----+ +-----+ +-----+ +-----+ +-----+ | | | |

| | | +-----+ | | |

| | +-----+ | |

| +-----+ |

| | |

| v |

| +-----+ +-----+ +-----+ |

| | RDS PostgreSQL | | ElastiCache | | Elasticsearch | |

| | (Multi-AZ) | | Redis (Cluster) | | (3 nodes) | |

| +-----+ +-----+ +-----+ |

| +-----+ +-----+ +-----+ |

| | DynamoDB | | MSK Kafka | | S3 | |

| | (Tables) | | (Brokers) | | (Buckets) | |

| +-----+ +-----+ +-----+ |

||

+-----+

D.4 Data Flow Diagram (Order Creation)

+-----+ +-----+ +-----+ +-----+

| Client |---->| API Gateway|---->| Order Service |---->| Order Database |

|||| (Spring Boot) || (PostgreSQL) |

+-----+ +-----+ +-----+ +-----+

||

| (Event) | (Transaction)

v v

+-----+ +-----+

| Kafka Topic || Order Items |

| order.created || Table |

+-----+ +-----+

|

| (Consumer)

v

+-----+

| Notification |

| Service |

| (Node.js) |

+-----+

|

| (Push/SMS/Email)

v

+-----+

| User Device |

| (Push Notif) |

+-----+

D.5 Security Architecture Diagram

+-----+

| Security Layers |

||

| Layer 1: Perimeter |

| +-----+ +-----+ +-----+ |

| | CloudFront | | AWS WAF | | AWS Shield | |

| | (DDoS, CDN) | | (Rules) | | (DDoS) | |

| +-----+ +-----+ +-----+ |

||

| Layer 2: Network |

| +-----+ +-----+ +-----+ |

| | VPC | | Security Groups | | NACLs | |

| | (Isolation) | | (Port/Protocol) | | (Subnet Rules) | |

| +-----+ +-----+ +-----+ |

| |

| Layer 3: Application |

| +-----+ +-----+ +-----+ |

| | HTTPS/TLS 1.3 | | JWT/OAuth2 | | Rate Limiting | |

| | (Encryption) | | (Auth) | | (Throttling) | |

| +-----+ +-----+ +-----+ |

| |

| Layer 4: Data |

| +-----+ +-----+ +-----+ |

| | AES-256 | | KMS | | Secrets Mgr | |

| | (Encryption) | | (Key Mgmt) | | (Secrets) | |

| +-----+ +-----+ +-----+ |

| |

| Layer 5: Monitoring |

| +-----+ +-----+ +-----+ |

| | GuardDuty | | CloudTrail | | Security Hub | |

| | (Threats) | | (Audit Logs) | | (Compliance) | |

| +-----+ +-----+ +-----+ |

| |

+-----+

D.6 Microservices Communication Pattern

+-----+

| Service Mesh (Istio) |

| |

| +-----+ +-----+ +-----+ |

| | Service A |<--->| Istio Proxy |<--->| Service B | |

| | (Spring Boot) | mTLS | (Sidecar) | mTLS | (Spring Boot) | |

| +-----+ +-----+ +-----+ |

| | |

| | (Async) | (Sync) |

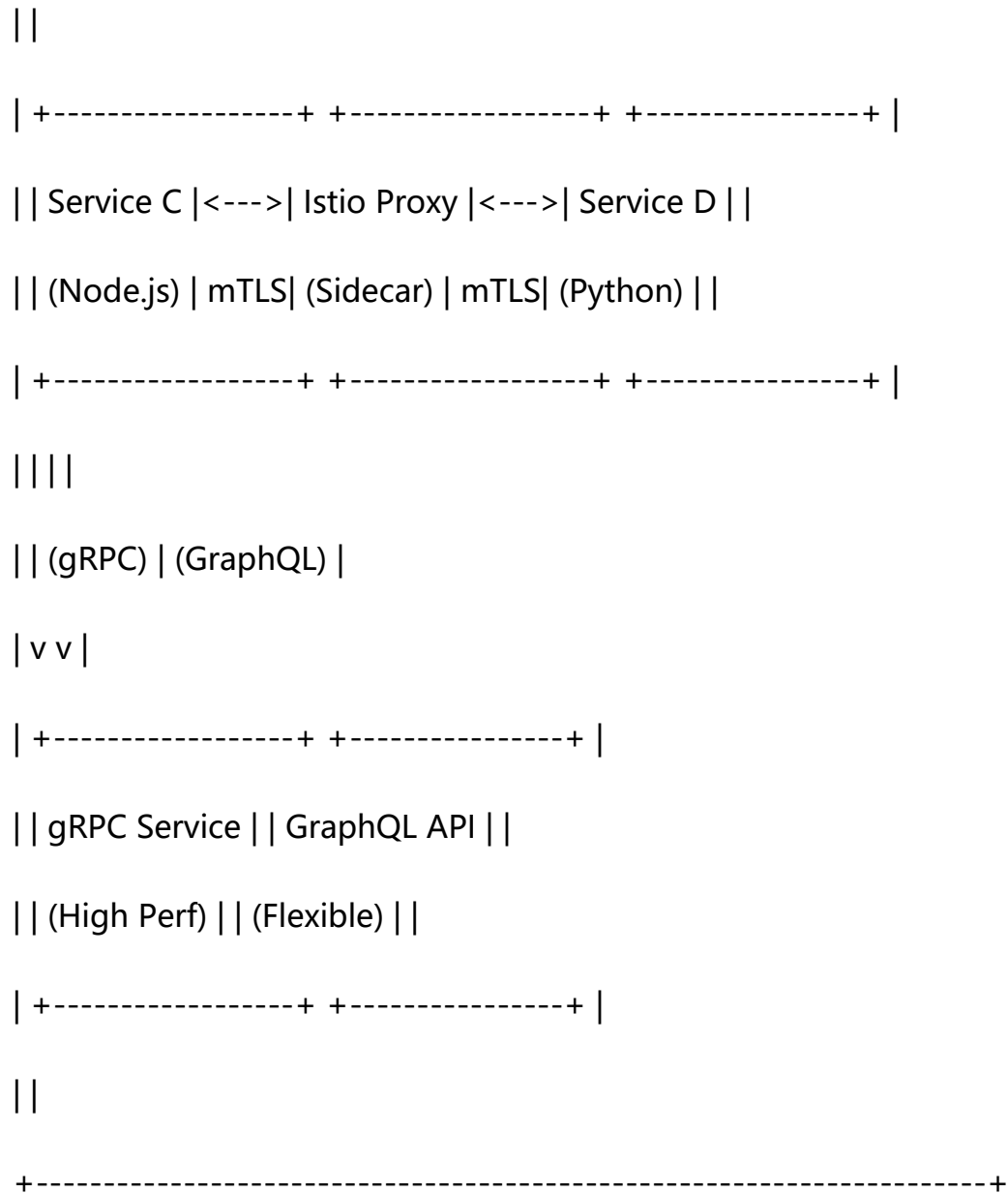
| v v |

| +-----+ +-----+ |

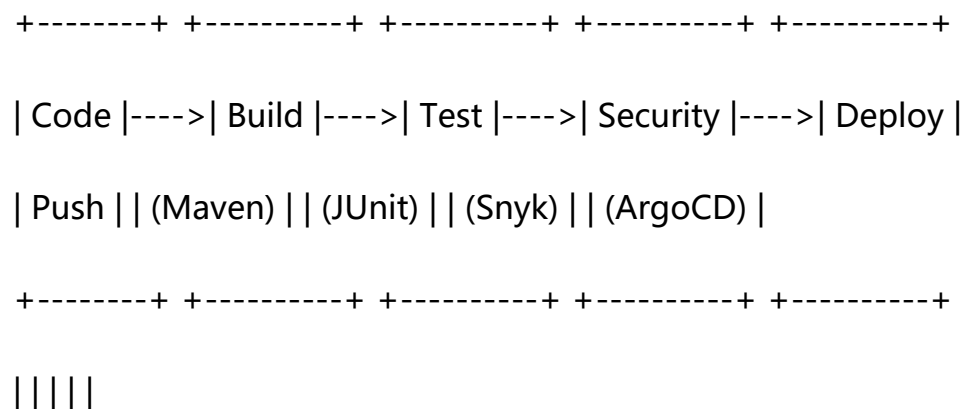
| | Kafka | | REST API | |

| | (Event Bus) | | (HTTP/JSON) | |

| +-----+ +-----+ |



D.7 CI/CD Pipeline Diagram



||||

v v v v v

+-----+ +-----+ +-----+ +-----+ +-----+

| GitHub | | Docker | | Sonar | | OWASP | | EKS |

| Actions| | Build | | Qube | | ZAP | | Cluster |

+-----+ +-----+ +-----+ +-----+ +-----+

||||

||||

v v v v v

+-----+ +-----+ +-----+ +-----+ +-----+

| Branch | | ECR | | Report | | Report | | Prod |

| Policy | | Push | | Quality | | Security| | Deploy |

+-----+ +-----+ +-----+ +-----+ +-----+

Appendix E: Change Log

Version	Date	Author	Changes
0.1	2024-01-15	Lead Architect	Initial draft - Chapters 1-5
0.2	2024-01-30	Lead Architect	Added Chapters 6-10 (Services, Data, Security)
0.3	2024-02-15	Security Architect	Added Chapters 11-15 (Security, API, Integration, Mobile, DevOps)

0.4	2024-03-01	DevOps Architect	Added Chapters 16-20 (Monitoring, Performance, Testing, Compliance)
0.5	2024-03-15	Lead Architect	Added Chapters 21-25 (Disaster Recovery, Scalability, Cost, Roadmap, Standards)
1.0	2024-03-30	Lead Architect	Complete document with all appendices, diagrams, glossary. Approved for use.
1.1	2024-04-15	Security Architect	Updated security architecture with new WAF rules, added threat model updates
1.2	2024-05-01	Data Architect	Updated database schema with new analytics tables, added partitioning strategy
1.3	2024-06-01	Lead Architect	Updated microservices architecture with new services (Emergency, ML, Analytics)
1.4	2024-07-01	DevOps Architect	Updated CI/CD pipeline with ArgoCD, added GitOps workflow
1.5	2024-08-01	Security Architect	Updated compliance section with new regulations, added penetration testing results
2.0	2024-09-01	Lead Architect	Major revision - added multi-region architecture, updated cost estimates, new diagrams

Document Control

Document Information

- Document Title: LocalConnect Enterprise System Architecture Document (ESAD)
- Document Version: 2.0
- Last Updated: 2024-09-01
- Document Owner: LocalConnect Architecture Team
- Document Status: Approved
- Classification: Confidential - Internal Use Only

- Distribution: LocalConnect Engineering Team, Leadership Team, External Auditors (under NDA)

Review Schedule

- Quarterly Review: Architecture review board meets quarterly to review and update this document
- Annual Review: Full document review and revision annually
- Trigger-Based Review: Major architectural changes, security incidents, compliance changes, technology updates
- Approval Required From: CTO, CISO, Lead Architect

Related Documents

- LocalConnect Security Policy Document (LSPD)
- LocalConnect Data Privacy Policy (LDPP)
- LocalConnect Incident Response Plan (LIRP)
- LocalConnect Business Continuity Plan (LBCP)
- LocalConnect API Documentation (OpenAPI Specs)
- LocalConnect Infrastructure Runbooks (Operational Guides)
- LocalConnect Development Guidelines (Coding Standards)
- LocalConnect Testing Strategy (Test Plans)
- LocalConnect Deployment Guide (DevOps Runbooks)

End of Document

© 2024 LocalConnect. All rights reserved. This document contains confidential and proprietary information. Unauthorized distribution or copying is strictly prohibited.

Chapter 2: Business Overview

2.1 Business Context

LocalConnect operates at the intersection of e-commerce, social networking, government digital services, and emergency response. Its primary users include:

- Local Business Owners — Small shops, restaurants, clinics, salons, artisans, craftsmen
- Community Members — Residents, students, families, professionals
- Government Officials — Municipal workers, local authorities, community organizers
- Event Organizers — Cultural groups, religious organizations, sports clubs, NGOs
- Emergency Services — Police, fire, ambulance, disaster response teams
- Delivery Partners — Independent drivers, courier companies, logistics providers
- Payment Providers — Banks, mobile money operators, fintech companies

2.2 Market Differentiation

Unlike generic social media or marketplace apps, LocalConnect:

- Focuses exclusively on hyper-local engagement (neighborhood-level)
- Integrates government services natively (not as third-party links)
- Provides real-time emergency alerting (not just messaging)
- Supports local languages (Sinhala, Tamil, English) natively in every feature
- Operates on low-bandwidth networks (2G/3G compatibility)
- Works offline with intelligent sync
- Community-driven governance (moderation, dispute resolution, rewards)

2.3 Revenue Model

- Freemium for Businesses: Basic listing free, premium features (promoted posts, analytics, CRM tools) paid
- Transaction Fees: 2.5% on marketplace purchases, 1.5% on service bookings
- Advertising: Hyper-local targeted ads (privacy-preserving, consent-based)
- Government Contracts: SaaS fees for municipal digital service integration
- Premium Subscriptions: "LocalConnect Plus" for power users (3/month—advanced features, no ads, priority support)3/month — advanced features, no ads, priority support)3/month— advanced features, no ads, priority support)
- Data Analytics: Anonymized community insights reports for urban planners, NGOs
- Payment Gateway: Micro-transactions on utility bill payments, government fee collection

2.4 Stakeholder Map

Stakeholder	Interest	Influence	Management Strategy
Community Members	App usability, free features, privacy	High	UX Research, Feedback Loops, Beta Testing
Business Owners	Customer acquisition, low fees, ROI	High	SMB Advisory Board, Success Metrics Dashboards
Government Officials	Transparency, service efficiency, data security	High	Dedicated GovTech Integration Team, Compliance Reports
Investors	Growth metrics, revenue, market share	High	Monthly Board Reports, KPI Dashboards

Emergency Services	Reliability, speed, accuracy	Critical	99.999% SLA, Dedicated Infrastructure, Priority Support
Payment Partners	Transaction volume, security, compliance	Medium	PCI DSS Compliance, API Rate Limits, Settlement Guarantees
NGOs	Community impact, accessibility, reach	Medium	Open Data APIs, Impact Reporting, Partnership Programs
Regulatory Bodies	Data protection, consumer rights, compliance	High	Legal Compliance Team, Automated Audit Trails, DPO

2.5 Success Metrics (KPIs)

Metric	Target (Year 1)	Target (Year 3)	Target (Year 5)
Registered Users	5 Million	50 Million	100 Million
DAU	1 Million	10 Million	25 Million
Business Listings	100,000	1 Million	5 Million
Monthly Transactions	500,000	10 Million	50 Million
Average Session Duration	8 minutes	12 minutes	15 minutes
Community Event Posts	50,000/month	500,000/month	2 Million/month
Emergency Alert Delivery Time	< 5 seconds	< 2 seconds	< 1 second
App Store Rating	4.5+	4.7+	4.8+
Customer Support Response Time	< 2 hours	< 30 minutes	< 10 minutes
Uptime SLA	99.95%	99.99%	99.999%

NPS Score	50+	60+	70+
Revenue	\$2M ARR	\$50M ARR	\$200M ARR

2.6 Competitive Analysis

Competitor	Strength	Weakness	LocalConnect Advantage
Facebook Groups	Massive reach, network effects	Not local-first, poor business tools, privacy concerns	Hyper-local focus, native business features, privacy by design
WhatsApp Business	Messaging, familiarity	No discovery, no marketplace, no government integration	Discovery, marketplace, integrated services, AI moderation
Google Maps	Search, reviews	No transactions, no community features, no events	End-to-end transactions, community events, emergency alerts
Local Directories (e.g., Yellow Pages)	Business listings	No engagement, outdated, no mobile-first	Dynamic content, social features, real-time updates, mobile-native
Uber/DoorDash	Logistics, payments	No community, no local business empowerment, high fees	Community-driven, lower fees, local business ownership
Government Portals	Official information	Terrible UX, no mobile, fragmented	Native mobile, unified, community feedback loops

Nextdoor (US)	Community focus	US-only, no government integration, no marketplace	Government integration, marketplace, multi-region, local languages
---------------	-----------------	--	--

2.7 Regulatory Landscape

- Sri Lanka: Personal Data Protection Act (PDPA), Computer Crime Act, Payment and Settlement Systems Act
- South Asia: India's DPDP Act, Bangladesh's DPA, Pakistan's Personal Data Protection Bill
- GDPR (EU): If expanding to EU markets, full GDPR compliance required
- PCI DSS: Level 1 compliance for payment processing
- ISO 27001: Information security management certification required
- Accessibility: WCAG 2.1 AA compliance, local disability access laws

Chapter 3: Business Goals (Strategic Objectives)

3.1 Primary Goals

BG-1: Community Empowerment

Enable every local community to have a digital presence, connecting residents, businesses, and services in a unified platform.

BG-2: Economic Growth

Support local businesses by providing digital tools for customer acquisition, transaction processing, and analytics at affordable pricing.

BG-3: Government Transparency

Integrate government services (permits, taxes, complaints, information) directly into the app, reducing friction and increasing civic engagement.

BG-4: Emergency Response

Provide a real-time emergency alerting system that can reach millions of users within seconds, with integrated response coordination.

BG-5: Digital Inclusion

Ensure the app works on low-end devices, low-bandwidth networks, and in multiple local languages, serving the underserved population.

3.2 Secondary Goals

BG-6: Platform Sustainability

Build a revenue model that sustains operations while keeping core community features free for users and businesses.

BG-7: Data Sovereignty

Ensure all citizen and community data remains within national borders, compliant with local data protection laws.

BG-8: Open Ecosystem

Provide open APIs for third-party developers, government systems, and NGO tools to integrate and extend the platform.

BG-9: Environmental Impact

Promote local consumption (reducing logistics carbon footprint), support green businesses, and integrate carbon offset programs.

BG-10: Knowledge Sharing

Create a community of practice where local developers, designers, and entrepreneurs can learn, share, and build on the platform.

3.3 Goal Mapping to Architecture

Business Goal	Architectural Enabler	Technology
Community Empowerment	Microservices for modularity, Event-driven for real-time	Kafka, WebSockets, CQRS
Economic Growth	Marketplace microservice, Payment gateway, Analytics	Stripe, PostgreSQL, ClickHouse
Government Transparency	GovTech Integration Service, API Gateway, Audit logging	REST APIs, OAuth 2.0, S3
Emergency Response	Priority messaging, Geo-distributed caching, Push notifications	SQS, Redis, Firebase, APNS
Digital Inclusion	Offline-first architecture, Progressive Web App, Image compression	PWA, Workbox, WebP, Lazy loading
Platform Sustainability	Multi-tenant SaaS, Usage-based billing, Cost optimization	Kubernetes HPA, Spot instances, Reserved capacity
Data Sovereignty	Region-specific deployment, Data residency controls	AWS regions, Encryption at rest, Geo-fencing
Open Ecosystem	Developer Portal, API Gateway, Rate limiting, API versioning	Kong, OpenAPI, GraphQL, OAuth 2.0

Environmental Impact	Carbon-aware scheduling, Green energy data centers, Optimized routing	AWS Carbon Footprint Tool, Green regions
Knowledge Sharing	Documentation platform, Community forums, Training modules	Docusaurus, Discourse, Moodle

Chapter 4: Functional Requirements (Epics & User Stories)

4.1 Epic: User Management & Authentication (FR-001 to FR-050)

FR-001: Multi-Method Authentication

Users can register/login via: Phone OTP, Email/Password, Google OAuth, Facebook OAuth, Apple Sign-In, Government ID (NIC/passport for verified accounts), Biometric (fingerprint/face on supported devices).

FR-002: Role-Based Access Control (RBAC)

System supports: Community Member, Business Owner, Government Official, Event Organizer, Emergency Responder, Admin, Super Admin, Moderator, Delivery Partner, Payment Agent.

FR-003: Profile Management

Users can create profiles with: Display name, Bio, Profile photo, Cover photo, Location (GPS or manual), Languages, Privacy settings, Notification preferences, Two-factor authentication settings, Linked accounts.

FR-004: KYC Verification

Businesses and government officials undergo KYC verification: Document upload (business registration, ID), AI-powered document verification, Manual review queue, Verification badge, Re-verification reminders.

FR-005: Account Security

Password complexity rules, Brute-force protection, Session management (view active sessions, revoke), Login history, Security alerts (new device, location), Account lockout, Recovery mechanisms.

FR-006: Anonymous Mode

Users can browse and interact in specific forums anonymously while maintaining anti-abuse measures.

FR-007: Family Accounts

Primary account holder can manage sub-accounts for family members (children, elderly) with restricted permissions.

FR-008: Accessibility Profiles

Screen reader support, High contrast mode, Font size adjustment, Voice navigation, Color blindness modes, Motor impairment support.

4.2 Epic: Local Business Discovery (FR-051 to FR-100)

FR-051: Business Search & Filter

Search by: Name, Category, Location (radius), Rating, Price range, Operating hours, Services offered, Products available, Availability (open now), Delivery support, Payment methods.

FR-052: Business Profile Pages

Rich profiles with: Photos/videos, Description, Contact info, Social links, Menu/catalog, Reviews, Q&A, Promotions, Operating hours, Location map, Accessibility info, COVID-19 safety measures.

FR-053: Real-Time Availability

Integration with business POS systems to show real-time inventory, table availability, appointment slots, queue status.

FR-054: Business Claim Process

Unclaimed businesses can be claimed by verified owners with document proof and AI verification.

FR-055: Business Categories

Hierarchical categories: Food & Beverage, Health & Wellness, Home Services, Retail, Education, Entertainment, Automotive, Professional Services, Agriculture, Government Services, NGOs, Religious Institutions.

FR-056: Promoted Listings

Businesses can pay for promoted placement in search results and category pages with transparent bidding system.

FR-057: Competitor Analysis Dashboard

Business owners can view anonymized market trends, competitor pricing, and customer sentiment in their area.

FR-058: Customer Relationship Management

Businesses can manage customer lists, send targeted promotions, track loyalty points, view customer analytics.

FR-059: Multi-Location Support

Businesses with multiple branches can manage all locations from a single dashboard with consolidated reporting.

FR-060: Booking Integration

Direct appointment booking with calendar sync (Google Calendar, Outlook), reminder notifications, and cancellation policies.

4.3 Epic: Community Events (FR-101 to FR-150)

FR-101: Event Creation

Organizers can create events with: Title, description, photos/videos, date/time, location (GPS + address), categories, capacity, ticket types (free/paid), registration form, QR code generation, recurring events.

FR-102: Event Discovery

Users can browse events by: Calendar view, Map view, Category, Location radius, Date range, Popularity, Friends attending, Recommended for you, Free vs paid.

FR-103: Event Registration

One-click registration, ticket purchase, add to calendar, share with friends, group registration, waitlist management.

FR-104: Event Management Dashboard

Organizers can manage: Attendee list, check-in (QR code scanning), real-time attendance, send updates, collect feedback, view analytics, export data.

FR-105: Live Event Streaming

For virtual/hybrid events: Live video streaming, chat, Q&A, polls, recording, replay access, monetization (pay-per-view, donations).

FR-106: Community-Driven Events

Users can propose events, community voting on proposals, crowdfunding for events, volunteer coordination.

FR-107: Event Safety

Emergency contact info at events, first aid station locations, crowd density monitoring (IoT integration), evacuation route maps.

FR-108: Event Reviews & Photos

Post-event reviews, photo sharing, highlight reels, attendee testimonials, organizer reputation building.

4.4 Epic: Community Forums (FR-151 to FR-200)

FR-151: Forum Structure

Hierarchical forums: Neighborhood forums, Interest-based forums, Emergency forums, Help & Support forums, Marketplace forums, Event forums, Government feedback forums.

FR-152: Post Types

Text posts, Photo albums, Videos, Polls, Surveys, Announcements, Questions, Recommendations, Lost & Found, Crime reports (anonymized), Safety alerts.

FR-153: Engagement Features

Likes, reactions, comments, nested replies, shares, bookmarks, save for later, report content, follow threads, mute threads, notification preferences.

FR-154: AI Moderation

Real-time AI content moderation: Hate speech detection, Spam detection, Misinformation flagging, Violence detection, Self-harm detection, NSFW detection, Sentiment analysis, Auto-escalation to human moderators.

FR-155: Reputation System

User karma/points, reputation badges, trusted contributor status, expert verification, community moderator elections, ban/suspend system with appeals.

FR-156: Anonymous Posting

Users can post anonymously in designated forums with anti-abuse measures (rate limiting, device fingerprinting, IP tracking for law enforcement).

FR-157: Real-Time Discussions

Live threads for breaking news, emergency situations, event discussions with real-time updates and WebSocket push notifications.

FR-158: Forum Analytics

Moderators can view: Engagement metrics, Top contributors, Content trends, Sentiment analysis, Report statistics, Growth metrics.

FR-159: Cross-Posting

Ability to cross-post relevant content between forums with attribution and comment aggregation.

FR-160: Forum Archiving

Old threads automatically archived, searchable archive, periodic cleanup of inactive forums, data retention policies.

4.5 Epic: Emergency Response (FR-201 to FR-250)

FR-201: Emergency Alert System

Government officials and authorized responders can send: Geofenced alerts (target specific areas), Multi-channel delivery (push, SMS, in-app, email, sirens), Priority levels (info, warning, critical), Rich media (photos, videos, maps), Acknowledgment tracking, Delivery confirmation.

FR-202: Emergency Reporting

Users can report: Medical emergencies, Fire, Crime, Accidents, Natural disasters, Infrastructure failures, Missing persons, Suspicious activity. Reports include: GPS location, Photo/video, Description, Anonymous option, Real-time chat with responder.

FR-203: Emergency Response Coordination

Dispatch system for: Police, Fire, Ambulance, Disaster response, NGOs.

Features: Real-time location tracking, Resource allocation, Route optimization, Status updates, Handoff protocols, Post-incident reporting.

FR-204: Safety Map

Real-time map showing: Active emergencies, Safe zones, Shelter locations, Hospital capacity, Police stations, Fire stations, Evacuation routes, Road closures, Weather alerts.

FR-205: SOS Feature

One-tap SOS button: Sends location to emergency contacts and responders, Activates audio/video recording, Initiates live chat, Triggers loud alarm, Flashes flashlight, Displays emergency info on lock screen.

FR-206: Emergency Contacts

Users can save emergency contacts, emergency services auto-detect based on location, one-tap calling.

FR-207: Disaster Preparedness

Educational content, checklists, emergency kit inventory, family reunification plans, offline access to critical info.

FR-208: Volunteer Coordination

During emergencies, coordinate volunteers: Skill matching, Location-based assignment, Shift management, Communication channels, Safety tracking, Post-event recognition.

FR-209: Emergency Drills

Scheduled drill notifications, drill participation tracking, feedback collection, performance analytics.

FR-210: Crisis Communication

Dedicated crisis communication channels: Official updates, Rumor control, Resource requests, Volunteer coordination, Family reunification, Media briefings.

4.6 Epic: Government Services (FR-251 to FR-300)

FR-251: Service Directory

Comprehensive directory of government services: Tax payments, License renewals, Permit applications, Complaint filing, Information requests, Appointment booking, Form downloads, Status tracking, Document verification.

FR-252: Tax Payment Integration

Direct integration with tax authorities: Property tax, Income tax, Business tax, VAT, Road tax, Penalty payments, Payment history, Receipt generation, Tax calculator.

FR-253: License & Permit Applications

Digital application workflows: Business licenses, Construction permits, Event permits, Driving licenses, Professional licenses, Application tracking, Renewal reminders, Digital certificates.

FR-254: Complaint Management

Citizen complaint filing: Category selection, Location tagging, Photo/video evidence, Anonymous option, Status tracking, Escalation workflows, Resolution confirmation, Satisfaction survey, Public complaint dashboards (anonymized).

FR-255: Government Notifications

Official government notifications: Policy updates, Public announcements, Election information, Census updates, Health advisories, Weather alerts, Traffic updates, Construction notices.

FR-256: Document Vault

Secure digital document storage: ID cards, Licenses, Certificates, Tax receipts, Utility bills, Medical records, Shareable via QR code, Expiration reminders, Backup to cloud.

FR-257: Public Consultations

Government can post proposals for public feedback: Comment periods, Surveys, Town hall scheduling, Feedback aggregation, Impact reports, Decision transparency.

FR-258: Service Chatbot

AI-powered chatbot for government services: FAQ, Form guidance, Status checks, Appointment scheduling, Multi-language support, Handoff to human agents.

FR-259: Open Data Portal

Public access to anonymized government data: Budgets, Spending, Projects, Statistics, Demographics, Environmental data, API access for developers.

FR-260: Inter-Government Integration

APIs for integration with: National ID systems, Tax databases, Land registries, Business registries, Court systems, Healthcare systems, Education systems.

4.7 Epic: Marketplace & Transactions (FR-301 to FR-350)

FR-301: Product Listings

Businesses can list products with: Photos, videos, 360-degree views, Variants (size, color, material), Inventory tracking, Pricing, Discounts, Shipping options, Return policies, SEO tags.

FR-302: Service Listings

Service providers can list services with: Description, pricing, availability, portfolio, reviews, service area, booking integration, packages.

FR-303: Shopping Cart

Persistent cart across sessions, save for later, wishlist, bundle deals, promo code application, shipping calculator, tax calculation.

FR-304: Checkout Flow

Guest checkout, saved addresses, payment method selection, order review, one-click checkout for returning customers, buy-now-pay-later integration, installment options.

FR-305: Payment Processing

Support for: Credit/debit cards, Bank transfers, Mobile money (e.g., mCash, eZ Cash), Digital wallets, Cryptocurrency (future), Cash on delivery, Installment payments, Subscription billing.

FR-306: Order Management

Order tracking, status updates, shipping notifications, delivery confirmation, proof of delivery (photo/signature), returns/refunds, order history, reorder functionality.

FR-307: Delivery Integration

Integration with delivery partners: Real-time tracking, Route optimization, Delivery scheduling, Contactless delivery, Delivery ratings, Driver management, Fleet analytics.

FR-308: Seller Dashboard

Analytics: Sales, revenue, customers, inventory, refunds, reviews, competitor analysis, marketing tools, payout management, tax reports.

FR-309: Buyer Protection

Dispute resolution, refund guarantee, fraud detection, purchase protection insurance, secure escrow for high-value items.

FR-310: Auction & Bidding

Auction listings, bidding system, reserve prices, auto-bidding, bid notifications, auction analytics, winner notification, payment collection.

4.8 Epic: Notifications & Communication (FR-351 to FR-400)

FR-351: Push Notifications

Rich push notifications with: Images, action buttons, deep links, priority levels, silent notifications, notification grouping, history, scheduled notifications.

FR-352: In-App Messaging

Direct messaging between users, business-to-customer messaging, group chats, message reactions, file sharing, voice messages, message encryption, disappearing messages, read receipts.

FR-353: SMS Fallback

Critical notifications sent via SMS when push delivery fails, with delivery tracking and retry logic.

FR-354: Email Notifications

Transactional emails, marketing emails, digest emails, customizable templates, A/B testing, deliverability monitoring, unsubscribe management.

FR-355: Notification Preferences

Granular notification settings per category, quiet hours, frequency caps, channel selection, batching, do-not-disturb.

FR-356: Announcements

System-wide announcements from platform, government, or businesses with acknowledgment tracking and priority display.

FR-357: Chatbot Integration

AI-powered chatbots for businesses, customer support, government services, community help, with NLP in local languages.

FR-358: Voice & Video Calls

End-to-end encrypted voice and video calls between users, group calls, call recording (with consent), screen sharing.

FR-359: Broadcasting

Businesses and government can broadcast messages to segments: Location-based, demographic-based, interest-based, engagement-based, with analytics.

FR-360: Message Scheduling

Schedule messages for future delivery, recurring messages, timezone-aware delivery, bulk scheduling.

4.9 Epic: Search & Discovery (FR-401 to FR-450)

FR-401: Universal Search

Search across all content types: Businesses, events, forum posts, products, services, government services, users, documents, with autocomplete and spell correction.

FR-402: AI Recommendations

Personalized recommendations based on: Location, behavior, preferences, social graph, time of day, weather, trending content, collaborative filtering, content-based filtering, hybrid models.

FR-403: Trending Content

Real-time trending topics, hashtags, businesses, events, with geo-fencing and category filtering.

FR-404: Map Discovery

Interactive map with layers: Businesses, events, emergencies, traffic, public transport, parking, WiFi hotspots, with AR navigation (future).

FR-405: Voice Search

Voice-activated search in multiple languages, with NLP understanding of local dialects and slang.

FR-406: Visual Search

Upload photos to search for similar products, identify businesses, find events, with computer vision and OCR.

FR-407: Saved Searches

Save and subscribe to search queries, get alerts for new matches, manage search history, export results.

FR-408: Search Analytics

For businesses and admins: Search terms, click-through rates, conversion rates, popular searches, failed searches, search trends.

FR-409: Content Discovery Feeds

Personalized feeds: Community feed, business feed, event feed, news feed, with infinite scroll, content filtering, and refresh mechanisms.

FR-410: Nearby Discovery

Discover nearby content as user moves: Proximity alerts, geo-fenced content, walking/driving mode, location history.

4.10 Epic: Analytics & Reporting (FR-451 to FR-500)

FR-451: User Analytics Dashboard

Real-time user metrics: DAU, MAU, retention, churn, session duration, screen flow, funnel analysis, cohort analysis, A/B test results.

FR-452: Business Analytics Dashboard

Business-specific metrics: Profile views, engagement, conversion, revenue, customer demographics, peak hours, competitor analysis, ROI on promotions.

FR-453: Government Analytics Dashboard

Service usage metrics: Complaint resolution times, service adoption, citizen satisfaction, tax collection efficiency, public consultation participation.

FR-454: Platform Health Dashboard

System metrics: Uptime, latency, error rates, throughput, resource utilization, security incidents, cost analysis.

FR-455: Custom Reports

Ad-hoc report builder with: Drag-and-drop interface, data source selection, filters, visualizations, scheduling, export (PDF, CSV, Excel), sharing.

FR-456: Predictive Analytics

AI-powered predictions: Demand forecasting, churn prediction, fraud detection, trend forecasting, resource optimization, emergency preparedness.

FR-457: Data Export

GDPR-compliant data export for users, businesses, and government entities, with data portability formats.

FR-458: Real-Time Analytics

Sub-second latency analytics for live events, emergency situations, flash sales, with streaming dashboards.

FR-459: Comparative Analytics

Benchmarking tools: Compare performance across regions, time periods, categories, competitors, with anonymized aggregate data.

FR-460: Impact Reporting

Social impact metrics for NGOs and government: Community engagement, economic impact, environmental impact, accessibility metrics, digital inclusion statistics.

Chapter 5: Non-Functional Requirements (NFRs)

5.1 Performance NFRs

ID	Requirement	Target	Measurement
NFR-P01	API Response Time (p50)	< 50ms	APM dashboards

NFR-P02	API Response Time (p95)	< 200ms	APM dashboards
NFR-P03	API Response Time (p99)	< 500ms	APM dashboards
NFR-P04	Page Load Time (Mobile)	< 2s	Lighthouse
NFR-P05	Page Load Time (Web)	< 1.5s	Lighthouse
NFR-P06	Image Load Time	< 500ms	RUM
NFR-P07	Video Start Time	< 2s	RUM
NFR-P08	Search Query Time	< 100ms	APM
NFR-P09	Chat Message Delivery	< 100ms	APM
NFR-P10	Push Notification Delivery	< 5s	Firebase/Apple metrics
NFR-P11	Emergency Alert Delivery	< 2s	Custom monitoring
NFR-P12	Database Query Time (p95)	< 10ms	RDS Performance Insights
NFR-P13	Cache Hit Rate	> 95%	Redis metrics
NFR-P14	CDN Cache Hit Rate	> 90%	CloudFront metrics
NFR-P15	Concurrent Users	10 Million	Load testing
NFR-P16	Transactions Per Second	100,000	Load testing

NFR-P17	Event Ingestion Rate	1 Million events/sec	Kafka metrics
NFR-P18	AI Inference Latency	< 200ms	SageMaker metrics

5.2 Availability & Reliability NFRs

ID	Requirement	Target	Measurement
NFR-A01	Overall Platform Uptime	99.99%	Status page
NFR-A02	Critical Services Uptime	99.999%	Status page
NFR-A03	Emergency Services Uptime	99.9999%	Dedicated monitoring
NFR-A04	Payment Services Uptime	99.999%	Stripe/Adyen metrics
NFR-A05	RTO (Recovery Time Objective)	< 15 minutes	DR drills
NFR-A06	RPO (Recovery Point Objective)	< 1 minute	Backup testing
NFR-A07	MTBF (Mean Time Between Failures)	> 720 hours	Incident tracking
NFR-A08	MTTR (Mean Time To Recovery)	< 30 minutes	Incident tracking
NFR-A09	Zero-Downtime Deployments	100% of deployments	CI/CD logs
NFR-A10	Failover Time	< 30 seconds	Chaos engineering
NFR-A11	Data Durability	99.999999999% (11 nines)	S3 durability metrics

NFR-A12	Backup Success Rate	100%	Backup monitoring
NFR-A13	Backup Verification	Weekly	Automated tests
NFR-A14	Circuit Breaker Activation	< 100ms	Service mesh metrics
NFR-A15	Graceful Degradation	All services	Chaos engineering

5.3 Scalability NFRs

ID	Requirement	Target	Measurement
NFR-S01	Horizontal Pod Scaling	0-1000 pods in 2 minutes	K8s HPA metrics
NFR-S02	Database Read Scaling	100 read replicas	RDS metrics
NFR-S03	Database Write Scaling	10 shards	Sharding metrics
NFR-S04	Cache Scaling	10 Redis clusters	ElastiCache metrics
NFR-S05	Message Queue Scaling	100 partitions	Kafka metrics
NFR-S06	CDN Scaling	10 Tbps	CloudFront metrics
NFR-S07	Storage Scaling	Unlimited	S3 metrics
NFR-S08	User Base Scaling	100 Million users	Load testing
NFR-S09	Geographic Scaling	50+ regions	Deployment metrics

NFR-S10	Feature Flag Scaling	1000+ flags	LaunchDarkly metrics
NFR-S11	Multi-Tenant Scaling	1 Million businesses	Tenant metrics
NFR-S12	API Rate Limit Scaling	10,000 req/sec per user	Gateway metrics

5.4 Security NFRs

ID	Requirement	Target	Measurement
NFR-SEC01	Data Encryption at Rest	AES-256	AWS KMS audit
NFR-SEC02	Data Encryption in Transit	TLS 1.3	SSL Labs rating
NFR-SEC03	mTLS Internal Services	100% of services	Istio metrics
NFR-SEC04	OAuth 2.0 + OIDC	All user auth	Auth0/Okta audit
NFR-SEC05	RBAC Granularity	Resource-level	Penetration testing
NFR-SEC06	API Security	OWASP Top 10	Quarterly pentests
NFR-SEC07	DDoS Protection	10 Tbps	AWS Shield metrics
NFR-SEC08	WAF Protection	All public endpoints	WAF metrics
NFR-SEC09	Vulnerability Scanning	Weekly	Trivy/Qualys reports
NFR-SEC10	Dependency Scanning	Every build	Snyk/Dependabot

NFR-SEC11	Secret Rotation	90 days	Vault audit logs
NFR-SEC12	Penetration Testing	Quarterly	Third-party reports
NFR-SEC13	Security Incident Response	< 15 minutes	SOC metrics
NFR-SEC14	Data Loss Prevention	100% of sensitive data	DLP monitoring
NFR-SEC15	Zero-Trust Architecture	All environments	Architecture review
NFR-SEC16	Identity Verification	99.9% accuracy	KYC provider metrics
NFR-SEC17	Fraud Detection	< 0.1% false positive	ML model metrics
NFR-SEC18	Audit Logging	100% of actions	Audit log completeness
NFR-SEC19	Data Residency	Per-country	Compliance audits
NFR-SEC20	Breach Notification	< 72 hours	Incident response plan

5.5 Maintainability NFRs

ID	Requirement	Target	Measurement
NFR-M01	Code Coverage	> 80%	SonarQube
NFR-M02	Test Coverage	Unit > 80%, Integration > 70%, E2E > 50%	CI/CD reports
NFR-M03	Documentation Coverage	100% of APIs	Swagger/OpenAPI

NFR-M04	Deployment Frequency	Multiple times per day	CI/CD metrics
NFR-M05	Lead Time for Changes	< 1 hour	DORA metrics
NFR-M06	Change Failure Rate	< 5%	DORA metrics
NFR-M07	Mean Time to Recovery	< 1 hour	DORA metrics
NFR-M08	Dependency Update Frequency	Weekly	Renovate/Bot metrics
NFR-M09	Technical Debt Budget	20% of sprint capacity	Jira tracking
NFR-M10	Service Documentation	100% of services	Confluence/Backstage
NFR-M11	API Versioning	2 major versions supported	API Gateway metrics
NFR-M12	Feature Flag Cleanup	Within 2 weeks of GA	LaunchDarkly audit
NFR-M13	Database Migration Time	< 5 minutes	Migration logs
NFR-M14	Rollback Time	< 2 minutes	CI/CD metrics
NFR-M15	Configuration Drift Detection	Real-time	Terraform/ArgoCD

5.6 Usability NFRs

ID	Requirement	Target	Measurement
NFR-U01	App Store Rating	> 4.5	App Store/Play Store

NFR-U02	NPS Score	> 50	Quarterly surveys
NFR-U03	Task Completion Rate	> 95%	UX analytics
NFR-U04	Time to First Action	< 30 seconds	Onboarding analytics
NFR-U05	Accessibility Compliance	WCAG 2.1 AA	Accessibility audits
NFR-U06	Screen Reader Support	100% of screens	Manual testing
NFR-U07	Localization	3 languages (Sinhala, Tamil, English)	Translation coverage
NFR-U08	Offline Functionality	Core features work offline	Offline testing
NFR-U09	Low-Bandwidth Mode	Functional on 2G	Network throttling tests
NFR-U10	Battery Efficiency	< 5% per hour of active use	Battery profiling
NFR-U11	App Size	< 50MB initial download	Build metrics
NFR-U12	Crash-Free Rate	> 99.9%	Crashlytics
NFR-U13	Customer Support Satisfaction	> 90%	Post-ticket surveys
NFR-U14	Help Documentation Completeness	100% of features	Documentation audit
NFR-U15	Onboarding Completion Rate	> 80%	Analytics

5.7 Compliance NFRs

ID	Requirement	Target	Measurement
NFR-C01	GDPR Compliance	Full compliance	Annual audit
NFR-C02	PCI DSS Level 1	Full compliance	Annual QSA audit
NFR-C03	ISO 27001	Certification	Annual certification audit
NFR-C04	SOC 2 Type II	Certification	Annual audit
NFR-C05	Data Protection Act (Sri Lanka)	Full compliance	Legal review
NFR-C06	Accessibility Standards	WCAG 2.1 AA	Third-party audit
NFR-C07	Data Retention	Per legal requirements	Data governance audit
NFR-C08	Right to Erasure	< 30 days	GDPR compliance testing
NFR-C09	Data Portability	Standard formats	Compliance testing
NFR-C10	Cross-Border Data Transfer	Legal mechanisms	Legal review
NFR-C11	Audit Trail Retention	7 years	Audit log verification
NFR-C12	Incident Reporting	< 72 hours	Incident response testing
NFR-C13	Third-Party Risk Assessment	Annual	Vendor management
NFR-C14	Business Continuity	ISO 22301	Annual certification

NFR-C15	Environmental Compliance	ISO 14001	Annual certification
---------	--------------------------	-----------	----------------------

5.8 Observability NFRs

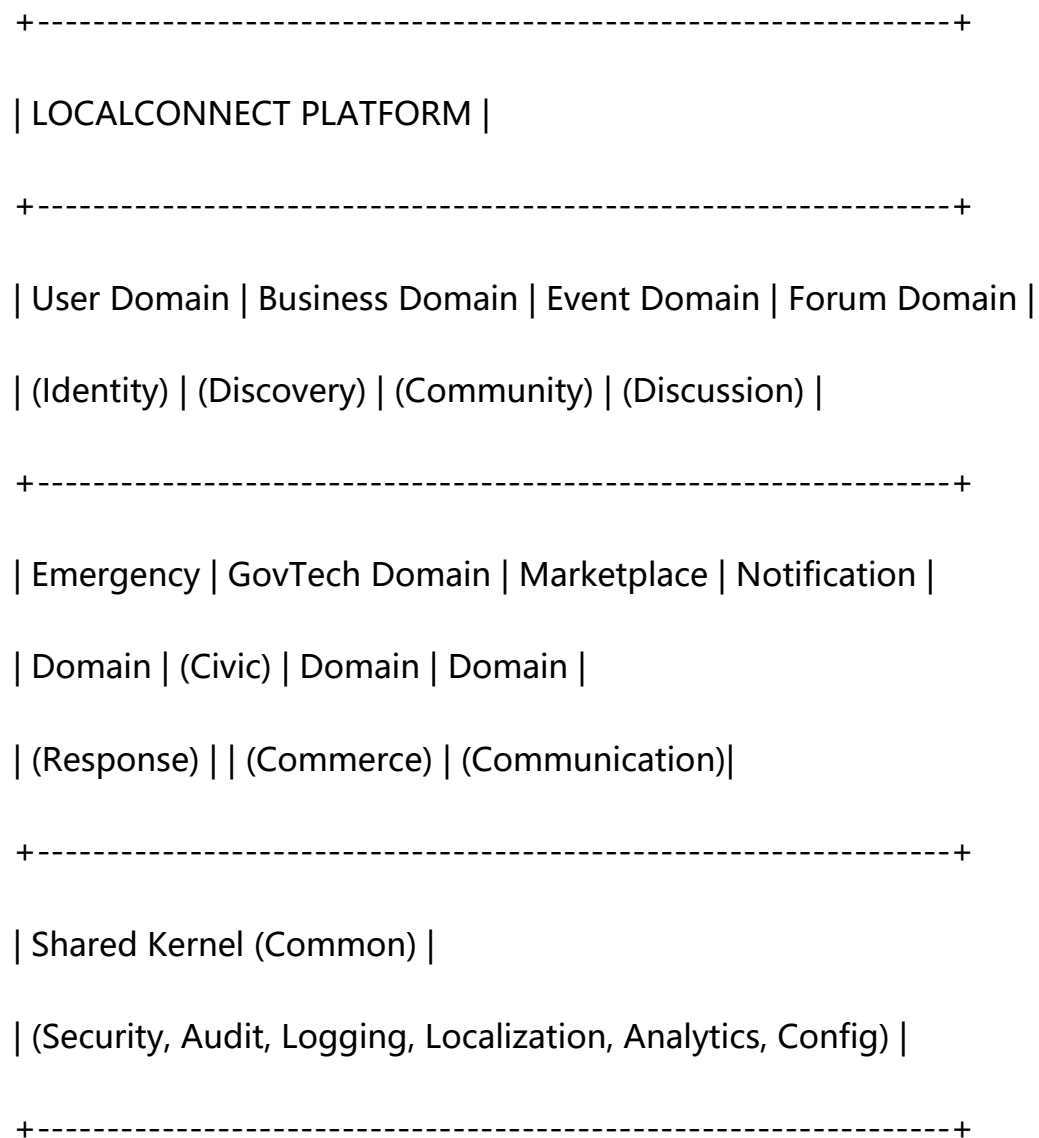
ID	Requirement	Target	Measurement
NFR-O01	Log Retention	90 days hot, 1 year warm, 7 years cold	ELK indices
NFR-O02	Metric Retention	15 months	Prometheus
NFR-O03	Trace Sampling	100% of errors, 1% of success	Jaeger config
NFR-O04	Alert Response Time	< 5 minutes	PagerDuty metrics
NFR-O05	Dashboard Refresh	< 5 seconds	Grafana config
NFR-O06	Log Correlation	100% of requests	Trace IDs
NFR-O07	Error Classification	Automated	Sentry/AI classification
NFR-O08	Capacity Planning	Weekly reviews	Capacity reports
NFR-O09	Cost Monitoring	Daily alerts	CloudHealth/FinOps
NFR-O10	Performance Regression	Auto-detected in CI	Benchmark testing

Chapter 6: Domain-Driven Design (DDD)

6.1 Strategic DDD

LocalConnect is decomposed into 8 core domains, each representing a major business capability. Each domain is implemented as a bounded context with its own ubiquitous language, data models, and API contracts.

Domain Map



6.2 Core Domains

6.2.1 User Domain (Core Domain)

- Purpose: User identity, authentication, authorization, profile management, KYC
- Ubiquitous Language: User, Account, Profile, Role, Permission, KYC, Verification, Session, Token, MFA, Identity Provider
- Aggregates: User Aggregate, Account Aggregate, Role Aggregate, Session Aggregate, KYC Aggregate
- Entities: User, Account, Profile, Role, Permission, VerificationDocument, Session, AuditLog
- Value Objects: Email, PhoneNumber, Address, Name, DateOfBirth, IdentityDocument, BiometricData
- Domain Events: UserRegistered, UserVerified, RoleAssigned, PermissionGranted, SessionCreated, MFAEnabled, KYCApproved, KYCRejected
- Business Rules:
 - Email must be unique and verified before account activation
 - Phone OTP must expire in 5 minutes
 - KYC documents must be validated within 24 hours
 - Admin roles require two-person approval
 - Biometric data must be encrypted with hardware security module
- Integration Patterns: Event publishing to Kafka, API calls from other domains, CQRS for read-heavy queries
- Data Model: PostgreSQL (users, accounts, roles), Redis (sessions, tokens), S3 (KYC documents)
- Team: 8 engineers (2 senior, 4 mid, 2 junior)

6.2.2 Business Domain (Core Domain)

- Purpose: Business discovery, profiles, categories, promotions, analytics
- Ubiquitous Language: Business, Listing, Category, Review, Promotion, Claim, Analytics, Competitor, Insight
- Aggregates: Business Aggregate, Category Aggregate, Review Aggregate, Promotion Aggregate, Claim Aggregate
- Entities: Business, Listing, Category, Review, Promotion, BusinessClaim, CompetitorInsight, CustomerAnalytics

- Value Objects: BusinessHours, Location, Rating, PriceRange, ContactInfo, SocialLinks, MediaAsset
- Domain Events: BusinessCreated, BusinessClaimed, ReviewSubmitted, PromotionStarted, CategoryUpdated, CompetitorAnalyzed
- Business Rules:
 - Business names must be unique within a 5km radius
 - Reviews can only be submitted by verified users who visited the business
 - Promotions must be approved by moderation AI before going live
 - Claimed businesses require re-verification annually
 - Competitor data is anonymized and aggregated
- Integration Patterns: Event-driven updates, GraphQL for complex queries, Elasticsearch for search
- Data Model: PostgreSQL (businesses, categories), MongoDB (reviews, analytics), Elasticsearch (search index), S3 (media)
- Team: 10 engineers (3 senior, 5 mid, 2 junior)

6.2.3 Event Domain (Core Domain)

- Purpose: Event creation, discovery, registration, management, live streaming
- Ubiquitous Language: Event, Organizer, Attendee, Ticket, Registration, Venue, Schedule, CheckIn, LiveStream, Recording
- Aggregates: Event Aggregate, Organizer Aggregate, Registration Aggregate, Ticket Aggregate, Venue Aggregate
- Entities: Event, Organizer, Attendee, Ticket, Registration, Venue, Schedule, CheckIn, LiveStream, Recording, Volunteer
- Value Objects: EventDateTime, EventLocation, TicketType, Capacity, Price, EventMedia, EventTag
- Domain Events: EventCreated, EventPublished, RegistrationOpened, TicketPurchased, AttendeeCheckedIn, LiveStreamStarted, EventCompleted, FeedbackSubmitted
- Business Rules:

- Events must be published at least 24 hours before start time
 - Ticket sales close 1 hour before event start
 - Refunds allowed up to 48 hours before event
 - Check-in requires QR code validation
 - Live streams require 2FA for organizer
- Integration Patterns: CQRS for event listings, Event sourcing for registration state, WebSocket for live updates
- Data Model: PostgreSQL (events, registrations), MongoDB (live stream metadata), S3 (recordings, media), Redis (real-time check-in)
- Team: 8 engineers (2 senior, 4 mid, 2 junior)

6.2.4 Forum Domain (Supporting Domain)

- Purpose: Community discussions, moderation, reputation, content management
- Ubiquitous Language: Forum, Thread, Post, Comment, Moderator, Reputation, Karma, Flag, Ban, Appeal, Tag
- Aggregates: Forum Aggregate, Thread Aggregate, Post Aggregate, Moderator Aggregate, Reputation Aggregate
- Entities: Forum, Thread, Post, Comment, Moderator, ReputationRecord, Flag, Ban, Appeal, Tag, Vote
- Value Objects: PostContent, ModerationDecision, ReputationScore, BanDuration, AppealReason, ContentFlag
- Domain Events: ForumCreated, ThreadStarted, PostSubmitted, CommentAdded, PostFlagged, ModeratorActionTaken, ReputationChanged, BanApplied, BanLifted, AppealResolved
- Business Rules:
 - New users cannot post for 24 hours after registration (anti-spam)
 - Posts flagged by 3+ users are auto-hidden pending review
 - All moderation decisions can be appealed within 48 hours
 - Moderators must provide reason for all actions
 - Reputation below -50 results in temporary posting restrictions
- Integration Patterns: Event-driven moderation, CQRS for thread listings, Redis for real-time vote counts

- Data Model: PostgreSQL (forums, threads, posts), MongoDB (content metadata), Redis (vote counts, hot feeds), S3 (media attachments)
- Team: 6 engineers (2 senior, 3 mid, 1 junior)

6.2.5 Emergency Domain (Core Domain)

- Purpose: Emergency alerting, reporting, response coordination, safety maps
- Ubiquitous Language: Emergency, Alert, Responder, Dispatch, Incident, SafetyZone, EvacuationRoute, SOS, Volunteer, Crisis, Drills
- Aggregates: EmergencyAggregate, AlertAggregate, IncidentAggregate, ResponderAggregate, SafetyMapAggregate
- Entities: Emergency, Alert, Incident, Responder, Dispatch, SafetyZone, EvacuationRoute, SOSRecord, Volunteer, CrisisCommunication, Drill
- Value Objects: EmergencyLocation, PriorityLevel, AlertType, ResponseTime, ResourceAllocation, IncidentStatus, GPSCoordinate
- Domain Events: EmergencyDeclared, AlertSent, IncidentReported, ResponderDispatched, SOSActivated, SafetyZoneUpdated, DrillInitiated, CrisisCommunicated, VolunteerAssigned
- Business Rules:
 - Critical alerts must be delivered within 2 seconds to 99.999% of recipients
 - Emergency services must acknowledge receipt within 30 seconds
 - SOS records all available data for 72 hours
 - Incident reports are immutable (audit trail)
 - Volunteer assignments match skills and location
- Integration Patterns: Priority message queue, Dedicated infrastructure, Circuit breaker with fast fail, Geo-replicated caching
- Data Model: PostgreSQL (emergencies, incidents), DynamoDB (alerts - high throughput), Redis (safety map - geo-replicated), S3 (incident evidence), Kafka (event stream)
- Team: 10 engineers (4 senior, 4 mid, 2 junior) - highest priority team

6.2.6 GovTech Domain (Supporting Domain)

- Purpose: Government service integration, citizen services, document vault, open data
- Ubiquitous Language: GovernmentService, Citizen, Tax, License, Permit, Complaint, DocumentVault, PublicConsultation, OpenData, Chatbot
- Aggregates: GovernmentServiceAggregate, TaxAggregate, LicenseAggregate, ComplaintAggregate, DocumentVaultAggregate
- Entities: GovernmentService, TaxPayment, License, Permit, Complaint, DocumentVault, PublicConsultation, OpenDataSet, GovernmentChatbot, ServiceForm
- Value Objects: TaxAmount, LicenseType, PermitStatus, ComplaintCategory, DocumentType, ConsultationPeriod, DataFormat, FormField
- Domain Events: ServiceAccessed, TaxPaid, LicenseApplied, ComplaintFiled, DocumentUploaded, ConsultationOpened, DataPublished, FormSubmitted
- Business Rules:
 - All government transactions require citizen ID verification
 - Complaint status must be updated within 48 hours
 - Tax payments generate immutable receipt
 - Document vault uses AES-256 encryption with citizen key
 - Public consultations require minimum 30-day comment period
- Integration Patterns: API gateway to government systems, Event-driven status updates, Secure document storage, CQRS for service catalogs
- Data Model: PostgreSQL (services, licenses, complaints), MongoDB (forms, consultations), S3 (documents, vault), DynamoDB (high-throughput tax payments)
- Team: 8 engineers (2 senior, 4 mid, 2 junior)

6.2.7 Marketplace Domain (Core Domain)

- Purpose: Product/service listings, transactions, payments, order management, delivery
- Ubiquitous Language: Product, Service, Listing, Cart, Order, Payment, Transaction, Refund, Delivery, Seller, Buyer, Inventory, Shipment, Invoice
- Aggregates: ProductAggregate, OrderAggregate, PaymentAggregate, DeliveryAggregate, SellerAggregate, InventoryAggregate
- Entities: Product, Service, Listing, Cart, Order, Payment, Transaction, Refund, Delivery, Seller, Buyer, Inventory, Shipment, Invoice, Review, Promotion
- Value Objects: Price, Quantity, SKU, PaymentMethod, ShippingAddress, OrderStatus, PaymentStatus, DeliveryStatus, TaxRate, DiscountCode
- Domain Events: ProductListed, CartUpdated, OrderPlaced, PaymentInitiated, PaymentCompleted, OrderShipped, DeliveryConfirmed, RefundProcessed, InventoryUpdated, InvoiceGenerated
- Business Rules:
 - Payments must be held in escrow until delivery confirmation
 - Refunds processed within 5 business days
 - Sellers must verify identity before listing products
 - Inventory updates are real-time (eventual consistency < 1 second)
 - High-value transactions require buyer/seller verification
- Integration Patterns: Saga pattern for order flow, Event sourcing for payment state, CQRS for inventory, Webhook for payment providers
- Data Model: PostgreSQL (orders, products, payments), MongoDB (catalog, seller profiles), Redis (cart, inventory cache), DynamoDB (high-throughput transactions), S3 (product images)
- Team: 12 engineers (4 senior, 6 mid, 2 junior)

6.2.8 Notification Domain (Generic Domain)

- Purpose: Multi-channel notifications, messaging, preferences, scheduling, broadcasts

- Ubiquitous Language: Notification, Message, Channel, Preference, Template, Campaign, Broadcast, Schedule, Digest, ChannelDelivery
- Aggregates: NotificationAggregate, MessageAggregate, PreferenceAggregate, TemplateAggregate, CampaignAggregate
- Entities: Notification, Message, Channel, Preference, Template, Campaign, Broadcast, Schedule, Digest, ChannelDelivery, DeliveryReceipt
- Value Objects: NotificationContent, ChannelType, Priority, DeliveryTime, ReadStatus, TemplateVariable, CampaignMetrics, MessageContent
- Domain Events: NotificationCreated, NotificationSent, MessageReceived, MessageRead, PreferenceUpdated, TemplateCreated, CampaignLaunched, BroadcastScheduled, DeliveryFailed, DeliveryRetried
- Business Rules:
 - Critical notifications bypass all preference settings
 - SMS fallback triggered if push not delivered within 30 seconds
 - Marketing messages require explicit opt-in
 - Notification frequency caps per user per channel
 - Message encryption for sensitive content
- Integration Patterns: Multi-channel adapter pattern, Priority queue routing, Batch processing for digests, Template engine with variables
- Data Model: PostgreSQL (notifications, templates), MongoDB (messages, campaigns), Redis (delivery queues, preferences cache), S3 (media attachments), DynamoDB (high-throughput delivery tracking)
- Team: 6 engineers (2 senior, 3 mid, 1 junior)

6.3 Shared Kernel (Common Domain)

6.3.1 Security Service

- Authentication, authorization, encryption, token management, audit logging
- Used by all domains

- Implemented as a shared library + microservice

6.3.2 Audit Service

- Immutable audit trail for all user actions, system events, data changes
- Compliance requirement (7-year retention)
- Write-once, read-many pattern

6.3.3 Localization Service

- Multi-language support (Sinhala, Tamil, English)
- RTL/LTR handling, date/time formatting, currency formatting, number formatting
- Content translation pipeline with AI + human review

6.3.4 Analytics Service

- Event tracking, metrics collection, user behavior analytics
- Privacy-compliant (anonymized, consent-based)
- Real-time and batch processing

6.3.5 Configuration Service

- Feature flags, A/B test configuration, system settings, rate limits
- Dynamic configuration without deployment
- Environment-specific overrides

6.3.6 Common Value Objects

- Money (amount, currency, precision)
- DateTime (UTC with timezone awareness)
- GeoLocation (latitude, longitude, accuracy, address)
- MediaReference (URL, type, size, format, CDN)
- ContactInfo (email, phone, address, social links)

- Status (enum with state machine transitions)

6.4 Domain Relationships

Context Map

User Domain <--[Customer-Supplier]--> Business Domain

User Domain <--[Customer-Supplier]--> Event Domain

User Domain <--[Customer-Supplier]--> Forum Domain

User Domain <--[Customer-Supplier]--> Emergency Domain

User Domain <--[Customer-Supplier]--> GovTech Domain

User Domain <--[Customer-Supplier]--> Marketplace Domain

User Domain <--[Shared Kernel]--> Notification Domain

Business Domain <--[Published Language]--> Event Domain

Business Domain <--[Published Language]--> Marketplace Domain

Business Domain <--[Anti-Corruption Layer]--> GovTech Domain

Event Domain <--[Published Language]--> Forum Domain

Event Domain <--[Published Language]--> Notification Domain

Forum Domain <--[Published Language]--> Emergency Domain

Forum Domain <--[Published Language]--> GovTech Domain

Emergency Domain <--[Customer-Supplier]--> Notification Domain

Emergency Domain <--[Anti-Corruption Layer]--> GovTech Domain

GovTech Domain <--[Published Language]--> Marketplace Domain

GovTech Domain <--[Anti-Corruption Layer]--> Notification Domain

Marketplace Domain <--[Customer-Supplier]--> Notification Domain

All Domains <--[Shared Kernel]--> Security, Audit, Localization, Analytics,
Config

Integration Patterns Summary

Relationship	Pattern	Communication	Data Consistency
User → All	Customer-Supplier	REST API + Events	Eventual
Business → Event	Published Language	Events	Eventual
Business → Marketplace	Published Language	Events	Eventual
Business → GovTech	Anti-Corruption Layer	REST API	Strong
Event → Forum	Published Language	Events	Eventual
Event → Notification	Published Language	Events	Eventual
Forum → Emergency	Published Language	Events	Eventual
Forum → GovTech	Published Language	Events	Eventual
Emergency → Notification	Customer-Supplier	Priority REST + Events	Strong

Emergency → GovTech	Anti-Corruption Layer	REST API	Strong
GovTech → Marketplace	Published Language	Events	Eventual
GovTech → Notification	Anti-Corruption Layer	REST API	Strong
Marketplace → Notification	Customer-Supplier	Events	Eventual
All → Shared Kernel	Shared Kernel	Library + REST	N/A

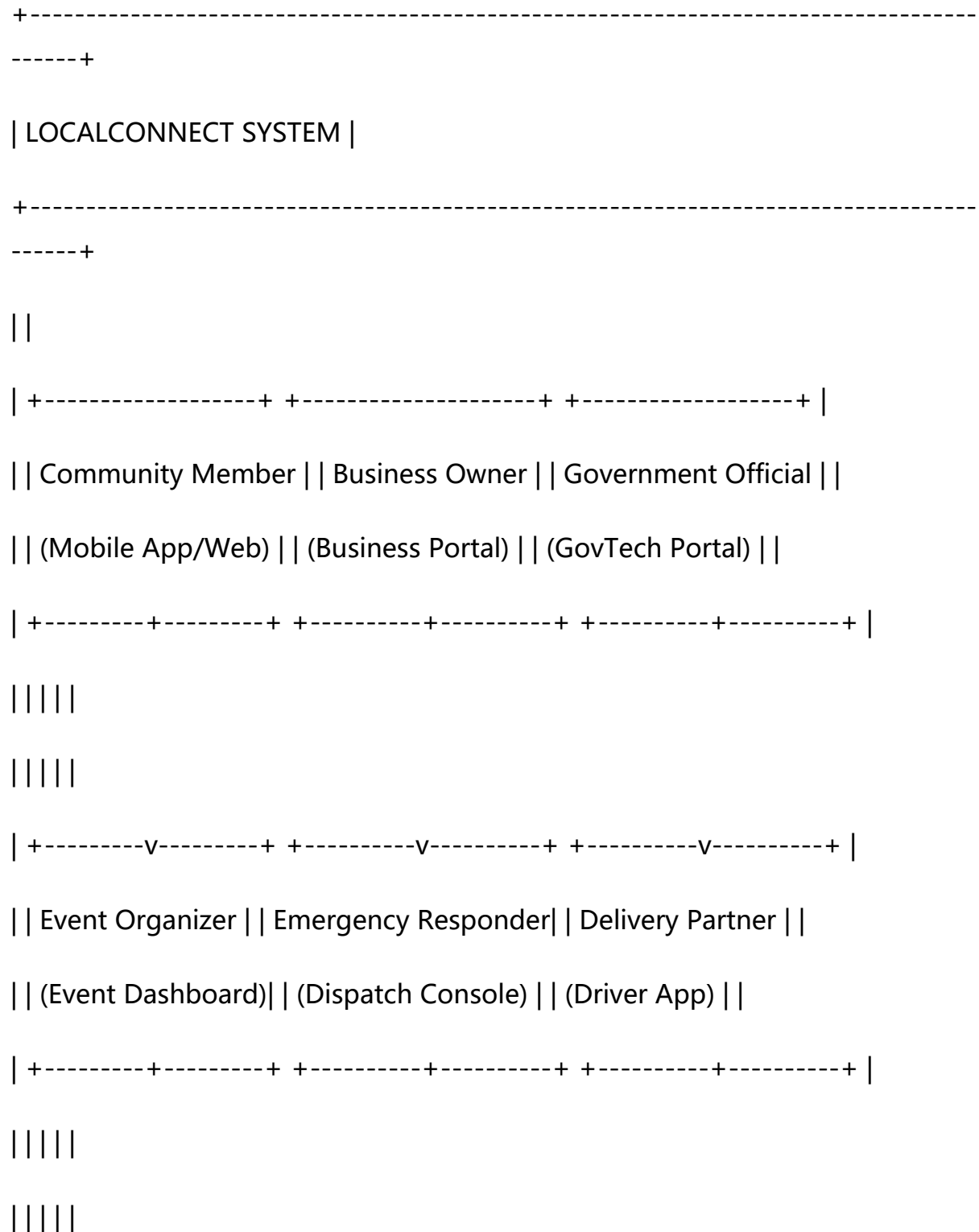
6.5 Domain Event Flow Example: User Places an Order

1. User Domain: UserAuthenticated (emitted)
2. Marketplace Domain: OrderPlaced (emitted) — User Domain consumes for audit
3. Marketplace Domain: PaymentInitiated (emitted) — triggers Payment Service
4. Payment Service: PaymentCompleted (emitted) — triggers Order fulfillment
5. Marketplace Domain: OrderShipped (emitted) — triggers Notification Domain
6. Notification Domain: NotificationSent (push to buyer, push to seller)
7. Marketplace Domain: DeliveryConfirmed (emitted) — triggers Payment release
8. Payment Service: FundsReleased (emitted) — seller receives payment
9. Audit Service: All events logged for compliance
10. Analytics Service: Purchase event tracked for recommendations

Chapter 7: C4 Architecture

The C4 Model (Context, Containers, Components, Code) provides a structured way to visualize software architecture at multiple levels of abstraction.

7.1 System Context Diagram (Level 1)



| +-----+-----+ |

|||

| v |

| +-----+ |

| | LOCALCONNECT PLATFORM | |

| | (Mobile, Web, APIs) | |

| +-----+ |

|||

|||

| +-----+-----+ |

|||||

| +-----v-----+ +-----v-----+ +-----v-----+ |

| | Payment Providers | | Government APIs | | Notification | |

| | (Stripe, Banks) | | (Tax, License) | | (SMS, Email, | |

| | | | | Push, Firebase) | |

| +-----+ +-----+ +-----+ |

||

| +-----+ +-----+ +-----+ |

| | CDN / CloudFront | | AI/ML Services | | Analytics | |

| | (Static Assets) | | (Bedrock, Sage) | | (ClickHouse, | |

||||| Mixpanel) ||

| +-----+ +-----+ +-----+ |

||

+-----
-----+

7.2 Container Diagram (Level 2)

+-----
-----+

| LOCALCONNECT PLATFORM CONTAINERS |

+-----
-----+

||

| USERS +-----+ |

| +-----+ +-----+ +-----+ | API Gateway (Kong) ||

| |Mobile| | Web| | Admin |-----> | - Authentication ||

| | App| | App| |Portal| | - Rate Limiting ||

| +---+---+ +---+---+ +---+---+ | - Routing ||

|||| - Load Balancing ||

|||| +-----+-----+ |

|||||

|||| +-----+-----+ |

|||||||

||| +----v----+ +-----v-----+ +-----v-----+ |

|||| REST API || GraphQL API || gRPC API ||

|||| (CRUD) || (Queries) || (Internal) |

||| +----+----+ +-----+-----+ +-----+-----+ |

|||||||

|||||||

||| +----v-----v-----v-----+ |

|||| SERVICE MESH (Istio) |

|||| - mTLS, Circuit Breaker, Retry, Timeout |

||| +----+-----+-----+-----+ |

|||||||

||| +----v----+ +-----v-----+ +-----v-----+ |

|||| User Svc || Business Svc || Event Svc ||

|||| (Auth) || (Discovery) || (Community) |

||| +----+----+ +-----+-----+ +-----+-----+ |

|||||||

||| +----v----+ +-----v-----+ +-----v-----+ |

|||| Forum Svc || Emergency Svc | GovTech Svc |

|||| (Discuss) || (Response) || (Civic) |

|||| +----+----+ +-----+-----+ +-----+-----+ |

|||||||

|||| +----v----+ +-----v-----+ +-----v-----+ |

|||| | Market Svc | | Notify Svc | | AI Svc | |

|||| | (Commerce) | | (Messaging) | | (ML/LLM) | |

|||| +----+----+ +-----+-----+ +-----+-----+ |

|||||||

|||| +----v----+ +-----v-----+ +-----v-----+ |

|||| | Search Svc | | Analytics Svc | | Config Svc | |

|||| | (Elastic) | | (Tracking) | | (Flags) | |

|||| +----+----+ +-----+-----+ +-----+-----+ |

|||||||

|||||||

|||| +----v-----v-----v-----+ |

|||| | DATA LAYER | |

|||| | PostgreSQL | MongoDB | Redis | DynamoDB | S3 | |

|||| +----+----+----+----+----+----+----+----+----+----+ |

|||||||

|||| +----v----+ +----v----+ +----v----+ +----v----+ |

|||| | Kafka | | ClickHouse | | SQS/SNS | | ElastiCache | |

|||| (Events)| |(Analytics)| |(Queues)| |(Sessions)| |

|||| +-----+-----+ +-----+-----+ +-----+-----+ +-----+-----+ |

|||||||

|||| +----v-----v-----v-----+ |

|||| INFRASTRUCTURE LAYER ||

|||| EKS | Lambda | CloudFront | Route53 | WAF | Shield| |

|||| +-----+-----+-----+-----+-----+-----+-----+-----+-----+ |

||

+-----
-----+

7.3 Component Diagram (Level 3) — Example: User Service

+-----
-----+

| USER SERVICE (Container: User Svc) |

+-----
-----+

||

| API LAYER |

| +-----+ +-----+ +-----+ +-----+ |

| | Auth Controller| | User Controller| | KYC Controller | | Admin Controller| |

|| - Login/Logout || - Profile CRUD || - Document Upload| | - User Mgmt | |

|| - OAuth || - Preferences || - Verification || - Role Mgmt | |

|| - MFA || - Search || - Status Check| | - Audit | |

| +-----+-----+ +-----+-----+ +-----+-----+ +-----+-----+ |

|||||

| +-----v-----v-----v-----v-----+ |

|| SERVICE LAYER ||

|| +-----+ +-----+ +-----+ +-----+ ||

||| Auth Service || User Service || KYC Service || Admin |||

||| - Token Mgmt || - Profile Mgmt || - Document Proc| Service |||

||| - Session Mgmt || - Privacy Ctrl || - Verification || - RBAC |||

||| - Password Mgmt| | - Deletion | | - Compliance | | - Audit |||

| | +-----+-----+ +-----+-----+ +-----+-----+ +-----+ ||

|||||||

| | +-----v-----v-----v-----v-----+ |

||| DOMAIN LAYER ||

||| +-----+ +-----+ +-----+ +-----+ ||

|||| User Aggregate || Account Agg. || KYC Aggregate || Role |||

|||| - User Entity || - Account Ent. || - Document Ent.| Agg. |||

|||| - Profile VO || - Session VO || - Status VO || - Perm |||

||| - Address VO || - Token VO || - Review VO || VO |||

|| +-----+-----+ +-----+-----+ +-----+-----+ +-----+ ||

|||||||

|| +-----+-----+-----+-----+-----+
|

||||

| +-----+-----+-----+-----+-----+ |

||||

| +-----v-----+-----+-----+ |

|| INFRASTRUCTURE LAYER ||

|| +-----+ +-----+ +-----+ +-----+ ||

||| User Repository| | KYC Repository | | Audit Repository| | Cache |||

||| (PostgreSQL) || (PostgreSQL) || (S3 + Dynamo) || (Redis) |||

|| +-----+ +-----+ +-----+ +-----+ ||

|| +-----+ +-----+ +-----+ +-----+ ||

||| Event Publisher| | Token Store | | Session Store | | Config |||

||| (Kafka) || (Redis) || (Redis) || (Consul) |||

|| +-----+ +-----+ +-----+ +-----+ ||

| +-----+
|

||

+-----+
-----+

7.4 Code Diagram (Level 4) — Example: User Aggregate

+-----+
-----+

| USER AGGREGATE (DDD) |

+-----+
-----+

||

| Aggregate Root: User |

| +-----+ |

|| class User { ||

|| UUID id; ||

|| Email email; ||

|| PhoneNumber phone; ||

|| UserStatus status; ||

|| Profile profile; ||

|| List<Role> roles; ||

|| LocalDateTime createdAt; ||

|| LocalDateTime updatedAt; ||

| | |

| | void register() | |

| | void verify() | |

| | void addRole(Role role) | |

| | void removeRole(Role role) | |

| | void updateProfile(...) | |

| | void deactivate() | |

| | void delete() | |

| | } | |

| +-----+ |

| |

| Entities: |

| +-----+ +-----+ +-----+
+ |

| | class Profile { | | class VerificationDocument { | | class Role { | |

| | UUID id; | | UUID id; | | UUID id; | |

| | Name displayName; | | DocumentType type; | | String name; | |

| | Bio bio; | | MediaReference document; | | List<Permission> | |

| | MediaReference avatar; | | VerificationStatus status; | | boolean isAdmin; | |

| | Location location; | | LocalDateTime submittedAt; | | } | |

| | PrivacySettings privacy; | | LocalDateTime reviewedAt; | +-----+ |

| | List<Language> languages; | | String reviewerNotes; | |

| | } | | }

| +-----+ +-----+ |

| |

| Value Objects: |

| +-----+ +-----+ +-----+ +-----+ |

| | class Email { | | class PhoneNumber { | | class Name { | | class Location { | |

| | String value; | | String country; | | String first; | | double lat; | |

| | boolean verified; | | String number; | | String last; | | double lng; | |

| | void verify() | | boolean verified; | | String display; | | String address; | |

| | } | | } | | } | | double accuracy; | |

| +-----+ +-----+ +-----+ +-----+ |

| |

| Domain Events: |

| +-----+ +-----+ +-----+
+ |

| | UserRegistered { | | UserVerified { | | RoleAssigned { | |

| | UUID userId; | | UUID userId; | | UUID userId; | |

| | Email email; | | VerificationMethod method; | | Role role; | |

```

| | LocalDateTime occurredAt; | | LocalDateTime occurredAt; | | LocalDateTime |
|

| | } | | } | | occurredAt; | |

| +-----+ +-----+ +-----+
+ |

| |

+-----+
-----+

```

Chapter 8: Complete Microservice Architecture

8.1 Service Catalog

LocalConnect comprises 40+ microservices organized by domain. Each service follows the 12-Factor App methodology and is independently deployable, scalable, and maintainable.

8.1.1 User Domain Services

Service	Tech	DB	Purpose	Instances	Scaling
auth-service	Node.js/Express	PostgreSQL, Redis	Authentication, OAuth, MFA, session management	3-20	CPU-based HPA
user-service	Java/Spring Boot	PostgreSQL, Redis	User profiles, preferences, privacy, deletion	3-20	CPU-based HPA

kyc-service	Python/FastAPI	PostgreSQL, S3	KYC verification, document processing, AI validation	2-10	Queue-based HPA
identity-service	Go	PostgreSQL, Redis	Identity provider, SSO, token management	2-10	CPU-based HPA
rbac-service	Java/Spring Boot	PostgreSQL	Role-based access control, permissions, policies	2-10	CPU-based HPA
session-service	Node.js	Redis	Session management, concurrent device tracking	3-15	Memory-based HPA

8.1.2 Business Domain Services

Service	Tech	DB	Purpose	Instances	Scaling
business-service	Java/Spring Boot	PostgreSQL, MongoDB	Business profiles, categories, claims	3-20	CPU-based HPA
review-service	Node.js	MongoDB, PostgreSQL	Reviews, ratings, sentiment analysis	3-20	CPU-based HPA
promotion-service	Python/FastAPI	PostgreSQL, Redis	Promotions, ads, bidding, analytics	2-10	CPU-based HPA

discovery-service	Go	Elasticsearch, Redis	Search, discovery, recommendations	3-20	CPU-based HPA
competitor-service	Python/FastAPI	PostgreSQL, ClickHouse	Competitor analysis, market insights	2-10	CPU-based HPA
booking-service	Java/Spring Boot	PostgreSQL, Redis	Appointment booking, calendar sync	2-10	CPU-based HPA

8.1.3 Event Domain Services

Service	Tech	DB	Purpose	Instances	Scaling
event-service	Java/Spring Boot	PostgreSQL, MongoDB	Event creation, management, discovery	3-20	CPU-based HPA
registration-service	Node.js	PostgreSQL, Redis	Event registration, ticketing, check-in	3-20	CPU-based HPA
livestream-service	Go	MongoDB, S3	Live streaming, recording, replay	2-10	GPU-based HPA
venue-service	Python/FastAPI	PostgreSQL	Venue management, capacity, amenities	2-10	CPU-based HPA

8.1.4 Forum Domain Services

Service	Tech	DB	Purpose	Instances	Scaling
---------	------	----	---------	-----------	---------

forum-service	Java/Spring Boot	PostgreSQL, MongoDB	Forum structure, threads, posts	3-20	CPU-based HPA
moderation-service	Python/FastAPI	PostgreSQL, Redis	AI moderation, human review queue, appeals	3-20	GPU-based HPA
reputation-service	Node.js	PostgreSQL, Redis	Karma, badges, reputation scoring	2-10	CPU-based HPA
content-service	Go	MongoDB, S3	Content management, media, archiving	3-20	CPU-based HPA

8.1.5 Emergency Domain Services

Service	Tech	DB	Purpose	Instances	Scaling
alert-service	Go	DynamoDB, Redis	Emergency alerting, geofencing, delivery	5-30	Custom HPA (alert volume)
incident-service	Java/Spring Boot	PostgreSQL, S3	Incident reporting, tracking, analytics	3-20	CPU-based HPA
dispatch-service	Go	PostgreSQL, Redis	Responder dispatch, resource allocation, routing	3-20	CPU-based HPA

safety-map-service	Python/FastAPI	Redis, PostgreSQL	Safety maps, zones, evacuation routes	2-10	CPU-based HPA
sos-service	Go	DynamoDB, S3	SOS activation, evidence recording, emergency chat	3-20	CPU-based HPA
volunteer-service	Node.js	PostgreSQL	Volunteer coordination, skill matching, shifts	2-10	CPU-based HPA

8.1.6 GovTech Domain Services

Service	Tech	DB	Purpose	Instances	Scaling
gov-service	Java/Spring Boot	PostgreSQL, MongoDB	Government service directory, integration	2-10	CPU-based HPA
tax-service	Python/FastAPI	PostgreSQL, DynamoDB	Tax payments, history, calculator	2-10	CPU-based HPA
license-service	Java/Spring Boot	PostgreSQL	License applications, renewals, tracking	2-10	CPU-based HPA
complaint-service	Node.js	PostgreSQL, MongoDB	Complaint filing, tracking, resolution	2-10	CPU-based HPA

document-vault-service	Go	S3, PostgreSQL	Secure document storage, sharing, encryption	2-10	CPU-based HPA
consultation-service	Python/FastAPI	PostgreSQL	Public consultations, feedback, surveys	2-10	CPU-based HPA
gov-chatbot-service	Python/FastAPI	MongoDB, Redis	AI chatbot for government services	2-10	GPU-based HPA
open-data-service	Node.js	PostgreSQL, ClickHouse	Open data portal, API access, analytics	2-10	CPU-based HPA

8.1.7 Marketplace Domain Services

Service	Tech	DB	Purpose	Instances	Scaling
catalog-service	Java/Spring Boot	PostgreSQL, MongoDB	Product catalog, categories, inventory	3-20	CPU-based HPA
inventory-service	Go	PostgreSQL, Redis	Real-time inventory tracking, updates	3-20	CPU-based HPA
order-service	Java/Spring Boot	PostgreSQL, DynamoDB	Order management, state machine, tracking	3-20	CPU-based HPA

payment-service	Go	PostgreSQL, Redis	Payment processing, escrow, refunds, fraud	3-20	CPU-based HPA
cart-service	Node.js	Redis	Shopping cart, wishlist, save for later	3-20	Memory-based HPA
delivery-service	Python/FastAPI	PostgreSQL, Redis	Delivery tracking, route optimization, driver mgmt	3-20	CPU-based HPA
seller-service	Java/Spring Boot	PostgreSQL, MongoDB	Seller profiles, dashboards, analytics	2-10	CPU-based HPA
auction-service	Node.js	PostgreSQL, Redis	Auctions, bidding, auto-bid, winner notification	2-10	CPU-based HPA
buyer-protection-service	Python/FastAPI	PostgreSQL	Dispute resolution, refund guarantee, fraud	2-10	CPU-based HPA

8.1.8 Notification Domain Services

Service	Tech	DB	Purpose	Instances	Scaling
notification-service	Java/Spring Boot	PostgreSQL, MongoDB	Notification creation, routing, delivery	3-20	CPU-based HPA

push-service	Go	Redis, DynamoDB	Push notification delivery (FCM, APNS)	3-20	Custom HPA (queue depth)
sms-service	Node.js	PostgreSQL	SMS delivery via Twilio, local providers	2-10	CPU-based HPA
email-service	Python/FastAPI	PostgreSQL, S3	Email delivery, templates, campaigns	2-10	CPU-based HPA
messaging-service	Go	MongoDB, Redis	In-app messaging, group chats, encryption	3-20	CPU-based HPA
chatbot-service	Python/FastAPI	MongoDB, Redis	AI chatbots, NLP, handoff to humans	2-10	GPU-based HPA
broadcast-service	Java/Spring Boot	PostgreSQL, Kafka	Mass broadcasts, campaigns, scheduling	2-10	CPU-based HPA

8.1.9 Shared Services

Service	Tech	DB	Purpose	Instances	Scaling
api-gateway	Kong (Lua)	PostgreSQL, Redis	API gateway, auth, rate limiting, routing	3-20	CPU-based HPA

search-service	Go	Elasticsearch, Redis	Full-text search, autocomplete, suggestions	3-20	CPU-based HPA
analytics-service	Python/FastAPI	ClickHouse, S3	Event tracking, metrics, dashboards	3-20	CPU-based HPA
config-service	Go	PostgreSQL, Redis	Feature flags, configuration, A/B tests	2-10	CPU-based HPA
audit-service	Java/Spring Boot	S3, DynamoDB	Immutable audit trail, compliance logging	2-10	CPU-based HPA
localization-service	Node.js	PostgreSQL, Redis	Multi-language, translation, formatting	2-10	CPU-based HPA
media-service	Go	S3, CloudFront	Image/video processing, optimization, CDN	3-20	CPU-based HPA
ai-service	Python/FastAPI	SageMaker, Redis	AI/ML inference, model serving, training	2-10	GPU-based HPA
health-service	Go	PostgreSQL	Service health checks, dependency mapping	2-10	CPU-based HPA

scheduler-service	Python/Celery	PostgreSQL, Redis	Background jobs, cron, task scheduling	2-10	Queue-based HPA
-------------------	---------------	-------------------	--	------	-----------------

8.2 Service Communication Patterns

8.2.1 Synchronous Communication

Pattern	Use Case	Technology	Timeout	Retry
REST API	CRUD operations, external APIs	HTTP/2, OpenAPI	5s	3x with backoff
GraphQL	Complex queries, mobile optimization	Apollo Server	10s	2x with backoff
gRPC	Internal service communication	HTTP/2, Protocol Buffers	3s	3x with backoff
WebSocket	Real-time chat, live updates, streaming	Socket.io	N/A	Auto-reconnect

8.2.2 Asynchronous Communication

Pattern	Use Case	Technology	Durability	Ordering
Event Pub/Sub	Domain events, notifications	Kafka	Persistent	Partition-ordered
Message Queue	Task processing, background jobs	SQS	Persistent	FIFO available
Priority Queue	Emergency alerts, critical tasks	Redis + Custom	In-memory	Priority-ordered
Stream Processing	Analytics, real-time aggregation	Kafka Streams	Persistent	Event-time ordered
Event Sourcing	Audit trails, state reconstruction	Kafka + PostgreSQL	Persistent	Strictly ordered

8.2.3 Saga Patterns

Saga Type	Use Case	Implementation	Compensation
Orchestration Saga	Order processing	Order Service orchestrates	Reverse payment, restore inventory, cancel delivery
Choreography Saga	User registration	Services react to events	Rollback email, delete profile, revoke KYC
Parallel Saga	Bulk operations	Parallel execution with coordinator	Individual compensations per sub-transaction

8.3 Service Mesh (Istio)

All services run within an Istio service mesh with the following capabilities:

8.3.1 Traffic Management

- Virtual Services: Route traffic based on headers, weights, user segments
- Destination Rules: Load balancing (least request, round robin), circuit breakers, outlier detection
- Traffic Splitting: Canary deployments (5% → 25% → 50% → 100%), A/B testing
- Retries: 3 retries with exponential backoff (base 100ms, max 5s)
- Timeouts: Per-service timeout configuration (default 5s, critical 10s)
- Rate Limiting: Per-user, per-service, per-IP (configurable via Envoy filters)

8.3.2 Security

- mTLS: Automatic mutual TLS for all service-to-service communication
- Authorization Policies: Namespace-level, service-level, method-level RBAC

- Ingress Gateway: TLS termination, certificate management, SNI routing
- Egress Gateway: Controlled external access, traffic monitoring, policy enforcement

8.3.3 Observability

- Distributed Tracing: Automatic Jaeger tracing for all HTTP/gRPC calls
- Metrics: Prometheus metrics for request rate, latency, errors, saturation
- Access Logs: Envoy access logs with correlation IDs for all traffic
- Service Graph: Kiali visualization of service dependencies and traffic flow

8.3.4 Resilience

- Circuit Breaker: Open after 50% error rate, half-open after 30s, close after 5 consecutive successes
- Outlier Detection: Eject hosts with 5 consecutive 5xx errors, minimum ejection 10%, max ejection 50%
- Fault Injection: Chaos testing with 1% delay (100ms) and 0.1% abort (503) in staging

8.4 API Design Standards

8.4.1 REST API Standards

- Versioning: URL path versioning (/v1/users, /v2/users)
- Resource Naming: Plural nouns (/users, /orders, /events)
- HTTP Methods: GET (read), POST (create), PUT (full update), PATCH (partial update), DELETE (remove)
- Status Codes: 200 (OK), 201 (Created), 204 (No Content), 400 (Bad Request), 401 (Unauthorized), 403 (Forbidden), 404 (Not Found), 409 (Conflict), 422 (Unprocessable), 429 (Too Many Requests), 500 (Internal Error), 503 (Service Unavailable)

- Pagination: Cursor-based pagination for large datasets, offset-based for small datasets
- Filtering: Query parameters with standardized operators (eq, ne, gt, lt, in, like, between)
- Sorting: sort=-createdAt (descending), sort=+name (ascending)
- Field Selection: fields=name,email,profile (sparse fieldsets)
- Error Format: RFC 7807 Problem Details (type, title, status, detail, instance, errors)
- Rate Limiting: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset headers
- Caching: ETag, Last-Modified, Cache-Control headers
- HATEOAS: Optional hypermedia links for navigation
- Content Negotiation: Accept: application/json, Content-Type: application/json
- Compression: Brotli (preferred), gzip fallback for responses > 1KB
- OpenAPI: Every API documented with OpenAPI 3.1 specification

8.4.2 GraphQL Standards

- Schema-First: GraphQL schema defines the contract
- Federation: Apollo Federation for distributed schema composition
- Query Complexity: Maximum query depth 10, maximum complexity 1000
- Persisted Queries: Allow-list for production queries to prevent abuse
- Caching: DataLoader for N+1 prevention, Redis for query result caching
- Subscriptions: WebSocket-based for real-time updates
- Introspection: Disabled in production
- Error Handling: Structured errors with extensions for error codes

8.4.3 gRPC Standards

- Proto3: All services defined in Protocol Buffers v3
- Unary Calls: Standard request/response for CRUD operations
- Server Streaming: For real-time updates and large result sets

- Client Streaming: For bulk uploads and batch processing
- Bidirectional Streaming: For chat and live collaboration
- Status Codes: Standard gRPC status codes mapped to HTTP equivalents
- Metadata: Correlation IDs, auth tokens, trace context passed via metadata
- Deadlines: Every call has a deadline (default 5s, max 30s)
- Interceptors: Authentication, logging, metrics, retry, circuit breaker
- Health Checks: Standard gRPC health checking protocol

8.4.4 WebSocket Standards

- Protocol: Socket.io with fallback to long-polling
- Authentication: JWT token in query parameter or header
- Rooms: Topic-based room subscription (user-specific, event-specific, forum-specific)
- Heartbeat: 25s ping/pong to detect dead connections
- Reconnection: Automatic reconnection with exponential backoff (max 5s, 10 attempts)
- Message Format: JSON with type, payload, timestamp, correlationId
- Rate Limiting: 100 messages/minute per connection, 1000 messages/minute per user
- Message Size: Maximum 1MB per message
- Connection Limit: Maximum 1000 concurrent connections per server instance
- Scaling: Redis adapter for horizontal scaling across multiple Socket.io nodes

8.5 Service Dependencies & Health Checks

8.5.1 Health Check Endpoints

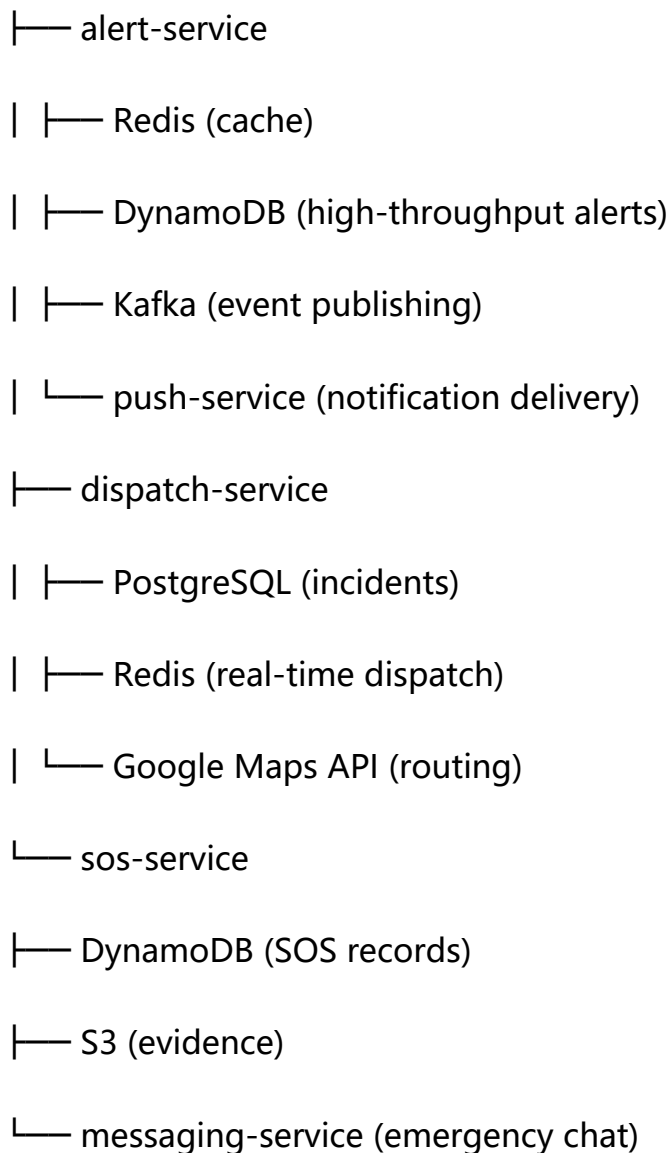
Every service exposes:

- GET /health/live — Liveness probe (service is running)

- GET /health/ready — Readiness probe (service can accept traffic)
- GET /health/startup — Startup probe (service initialization complete)
- GET /health/deep — Deep health check (includes downstream dependencies)
- GET /health/dependencies — Dependency status (database, cache, queue, external APIs)

8.5.2 Dependency Chain

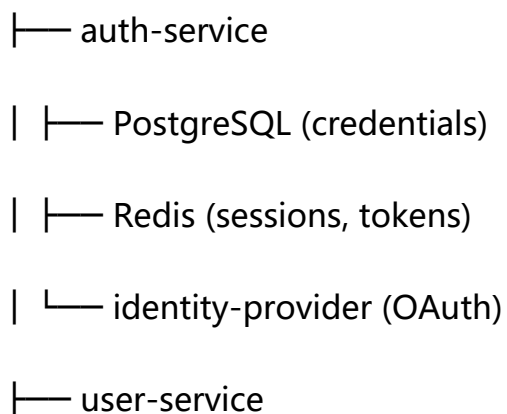
Emergency Services (Critical Path)

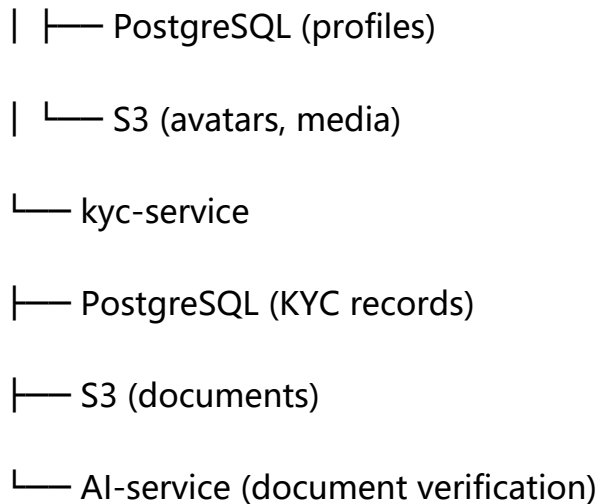


Marketplace Services (High Volume)



User Services (Foundation)





8.6 Data Consistency Strategy

8.6.1 Eventual Consistency (Default)

- Use Case: Non-critical operations, read-heavy data, analytics
- Implementation: Async events via Kafka, read from read replicas
- SLA: < 1 second consistency for 95% of operations, < 5 seconds for 99.9%
- Conflict Resolution: Last-write-wins with vector clocks, manual reconciliation for critical data

8.6.2 Strong Consistency (Critical)

- Use Case: Payments, orders, inventory, emergency alerts, KYC
- Implementation: Saga pattern with compensation, 2PC for database transactions, distributed locks via Redis Redlock
- SLA: Immediate consistency for all operations
- Conflict Resolution: Pessimistic locking, atomic operations, retry with idempotency keys

8.6.3 Causal Consistency

- Use Case: Social features, comments, likes, forum posts

- Implementation: Event sourcing with Kafka Streams, causal ordering via Lamport timestamps
- SLA: Causal ordering guaranteed, eventual visibility < 500ms

8.6.4 Session Consistency

- Use Case: User sessions, shopping carts, real-time chat
- Implementation: Sticky sessions or session affinity, read-after-write consistency via Redis
- SLA: Immediate consistency for same session, eventual for cross-session